

Temporal Planning Semantics

Mohammad Abdulaziz Lukas Koller

This is a formalisation of the semantics of temporal planning as used in the Planning Domain Definition Language. Based on that, we also develop a plan validator which is verified w.r.t. those semantics. It is based on an earlier formalisation of the semantics of classical planning [2]. This work was described in an earlier publication [1].

Contents

1	Temporal PDDL Semantics	3
1.1	Utility Functions	3
1.2	Abstract Syntax	4
1.2.1	Generic Entities	4
1.2.2	Domains	4
1.2.3	Problems	5
1.2.4	Plans	6
1.2.5	Ground Actions	6
1.3	Closed-World Assumption, Equality, and Negation	6
1.3.1	Proper Generalization	8
1.4	Happening Execution Semantics	9
1.5	Well-Formedness of PDDL	10
1.6	PDDL Semantics	15
1.7	Preservation of Well-Formedness	21
1.7.1	Well-Formed Action Instances	21
1.7.2	Preservation	27
1.7.3	Completeness of Semantics	31
2	Executable Temporal PDDL Checker	47
2.1	Generic DFS Reachability Checker	47
2.2	Implementation Refinements	50
2.2.1	Of-Type	50
2.2.2	Ground Action Interference & Application of Effects	53
2.2.3	Well-Formedness	54
2.2.4	Happening Execution	56
2.2.5	Construction of Induced Happening Sequence	57
2.3	Executable Plan Checker	95

2.4	Code Setup	97
2.4.1	Code Equations	97
2.4.2	Setup for Containers Framework	99
2.4.3	More Efficient Distinctness Check for Linorders	99
2.4.4	Parsing Rational Numbers	99
2.4.5	Code Generation	101
3	Connecting the Semantics to State-Transition Based Semantics	102
3.1	State-Transition Based Semantics	105
3.2	Connecting the State-Transition Based Semantics to Happening Sequence Based Semantics	106

1 Temporal PDDL Semantics

theory *TEMPORAL-PDDL-Semantics*

imports

Propositional-Proof-Systems.Formulas
Propositional-Proof-Systems.Sema
Propositional-Proof-Systems.Consistency
Automatic-Refinement.Misc
Automatic-Refinement.Refine-Util
HOL.Rat

begin

no-notation *insert* ($- \triangleright - [56, 55] \ 55$)

1.1 Utility Functions

abbreviation *strict-sorted::'a::{\linorder} list $\Rightarrow$ bool* **where** *strict-sorted* $\equiv$ *sorted-wrt* ($<$)

definition *index-by f l* $\equiv$ *map-of* (*map* ($\lambda x. (f \ x, x)$) *l*)

lemma *index-by-eq-Some-eq[simp]:*

assumes *distinct* (*map f l*)

shows *index-by f l n = Some x* $\longleftrightarrow$ ($x \in \text{set } l \wedge f \ x = n$)

unfolding *index-by-def*

using *assms*

by (*auto simp: o-def*)

lemma *index-by-eq-SomeD:*

shows *index-by f l n = Some x* $\implies$ ($x \in \text{set } l \wedge f \ x = n$)

unfolding *index-by-def*

by (*auto dest: map-of-SomeD*)

lemma *lookup-zip-idx-eq:*

assumes *length params = length args*

assumes $i < \text{length } \text{args}$

assumes *distinct params*

assumes $k = \text{params } ! \ i$

shows *map-of* (*zip params args*) $k = \text{Some } (\text{args } ! \ i)$

using *assms*

by (*auto simp: in-set-conv-nth*)

lemma *rtrancl-image-idem[simp]:* $R^* \circ R^* \circ s = R^* \circ s$

by (*metis relcomp-Image rtrancl-idemp-self-comp*)

1.2 Abstract Syntax

1.2.1 Generic Entities

Defines the time number type

type-synonym *time* = *rat*

type-synonym *name* = *string*

datatype *predicate* = *Pred* (*name*: *name*)

datatype *func* = *Func* (*name*: *name*)

Some of the AST entities are defined over a polymorphic '*val*' type, which gets either instantiated by variables (for domains) or objects (for problems).

An atom is either a predicate with arguments, or an equality statement.

datatype '*ent atom* = *predAtm* (*predicate*: *predicate*) (*arguments*: '*ent list*)
| *eqAtm* (*lhs*: '*ent*) (*rhs*: '*ent*)

A type is a list of primitive type names. To model a primitive type, we use a singleton list.

datatype *type* = *Either* (*primitives*: *name list*)

An effect contains a list of values to be added, and a list of values to be removed.

datatype '*ent ast-effect* = *Effect* (*adds*: ('*ent atom formula*) *list*)
(*dels*: ('*ent atom formula*) *list*)

Variables are identified by their names.

datatype *variable* = *Var* (*varname*: *name*)

Objects and constants are identified by their names

datatype *object* = *Obj* (*name*: *name*) | *FuncEnt* (*func*: *func*) (*arguments*: *object list*) | *TimeEnt* *time*

datatype *term* = *VAR* *variable* | *CONST* *object*

hide-const (**open**) *VAR CONST* — Refer to constructors by qualified names only

1.2.2 Domains

A time specifier specifies the time point of conditions and effects in a durative action

datatype *temporal-annotation* = *At-Start* | *Over-All* | *At-End*

datatype *duration-op* = *EQ* | *LEQ* | *GEQ*

```

datatype 'ent duration-constraint =
  No-Const
| Time-Const (d-op: duration-op) time
| Func-Const (d-op: duration-op) (func: func) (arguments: 'ent list)

```

An action schema has a name, a typed parameter list, a precondition, and an effect. In case of a durative action schema there is additionally a duration constraint, and the conditions and effects are timed with a time specifier

```

datatype ast-action-schema =
  Simple-Action-Schema
    (name: name)
    (parameters: (variable  $\times$  type) list)
    (precondition: term atom formula)
    (effect: term ast-effect)
| Durative-Action-Schema
    (name: name)
    (parameters: (variable  $\times$  type) list)
    (duration-constraint: term duration-constraint)
    (condition: (temporal-annotation  $\times$  (term atom formula)) list)
    (durative-effect: (temporal-annotation  $\times$  (term ast-effect)) list)

```

A predicate declaration contains the predicate's name and its argument types.

```

datatype predicate-decl = PredDecl (pred: predicate) (argTs: type list)

```

```

datatype function-decl = FuncDecl (func: func) (argTs: type list)

```

A domain contains the declarations of primitive types, predicates, and action schemas.

```

datatype ast-domain = Domain
  (types: (name  $\times$  name) list) — (type, supertype) declarations.
  (predicates: predicate-decl list)
  (functions: function-decl list)
  (consts: (object  $\times$  type) list)
  (actions: ast-action-schema list)

```

1.2.3 Problems

A fact is a predicate applied to objects.

```

type-synonym fact = predicate  $\times$  object list

```

A problem consists of a domain, a list of objects, a description of the initial state, and a description of the goal state.

```

datatype ast-problem = Problem
  (domain: ast-domain)
  (objects: (object  $\times$  type) list)

```

(*init: object atom formula list*)
 (*goal: object atom formula*)

1.2.4 Plans

datatype *plan-action* =
 Simple-Plan-Action (*name: name*) (*arguments: object list*)
 | *Durative-Plan-Action* (*name: name*) (*arguments: object list*) (*duration: time*)

type-synonym *plan* = (*time* × *plan-action*) *list*

1.2.5 Ground Actions

The following datatype represents an action scheme that has been instantiated by replacing the arguments with concrete objects, also called ground action.

datatype *ground-action* = *Ground-Action*
 (*precondition: (object atom) formula*)
 (*effect: object ast-effect*)

A happening represents a collection of grounded actions that are all executed simultaneously at a specified time point.

type-synonym *happening* = *time* × (*ground-action list*)

1.3 Closed-World Assumption, Equality, and Negation

Discriminator for atomic predicate formulas.

fun *is-predAtom* **where**
 is-predAtom (*Atom* (*predAtm* - -)) = *True* | *is-predAtom* - = *False*

fun *is-eqAtom* **where**
 is-eqAtom (*Atom* (*eqAtm* - -)) = *True* | *is-eqAtom* - = *False*

The world model is a set of (atomic) formulas

type-synonym *world-model* = *object atom formula set*

It is basic, if it only contains atoms

definition *wm-basic* $M \equiv \forall a \in M. \text{is-predAtom } a$

A valuation extracted from the atoms of the world model

definition *valuation* :: *world-model* $\Rightarrow$ *object atom valuation*
where *valuation* $M \equiv \lambda \text{predAtm } p \text{ } xs \Rightarrow \text{Atom } (\text{predAtm } p \text{ } xs) \in M \mid \text{eqAtm } a \text{ } b \Rightarrow a=b$

Augment a world model by adding negated versions of all atoms not contained in it, as well as interpretations of equality.

definition *close-world* :: *world-model* $\Rightarrow$ *world-model* **where**

close-world $M =$

$M \cup \{\neg(\text{Atom } (\text{predAtm } p \text{ as})) \mid p \text{ as. Atom } (\text{predAtm } p \text{ as}) \notin M\}$
 $\cup \{\text{Atom } (\text{eqAtm } a \text{ a}) \mid a. \text{True}\} \cup \{\neg(\text{Atom } (\text{eqAtm } a \text{ b})) \mid a \text{ b. } a \neq b\}$

definition *close-neg* $M \equiv M \cup \{\neg(\text{Atom } a) \mid a. \text{Atom } a \notin M\}$

lemma *wm-basic* $M \Longrightarrow \text{close-world } M = \text{close-neg } (M \cup \{\text{Atom } (\text{eqAtm } a \text{ a}) \mid$
a. True\})

unfolding *close-world-def close-neg-def wm-basic-def*

apply *clarsimp*

apply (*auto 0 3*)

by (*metis atom.exhaust*)

abbreviation *cw-entailment* (**infix** $^c \models =$ 53) **where**

$M ^c \models = \varphi \equiv \text{close-world } M \models \varphi$

lemma

close-world-extensive: $M \subseteq \text{close-world } M$ **and**

close-world-idem[simp]: $\text{close-world } (\text{close-world } M) = \text{close-world } M$

by (*auto simp: close-world-def*)

lemma *in-close-world-conv*:

$\varphi \in \text{close-world } M \longleftrightarrow ($

$\varphi \in M$

$\vee (\exists p \text{ as. } \varphi = \neg(\text{Atom } (\text{predAtm } p \text{ as})) \wedge \text{Atom } (\text{predAtm } p \text{ as}) \notin M)$

$\vee (\exists a. \varphi = \text{Atom } (\text{eqAtm } a \text{ a}))$

$\vee (\exists a \text{ b. } \varphi = \neg(\text{Atom } (\text{eqAtm } a \text{ b})) \wedge a \neq b)$

$)$

by (*auto simp: close-world-def*)

lemma *valuation-aux-1*:

fixes $M :: \text{world-model}$ **and** $\varphi :: \text{object atom formula}$

defines $C \equiv \text{close-world } M$

assumes $A: \forall \varphi \in C. \mathcal{A} \models \varphi$

shows $\mathcal{A} = \text{valuation } M$

using A **unfolding** $C\text{-def}$

apply (*auto simp: in-close-world-conv valuation-def Ball-def intro!: ext split:*
atom.split)

apply (*metis formula-semantics.simps(1) formula-semantics.simps(3)*)

apply (*metis formula-semantics.simps(1) formula-semantics.simps(3)*)

apply (*metis atom.collapse(2) formula-semantics.simps(1) is-predAtm-def*)

done

lemma *valuation-aux-2*:

assumes *wm-basic* M

shows $(\forall G \in \text{close-world } M. \text{valuation } M \models G)$

using *assms* **unfolding** *wm-basic-def*

by (*force simp: in-close-world-conv valuation-def elim: is-predAtm.elims is-eqAtm.elims*)

lemma *val-imp-close-world*: *valuation* $M \models \varphi \implies M^c \models \varphi$
unfolding *entailment-def*
using *valuation-aux-1*
by *blast*

lemma *close-world-imp-val*:
 $wm\text{-}basic\ M \implies M^c \models \varphi \implies valuation\ M \models \varphi$
unfolding *entailment-def* **using** *valuation-aux-2* **by** *blast*

Main theorem of this section: If a world model M contains only atoms, its induced valuation satisfies a formula φ if and only if the closure of M entails φ .

Note that there are no syntactic restrictions on φ , in particular, φ may contain negation.

theorem *valuation-iff-close-world*:
assumes *wm-basic* M
shows $valuation\ M \models \varphi \longleftrightarrow M^c \models \varphi$
using *assms val-imp-close-world close-world-imp-val* **by** *blast*

1.3.1 Proper Generalization

Adding negation and equality is a proper generalization of the case without negation and equality

fun *is-STRIPS-fmla* :: '*ent atom formula* $\Rightarrow$ *bool* **where**
is-STRIPS-fmla (*Atom* (*predAtm* -)) $\longleftrightarrow$ *True*
| *is-STRIPS-fmla* ($\perp$) $\longleftrightarrow$ *True*
| *is-STRIPS-fmla* ($\varphi_1 \wedge \varphi_2$) $\longleftrightarrow$ *is-STRIPS-fmla* $\varphi_1 \wedge$ *is-STRIPS-fmla* φ_2
| *is-STRIPS-fmla* ($\varphi_1 \vee \varphi_2$) $\longleftrightarrow$ *is-STRIPS-fmla* $\varphi_1 \wedge$ *is-STRIPS-fmla* φ_2
| *is-STRIPS-fmla* ($\neg \perp$) $\longleftrightarrow$ *True*
| *is-STRIPS-fmla* - $\longleftrightarrow$ *False*

lemma *aux1*: $\llbracket wm\text{-}basic\ M; is\text{-}STRIPS\text{-}fmla\ \varphi; valuation\ M \models \varphi; \forall G \in M. \mathcal{A} \models G \rrbracket \implies \mathcal{A} \models \varphi$
apply (*induction* φ *rule*: *is-STRIPS-fmla.induct*)
by (*auto simp*: *valuation-def*)

lemma *aux2*: $\llbracket wm\text{-}basic\ M; is\text{-}STRIPS\text{-}fmla\ \varphi; \forall \mathcal{A}. (\forall G \in M. \mathcal{A} \models G) \longrightarrow \mathcal{A} \models \varphi \rrbracket \implies valuation\ M \models \varphi$
apply (*induction* φ *rule*: *is-STRIPS-fmla.induct*)
apply *simp-all*
apply (*metis in-close-world-conv valuation-aux-2*)
using *in-close-world-conv valuation-aux-2* **apply** *blast*
using *in-close-world-conv valuation-aux-2* **by** *auto*

lemma *valuation-iff-STRIPS*:
assumes *wm-basic* M
assumes *is-STRIPS-fmla* φ


```

shows valuation  $M \models \varphi \longleftrightarrow M \models \varphi$ 
proof –
  have aux1:  $\bigwedge \mathcal{A}. [\text{valuation } M \models \varphi; \forall G \in M. \mathcal{A} \models G] \implies \mathcal{A} \models \varphi$ 
    using assms apply (induction  $\varphi$  rule: is-STRIPS-fmla.induct)
    by (auto simp: valuation-def)
  have aux2:  $[\forall \mathcal{A}. (\forall G \in M. \mathcal{A} \models G) \longrightarrow \mathcal{A} \models \varphi] \implies \text{valuation } M \models \varphi$ 
    using assms
    apply (induction  $\varphi$  rule: is-STRIPS-fmla.induct)
    apply simp-all
    apply (metis in-close-world-conv valuation-aux-2)
    using in-close-world-conv valuation-aux-2 apply blast
    using in-close-world-conv valuation-aux-2 by auto
  show ?thesis
    by (auto simp: entailment-def intro: aux1 aux2)
qed

```

Our extension to negation and equality is a proper generalization of the standard STRIPS semantics for formula without negation and equality

theorem *proper-STRIPS-generalization:*
 $[\text{wm-basic } M; \text{is-STRIPS-fmla } \varphi] \implies M^c \models \varphi \longleftrightarrow M \models \varphi$
by (*simp add: valuation-iff-close-world[symmetric] valuation-iff-STRIPS*)

1.4 Happening Execution Semantics

For this section, we fix a domain D , using Isabelle’s locale mechanism.

```

locale ast-domain =
  fixes  $D :: \text{ast-domain}$ 
begin

```

Check if two ground actions are interfering. This definition is taken from [FL03]

```

definition acts-non-intrf :: ground-action  $\Rightarrow$  ground-action  $\Rightarrow$  bool where
  acts-non-intrf  $a \longleftrightarrow$  (
    let
       $\text{add}_a = \text{set}(\text{adds}(\text{effect } a)); \text{del}_a = \text{set}(\text{dels}(\text{effect } a)); \text{pre}_a = \text{Atom } \text{'atoms}$ 
      (precondition  $a$ );
       $\text{add}_b = \text{set}(\text{adds}(\text{effect } b)); \text{del}_b = \text{set}(\text{dels}(\text{effect } b)); \text{pre}_b = \text{Atom } \text{'atoms}$ 
      (precondition  $b$ )
    in
       $\text{pre}_a \cap (\text{add}_b \cup \text{del}_b) = \{\}$   $\wedge$ 
       $\text{pre}_b \cap (\text{add}_a \cup \text{del}_a) = \{\}$   $\wedge$ 
       $\text{add}_a \cap \text{del}_b = \{\}$   $\wedge$ 
       $\text{add}_b \cap \text{del}_a = \{\}$ 
  )

```

lemma *acts-non-intrf-symmetric:*
 $\forall a \ b. \text{acts-non-intrf } a \ b \longleftrightarrow \text{acts-non-intrf } b \ a$
unfolding *acts-non-intrf-def* **by** (*auto simp: Let-def*)

It seems to be agreed upon that, in case of a contradictory effect, addition overrides deletion. We model this behaviour by first executing the deletions, and then the additions.

```
fun apply-happ :: happening  $\Rightarrow$  world-model  $\Rightarrow$  world-model where
  apply-happ (ti, Ai) M =
    (M -  $\bigcup$  (set (map (set o dels o effect) Ai)))  $\cup$   $\bigcup$  (set (map (set o adds o effect) Ai))
```

Predicate to model that the given list happenings is executable, and transforms an initial world model M into a final model M' .

Note that this definition over the list structure is more convenient in HOL than to explicitly define an indexed sequence $M_0 \dots M_N$ of intermediate world models, as done in [Lif87].

```
fun valid-happ-seq :: world-model  $\Rightarrow$  happening list  $\Rightarrow$  world-model  $\Rightarrow$  bool where
  valid-happ-seq M [] M'  $\longleftrightarrow$  (M = M')
| valid-happ-seq M ((ti, Ai) # hs) M'  $\longleftrightarrow$ 
  ( $\forall a \in \text{set } A_i. M \models^c \text{precondition } a$ )
   $\wedge$  (pairwise acts-non-intrf (set Ai))
   $\wedge$  valid-happ-seq (apply-happ (ti, Ai) M) hs M'
```

```
lemma valid-happ-seq-Mj-is-Mi:
  assumes  $\forall h \in \text{set } hs. \text{apply-happ } h \ M_i = M_i$ 
  and valid-happ-seq Mi hs Mj
  shows Mi = Mj
using assms by (induction Mi hs Mj rule: valid-happ-seq.induct) auto
```

end — Context of *ast-domain*

1.5 Well-Formedness of PDDL

```
fun ty-term where
  ty-term varT objT (term.VAR v) = varT v
| ty-term varT objT (term.CONST c) = objT c
```

```
lemma ty-term-mono: varT  $\subseteq_m$  varT'  $\Longrightarrow$  objT  $\subseteq_m$  objT'  $\Longrightarrow$ 
  ty-term varT objT  $\subseteq_m$  ty-term varT' objT'
apply (rule map-leI)
subgoal for x v
  apply (cases x)
  apply (auto dest: map-leD)
done
done
```

context *ast-domain* **begin**

The signature is a partial function that maps the predicates of the domain

to lists of argument types.

definition $\text{sig} :: \text{predicate} \rightarrow \text{type list}$ **where**
 $\text{sig} \equiv \text{map-of } (\text{map } (\lambda \text{PredDecl } p \ n \Rightarrow (p, n)) \ (\text{predicates } D))$

definition $\text{func-sig} :: \text{func} \rightarrow \text{type list}$ **where**
 $\text{func-sig} \equiv \text{map-of } (\text{map } (\lambda \text{FuncDecl } f \ n \Rightarrow (f, n)) \ (\text{functions } D))$

We use a flat subtype hierarchy, where every type is a subtype of object, and there are no other subtype relations.

Note that we do not need to restrict this relation to declared types, as we will explicitly ensure that all types used in the problem are declared.

fun subtype-edge **where**
 $\text{subtype-edge } (ty, \text{superty}) = (\text{superty}, ty)$

definition $\text{subtype-rel} \equiv \text{set } (\text{map } \text{subtype-edge } (\text{types } D))$

definition $\text{of-type} :: \text{type} \Rightarrow \text{type} \Rightarrow \text{bool}$ **where**
 $\text{of-type } oT \ T \equiv \text{set } (\text{primitives } oT) \subseteq \text{subtype-rel}^* \text{ “ set } (\text{primitives } T)$

This checks that every primitive on the LHS is contained in or a subtype of a primitive on the RHS

For the next few definitions, we fix a partial function that maps a polymorphic entity type $'e$ to types. An entity can be instantiated by variables or objects later.

context
fixes $\text{ty-ent} :: 'ent \rightarrow \text{type}$ — Entity's type, None if invalid
begin

Checks whether an entity has a given type

definition $\text{is-of-type} :: 'ent \Rightarrow \text{type} \Rightarrow \text{bool}$ **where**
 $\text{is-of-type } v \ T \longleftrightarrow ($
 $\text{case } \text{ty-ent } v \text{ of}$
 $\text{Some } vT \Rightarrow \text{of-type } vT \ T$
 $| \text{None} \Rightarrow \text{False})$

fun $\text{wf-pred-atom} :: \text{predicate} \times 'ent \text{ list} \Rightarrow \text{bool}$ **where**
 $\text{wf-pred-atom } (p, vs) \longleftrightarrow ($
 $\text{case } \text{sig } p \text{ of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } Ts \Rightarrow \text{list-all2 } \text{is-of-type } vs \ Ts)$

fun $\text{wf-func-args} :: \text{func} \times 'ent \text{ list} \Rightarrow \text{bool}$ **where**
 $\text{wf-func-args } (f, vs) \longleftrightarrow ($
 $\text{case } \text{func-sig } f \text{ of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } Ts \Rightarrow \text{list-all2 } \text{is-of-type } vs \ Ts)$

Predicate-atoms are well-formed if their arguments match the signature, equalities are well-formed if the arguments are valid objects (have a type).

TODO: We could check that types may actually overlap

```
fun wf-atom :: 'ent atom  $\Rightarrow$  bool where
  wf-atom (predAtm p vs)  $\longleftrightarrow$  wf-pred-atom (p,vs)
| wf-atom (eqAtm a b)  $\longleftrightarrow$  ty-ent a  $\neq$  None  $\wedge$  ty-ent b  $\neq$  None
```

A formula is well-formed if it consists of valid atoms, and does not contain negations, except for the encoding $\neg \perp$ of true.

```
fun wf-fmla :: ('ent atom) formula  $\Rightarrow$  bool where
  wf-fmla (Atom a)  $\longleftrightarrow$  wf-atom a
| wf-fmla ( $\perp$ )  $\longleftrightarrow$  True
| wf-fmla ( $\varphi 1 \wedge \varphi 2$ )  $\longleftrightarrow$  (wf-fmla  $\varphi 1 \wedge$  wf-fmla  $\varphi 2$ )
| wf-fmla ( $\varphi 1 \vee \varphi 2$ )  $\longleftrightarrow$  (wf-fmla  $\varphi 1 \wedge$  wf-fmla  $\varphi 2$ )
| wf-fmla ( $\neg \varphi$ )  $\longleftrightarrow$  wf-fmla  $\varphi$ 
| wf-fmla ( $\varphi 1 \rightarrow \varphi 2$ )  $\longleftrightarrow$  (wf-fmla  $\varphi 1 \wedge$  wf-fmla  $\varphi 2$ )
```

```
lemma wf-fmla  $\varphi = (\forall a \in \text{atoms } \varphi. \text{wf-atom } a)$ 
by (induction  $\varphi$ ) auto
```

Special case for a well-formed atomic predicate formula

```
fun wf-fmla-atom where
  wf-fmla-atom (Atom (predAtm a vs))  $\longleftrightarrow$  wf-pred-atom (a,vs)
| wf-fmla-atom -  $\longleftrightarrow$  False
```

```
lemma wf-fmla-atom-alt: wf-fmla-atom  $\varphi \longleftrightarrow \text{is-predAtom } \varphi \wedge \text{wf-fmla } \varphi$ 
by (cases  $\varphi$  rule: wf-fmla-atom.cases) auto
```

An effect is well-formed if the added and removed formulas are atomic

```
fun wf-effect where
  wf-effect (Effect a d)  $\longleftrightarrow (\forall ae \in \text{set } a. \text{wf-fmla-atom } ae) \wedge (\forall de \in \text{set } d. \text{wf-fmla-atom } de)$ 
```

```
fun wf-duration-const :: 'ent duration-constraint  $\Rightarrow$  bool where
  wf-duration-const No-Const  $\longleftrightarrow$  True
| wf-duration-const (Time-Const op d)  $\longleftrightarrow d \geq 0$ 
| wf-duration-const (Func-Const op f vs)  $\longleftrightarrow$ 
  (case func-sig f of
    None  $\Rightarrow$  False
  | Some Ts  $\Rightarrow$  list-all2 is-of-type vs Ts)
```

end — Context fixing *ty-ent*

```
definition constT :: object  $\rightarrow$  type where
  constT  $\equiv$  map-of (consts D)
```

An action schema is well-formed if the parameter names are distinct, and the precondition and effect is well-formed wrt. the parameters.

```

fun wf-action-schema :: ast-action-schema  $\Rightarrow$  bool where
  wf-action-schema (Simple-Action-Schema n params pre eff)  $\longleftrightarrow$  (
    let tyt = ty-term (map-of params) constT in
    distinct (map fst params)
     $\wedge$  wf-fmla tyt pre
     $\wedge$  wf-effect tyt eff)
| wf-action-schema (Durative-Action-Schema n params d cond eff)  $\longleftrightarrow$  (
  let tyt = ty-term (map-of params) constT in
  distinct (map fst params)
   $\wedge$  ( $\forall (t, c) \in \text{set cond. wf-fmla tyt } c$ )
   $\wedge$  ( $\forall (t, e) \in \text{set eff. wf-effect tyt } e \wedge t \neq \text{Over-All}$ )
   $\wedge$  wf-duration-const tyt d)

```

A type is well-formed if it consists only of declared primitive types, and the type object.

```

fun wf-type where
  wf-type (Either Ts)  $\longleftrightarrow$  set Ts  $\subseteq$  insert "object" (fst'set (types D))

```

A predicate is well-formed if its argument types are well-formed.

```

fun wf-predicate-decl where
  wf-predicate-decl (PredDecl p Ts)  $\longleftrightarrow$  ( $\forall T \in \text{set Ts. wf-type } T$ )

```

A function is well-formed if its argument types are well-formed.

```

fun wf-function-decl where
  wf-function-decl (FuncDecl f Ts)  $\longleftrightarrow$  ( $\forall T \in \text{set Ts. wf-type } T$ )

```

The types declaration is well-formed, if all supertypes are declared types (or object)

```

definition wf-types  $\equiv$  snd'set (types D)  $\subseteq$  insert "object" (fst'set (types D))

```

A domain is well-formed if

- there are no duplicate declared predicate names,
- all declared predicates are well-formed,
- there are no duplicate declared function names,
- all declared functions are well-formed,
- there are no duplicate action names,
- and all declared actions are well-formed

```

definition wf-domain :: bool where
  wf-domain  $\equiv$ 
    wf-types
     $\wedge$  distinct (map (predicate-decl.pred) (predicates D))

```

```

 $\wedge (\forall p \in \text{set } (\text{predicates } D). \text{ wf-predicate-decl } p)$ 
 $\wedge \text{distinct } (\text{map } (\text{function-decl.func}) (\text{functions } D))$ 
 $\wedge (\forall f \in \text{set } (\text{functions } D). \text{ wf-function-decl } f)$ 
 $\wedge \text{distinct } (\text{map fst } (\text{consts } D))$ 
 $\wedge (\forall (n, T) \in \text{set } (\text{consts } D). \text{ wf-type } T)$ 
 $\wedge \text{distinct } (\text{map ast-action-schema.name } (\text{actions } D))$ 
 $\wedge (\forall a \in \text{set } (\text{actions } D). \text{ wf-action-schema } a)$ 

```

end — locale *ast-domain*

We fix a problem, and also include the definitions for the domain of this problem.

```

locale ast-problem = ast-domain domain P
  for  $P :: \text{ast-problem}$ 
begin

```

We refer to the problem domain as D

abbreviation $D \equiv \text{ast-problem.domain } P$

definition $\text{objT} :: \text{object} \rightarrow \text{type}$ **where**
 $\text{objT} \equiv \text{map-of } (\text{objects } P) ++ \text{constT}$

lemma *objT-alt*: $\text{objT} = \text{map-of } (\text{consts } D @ \text{objects } P)$

unfolding *objT-def constT-def*

apply (*clarsimp*)

done

definition $\text{wf-fact} :: \text{fact} \Rightarrow \text{bool}$ **where**

$\text{wf-fact} = \text{wf-pred-atom objT}$

fun *wf-func-assign* :: $\text{object atom formula} \Rightarrow \text{bool}$ **where**

$\text{wf-func-assign } (\text{Atom } (\text{eqAtm } (\text{FuncEnt } f \text{ args}) (\text{TimeEnt } d))) \longleftrightarrow \text{wf-func-args}$
 $\text{objT } (f, \text{args})$

| $\text{wf-func-assign} - \longleftrightarrow \text{False}$

lemma *wf-func-assign-is-eqAtom*: $\text{wf-func-assign } f \Longrightarrow \text{is-eqAtom } f$

apply (*cases f*)

apply *auto*

apply (*metis is-eqAtom.simps(1) wf-func-assign.elims(2)*)

done

This definition is needed for well-formedness of the initial model, and forward-references to the concept of world model.

definition *wf-world-model* **where**

$\text{wf-world-model } M = (\forall f \in M. \text{ wf-fmla-atom objT } f)$

definition *wf-problem* **where**

wf-problem $\equiv$
wf-domain
 $\wedge \text{distinct } (\text{map fst } (\text{objects } P) \text{ @ } \text{map fst } (\text{consts } D))$
 $\wedge (\forall (n, T) \in \text{set } (\text{objects } P). \text{wf-type } T)$
 $\wedge \text{distinct } (\text{init } P)$
 $\wedge (\forall f \in \text{set } (\text{init } P). \text{wf-fmla-atom objT } f \vee \text{wf-func-assign } f)$
 $\wedge \text{wf-fmla objT } (\text{goal } P)$

fun *wf-effect-inst* :: *object ast-effect* $\Rightarrow$ *bool* **where**

wf-effect-inst (*Effect* (*a*) (*d*)) $\longleftrightarrow (\forall a \in \text{set } a \cup \text{set } d. \text{wf-fmla-atom objT } a)$

lemma *wf-effect-inst-alt*: *wf-effect-inst eff* = *wf-effect objT eff*

by (*cases eff*) *auto*

end — locale *ast-problem*

Locale to express a well-formed domain

locale *wf-ast-domain* = *ast-domain* +
assumes *wf-domain*: *wf-domain*

Locale to express a well-formed problem

locale *wf-ast-problem* = *ast-problem P* **for** *P* +
assumes *wf-problem*: *wf-problem*
begin
sublocale *wf-ast-domain domain P*
apply *unfold-locales*
using *wf-problem*
unfolding *wf-problem-def* **by** *simp*

end — locale *wf-ast-problem*

1.6 PDDL Semantics

context *ast-domain* **begin**

definition *resolve-action-schema* :: *name* $\rightarrow$ *ast-action-schema* **where**

resolve-action-schema n = *index-by ast-action-schema.name (actions D) n*

fun *subst-term* **where**

subst-term psubst (*term.VAR x*) = *psubst x*
| *subst-term psubst* (*term.CONST c*) = *c*

To instantiate an action schema, we first compute a substitution from parameters to objects, and then apply this substitution to the precondition and effect. The substitution is applied via the *map-xxx* functions generated by the datatype package.

fun *instantiate-action-schema* :: *ast-action-schema* $\Rightarrow$ *object list* $\Rightarrow$ *ground-action*
where

```

instantiate-action-schema (Simple-Action-Schema n params pre eff) args =
  (let
    tsubst = subst-term (the o (map-of (zip (map fst params) args)));
    pre-inst = (map-formula o map-atom) tsubst pre;
    eff-inst = (map-ast-effect) tsubst eff
  in
    Ground-Action pre-inst eff-inst
  )

```

Filter a given timed-list by a time specifier. (**at start ...**), (**at end ...**), or (**over all ...**) conditions/effects

definition *filter-time-spec* :: temporal-annotation $\Rightarrow$ (temporal-annotation $\times$ 'a) list $\Rightarrow$ 'a list **where**

```

filter-time-spec t l = map snd (filter (((=) t) o fst) l)

```

lemma *filter-time-spec-correct*: $\forall x \in \text{set } (\text{filter-time-spec } t \text{ } xs). (t, x) \in \text{set } xs$

unfolding *filter-time-spec-def*

by (induction *xs* arbitrary: *t*) auto

auxiliary-function to conjunct multiple effects into one single effect. Needed to construct effect after filtering by a time specifier

definition *conjunct-effects* :: ('a ast-effect) list $\Rightarrow$ 'a ast-effect ($\bigwedge_{eff} - [40] 40$) **where**

```

conjunct-effects l = Effect ((concat o (map adds)) l) ((concat o (map dels)) l)

```

instantiate a given Durative Action Schema; Note: if an action schema is well formed then the effect for the snap action for the temporal annotation (**over all ...**) will be an empty effect (**Effect [] []**)

fun *inst-snap-action* :: ast-action-schema $\Rightarrow$ object list $\Rightarrow$ temporal-annotation $\Rightarrow$ ground-action **where**

```

inst-snap-action (Durative-Action-Schema n params d cond eff) args ta =
  (let
    tsubst = subst-term (the o (map-of (zip (map fst params) args)));
    prei = (map-formula o map-atom) tsubst (BigAnd (filter-time-spec ta cond));
    effi = (map-ast-effect) tsubst ( $\bigwedge_{eff}$  (filter-time-spec ta eff))
  in
    Ground-Action prei effi
  )

```

end — Context of *ast-domain*

context *wf-ast-domain* **begin**

Resolving an action yields a well-founded action schema.

lemma *resolve-action-wf*:

assumes *resolve-action-schema* *n* = Some *a*

shows *wf-action-schema* *a*

proof —


```

from wf-domain have
  X1: distinct (map ast-action-schema.name (actions D))
  and X2:  $\forall a \in \text{set } (\text{actions } D). \text{wf-action-schema } a$ 
  unfolding wf-domain-def by auto

show ?thesis
  using assms unfolding resolve-action-schema-def
  by (auto simp add: index-by-eq-Some-eq[OF X1] X2)
qed

end — Context of ast-domain

context ast-problem begin

Initial model

definition I :: world-model where
  I  $\equiv \text{set } (\text{filter is-predAtom } (\text{init } P))$ 

Resolve a plan action and instantiate the referenced action schema.

fun res-inst :: plan-action  $\rightarrow$  ground-action where
  res-inst (Simple-Plan-Action n args) =
    Some (instantiate-action-schema (the (resolve-action-schema n)) args)
  | res-inst (Durative-Plan-Action - -) = None

fun res-inst-snap-action :: plan-action  $\Rightarrow$  temporal-annotation  $\rightarrow$  ground-action
where
  res-inst-snap-action (Durative-Plan-Action n args d) ta =
    Some (inst-snap-action (the (resolve-action-schema n)) args ta)
  | res-inst-snap-action (Simple-Plan-Action - -) = None

Check whether object has specified type

definition is-obj-of-type n T  $\equiv \text{case objT } n \text{ of}$ 
  None  $\Rightarrow$  False
  | Some oT  $\Rightarrow$  of-type oT T

We can also use the generic is-of-type function.

lemma is-obj-of-type-alt: is-obj-of-type = is-of-type objT
apply (intro ext)
unfolding is-obj-of-type-def is-of-type-def by auto

HOL encoding of matching an action's formal parameters against an argument list. The parameters of the action are encoded as a list of name $\times$ type pairs, such that we map it to a list of types first. Then, the list relator list-all2 checks that arguments and types have the same length, and each matching pair of argument and type satisfies the predicate is-obj-of-type.

definition action-params-match a args
   $\equiv \text{list-all2 is-obj-of-type args } (\text{map snd } (\text{parameters } a))$ 

```

All functions are required to be constant and defined in the initial state. Therefore all duration constraints must be satisfied by the initial state.

definition *func-eval* :: *object* $\rightarrow$ *rat* **where**
func-eval $\equiv$ *map-of* (*fold* (λa *fev*. *case a of* *Atom* (*eqAtm f (TimeEnt t)*) $\Rightarrow$ (*f,t*)#*fev* | $- \Rightarrow$ *fev*) (*init P*) [])

fun *duration-matches* :: *time* $\Rightarrow$ *term duration-constraint* $\Rightarrow$ (*variable* $\times$ *type*)
list $\Rightarrow$ *object list* $\Rightarrow$ *bool* **where**
duration-matches d No-Const params args $\longleftrightarrow d \geq 0$
| *duration-matches d (Time-Const op d')* *params args* $\longleftrightarrow$
(*case op of*
LEQ $\Rightarrow d \leq d'$
| *EQ* $\Rightarrow d = d'$
| *GEQ* $\Rightarrow d \geq d'$)
| *duration-matches d (Func-Const op f vs)* *params args* $\longleftrightarrow$
(*let tsubst* = *subst-term* (*the o* (*map-of* (*zip* (*map fst params*) *args*))) *in*
case func-eval (FuncEnt f (map tsubst vs)) of
None $\Rightarrow$ *False*
| *Some d'* $\Rightarrow$ (*case op of*
LEQ $\Rightarrow d \leq d'$
| *EQ* $\Rightarrow d = d'$
| *GEQ* $\Rightarrow d \geq d'$))

At this point, we can define well-formedness of a plan action: The action must refer to a declared action schema, the arguments must be compatible with the formal parameters' types.

fun *wf-plan-action* :: *plan-action* $\Rightarrow$ *bool* **where**
wf-plan-action (Simple-Plan-Action n args) = (
case resolve-action-schema n of
None $\Rightarrow$ *False*
| *Some (Simple-Action-Schema n params pre eff)* $\Rightarrow$
action-params-match (Simple-Action-Schema n params pre eff) args
| *Some (Durative-Action-Schema n params d cond eff)* $\Rightarrow$ *False*)
| *wf-plan-action (Durative-Plan-Action n args d)* = (
case resolve-action-schema n of
None $\Rightarrow$ *False*
| *Some (Simple-Action-Schema n params pre eff)* $\Rightarrow$ *False*
| *Some (Durative-Action-Schema n params dconst cond eff)* $\Rightarrow$
action-params-match (Durative-Action-Schema n params dconst cond eff)
args
 $\wedge$ *duration-matches d dconst params args* $\wedge d \geq 0$)

A plan is wellformed if all plan actions are wellformed and all starting time points are greater-equal 0.

definition *wf-plan* :: *plan* $\Rightarrow$ *bool* **where**
wf-plan $\pi s \longleftrightarrow (\forall (t, \pi) \in \text{set } \pi s. \text{wf-plan-action } \pi \wedge t \geq 0)$

definition *is-act-simple* :: *plan-action* $\Rightarrow$ *bool* **where**

is-act-simple $\pi \longleftrightarrow (\text{case } \pi \text{ of } (\text{Simple-Plan-Action } -) \Rightarrow \text{True} \mid - \Rightarrow \text{False})$

definition *simple-acts* :: *plan* $\Rightarrow$ (*time* $\times$ *plan-action*) *set* **where**
simple-acts $\pi s = \text{set } (\text{filter } (\text{is-act-simple } o \text{ snd}) \pi s)$

lemma *simple-acts-subset*: *simple-acts* $\pi s \subseteq \text{set } \pi s$
unfolding *simple-acts-def* **by** *auto*

definition *durative-acts* :: *plan* $\Rightarrow$ (*time* $\times$ *plan-action*) *set* **where**
durative-acts $\pi s = \text{set } (\text{filter } ((\text{HOL.Not}) o \text{is-act-simple } o \text{ snd}) \pi s)$

lemma *dur-acts-subset*: *durative-acts* $\pi s \subseteq \text{set } \pi s$
unfolding *durative-acts-def* **by** *auto*

Definition of a happening time point. Happening time point specify time points at, which the current state can change. Therefore every start and end of any plan action is a happening time point.

definition *is-htp* :: *plan* $\Rightarrow$ *time* $\Rightarrow$ *bool* **where**
is-htp $\pi s \ t_i \longleftrightarrow (\exists \pi. (t_i, \pi) \in \text{set } \pi s) \vee (\exists (t_\pi, \pi) \in \text{durative-acts } \pi s. t_i = t_\pi + (\text{duration } \pi))$

Predicate for sequence of happening time points.

definition *htps-seq* :: *plan* $\Rightarrow$ *time list* $\Rightarrow$ *bool* **where**
htps-seq $\pi s \ \text{htps} \longleftrightarrow \text{strict-sorted } \text{htps} \wedge (\forall t. t \in \text{set } \text{htps} \longleftrightarrow \text{is-htp } \pi s \ t)$

Predicate for consecutive happening time points: There is no happening time point in between.

definition *consec-htps* :: *plan* $\Rightarrow$ *time* $\Rightarrow$ *time* $\Rightarrow$ *bool* **where**
consec-htps $\pi s \ t_i \ t_j \longleftrightarrow (t_i < t_j \wedge \text{is-htp } \pi s \ t_i \wedge \text{is-htp } \pi s \ t_j \wedge (\forall t. \text{is-htp } \pi s \ t \longrightarrow t \leq t_i \vee t_j \leq t))$

Predicate to check is a grounded action is a instantiation of a plan action for an induced happening sequence.

fun *inst-of-plan-action* :: *plan* $\Rightarrow$ *time* $\times$ *plan-action* $\Rightarrow$ *time* $\times$ *ground-action* $\Rightarrow$ *bool* **where**
inst-of-plan-action $\pi s \ (t_\pi, \pi) \ (t_a, a) \longleftrightarrow (\text{res-inst } \pi = \text{Some } a \wedge t_\pi = t_a) \vee (\text{res-inst-snap-action } \pi \ \text{At-Start} = \text{Some } a \wedge t_\pi = t_a) \vee (\text{res-inst-snap-action } \pi \ \text{At-End} = \text{Some } a \wedge t_\pi + (\text{duration } \pi) = t_a) \vee (\text{res-inst-snap-action } \pi \ \text{Over-All} = \text{Some } a \wedge (\exists t_i \ t_j. \text{consec-htps } \pi s \ t_i \ t_j \wedge t_\pi \leq t_i \wedge t_j \leq t_\pi + (\text{duration } \pi) \wedge t_i < t_a \wedge t_a < t_j)))$

abbreviation *happ-member* (**infix** $\in_h$ 53) **where**

$$ta \in_h hs \equiv (\text{case } ta \text{ of } (t,a) \Rightarrow (\exists as. (t,as) \in set\ hs \wedge a \in set\ as))$$

Definition of an induced happening sequence.

- for every simple action (t, π) there is an instantiation at time point t
- for every durative action (t, π)
 - there is a ground action at time point t , which contains all conditions and effect of π with time specifier (**at start** ...)
 - there is a ground action at time point $t + (\text{duration } \pi)$, which contains all conditions and effect of π with time specifier (**at end** ...)
 - in between every consecutive happening-time-points $t_i\ t_j$ during the execution of π there is a ground action at some time point t' (in between t_i & t_j), which contains all conditions of π with time specifier (**over all** ...), the effect of is empty
- additionally an induced happening sequence is strictly sorted and contains no other actions besides the ones specified above.

definition *ind-happ-seq* :: *plan* $\Rightarrow$ *happening list* $\Rightarrow$ *bool* **where**

$$\begin{aligned} \text{ind-happ-seq } \pi s\ hs \iff & (\text{strict-sorted } (\text{map fst } hs)) \\ & \wedge (\forall (t_\pi, \pi) \in \text{simple-acts } \pi s. \\ & \quad \text{let } g_a = \text{the } (\text{res-inst } \pi) \text{ in} \\ & \quad \exists A. (t_\pi, A) \in set\ hs \wedge g_a \in set\ A) \\ & \wedge (\forall (t_\pi, \pi) \in \text{durative-acts } \pi s. \\ & \quad \text{let } \pi_{start} = \text{the } (\text{res-inst-snap-action } \pi \text{ At-Start}); \\ & \quad \pi_{end} = \text{the } (\text{res-inst-snap-action } \pi \text{ At-End}); \\ & \quad \pi_{inv} = \text{the } (\text{res-inst-snap-action } \pi \text{ Over-All}) \text{ in} \\ & \quad (\exists A. (t_\pi, A) \in set\ hs \wedge \pi_{start} \in set\ A) \wedge \\ & \quad (\exists A. (t_\pi + (\text{duration } \pi), A) \in set\ hs \wedge \pi_{end} \in set\ A) \wedge \\ & \quad (\forall t_i\ t_j. (\text{consec-htps } \pi s\ t_i\ t_j \wedge t_\pi \leq t_i \wedge t_j \leq t_\pi + (\text{duration } \pi)) \\ & \quad \longrightarrow (\exists (t', A) \in set\ hs. t_i < t' \wedge t' < t_j \wedge \pi_{inv} \in set\ A))) \\ & \wedge (\forall (t_i, A_i) \in set\ hs. A_i \neq [] \wedge \\ & \quad (\forall a \in set\ A_i. \exists (t_\pi, \pi) \in set\ \pi s. \text{inst-of-plan-action } \pi s\ (t_\pi, \pi)\ (t_i, a)))) \end{aligned}$$

lemma *ind-happ-seq-props*:

assumes *ind-happ-seq* $\pi s\ hs$

shows *strict-sorted* (*map fst* *hs*)

$$(t_\pi, \pi) \in \text{simple-acts } \pi s \implies \exists A. (t_\pi, A) \in set\ hs \wedge \text{the } (\text{res-inst } \pi) \in set\ A$$

$$(t_\pi, \pi) \in \text{durative-acts } \pi s \implies \exists A. (t_\pi, A) \in set\ hs \wedge \text{the } (\text{res-inst-snap-action } \pi \text{ At-Start}) \in set\ A$$

$$(t_\pi, \pi) \in \text{durative-acts } \pi s \implies \exists A. (t_\pi + (\text{duration } \pi), A) \in set\ hs \wedge \text{the } (\text{res-inst-snap-action } \pi \text{ At-End}) \in set\ A$$

$$[(t_\pi, \pi) \in \text{durative-acts } \pi s; \text{consec-htps } \pi s\ t_i\ t_j; t_\pi \leq t_i; t_j \leq t_\pi + (\text{duration } \pi)]$$

$\implies \exists (t', A) \in \text{set } hs. t_i < t' \wedge t' < t_j \wedge \text{the } (res-inst-snap-action \pi$
Over-All) $\in \text{set } A$
 $(t_i, A_i) \in \text{set } hs \implies A_i \neq []$
 $[(t_i, A_i) \in \text{set } hs ; a \in \text{set } A_i] \implies \exists (t_\pi, \pi) \in \text{set } \pi s. inst-of-plan-action \pi s$
 $(t_\pi, \pi) (t_i, a)$
using *assms*
by(*auto simp: ind-happ-seq-def*)

A sequence of plan actions form a path, if they are well-formed and their instantiations form a path.

definition *plan-happ-path* :: *world-model* $\Rightarrow$ *plan* $\Rightarrow$ *world-model* $\Rightarrow$ *bool* **where**
plan-happ-path *M* πs *M'* $\longleftrightarrow (\exists hs. ind-happ-seq \pi s hs \wedge valid-happ-seq M hs$
M')

A plan is valid wrt. a given initial model, if it forms a path to a goal model

definition *valid-plan-from2* :: *world-model* $\Rightarrow$ *plan* $\Rightarrow$ *bool* **where**
valid-plan-from2 *M* $\pi s \longleftrightarrow (wf-plan \pi s \wedge (\exists M'. plan-happ-path M \pi s M' \wedge$
M' c $\models (goal P)))$

Finally, a plan is valid if it is valid wrt. the initial world model *I*

definition *valid-plan2* :: *plan* $\Rightarrow$ *bool*
where *valid-plan2* $\equiv valid-plan-from2 I$

Concise definition used in paper:

lemma *valid-plan2* $\pi s \equiv wf-plan \pi s \wedge (\exists M'. plan-happ-path I \pi s M' \wedge M' c \models$
 $(goal P))$
unfolding *valid-plan2-def valid-plan-from2-def* **by** *auto*

end — Context of *ast-problem*

1.7 Preservation of Well-Formedness

1.7.1 Well-Formed Action Instances

The goal of this section is to establish that well-formedness of world models is preserved by execution of well-formed plan actions.

context *ast-problem* **begin**

As plan actions are executed by first instantiating them, and then executing the action instance, it is natural to define a well-formedness concept for action instances.

fun *wf-ground-action* :: *ground-action* $\Rightarrow$ *bool* **where**
wf-ground-action (*Ground-Action pre eff*) $\longleftrightarrow (wf-fmla objT pre \wedge wf-effect$
 $objT eff)$

We first prove that instantiating a well-formed action schema will yield a well-formed action instance.

We begin with some auxiliary lemmas before the actual theorem.

lemma (in *ast-domain*) *of-type-refl*[simp, intro!]: *of-type* T T
unfolding *of-type-def* **by** *auto*

lemma (in *ast-domain*) *of-type-trans*[trans]:
of-type $T1$ $T2 \implies of-type\ T2\ T3 \implies of-type\ T1\ T3$
unfolding *of-type-def*
by *clarsimp* (meson in-mono rtrancl-image-unfold-right rtrancl-reachable-induct)

lemma *is-of-type-map-ofE*:
assumes *is-of-type* (map-of params) x T
obtains i xT **where** $i < \text{length params}$ $\text{params}!i = (x, xT)$ *of-type* xT T
using *assms*
unfolding *is-of-type-def*
by (auto split: option.splits dest!: map-of-SomeD simp: in-set-conv-nth)

lemma *wf-atom-mono*:
assumes *SS*: $\text{tys} \subseteq_m \text{tys}'$
assumes *WF*: *wf-atom* tys a
shows *wf-atom* tys' a
proof –
have *list-all2* (*is-of-type* tys') xs Ts **if** *list-all2* (*is-of-type* tys) xs Ts **for** xs Ts
using *that*
apply *induction*
by (auto simp: *is-of-type-def* split: option.splits dest: map-leD[OF *SS*])
with *WF* **show** ?thesis
by (cases a) (auto split: option.splits dest: map-leD[OF *SS*])
qed

lemma *wf-fmla-atom-mono*:
assumes *SS*: $\text{tys} \subseteq_m \text{tys}'$
assumes *WF*: *wf-fmla-atom* tys a
shows *wf-fmla-atom* tys' a
proof –
have *list-all2* (*is-of-type* tys') xs Ts **if** *list-all2* (*is-of-type* tys) xs Ts **for** xs Ts
using *that*
apply *induction*
by (auto simp: *is-of-type-def* split: option.splits dest: map-leD[OF *SS*])
with *WF* **show** ?thesis
by (cases a rule: *wf-fmla-atom.cases*) (auto split: option.splits dest: map-leD[OF *SS*])
qed

lemma *constT-ss-objT*: *constT* $\subseteq_m \text{objT}$
unfolding *constT-def* *objT-def*
apply *rule*
by (auto simp: map-add-def split: option.split)

lemma *wf-atom-constT-imp-objT*: *wf-atom (ty-term Q constT) a $\implies$ wf-atom (ty-term Q objT) a*
apply (*erule wf-atom-mono[rotated]*)
apply (*rule ty-term-mono*)
by (*simp-all add: constT-ss-objT*)

lemma *wf-fmla-atom-constT-imp-objT*: *wf-fmla-atom (ty-term Q constT) a $\implies$ wf-fmla-atom (ty-term Q objT) a*
apply (*erule wf-fmla-atom-mono[rotated]*)
apply (*rule ty-term-mono*)
by (*simp-all add: constT-ss-objT*)

context
fixes *Q and f :: variable $\Rightarrow$ object*
assumes *INST: is-of-type Q x T $\implies$ is-of-type objT (f x) T*
begin

lemma *is-of-type-var-conv*: *is-of-type (ty-term Q objT) (term.VAR x) T $\longleftrightarrow$ is-of-type Q x T*
unfolding *is-of-type-def* **by** (*auto*)

lemma *is-of-type-const-conv*: *is-of-type (ty-term Q objT) (term.CONST x) T $\longleftrightarrow$ is-of-type objT x T*
unfolding *is-of-type-def*
by (*auto split: option.split*)

lemma *INST'*: *is-of-type (ty-term Q objT) x T $\implies$ is-of-type objT (subst-term f x) T*
apply (*cases x*) **using** *INST* **apply** (*auto simp: is-of-type-var-conv is-of-type-const-conv*)
done

lemma *wf-inst-eq-aux*: *Q x = Some T $\implies$ objT (f x) $\neq$ None*
using *INST[of x T]* **unfolding** *is-of-type-def*
by (*auto split: option.splits*)

lemma *wf-inst-eq-aux'*: *ty-term Q objT x = Some T $\implies$ objT (subst-term f x) $\neq$ None*
by (*cases x*) (*auto simp: wf-inst-eq-aux*)

lemma *wf-inst-atom*:
assumes *wf-atom (ty-term Q constT) a*
shows *wf-atom objT (map-atom (subst-term f) a)*
proof –
have *X1: list-all2 (is-of-type objT) (map (subst-term f) xs) Ts* **if**
list-all2 (is-of-type (ty-term Q objT)) xs Ts **for** *xs Ts*
using *that*

```

apply induction
using INST'
by auto
then show ?thesis
using assms[THEN wf-atom-constT-imp-objT] wf-inst-eq-aux'
by (cases a; auto split: option.splits)

```

qed

```

lemma wf-inst-formula-atom:
assumes wf-fmla-atom (ty-term Q constT) a
shows wf-fmla-atom objT ((map-formula o map-atom o subst-term) f a)
using assms[THEN wf-fmla-atom-constT-imp-objT] wf-inst-atom
apply (cases a rule: wf-fmla-atom.cases; auto split: option.splits)
by (simp add: INST' list.rel-map(1) list-all2-mono)

```

```

lemma wf-inst-effect:
assumes wf-effect (ty-term Q constT)  $\varphi$ 
shows wf-effect objT ((map-ast-effect o subst-term) f  $\varphi$ )
using assms
proof (induction  $\varphi$ )
  case (Effect x1a x2a)
  then show ?case using wf-inst-formula-atom by auto
qed

```

```

lemma wf-inst-formula:
assumes wf-fmla (ty-term Q constT)  $\varphi$ 
shows wf-fmla objT ((map-formula o map-atom o subst-term) f  $\varphi$ )
using assms
by (induction  $\varphi$  (auto simp: wf-inst-atom dest: wf-inst-eq-aux))

```

```

lemma wf-BigAnd-aux:  $\forall x \in \text{set } xs. \text{wf-fmla } \text{tyt } x \implies \text{wf-fmla } \text{tyt } (\text{BigAnd } xs)$ 
by (induction xs auto)

```

```

lemma wf-BigAnd-filter-time-spec:
fixes n params d cond eff
defines a  $\equiv$  Durative-Action-Schema n params d cond eff
assumes wf-action-schema a
shows wf-fmla (ty-term (map-of params) constT) (BigAnd (filter-time-spec ta cond))
using assms filter-time-spec-correct wf-BigAnd-aux
by (fastforce simp: Let-def)

```

```

lemma wf-conjunct-effects-aux:  $\forall x \in \text{set } xs. \text{wf-effect } \text{tyt } x \implies \text{wf-effect } \text{tyt } (\bigwedge_{eff} xs)$ 
unfolding conjunct-effects-def
apply (induction xs)
apply auto
apply (metis ast-effect.collapse wf-effect.simps)+

```



```

done

lemma wf-conj-eff-filter-time-spec:
  fixes n params d cond eff
  defines a  $\equiv$  Durative-Action-Schema n params d cond eff
  assumes wf-action-schema a
  shows wf-effect (ty-term (map-of params) constT) ( $\bigwedge_{eff}$  (filter-time-spec ta
eff))
  using assms filter-time-spec-correct wf-conjunct-effects-aux
  by (fastforce simp: Let-def)

end

```

Next two theorems show that, instantiating a well-formed action schema with compatible arguments will yield a well-formed action instance.

```

theorem wf-inst-action-schema:
  fixes n params pre eff
  defines a  $\equiv$  Simple-Action-Schema n params pre eff
  assumes action-params-match a args
  assumes wf-action-schema a
  shows wf-ground-action (instantiate-action-schema a args)
proof -
  have INST:
    is-of-type objT ((the  $\circ$  map-of (zip (map fst params) args)) x) T
  if is-of-type (map-of params) x T for x T
  using that
  apply (rule is-of-type-map-ofE)
  using assms
  apply (clarsimp simp: Let-def)
  subgoal for i xT
    unfolding action-params-match-def
    apply (subst lookup-zip-idx-eq[where i=i];
      (clarsimp simp: list-all2-lengthD)?)
    apply (frule list-all2-nthD2[where p=i]; simp?)
    apply (auto
      simp: is-obj-of-type-alt is-of-type-def
      intro: of-type-trans
      split: option.splits)
  done
done
then show ?thesis
  using assms wf-inst-formula wf-inst-effect
  by (fastforce split: term.splits simp: Let-def comp-apply[abs-def])
qed

```

```

theorem wf-inst-durative-action-schema:
  fixes n params d cond eff
  defines a  $\equiv$  Durative-Action-Schema n params d cond eff
  assumes action-params-match a args

```

```

assumes wf-action-schema a
shows wf-ground-action (inst-snap-action a args ta)
proof –
  have INST:
    is-of-type objT ((the ◦ map-of (zip (map fst params) args)) x) T
    if is-of-type (map-of params) x T for x T
    using that
    apply (rule is-of-type-map-ofE)
    using assms
    apply (clarsimp simp: Let-def)
    subgoal for i xT
      unfolding action-params-match-def
      apply (subst lookup-zip-idx-eq[where i=i];
        (clarsimp simp: list-all2-lengthD)?)
      apply (frule list-all2-nthD2[where p=i]; simp?)
      apply (auto
        simp: is-obj-of-type-alt is-of-type-def
        intro: of-type-trans
        split: option.splits)
    done
  done
then show ?thesis
using assms
  wf-inst-formula[OF - wf-BigAnd-filter-time-spec,
    where f1=λx. the (map-of (zip (map fst params) args) x)
    and params1=params and Q1=(map-of params)]
  wf-inst-effect[OF - wf-conj-eff-filter-time-spec,
    where f1=λx. the (map-of (zip (map fst params) args) x)
    and params1=params and Q1=(map-of params)]
apply (auto split: term.splits simp: Let-def comp-apply[abs-def])
done
qed

lemma wf-over-all-empty-eff:
  assumes wf-plan-action π
    and wf-action-schema aschema
    and Some aschema = resolve-action-schema (name π)
    and Some a = res-inst-snap-action π Over-All
  shows effect a = Effect [] []
using assms
proof (cases aschema)
case (Simple-Action-Schema n params pre eff)
then show ?thesis
  using assms
  by (cases π) (auto split: option.splits)
next
case (Durative-Action-Schema n params d cond eff)
then have ∀ (t,e) ∈ set eff. t ≠ Over-All
  using ⟨wf-action-schema aschema⟩ by (auto simp: Let-def)

```

```

then have ( $\bigwedge_{eff} (filter-time-spec \text{ Over-All } eff) = Effect$ ) [] []
  unfolding filter-time-spec-def conjunct-effects-def by auto
then show ?thesis
  using assms Durative-Action-Schema
  by (cases  $\pi$ ) (auto split: option.splits)
qed

```

end — Context of *ast-problem*

1.7.2 Preservation

context *ast-problem* **begin**

```

fun happ-precond :: happening  $\Rightarrow$  world-model  $\Rightarrow$  bool where
  happ-precond (t,A) M  $\longleftrightarrow$  ( $\forall a \in set A. M \models_{\text{c}} precondition\ a$ )

```

```

fun happ-non-intrf :: happening  $\Rightarrow$  bool where
  happ-non-intrf (t,A)  $\longleftrightarrow$  ( $\forall a \in set A. \forall b \in set A. a \neq b \longrightarrow acts-non-intrf\ a\ b$ )

```

Shorthand for enabled happening: all preconditions satisfied, no interfering actions, & all ground actions are wellformed .

```

fun happ-enabled :: happening  $\Rightarrow$  world-model  $\Rightarrow$  bool where
  happ-enabled (t,A) M  $\longleftrightarrow$  (happ-precond (t,A) M  $\wedge$  happ-non-intrf (t,A)  $\wedge$ 
( $\forall a \in set A. wf-ground-action\ a$ ))

```

Shorthand for wellformed-ness of a happening sequence: all ground actions wellformed .

```

fun wf-happ-seq :: happening list  $\Rightarrow$  bool where
  wf-happ-seq hs  $\longleftrightarrow$  ( $\forall (t,A) \in set\ hs. (\forall a \in set A. wf-ground-action\ a)$ )

```

end — Context of *ast-problem*

context *wf-ast-problem* **begin**

The initial world model is well-formed

```

lemma wf-I: wf-world-model I
  using wf-problem
  unfolding I-def wf-world-model-def wf-problem-def
  apply(safe)
  subgoal for f
    apply (induction f)
    apply auto
    by (metis is-predAtom.elims(2) wf-func-assign.simps(2))
  done

```

```

lemma wf-ground-action-wf-fmla-atom:
  wf-ground-action a  $\implies \forall x \in set ((adds\ o\ effect)\ a). wf-fmla-atom\ objT\ x$ 

```

using *wf-effect-inst-alt*[*symmetric*] *wf-effect.elims*(2)
by (*cases a*) *force*

The Application of a happening preserves well-formedness

theorem *wf-apply-happ*:
assumes *happ-enabled* (*t,as*) *s*
and *wf-world-model* *s*
shows *wf-world-model* (*apply-happ* (*t,as*) *s*)
using *assms wf-ground-action-wf-fmla-atom* **by** (*auto simp: wf-world-model-def*)

theorem *wf-apply-happ-compact-notation*:
happ-enabled (*t,as*) *s* $\implies$ *wf-world-model* *s* $\implies$ *wf-world-model* (*apply-happ* (*t,as*) *s*)
by (*rule wf-apply-happ*)

corollary *wf-valid-happ-seq*:
assumes *wf-world-model* *M*
and *wf-happ-seq* *hs*
and *valid-happ-seq* *M hs M'*
shows *wf-world-model* *M'*
using *assms wf-apply-happ* **by** (*induction hs arbitrary: M*) (*auto split: prod.splits simp: pairwise-def*)

In a wellformed plan the action parameters match for every plan action

lemma *wf-plan-action-params-match*:
assumes *wf-plan* πs
and (*t,a*) $\in$ *set* πs
and *a_{schema}* = *the* (*resolve-action-schema* (*name a*))
shows *action-params-match* *a_{schema}* (*arguments a*)
using *assms* **by** (*cases a*) (*auto simp: wf-plan-def split: ast-action-schema.splits option.splits*)

lemma *simple-plan-action-schema-type1*:
assumes *wf-plan-action* (*Simple-Plan-Action* *n args*)
shows $\exists$ *params pre eff. resolve-action-schema n = Some (Simple-Action-Schema*
n params pre eff)
proof (*cases resolve-action-schema n*)
case *None*
then show *?thesis*
using *assms* **by** *auto*
next
case (*Some a*)
from *wf-domain* **have** [*simp*]: *distinct* (*map ast-action-schema.name* (*actions*
D))
unfolding *wf-domain-def* **by** *auto*
then show *?thesis*
using *assms* $\langle$ *resolve-action-schema n = Some a* $\rangle$ *resolve-action-schema-def*
by (*cases a*) *fastforce+*
qed

```

lemma durative-plan-action-schema-type1:
  assumes wf-plan-action (Durative-Plan-Action n args t')
  shows  $\exists$  params d cond eff. resolve-action-schema n
    = Some (Durative-Action-Schema n params d cond eff)
proof (cases resolve-action-schema n)
  case None
  then show ?thesis
    using assms by auto
next
  case (Some a)
  from wf-domain have [simp]: distinct (map ast-action-schema.name (actions
D))
    unfolding wf-domain-def by auto
  then show ?thesis
    using assms  $\langle$ resolve-action-schema n = Some a $\rangle$  resolve-action-schema-def
    by (cases a) fastforce +
qed

```

Instantiating a wellformed plan action always produces a wellformed ground action

```

lemma wf-inst-of-plan-action:
  assumes wf-plan  $\pi$  s
    and  $(t_\pi, \pi) \in \text{set } \pi s$ 
    and inst-of-plan-action  $\pi s (t_\pi, \pi) (t_i, a)$ 
  shows wf-ground-action a
  using assms
proof –
  have wf- $\pi$ : wf-plan-action  $\pi$ 
    using assms wf-plan-def by blast
  obtain ta where res-inst  $\pi$  = Some a  $\vee$  res-inst-snap-action  $\pi$  ta = Some a
    using assms by auto
  then show ?thesis
proof (elim disjE)
  assume case-smpl-act: res-inst  $\pi$  = Some a
  then show ?thesis
    proof (cases  $\pi$ )
    case (Simple-Plan-Action n args)
    then obtain params pre eff where
      aschema-def: resolve-action-schema n = Some (Simple-Action-Schema
n params pre eff)
    using wf- $\pi$  simple-plan-action-schema-type1 Simple-Plan-Action by blast
    with wf-domain have wf-action-schema (Simple-Action-Schema n params
pre eff)
    unfolding wf-domain-def using resolve-action-schema-def by force
    then show ?thesis
      using assms wf- $\pi$  case-smpl-act aschema-def Simple-Plan-Action
wf-inst-action-schema
      by force

```

```

next
  case (Durative-Plan-Action n args d)
  then show ?thesis using case-smpl-act by auto
qed
next
assume case-snp-act: res-inst-snap-action  $\pi$  ta = Some a
then show ?thesis
proof (cases  $\pi$ )
  case (Simple-Plan-Action n args)
  then show ?thesis using case-snp-act by auto
next
  case (Durative-Plan-Action n args d)
  then obtain params dcond cond eff where
    aschema-def: resolve-action-schema n = Some (Durative-Action-Schema
n params dcond cond eff)
  using wf- $\pi$  durative-plan-action-schema-type1 by blast
  with wf-domain have wf-action-schema (Durative-Action-Schema n params
dcond cond eff)
  unfolding wf-domain-def using resolve-action-schema-def by force

  then show ?thesis
    using assms wf- $\pi$  case-snp-act aschema-def Durative-Plan-Action
wf-inst-durative-action-schema
    by force
qed
qed
qed

```

Every ground action in an induced happening sequence for a wellformed plan is wellformed.

```

lemma happ-seq-wf-ground-action:
  assumes wf-plan  $\pi$ s
    and ind-happ-seq  $\pi$ s hs
    and  $(t_i, A_i) \in \text{set } hs$ 
    and  $a \in \text{set } A_i$ 
  shows wf-ground-action a
  unfolding ind-happ-seq-def
proof -
  have  $\exists (t_\pi, \pi) \in \text{set } \pi.s. \text{inst-of-plan-action } \pi.s (t_\pi, \pi) (t_i, a)$ 
    using assms ind-happ-seq-def by blast
  then obtain  $t_\pi \pi$  where  $(t_\pi, \pi) \in \text{set } \pi.s$  and  $\text{inst-of-plan-action } \pi.s (t_\pi, \pi)$ 
    (ti, a)
    by auto
  then show ?thesis
    using assms wf-inst-of-plan-action
    by blast
qed

```

An induced happening sequence for a wellformed plan is always a wellformed

happening sequence.

lemma *inst-wf-plan-wf-happ-seq*:
assumes *wf-plan* πs
and *ind-happ-seq* πs hs
shows *wf-happ-seq* hs
unfolding *Let-def simple-acts-def durative-acts-def is-act-simple-def*
using *assms happ-seq-wf-ground-action* **by** *auto*

Execution of a plan preserves well-formedness

corollary *wf-plan-happ-path*:
assumes *wf-world-model* M **and** *wf-plan* πs **and** *plan-happ-path* M πs M'
shows *wf-world-model* M'
using *assms inst-wf-plan-wf-happ-seq wf-valid-happ-seq*
unfolding *plan-happ-path-def* **by** (*induction* πs *arbitrary*: M) *blast+*

end — Context of *wf-ast-problem*

Auxiliary function that are used to divide happening sequences.

definition *leq* $\equiv \lambda t_i. \lambda(t_a, as). t_a \leq t_i$

definition *le* $\equiv \lambda t_i. \lambda(t_a, as). t_a < t_i$

lemma *strict-sorted-app-iff*: *strict-sorted* $a \wedge$ *strict-sorted* $b \wedge (\forall x \in \text{set } a. \forall y \in \text{set } b. x < y)$
 $\longleftrightarrow$ *strict-sorted* ($a @ b$)
by (*induction* a *arbitrary*: b) *auto*

lemma *empty-filter-conv*: $[] = \text{filter } P \text{ } xs \longleftrightarrow (\forall x \in \text{set } xs. \neg P x)$
by(*auto* *dest*: *sym simp add*: *filter-empty-conv*)

1.7.3 Completeness of Semantics

context *wf-ast-problem*
begin

fun *consume2* **where**
consume2 $[] [] = 0$
 $| \text{consume2 } (x \# xs) (y \# ys) = 0 + \text{consume2 } xs \ ys$

lemma *empty-happ*:
 $(\bigwedge a. a \in \text{set } A \implies \text{effect } a = \text{Effect } [] []) \implies \text{apply-happ } (t, A) \ M = M$
by (*induction* A) *auto*

lemma *valid-happ-seq-append-1*:
 $\llbracket \bigwedge t \ A \ a. \llbracket (t, A) \in \text{set } hs1; a \in \text{set } A \rrbracket \implies \text{effect } a = \text{Effect } [] [];$
 $\text{valid-happ-seq } M \ hs2 \ M';$
 $(\bigwedge t \ A \ a. \llbracket (t, A) \in \text{set } hs1; a \in \text{set } A \rrbracket \implies M \models^c \text{precondition } a);$
 $(\bigwedge t \ A \ a. (t, A) \in \text{set } hs1 \implies \text{pairwise acts-non-intrf } (\text{set } A)) \rrbracket \implies$
 $\text{valid-happ-seq } M \ (hs1 @ hs2) \ M'$

by (induction $hs1$ arbitrary: M) (fastforce simp del: apply-happ.simps simp add: empty-happ)+

lemma valid-happ-seq-append-2:

$\llbracket \text{valid-happ-seq } M (hs1 @ hs2) M'; \bigwedge t A a. \llbracket (t, A) \in \text{set } hs1; a \in \text{set } A \rrbracket \implies \text{effect } a = \text{Effect } [] [] \implies \text{valid-happ-seq } M hs2 M' \rrbracket$

by (induction $hs1$ arbitrary: M) (fastforce simp del: apply-happ.simps simp add: empty-happ)+

lemma valid-happ-seq-append-2':

$\llbracket \text{valid-happ-seq } M (hs1 @ hs2) M'; \bigwedge t A a. \llbracket (t, A) \in \text{set } hs2; a \in \text{set } A \rrbracket \implies \text{effect } a = \text{Effect } [] [] \implies \text{valid-happ-seq } M hs1 M' \rrbracket$

apply (induction $hs1$ arbitrary: $hs2 M M'$)

apply (auto simp del: apply-happ.simps simp add: empty-happ)

by (metis empty-happ old.prod.exhaust valid-happ-seq- M_j -is- M_i)+

lemma valid-happ-seq-append-3:

$\llbracket (t, A) \in \text{set } hs1; a \in \text{set } A; \text{valid-happ-seq } M (hs1 @ hs2) M'; \bigwedge t A a. \llbracket (t, A) \in \text{set } hs1; a \in \text{set } A \rrbracket \implies \text{effect } a = \text{Effect } [] [] \implies M \models^c \text{precondition } a \rrbracket$

by (induction $hs1$ arbitrary: M) (fastforce simp del: apply-happ.simps simp add: empty-happ)+

lemma valid-happ-seq-append-4:

$\llbracket \text{valid-happ-seq } M (hs1 @ hs2) M'; \bigwedge t A a. \llbracket (t, A) \in \text{set } hs1; a \in \text{set } A \rrbracket \implies \text{effect } a = \text{Effect } [] [] \implies$

$(\bigwedge t A a. (t, A) \in \text{set } hs1 \implies \text{pairwise acts-non-intrf } (\text{set } A)) \rrbracket$

by (induction $hs1$ arbitrary: M) (fastforce simp del: apply-happ.simps simp add: empty-happ)+

lemma valid-happ-seq-empty-eff:

$\llbracket \bigwedge t A a. \llbracket (t, A) \in \text{set } hs; a \in \text{set } A \rrbracket \implies \text{effect } a = \text{Effect } [] []; (\bigwedge t A a. \llbracket (t, A) \in \text{set } hs; a \in \text{set } A \rrbracket \implies M \models^c \text{precondition } a); (\bigwedge t A a. (t, A) \in \text{set } hs \implies \text{pairwise acts-non-intrf } (\text{set } A)) \rrbracket \implies \text{valid-happ-seq } M hs M \rrbracket$

by (induction hs) (fastforce simp del: apply-happ.simps simp add: empty-happ)+

lemma valid-happ-seq-append-5:

$\llbracket \bigwedge t A a. \llbracket (t, A) \in \text{set } hs2; a \in \text{set } A \rrbracket \implies \text{effect } a = \text{Effect } [] []; \text{valid-happ-seq } M hs1 M'; (\bigwedge t A a. \llbracket (t, A) \in \text{set } hs2; a \in \text{set } A \rrbracket \implies M' \models^c \text{precondition } a); (\bigwedge t A a. (t, A) \in \text{set } hs2 \implies \text{pairwise acts-non-intrf } (\text{set } A)) \rrbracket \implies \text{valid-happ-seq } M (hs1 @ hs2) M' \rrbracket$

by (induction $hs1$ arbitrary: M)

(auto simp del: apply-happ.simps simp add: empty-happ intro!: valid-happ-seq-empty-eff)

lemma strict-sorted-cons: $\llbracket \text{strict-sorted } (\text{map } f (x \# xs)); x' \in \text{set } xs \rrbracket \implies f x \neq$

$f\ x'$
by (*induction xs*) *auto*

lemma *strict-sorted-order*:
 $\llbracket (\bigwedge x. x \in s \implies \exists y \in \text{set } (x1 \# xs1 @ x2 \# xs2). f\ y = x); \text{strict-sorted } (\text{map } f\ (x1 \# xs1 @ x2 \# xs2));$
 $(\bigwedge y. y \in \text{set } xs1 \implies f\ y \notin s); x \in s; x < f\ x2 \rrbracket \implies x = f\ x1$
by (*fastforce simp: image-def strict-sorted-app-iff[symmetric]*)

lemma *in-suff*:
 $\llbracket \bigwedge x. x \in \text{set } xs1 \implies x \neq x2; x2 \in \text{set } (xs1 @ xs2) \rrbracket \implies x2 \in \text{set } xs2$
by *auto*

lemma *strict-sorted-order-in-suff*:
 $\llbracket \text{strict-sorted } (xs1 @ xs2); x1 \in \text{set } xs1; x2 \in \text{set } xs2 \rrbracket \implies x1 < x2$
by (*induction xs1*) (*auto simp: image-def strict-sorted-app-iff[symmetric]*)

lemma *empty-eff-non-intrf*:
 $\llbracket \bigwedge a. a \in A \implies \text{effect } a = \text{Effect } [] \rrbracket \implies \text{pairwise acts-non-intrf } A$
by (*auto simp: pairwise-def acts-non-intrf-def*)

lemma *filter-eq-consD*: $x \# l' = \text{filter } Q\ l \implies x \in \text{set } l \wedge Q\ x$
by (*induction l*) (*auto split: if-splits*)

lemma *valid-happ-seq-construct*:
assumes $\bigwedge t. t \in \text{htps} \implies \exists A. (t, A) \in \text{set } \text{hs}$
and $\bigwedge t. t \in \text{htps} \implies \exists A. (t, A) \in \text{set } \text{hs}'$
and *strict-sorted* (*map fst hs*)
and *strict-sorted* (*map fst hs'*)
and $\bigwedge t\ A\ A'. \llbracket t \in \text{htps}; (t, A) \in \text{set } \text{hs}; (t, A') \in \text{set } \text{hs}' \rrbracket \implies \text{set } A = \text{set } A'$
and $\bigwedge t\ A\ A'\ a. \llbracket t \notin \text{htps}; (t, A) \in \text{set } \text{hs}; a \in \text{set } A \rrbracket \implies \text{effect } a = \text{Effect } []$
and $\bigwedge t\ A\ A'\ a. \llbracket t \notin \text{htps}; (t, A) \in \text{set } \text{hs}'; a \in \text{set } A \rrbracket \implies \text{effect } a = \text{Effect } []$
and $\bigwedge t\ A\ t_1\ t_2\ a. \llbracket t \notin \text{htps}; t_1 \in \text{htps}; t_2 \in \text{htps}; t \in \{t_1 <..< t_2\}; (t, A) \in \text{set } \text{hs}; a \in \text{set } A \rrbracket \implies$
 $\exists t' \in \{t_1 <..< t_2\}. \exists A'. (t', A') \in \text{set } \text{hs}' \wedge a \in \text{set } A'$
and $\bigwedge t\ A\ t_1\ t_2\ a. \llbracket t \notin \text{htps}; t_1 \in \text{htps}; t_2 \in \text{htps}; t \in \{t_1 <..< t_2\}; (t, A) \in \text{set } \text{hs}'; a \in \text{set } A \rrbracket \implies$
 $\exists t' \in \{t_1 <..< t_2\}. \exists A'. (t', A') \in \text{set } \text{hs} \wedge a \in \text{set } A'$
and *valid-happ-seq* *M hs M'*
and $\text{hs} \neq [] \implies (\text{fst } (\text{hd } \text{hs})) \in \text{htps}$
and $\text{hs}' \neq [] \implies (\text{fst } (\text{hd } \text{hs}')) \in \text{htps}$
and $\text{hs} \neq [] \implies (\text{fst } (\text{last } \text{hs})) \in \text{htps}$
and $\text{hs}' \neq [] \implies (\text{fst } (\text{last } \text{hs}')) \in \text{htps}$
shows *valid-happ-seq* *M hs' M'*
using *assms*
proof(*induction filter* ($\lambda h. \text{fst } h \in \text{htps}$) *hs filter* ($\lambda h. \text{fst } h \in \text{htps}$) *hs'*
arbitrary: M hs hs' htps rule: consume2.induct)

```

case (2 h xs h' ys hs hs' https M)
then obtain hs1 hs2 hs1' hs2'
  where hs = hs1 @ h # hs2 (∀ h ∈ set hs1. fst h ∉ https)
    fst h ∈ https xs = filter (λh. fst h ∈ https) hs2

    hs' = hs1' @ h' # hs2' (∀ h ∈ set hs1'. fst h ∉ https)
    fst h' ∈ https ys = filter (λh. fst h ∈ https) hs2'
  by (auto simp: Cons-eq-filter-iff)
hence [simp]: hs1 = [] hs1' = []
  using ⟨hs' ≠ [] ⟹ fst (hd hs') ∈ https⟩ ⟨hs ≠ [] ⟹ fst (hd hs) ∈ https⟩
  apply -
  by (rule list.exhaust, auto)+

note [simp] = ⟨hs = hs1 @ h # hs2⟩ ⟨hs' = hs1' @ h' # hs2'⟩

obtain t t' A A' where *[simp]: h = (t, A) h' = (t', A')
  by (cases h, cases h', auto)
hence in-https: t ∈ https t' ∈ https
  using ⟨fst h ∈ https⟩ ⟨fst h' ∈ https⟩
  by auto
moreover have ∧ t. t ∈ https ⟹ ∃ A. (t, A) ∈ set (h # hs2)
  ∧ t. t ∈ https ⟹ ∃ A. (t, A) ∈ set (h' # hs2')
  using 2(4,5) ⟨(∀ h ∈ set hs1. fst h ∉ https)⟩ ⟨(∀ h ∈ set hs1'. fst h ∉ https)⟩
  by fastforce+
moreover have strict-sorted (map fst (h # hs2)) strict-sorted (map fst (h' # hs2'))
  using 2(6,7)
  by (auto simp: strict-sorted-app-iff[symmetric])
ultimately have [simp]: t = t'
  by force
hence A-eq: set A = set A'
  using 2(8) in-https
  by auto
moreover have valid-happ-seq M (h # hs2) M'
  using 2(9) ⟨valid-happ-seq M hs M'⟩ ⟨(∀ h ∈ set hs1. fst h ∉ https)⟩
  by (force simp: intro!: valid-happ-seq-append-2[where ?hs1.0 = hs1])+
ultimately have valid-happ-seq M [h] (apply-happ (t, A') M)
  using ⟨h = (t, A)⟩ A-eq
  by auto
hence valid-happ-seq M [h'] (apply-happ (t, A') M)
  using ⟨h = (t, A)⟩ A-eq
  by auto

show ?case
proof(cases xs)
  case Nil
  hence set (map fst hs2) ∩ https = {}
  using ⟨xs = filter (λh. fst h ∈ https) hs2⟩
  by (auto simp add: filter-empty-conv)
  hence M' = (apply-happ (t, A') M)

```

```

    using ⟨valid-happ-seq M hs M'⟩ ⟨valid-happ-seq M [h] (apply-happ (t, A') M)⟩
      ⟨hs ≠ [] ⟹ fst (last hs) ∈ htps⟩
    by (auto split: if-splits)
  moreover from ⟨set (map fst hs2) ∩ htps = {}⟩ have set (map fst hs2') ∩
htps = {}
    using ⟨∧t. t ∈ htps ⟹ ∃ A. (t, A) ∈ set (h # hs2)⟩ ⟨fst h' ∈ htps⟩
      strict-sorted-cons[OF ⟨strict-sorted (map fst (h' # hs2'))⟩]
    by (simp add: disjoint-iff-not-equal) (metis fst-conv)
  ultimately show ?thesis
    using ⟨hs' ≠ [] ⟹ fst (last hs') ∈ htps⟩ ⟨valid-happ-seq M [h'] (apply-happ
(t, A') M)⟩
    by (auto split: if-splits)
next
case (Cons h2 xs')
then obtain hs3 hs4
  where hs2 = hs3 @ h2 # hs4 (∀ h ∈ set hs3. fst h ∉ htps)
    fst h2 ∈ htps xs' = filter (λh. fst h ∈ htps) hs4
  using ⟨xs = filter (λh. fst h ∈ htps) hs2⟩
  by (auto simp: filter-eq-Cons-iff)
moreover hence fst h2 ≠ fst h
  using ⟨strict-sorted (map fst (h # hs2))⟩
  by auto
hence fst h2 ≠ fst h'
  using ⟨t = t'⟩
  by auto
ultimately obtain A2' where (fst h2, A2') ∈ set hs2'
  using ⟨∧t. t ∈ htps ⟹ ∃ A. (t, A) ∈ set (h' # hs2')⟩
  by (cases h2, cases h', auto simp add: disjoint-iff-not-equal)
then obtain hs3' hs4' where [simp]: hs2' = hs3' @ (fst h2, A2') # hs4'
  by (auto dest: split-list-first)

note[simp] = ⟨hs2 = hs3 @ h2 # hs4⟩ strict-sorted-app-iff[symmetric]

have fst-h3'-n-htp: fst h3' ∉ htps if h3' ∈ set hs3' for h3'
proof(rule ccontr, safe)
  assume fst h3' ∈ htps
  hence fst h3' < fst h2
    using that ⟨strict-sorted (map fst hs')⟩
    by auto
  moreover have x ∈ htps ⟹ ∃ y ∈ set (h # hs3 @ h2 # hs4). fst y = x for x
    using ⟨∧t. t ∈ htps ⟹ ∃ A. (t, A) ∈ set (h # hs2)⟩
    by force
  ultimately have fst h3' = fst h
    using ⟨fst h3' ∈ htps⟩ ⟨∀ h ∈ set hs3. fst h ∉ htps⟩ ⟨strict-sorted (map fst hs)⟩
    by (auto simp: simp del: *)
    intro!: strict-sorted-order[where s = htps and ?xs1.0 = hs3 and
?xs2.0 = h2 and ?xs2.0 = hs4])+
  thus False
    using that ⟨strict-sorted (map fst hs')⟩

```

```

    by auto
qed

define hs-IH where hs-IH  $\equiv$  h2 # hs4
define hs'-IH where hs'-IH  $\equiv$  (fst h2, A2') # hs4'
define htps-IH where htps-IH  $\equiv$  htps - {t}
define M-IH where M-IH  $\equiv$  (apply-happ (t, A') M)
have [dest]:  $\bigwedge h'. h' \in \text{set } (hs-IH) \implies t \neq \text{fst } h'$ 
  using  $\langle \text{strict-sorted } (\text{map } \text{fst } hs) \rangle$ 
  by (auto simp: hs-IH-def)
have  $\bigwedge h'. h' \in \text{set } (hs'-IH) \implies t' \neq \text{fst } h'$ 
  using  $\langle \text{strict-sorted } (\text{map } \text{fst } hs') \rangle$ 
  by (auto simp: hs'-IH-def)

  have xs = filter ( $\lambda h. \text{fst } h \in \text{htps-IH}$ ) (hs-IH) (is ?xs = ?fil-xs ( $\lambda h. \text{fst } h \in$ 
htps-IH))
  proof-
    have ?fil-xs ( $\lambda h. \text{fst } h \in \text{htps-IH}$ ) = ?fil-xs ( $\lambda h. \text{fst } h \in \text{htps}$ )
      by (fastforce intro!: filter-cong simp: htps-IH-def)
    also have ... = filter ( $\lambda h. \text{fst } h \in \text{htps}$ ) hs2
      using Cons  $\langle (\forall h \in \text{set } hs3. \text{fst } h \notin \text{htps}) \rangle$ 
      by (auto simp: hs-IH-def)
    finally show ?thesis
      using Cons
      by (simp add: xs = filter ( $\lambda h. \text{fst } h \in \text{htps}$ ) hs2)
  qed
  moreover have ys = filter ( $\lambda h. \text{fst } h \in \text{htps-IH}$ ) (hs'-IH) (is ?ys = ?fil-ys ( $\lambda h.$ 
fst h  $\in$  htps-IH))
  proof-
    have ?fil-ys ( $\lambda h. \text{fst } h \in \text{htps} - \{t'\}$ ) = ?fil-ys ( $\lambda h. \text{fst } h \in \text{htps}$ )
      using  $\langle \bigwedge h'. h' \in \text{set } (hs'-IH) \implies t' \neq \text{fst } h' \rangle$ 
      by (force intro!: filter-cong)
    also have ... = filter ( $\lambda h. \text{fst } h \in \text{htps}$ ) hs2'
      using Cons fst-h3'-n-htp
      by (auto simp: hs'-IH-def)
    finally show ?thesis
      using Cons
      by (simp add: ys = filter ( $\lambda h. \text{fst } h \in \text{htps}$ ) hs2') htps-IH-def)
  qed
  moreover have  $\bigwedge t'. t' \in \text{htps-IH} \implies \exists A. (t', A) \in \text{set } (hs-IH)$ 
    using  $\langle (\forall h \in \text{set } hs3. \text{fst } h \notin \text{htps}) \rangle$ 
     $\langle \bigwedge t. t \in \text{htps} \implies \exists A. (t, A) \in \text{set } (h \# hs2) \rangle$ 
     $\langle \bigwedge h'. h' \in \text{set } (hs-IH) \implies t \neq \text{fst } h' \rangle$ 
    by (fastforce simp: hs-IH-def htps-IH-def)
  moreover have  $\bigwedge t''. t'' \in \text{htps-IH} \implies \exists A. (t'', A) \in \text{set } (hs'-IH)$ 
    using fst-h3'-n-htp
     $\langle \bigwedge t. t \in \text{htps} \implies \exists A. (t, A) \in \text{set } (h' \# hs2') \rangle$ 
    by (fastforce simp: hs'-IH-def htps-IH-def)
  moreover have strict-sorted (map fst (hs-IH))

```

using $\langle \text{strict-sorted } (\text{map fst } \text{hs}) \rangle$
 by $(\text{auto simp: } \text{hs-IH-def } \text{https-IH-def})$
 moreover have $\text{strict-sorted } (\text{map fst } (\text{hs}'\text{-IH}))$
 using $\langle \text{strict-sorted } (\text{map fst } \text{hs}') \rangle$
 by $(\text{auto simp: } \text{hs}'\text{-IH-def } \text{https-IH-def})$
 moreover have
 $\bigwedge t'' A'' A'''. \llbracket t'' \in \text{https-IH}; (t'', A'') \in \text{set } (\text{hs-IH}); (t'', A''') \in \text{set } (\text{hs}'\text{-IH}) \rrbracket$
 $\implies \text{set } A'' = \text{set } A'''$
 using 2(8)
 by $(\text{auto simp: } \text{https-IH-def } \text{hs-IH-def } \text{hs}'\text{-IH-def})$
 moreover have $\bigwedge t'' A'' a. \llbracket t'' \notin \text{https-IH}; (t'', A'') \in \text{set } (\text{hs-IH}); a \in \text{set } A'' \rrbracket$
 $\implies \text{effect } a = \text{Effect } [] []$
 using 2(9) $\langle \text{fst } h2 \in \text{https} \rangle \langle \bigwedge h'. h' \in \text{set } (\text{hs-IH}) \implies t \neq \text{fst } h' \rangle$
 by $(\text{fastforce simp add: } \text{https-IH-def } \text{hs-IH-def})$
 moreover have $\bigwedge t'' A'' a. \llbracket t'' \notin \text{https-IH}; (t'', A'') \in \text{set } (\text{hs}'\text{-IH}); a \in \text{set } A'' \rrbracket \implies \text{effect } a = \text{Effect } [] []$
 using 2(10) $\langle \text{fst } h2 \in \text{https} \rangle \langle \bigwedge h'. h' \in \text{set } (\text{hs}'\text{-IH}) \implies t' \neq \text{fst } h' \rangle$
 by $(\text{force simp add: } \text{https-IH-def } \text{hs}'\text{-IH-def})$
 moreover have $\exists t' \in \{t_1 < .. < t_2\}. \exists A'. (t', A') \in \text{set } \text{hs}'\text{-IH} \wedge a \in \text{set } A'$
 if $t'' \notin \text{https-IH } t_1 \in \text{https-IH } t_2 \in \text{https-IH } t'' \in \{t_1 < .. < t_2\} (t'', A'') \in \text{set } \text{hs-IH}$
 $a \in \text{set } A''$
 for $t'' A'' t_1 t_2 a$
 proof-
 have $t'' \notin \text{https}$
 using $\langle \bigwedge h'. h' \in \text{set } (\text{hs-IH}) \implies t \neq \text{fst } h' \rangle \langle t'' \notin \text{https-IH} \rangle \langle (t'', A'') \in \text{set } \text{hs-IH} \rangle$
 by $(\text{fastforce simp: } \text{https-IH-def})$
 hence $\exists t' \in \{t_1 < .. < t_2\}. \exists A'. (t', A') \in \text{set } \text{hs}' \wedge a \in \text{set } A'$
 using that
 by $(\text{auto simp del: } \langle \text{hs}' = \text{hs1}' @ h' \# \text{hs2}' \rangle \text{simp: } \text{https-IH-def } \text{hs-IH-def})$
 intro!: 2(11))
 then obtain $t''' A'''$ where $t''' \in \{t_1 < .. < t_2\} (t''', A''') \in \text{set } \text{hs}' a \in \text{set } A'''$
 by $(\text{auto simp del: } \langle \text{hs}' = \text{hs1}' @ h' \# \text{hs2}' \rangle)$
 hence $t_1 < t'''$
 by auto
 moreover have $t' < t_1$
 using $\langle t_1 \in \text{https-IH} \rangle \langle \bigwedge t. t \in \text{https} \implies \exists A. (t, A) \in \text{set } (h \# \text{hs2}) \rangle$
 $\langle \text{strict-sorted } (\text{map fst } \text{hs}) \rangle$
 by $(\text{fastforce simp: } \text{https-IH-def})$
 hence $t_1 \in \text{set } (\text{map fst } \text{hs}'\text{-IH})$
 using $\text{fst-h3'-n-htp } \langle t_1 \in \text{https-IH} \rangle \langle \bigwedge t''. t'' \in \text{https-IH} \implies \exists A. (t'', A) \in \text{set } \text{hs}'\text{-IH} \rangle$
 by $(\text{fastforce simp: } \text{image-def } \text{hs}'\text{-IH-def } \text{https-IH-def})$
 hence $\bigwedge h3'. h3' \in \text{set } \text{hs3}' \implies \text{fst } h3' < t_1$
 using $\langle \text{strict-sorted } (\text{map fst } \text{hs}') \rangle$
 by $(\text{auto simp: } \text{hs}'\text{-IH-def})$
 ultimately have $t''' \notin \text{set } (\text{map fst } \text{hs3}')$
 by fastforce

thus *?thesis*
using $\langle (t''', A''') \in \text{set } hs' \rangle \langle t' < t_1 \rangle \langle t_1 < t''' \rangle \langle t''' \in \{t_1 < .. < t_2\} \rangle \langle a \in \text{set } A''' \rangle$
by (*auto simp: hs'-IH-def*) +
qed

moreover have $\exists t' \in \{t_1 < .. < t_2\}. \exists A'. (t', A') \in \text{set } hs\text{-}IH \wedge a \in \text{set } A'$
if $t'' \notin \text{https-IH}$ $t_1 \in \text{https-IH}$ $t_2 \in \text{https-IH}$ $t'' \in \{t_1 < .. < t_2\}$ $(t'', A'') \in \text{set } hs'\text{-}IH$
 $a \in \text{set } A''$
for $t'' A'' t_1 t_2 a$
proof–
have $t'' \notin \text{https}$
using $\langle \bigwedge h'. h' \in \text{set } (hs'\text{-}IH) \implies t' \neq \text{fst } h' \rangle \langle t'' \notin \text{https-IH} \rangle \langle (t'', A'') \in \text{set } hs'\text{-}IH \rangle$
by (*fastforce simp: https-IH-def*)
hence $\exists t' \in \{t_1 < .. < t_2\}. \exists A'. (t', A') \in \text{set } hs \wedge a \in \text{set } A'$
using *that*
by (*auto simp del: $\langle hs = hs1 @ h \# hs2 \rangle$ simp: https-IH-def hs'-IH-def intro!: 2(12)*)
then obtain $t''' A'''$ **where** $t''' \in \{t_1 < .. < t_2\}$ $(t''', A''') \in \text{set } hs$ $a \in \text{set } A'''$
by (*auto simp del: $\langle hs = hs1 @ h \# hs2 \rangle$*)
hence $t_1 < t'''$
by *auto*
moreover have $t' < t_1$
using $\langle t_1 \in \text{https-IH} \rangle \langle \bigwedge t. t \in \text{https} \implies \exists A. (t, A) \in \text{set } (h \# hs2) \rangle$
 $\langle \text{strict-sorted } (\text{map } \text{fst } hs) \rangle$
by (*fastforce simp: https-IH-def*)
hence $t_1 \in \text{set } (\text{map } \text{fst } hs\text{-}IH)$
using fst-h3'-n-htp $\langle t_1 \in \text{https-IH} \rangle \langle hs1 = [] \rangle \langle \bigwedge t''. t'' \in \text{https-IH} \implies \exists A. (t'', A) \in \text{set } hs\text{-}IH \rangle$
by (*fastforce simp: image-def hs'-IH-def https-IH-def*) +
hence $\bigwedge h3'. h3' \in \text{set } hs3 \implies \text{fst } h3' < t_1$
using $\langle \text{strict-sorted } (\text{map } \text{fst } hs) \rangle$
by (*auto simp: hs-IH-def*)
ultimately have $t''' \notin \text{set } (\text{map } \text{fst } hs3)$
by *fastforce*
thus *?thesis*
using $\langle (t''', A''') \in \text{set } hs \rangle \langle t''' \in \{t_1 < .. < t_2\} \rangle \langle a \in \text{set } A''' \rangle \langle t' < t_1 \rangle \langle t_1 < t''' \rangle$
by (*auto simp: hs-IH-def*)
qed

moreover have *valid-happ-seq* $M\text{-}IH$ $hs\text{-}IH$ M'
using $\langle \text{valid-happ-seq } M \text{ } hs \text{ } M' \rangle \langle \text{valid-happ-seq } M [h] (\text{apply-happ } (t, A') M) \rangle$
 $\langle (\forall h \in \text{set } hs3. \text{fst } h \notin \text{https}) \rangle$
by (*auto simp: M-IH-def hs-IH-def intro: 2(9) dest!: valid-happ-seq-append-2*)
moreover have $(\text{fst } (\text{hd } hs\text{-}IH)) \in \text{https-IH}$
using $\langle \text{fst } h2 \in \text{https} \rangle$
by (*auto simp: hs-IH-def https-IH-def $\langle \bigwedge h'. h' \in \text{set } hs\text{-}IH \implies t \neq \text{fst } h' \rangle$*)

```

simp del: ⟨t = t'⟩
  moreover have (fst (hd hs'-IH)) ∈ https-IH
  using ⟨fst h2 ∈ https⟩ ⟨fst h2 ≠ fst h'⟩
  by (auto simp: hs'-IH-def https-IH-def)
  moreover have (fst (last hs-IH)) ∈ https-IH
  using ⟨hs ≠ [] ⟹ fst (last hs) ∈ https⟩
  by (auto simp: hs-IH-def https-IH-def ⟨∧ h'. h' ∈ set hs-IH ⟹ t ≠ fst h'⟩)
simp del: ⟨t = t'⟩
  moreover have (fst (last hs'-IH)) ∈ https-IH
  using ⟨fst h2 ∈ https⟩ ⟨hs' ≠ [] ⟹ fst (last hs') ∈ https⟩ ⟨∧ h'. h' ∈ set hs'-IH ⟹ t' ≠ fst h'⟩
  by (auto simp: https-IH-def hs'-IH-def split: if_splits)+
ultimately have valid-happ-seq M-IH hs'-IH M'
  by (rule 2)
moreover have (apply-happ (t, A') M) c⊨ precondition a
  if a ∈ set A'' (t'', A'') ∈ set hs3'
  for t'' A'' a
proof-
  have t'' ∉ https
  using ⟨(t'', A'') ∈ set hs3'⟩ fst-h3'-n-htp
  by auto
  moreover have t ∈ https
  using ⟨t ∈ https⟩ .
  moreover have fst h2 ∈ https
  using ⟨fst h2 ∈ https⟩ .
  moreover have t'' ∈ {t <..

```

A'')
using *fst-h3'-n-htp* 2(10)
by (*force simp: intro!: empty-eff-non-intrf*)
ultimately show ?thesis
using $\langle \text{valid-happ-seq } M \text{ [} h \text{]} \text{ (apply-happ } (t, A') M) \rangle$ *fst-h3'-n-htp*
by (*auto simp add: M-IH-def hs'-IH-def intro!: valid-happ-seq-append-1* 2(10))
qed
qed (*fastforce simp: empty-filter-conv dest!: filter-eq-consD*)+

lemma *not-htp-empty-effect*:
assumes *wf-plan* πs
and $\neg \text{is-htp } \pi s \ t_i$
and *ind-happ-seq* $\pi s \ hs$
and $(t_i, A_i) \in \text{set } hs$
and $a \in \text{set } A_i$
shows (*effect* a) = *Effect* [] []
proof –
have *: $(t_i, \pi) \notin \text{set } \pi s$
 $\bigwedge t_\pi. (t_\pi, \pi) \in \text{durative-acts } \pi s \implies t_i \neq t_\pi + (\text{duration } \pi)$
for π
using $\langle \neg \text{is-htp } \pi s \ t_i \rangle$
by (*auto simp: is-htp-def*)
obtain $t_\pi \ \pi$ **where** $(t_\pi, \pi) \in \text{set } \pi s$ *inst-of-plan-action* $\pi s \ (t_\pi, \pi) \ (t_i, a)$
using *ind-happ-seq-props*(7)[*OF assms*(3–5)]
by *auto*
have *wf-plan-action* π
using $\langle \text{wf-plan } \pi s \rangle \langle (t_\pi, \pi) \in \text{set } \pi s \rangle$
by (*auto simp: wf-plan-def*)
moreover then obtain a_{schema} **where**
 $\text{wf-action-schema } a_{\text{schema}}$
 $\text{Some } a_{\text{schema}} = \text{resolve-action-schema } (\text{plan-action.name } \pi)$
by (*cases* π) (*auto split: option.splits dest: resolve-action-wf*)
moreover have $\text{Some } a = \text{res-inst-snap-action } \pi \ \text{Over-All}$
using * $\langle (t_\pi, \pi) \in \text{set } \pi s \rangle \langle \text{inst-of-plan-action } \pi s \ (t_\pi, \pi) \ (t_i, a) \rangle$
by (*cases* π) (*fastforce simp: durative-acts-def is-act-simple-def*)+
ultimately show ?thesis
by (*rule wf-over-all-empty-eff*)
qed

lemma *unique-happ*:
assumes *ind-happ-seq* $\pi s \ hs$
shows $\llbracket (t, A) \in \text{set } hs; (t, A') \in \text{set } hs \rrbracket \implies A = A'$
using *ind-happ-seq-props*(1)[*OF assms*]
by (*induction* hs) *auto*

lemma *inst-of-plan-actionE*:
 $\bigwedge t_\pi \ \pi. \llbracket \text{inst-of-plan-action } \pi s \ (t_\pi, \pi) \ (t_a, a);$
 $\llbracket \text{res-inst } \pi = \text{Some } a; t_\pi = t_a \rrbracket \implies Q;$

$\llbracket \text{res-inst-snap-action } \pi \text{ At-Start} = \text{Some } a; t_\pi = t_a \rrbracket \implies Q;$
 $\llbracket \text{res-inst-snap-action } \pi \text{ At-End} = \text{Some } a; t_\pi + (\text{duration } \pi) = t_a \rrbracket \implies Q;$
 $\bigwedge t_i t_j. \llbracket \text{res-inst-snap-action } \pi \text{ Over-All} = \text{Some } a; \text{consec-htps } \pi s \ t_i \ t_j;$
 $t_\pi \leq t_i; t_j \leq t_\pi + (\text{duration } \pi); t_i < t_a; t_a < t_j \rrbracket \implies Q$
 $\implies Q$
by *auto*

lemma *ind-happ-seq-happ-at-htps*:

assumes *is-htp* $\pi s \ t$
and *ind-happ-seq* $\pi s \ hs$
and *ind-happ-seq* $\pi s \ hs'$
and $(t, A) \in \text{set } hs$
and $(t, A') \in \text{set } hs'$
and $a \in \text{set } A$
shows $a \in \text{set } A'$

proof–

obtain $t_\pi \ \pi$ **where** $(t_\pi, \pi) \in \text{set } \pi s$
 $(\text{res-inst } \pi = \text{Some } a \wedge t_\pi = t)$
 $\vee (\text{res-inst-snap-action } \pi \text{ At-Start} = \text{Some } a \wedge t_\pi = t)$
 $\vee (\text{res-inst-snap-action } \pi \text{ At-End} = \text{Some } a \wedge t_\pi + (\text{duration } \pi) = t)$
using *ind-happ-seq-props*(7)[*OF* $\langle \text{ind-happ-seq } \pi s \ hs \rangle \langle (t, A) \in \text{set } hs \rangle \langle a \in \text{set } A \rangle$]
 $\langle \text{is-htp } \pi s \ t \rangle$
by (*auto simp: ast-problem.consec-htps-def*)
then obtain A'' **where** $(t, A'') \in \text{set } hs' \ a \in \text{set } A''$
using *ind-happ-seq-props*(2–4)[*OF* $\langle \text{ind-happ-seq } \pi s \ hs' \rangle$]
by (*force simp: durative-acts-def simple-acts-def is-act-simple-def split: plan-action.splits*)
moreover hence $A'' = A'$
using $\langle \text{ind-happ-seq } \pi s \ hs' \rangle \langle (t, A') \in \text{set } hs' \rangle$
by (*auto intro: unique-happ*)
ultimately show *?thesis*
by *auto*
qed

lemma *ind-happ-seq-happ-at-htps'*:

assumes *is-htp* $\pi s \ t$
and *ind-happ-seq* $\pi s \ hs$
and *ind-happ-seq* $\pi s \ hs'$
and $(t, A) \in \text{set } hs$
and $(t, A') \in \text{set } hs'$
shows $\text{set } A = \text{set } A'$
using *assms ind-happ-seq-happ-at-htps*
by *blast*

lemma *htp-in-ind-happ-seq*:

$\llbracket \text{is-htp } \pi s \ t; \text{ind-happ-seq } \pi s \ hs \rrbracket \implies \exists A. (t, A) \in \text{set } hs$
by (*fastforce simp: is-htp-def ind-happ-seq-def durative-acts-def simple-acts-def split: prod.splits*)

lemma *sorted-takeWhile: strict-sorted xs $\implies x \in \text{set } (\text{takeWhile } ((\lambda x. x < c)) \text{ } xs)$*
 $= (x \in \text{set } xs \wedge (x < c))$
by (*induction xs arbitrary: c x*) *auto*

lemma *sorted-dropWhile: strict-sorted xs $\implies x \in \text{set } (\text{dropWhile } ((\lambda x. x \leq c)) \text{ } xs)$*
 $= (x \in \text{set } xs \wedge (c < x))$
by (*induction xs arbitrary: c x*) *auto*

lemma *map-fst-dropWhile: map fst (dropWhile (leq t) hs) = (dropWhile (($\lambda x. x \leq t$)) (map fst hs))*
by (*induction hs*) (*auto simp: leq-def*)

lemma *map-fst-takeWhile: map fst (takeWhile (le t) hs) = (takeWhile (($\lambda x. x < t$)) (map fst hs))*
by (*induction hs*) (*auto simp: le-def*)

lemma *strict-sorted-fst-unique-snd:*
 $\llbracket \text{strict-sorted } (\text{map fst } hs); (t, A) \in \text{set } hs; (t, A') \in \text{set } hs \rrbracket \implies A = A'$
by (*induction hs*) *auto*

lemma *strict-sorted-dropWhile:*
 $\text{strict-sorted } xs \implies \text{strict-sorted } (\text{dropWhile } Q \text{ } xs)$
by (*induction xs*) *auto*

lemma *strict-sorted-takeWhile:*
 $\text{strict-sorted } xs \implies \text{strict-sorted } (\text{takeWhile } Q \text{ } xs)$
by (*induction xs*) (*auto dest!: set-takeWhileD*)

lemma *strict-sorted-drop-leq-take-le':*
assumes *strict-sorted (map fst hs)*
shows $(t_a, A) \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) \text{ } hs)) \longleftrightarrow$
 $(t_i < t_a \wedge t_a < t_j \wedge (t_a, A) \in \text{set } hs) \text{ (is } ?l \longleftrightarrow ?r)$

proof
assume *?l*
hence $t_a \in \text{set } (\text{map fst } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) \text{ } hs)))$
by *auto*
hence $(t_i < t_a) \wedge t_a < t_j$
using *strict-sorted-takeWhile[OF $\langle \text{strict-sorted } (\text{map fst } hs) \rangle \langle \text{strict-sorted } (\text{map fst } hs) \rangle$*
 $\text{fst } hs \rangle$
by (*auto simp: sorted-takeWhile sorted-dropWhile map-fst-takeWhile map-fst-dropWhile*) +
thus *?r*
using $\langle ?l \rangle$
by (*auto dest: set-dropWhileD set-takeWhileD*)
next
assume *?r*
hence $t_a \in \text{set } (\text{map fst } hs)$
by *auto*
hence $t_a \in \text{set } (\text{map fst } ((\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) \text{ } hs))))$

using *strict-sorted-takeWhile*[*OF* $\langle \text{strict-sorted} (\text{map fst } hs) \rangle$] $\langle \text{strict-sorted} (\text{map fst } hs) \rangle$ $\langle ?r \rangle$
by (*auto simp: sorted-takeWhile sorted-dropWhile map-fst-takeWhile map-fst-dropWhile*)
then obtain A' **where** $(t_a, A') \in \text{set } ((\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs)))$ $(t_a, A') \in \text{set } hs$
by (*fastforce dest: set-dropWhileD set-takeWhileD*)
thus $?l$
using $\langle ?r \rangle$ *strict-sorted-fst-unique-snd* $\langle \text{strict-sorted} (\text{map fst } hs) \rangle$
by force
qed

lemma *ind-happ-seqs-same-acts-leq_i-le_j'*:

assumes *is-htp* $\pi s t_i$
and *is-htp* $\pi s t_j$
and *ind-happ-seq* $\pi s hs_1$
and *ind-happ-seq* $\pi s hs_2$
and $(t_a, A) \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs_1))$
and $a \in \text{set } A$
shows $\exists t_a' A'. (t_a', A') \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs_2)) \wedge a \in \text{set } A'$
proof –
let $?hs_1' = \text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs_1)$
let $?hs_2' = \text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs_2)$
have $(t_a, A) \in \text{set } hs_1$
using $\langle (t_a, A) \in \text{set } ?hs_1' \rangle$ **by** (*meson set-dropWhileD set-takeWhileD*)
have *strict-sorted* $(\text{map fst } hs_1)$ **and** *strict-sorted* $(\text{map fst } hs_2)$
using $\langle \text{ind-happ-seq } \pi s hs_1 \rangle$ $\langle \text{ind-happ-seq } \pi s hs_2 \rangle$ **unfolding** *ind-happ-seq-def*
by *auto*
moreover have $t_i < t_a \wedge t_a < t_j$
using $\langle (t_a, A) \in \text{set } ?hs_1' \rangle$ $\langle \text{strict-sorted} (\text{map fst } hs_1) \rangle$ $\langle (t_a, A) \in \text{set } hs_1 \rangle$
by (*fastforce simp: strict-sorted-drop-leq-take-le'*)
moreover obtain $t_\pi \pi$ **where** $(t_\pi, \pi) \in \text{set } \pi s$ **and** *inst-of-plan-action* $\pi s (t_\pi, \pi)$ (t_a, a)
using $\langle \text{ind-happ-seq } \pi s hs_1 \rangle$ $\langle (t_a, A) \in \text{set } hs_1 \rangle$ $\langle a \in \text{set } A \rangle$
by (*fastforce simp: ind-happ-seq-def*)
ultimately show $?thesis$
proof (*elim inst-of-plan-actionE, goal-cases*)
case $(\text{?} t_i' t_j')$
let $?a_{inv} = \text{the } (\text{res-inst-snap-action } \pi \text{ Over-All})$
have $(t_\pi, \pi) \in \text{durative-acts } \pi s$
unfolding *durative-acts-def is-act-simple-def*
using $\text{?} \langle (t_\pi, \pi) \in \text{set } \pi s \rangle$ **by** (*cases* π) *auto*
then have $\exists (t', as) \in \text{set } hs_2. t_i' < t' \wedge t' < t_j' \wedge ?a_{inv} \in \text{set } as$
using $\langle \text{ind-happ-seq } \pi s hs_2 \rangle$
using $\langle \text{consec-htps } \pi s t_i' t_j' \rangle$? **by** (*auto simp: ind-happ-seq-def*)
then obtain $t_a' A'$ **where** $(t_a', A') \in \text{set } hs_2$ **and** $t_i' < t_a' \wedge t_a' < t_j'$ **and** $?a_{inv} \in \text{set } A'$
by *auto*
then have $(t_a', A') \in \text{set } hs_2$

```

    using 4 by auto
  moreover have  $t_i < t_a' \wedge t_a' < t_j$ 
  using assms  $\langle \text{consec-https } \pi s \ t_i' \ t_j' \rangle \langle t_i < t_a \wedge t_a < t_j \rangle \langle t_i' < t_a \rangle \langle t_a < t_j' \rangle$ 
     $\langle t_i' < t_a' \wedge t_a' < t_j' \rangle$ 
  by (auto simp: consec-https-def)
  ultimately show ?thesis
    using 4  $\langle ?a_{inv} \in \text{set } A' \rangle \langle \text{strict-sorted } (\text{map fst } \text{hs}_2) \rangle$ 
    by (fastforce simp: strict-sorted-drop-leq-take-le')
qed (insert  $\langle \text{ind-happ-seq } \pi s \ \text{hs}_2 \rangle$ , cases  $\pi$ ;
    fastforce simp: strict-sorted-drop-leq-take-le' ind-happ-seq-def simple-acts-def
    durative-acts-def is-act-simple-def)+
qed

```

lemma *happs-same-between-https:*

```

  assumes ind-happ-seq  $\pi s \ \text{hs}$ 
    and ind-happ-seq  $\pi s \ \text{hs}'$ 
    and is-htp  $\pi s \ t_1$  is-htp  $\pi s \ t_2$ 
    and  $(t, A) \in \text{set } \text{hs}$ 
    and  $\neg \text{is-htp } \pi s \ t$ 
    and  $t_1 < t$ 
    and  $t < t_2$ 
    and  $a \in \text{set } A$ 
  shows  $\exists t' \in \{t_1 < .. < t_2\}. \exists A'. (t', A') \in \text{set } \text{hs}' \wedge a \in \text{set } A'$ 
proof-
  have  $\exists t_a' A'. (t_a', A') \in \text{set } (\text{dropWhile } (\text{leq } t_1) (\text{takeWhile } (\text{le } t_2) \ \text{hs}')) \wedge a \in \text{set } A'$ 
  using ind-happ-seq-props(1)[OF  $\langle \text{ind-happ-seq } \pi s \ \text{hs} \rangle$ ] assms
  by (fastforce simp: strict-sorted-drop-leq-take-le' ind-happ-seqs-same-acts-leqi-lej')
  thus ?thesis
    using  $\langle (t, A) \in \text{set } \text{hs} \rangle$  ind-happ-seq-props(1)[OF  $\langle \text{ind-happ-seq } \pi s \ \text{hs}' \rangle$ ]
    by (auto simp: strict-sorted-drop-leq-take-le')
qed

```

lemma *consec-https-props:*

```

  assumes consec-https  $\pi s \ t_i \ t_j$ 
  shows  $t_i < t_j$ 
    is-htp  $\pi s \ t_i$ 
    is-htp  $\pi s \ t_j$ 
    is-htp  $\pi s \ t \implies t \leq t_i \vee t_j \leq t$ 
  using assms
  by (auto simp: consec-https-def)

```

lemma *ind-happ-seq-hd-htp:*

```

  assumes ind-happ-seq  $\pi s \ \text{hs} \ \text{hs}' \neq []$ 
  shows is-htp  $\pi s \ (\text{fst } (\text{hd } \text{hs}))$ 
proof-
  obtain  $h \ \text{hs}'$  where  $\text{hs}[\text{simp}]: \text{hs} = h \# \text{hs}'$ 
  using  $\langle \text{hs} \neq [] \rangle$ 
  by (auto simp: neq-Nil-conv)

```

then obtain $t \ A$ **where** $h[simp]: h = (t, A)$
 by (*elim prod.exhaust*)
then obtain a **where** $a \in \text{set } A$
 using *ind-happ-seq-props(6)*[*OF* $\langle \text{ind-happ-seq } \pi s \ hs \rangle$]
 by *auto*
then obtain $t_\pi \ \pi$ **where** $(t_\pi, \pi) \in \text{set } \pi s$ **and** *inst-of-plan-action* $\pi s \ (t_\pi, \pi) \ (t, a)$
 using $\langle \text{ind-happ-seq } \pi s \ hs \rangle \ \langle a \in \text{set } A \rangle$
 by (*fastforce simp: ind-happ-seq-def*)
then show *?thesis*
 using *ind-happ-seq-props(1)*[*OF* $\langle \text{ind-happ-seq } \pi s \ hs \rangle$]
 by(*force simp: durative-acts-def is-act-simple-def is-htp-def*
 dest!: *consec-htps-props(2)* *htp-in-ind-happ-seq*[*OF* - $\langle \text{ind-happ-seq } \pi s$
 $hs \rangle$]
 elim!: *res-inst-snap-action.elims*)
qed

lemma *sorted-last*: $\llbracket \text{strict-sorted } xs; x \in \text{set } xs \rrbracket \implies x \leq \text{last } xs$
 by (*induction xs*) (*auto simp: dest!:* *last-in-set*)

lemma *ind-happ-seq-last-htp*:
 assumes *ind-happ-seq* $\pi s \ hs \ hs \neq []$
 shows *is-htp* $\pi s \ (\text{fst } (\text{last } hs))$
proof–
obtain $h \ hs'$ **where** $hs[simp]: hs = hs' @ [h]$
 using $\langle hs \neq [] \rangle$
 by (*auto simp: neq-Nil-rev-conv*)
then obtain $t \ A$ **where** $h[simp]: h = (t, A)$
 by (*elim prod.exhaust*)
then obtain a **where** $a \in \text{set } A$
 using *ind-happ-seq-props(6)*[*OF* $\langle \text{ind-happ-seq } \pi s \ hs \rangle$]
 by *auto*
then obtain $t_\pi \ \pi$ **where** $(t_\pi, \pi) \in \text{set } \pi s$ **and** *inst-of-plan-action* $\pi s \ (t_\pi, \pi) \ (t, a)$
 using $\langle \text{ind-happ-seq } \pi s \ hs \rangle \ \langle a \in \text{set } A \rangle$
 by (*fastforce simp: ind-happ-seq-def*)
then show *?thesis*
 using *ind-happ-seq-props(1)*[*OF* $\langle \text{ind-happ-seq } \pi s \ hs \rangle$] *sorted-last*
 by (*fastforce simp: durative-acts-def is-act-simple-def is-htp-def*
 dest!: *consec-htps-props(3)* *htp-in-ind-happ-seq*[*OF* - $\langle \text{ind-happ-seq } \pi s \ hs \rangle$]
 elim!: *res-inst-snap-action.elims*)
qed

definition $htps' \ \pi s \equiv \{t \ . \ \text{is-htp } \pi s \ t\}$

lemma *htps-props*: $t \in htps' \ \pi s \implies \text{is-htp } \pi s \ t$
 by(*auto simp: htps'-def*)

lemma *ind-happ-seq-unique-final-state'*:
 assumes *ind-happ-seq* $\pi s \ hs$ **and** *ind-happ-seq* $\pi s \ hs'$
 shows $\llbracket \text{wf-plan } \pi s; \text{valid-happ-seq } M_0 \ hs \ M_n \rrbracket \implies \text{valid-happ-seq } M_0 \ hs' \ M_n$

```

using not-htp-empty-effect assms
by (auto intro!: ind-happ-seq-last-htp htp-in-ind-happ-seq ind-happ-seq-hd-htp
ind-happ-seq-props(1)
      valid-hap-seq-construct[where https = https'  $\pi s$  and hs = hs]
      intro: happs-same-between-https
      simp: https'-def ind-happ-seq-happ-at-https'[OF https-props ⟨ind-happ-seq  $\pi s$ 
hs⟩ ⟨ind-happ-seq  $\pi s$  hs'⟩])

end

end — Theory
theory Error-Monad-Add
imports
  Certification-Monads.Check-Monad
  Show.Show-Instances
begin

abbreviation assert-opt  $\Phi \equiv$  if  $\Phi$  then Some () else None

definition lift-opt  $m\ e \equiv$  case  $m$  of Some  $x \Rightarrow$  Error-Monad.return  $x$  | None  $\Rightarrow$ 
Error-Monad.error  $e$ 

lemma lift-opt-simps[simp]:
  lift-opt None  $e =$  error  $e$ 
  lift-opt (Some  $v$ )  $e =$  return  $v$ 
by (auto simp: lift-opt-def)

lemma reflcl-image-iff[simp]:  $R^= \text{“} S = S \cup R \text{“} S$  by blast

named-theorems return-iff

lemma bind-return-iff[return-iff]: Error-Monad.bind  $m\ f =$  Inr  $y \longleftrightarrow (\exists x. m$ 
 $=$  Inr  $x \wedge f\ x =$  Inr  $y)$ 
by auto

lemma lift-opt-return-iff[return-iff]: lift-opt  $m\ e =$  Inr  $x \longleftrightarrow m =$  Some  $x$ 
unfolding lift-opt-def by (auto split: option.split)

lemma check-return-iff[return-iff]: check  $\Phi\ e =$  Inr  $uu \longleftrightarrow \Phi$ 
by (auto simp: check-def)

lemma check-simps[simp]:
  check True  $e =$  succeed
  check False  $e =$  error  $e$ 
by (auto simp: check-def)

lemma Let-return-iff[return-iff]: (let  $x=v$  in  $f\ x$ ) = Inr  $w \longleftrightarrow f\ v =$  Inr  $w$  by

```

simp

abbreviation $ERR :: shows \Rightarrow (unit \Rightarrow shows)$ **where** $ERR\ s \equiv \lambda-. s$
abbreviation $ERRS :: string \Rightarrow (unit \Rightarrow shows)$ **where** $ERRS\ s \equiv ERR\ (shows\ s)$

lemma *error-monad-bind-split*: $P\ (bind\ m\ f) \longleftrightarrow (\forall v. m = Inl\ v \longrightarrow P\ (Inl\ v))$
 $\wedge (\forall v. m = Inr\ v \longrightarrow P\ (f\ v))$
by (*cases m*) *auto*

lemma *error-monad-bind-split-asm*: $P\ (bind\ m\ f) \longleftrightarrow \neg (\exists x. m = Inl\ x \wedge \neg P\ (Inl\ x) \vee (\exists x. m = Inr\ x \wedge \neg P\ (f\ x)))$
by (*cases m*) *auto*

lemmas *error-monad-bind-splits = error-monad-bind-split error-monad-bind-split-asm*

end

2 Executable Temporal PDDL Checker

theory *TEMPORAL-PDDL-Checker*

imports

TEMPORAL-PDDL-Semantics

Error-Monad-Add

HOL.String

HOL-Library.Code-Target-Nat

HOL-Library.While-Combinator

Containers.Containers

HOL-Library.Tree

begin

2.1 Generic DFS Reachability Checker

Used for subtype checks

definition *E-of-succ succ* $\equiv \{ (u, v). v \in set\ (succ\ u) \}$

lemma *succ-as-E*: $set\ (succ\ x) = E\text{-of-succ}\ succ\ \{\{x\}$

unfolding *E-of-succ-def* **by** *auto*

context

fixes $succ :: 'a \Rightarrow 'a\ list$

begin

private abbreviation (*input*) $E \equiv E\text{-of-succ}\ succ$

definition *dfs-reachable* $D\ w \equiv$
 $\text{let } (V, w, brk) = \text{while } (\lambda(V, w, brk). \neg brk \wedge w \neq []) (\lambda(V, w, -). \\$
 $\text{case } w \text{ of } v \# w \Rightarrow \\$
 $\text{if } D\ v \text{ then } (V, v \# w, \text{True}) \\$
 $\text{else if } v \in V \text{ then } (V, w, \text{False}) \\$
 $\text{else} \\$
 $\text{let } V = \text{insert } v\ V \text{ in} \\$
 $\text{let } w = \text{succ } v\ @\ w \text{ in} \\$
 $(V, w, \text{False}) \\$
 $)\ (\{\}, w, \text{False}) \\$
 $\text{in } brk$

context
fixes $w_0 :: 'a\ \text{list}$
assumes *finite-dfs-reachable*[*simp*, *intro*!]: *finite* (E^* “ *set* w_0)
begin

private abbreviation (*input*) $W_0 \equiv \text{set } w_0$

definition *dfs-reachable-invar* $D\ V\ W\ brk \longleftrightarrow$
 $W_0 \subseteq W \cup V \\$
 $\wedge\ W \cup V \subseteq E^* \text{ “ } W_0 \\$
 $\wedge\ E \text{ “ } V \subseteq W \cup V \\$
 $\wedge\ \text{Collect } D \cap V = \{\} \\$
 $\wedge\ (brk \longrightarrow \text{Collect } D \cap E^* \text{ “ } W_0 \neq \{\})$

lemma *card-decreases*:
 $\llbracket \text{finite } V; y \notin V; \text{dfs-reachable-invar } D\ V\ (\text{Set.insert } y\ W)\ brk \rrbracket \\$
 $\implies \text{card } (E^* \text{ “ } W_0 - \text{Set.insert } y\ V) < \text{card } (E^* \text{ “ } W_0 - V) \\$
apply (*rule psubset-card-mono*)
apply (*auto simp: dfs-reachable-invar-def*)
done

lemma *all-neq-Cons-is-Nil*[*simp*]:
 $(\forall y\ ys. x2 \neq y \# ys) \longleftrightarrow x2 = []$ **by** (*cases x2*) *auto*

lemma *dfs-reachable-correct*: *dfs-reachable* $D\ w_0 \longleftrightarrow \text{Collect } D \cap E^* \text{ “ } \text{set } w_0 \neq \{\}$
unfolding *dfs-reachable-def*
apply (*rule while-rule* [**where**
 $P = \lambda(V, w, brk). \text{dfs-reachable-invar } D\ V\ (\text{set } w)\ brk \wedge \text{finite } V \\$
 $\text{and } r = \text{measure } (\lambda V. \text{card } (E^* \text{ “ } (\text{set } w_0) - V)) < *lex* > \text{measure length} \\$
 $< *lex* > \text{measure } (\lambda \text{True} \Rightarrow 0 \mid \text{False} \Rightarrow 1) \\$
 $\rrbracket$])
subgoal by (*auto simp: dfs-reachable-invar-def*)
subgoal


```

    apply (auto simp: neq-Nil-conv succ-as-E[of succ] split: if-splits)
  by (auto simp: dfs-reachable-invar-def Image-iff intro: rtrancl.rtrancl-into-rtrancl)
subgoal by (fastforce simp: dfs-reachable-invar-def dest: Image-closed-trancl)
subgoal by blast
subgoal by (auto simp: neq-Nil-conv card-decreases)
done

```

end

definition *tab-succ* $l \equiv \text{Mapping.lookup-default } [] \text{ (fold } (\lambda(u,v). \text{ Mapping.map-default } u \text{ } [] \text{ (Cons } v)) \text{ } l \text{ Mapping.empty)}$

lemma *Some-eq-map-option* [iff]: $(\text{Some } y = \text{map-option } f \text{ } xo) = (\exists z. xo = \text{Some } z \wedge f \text{ } z = y)$
 by (auto simp add: map-option-case split: option.split)

lemma *tab-succ-correct*: $E\text{-of-succ } (\text{tab-succ } l) = \text{set } l$

proof –

```

  have set (Mapping.lookup-default [] (fold (\lambda(u,v). Mapping.map-default u [] (Cons
v)) l m) u) = set l “ {u} ∪ set (Mapping.lookup-default [] m u)
  for m u
  apply (induction l arbitrary: m)
  by (auto
    simp: Mapping.lookup-default-def Mapping.map-default-def Mapping.default-def
    simp: lookup-map-entry' lookup-update' keys-is-none-rep Option.is-none-def
    split: if-splits
  )
  from this[where m=Mapping.empty] show ?thesis
  by (auto simp: E-of-succ-def tab-succ-def lookup-default-empty)

```

qed

end

lemma *finite-imp-finite-dfs-reachable*:

```

[[finite E; finite S]] ⟹ finite (E* “S)
  apply (rule finite-subset[where B=S ∪ (Relation.Domain E ∪ Relation.Range
E)])
  apply (auto simp: intro: finite-Domain finite-Range elim: rtranclE)
done

```

lemma *dfs-reachable-tab-succ-correct*: $\text{dfs-reachable } (\text{tab-succ } l) \text{ } D \text{ } vs_0 \longleftrightarrow \text{Collect } D \cap (\text{set } l)^* \text{ “set } vs_0 \neq \{\}$

```

  apply (subst dfs-reachable-correct)
  by (simp-all add: tab-succ-correct finite-imp-finite-dfs-reachable)

```

2.2 Implementation Refinements

2.2.1 Of-Type

definition *of-type-impl* $G \text{ } oT \text{ } T \equiv (\forall pt \in \text{set } (\text{primitives } oT). \text{dfs-reachable } G ((=) pt) (\text{primitives } T))$

fun *ty-term'* **where**

ty-term' *varT objT* (*term.VAR* *v*) = *varT v*
| *ty-term'* *varT objT* (*term.CONST* *c*) = *Mapping.lookup objT c*

lemma *ty-term'-correct-aux*: *ty-term'* *varT objT t* = *ty-term varT (Mapping.lookup objT) t*
by (*cases t*) *auto*

lemma *ty-term'-correct[simp]*: *ty-term'* *varT objT* = *ty-term varT (Mapping.lookup objT)*
using *ty-term'-correct-aux* **by** *auto*

context *ast-domain* **begin**

definition *of-type1* $pt \text{ } T \longleftrightarrow pt \in \text{subtype-rel}^* \text{ `` } \text{set } (\text{primitives } T)$

lemma *of-type-refine1*: *of-type* $oT \text{ } T \longleftrightarrow (\forall pt \in \text{set } (\text{primitives } oT). \text{of-type1 } pt T)$
unfolding *of-type-def* *of-type1-def* **by** *auto*

definition *STG* $\equiv (\text{tab-succ } (\text{map } \text{subtype-edge } (\text{types } D)))$

lemma *subtype-rel-impl*: *subtype-rel* = *E-of-succ (tab-succ (map subtype-edge (types D)))*
by (*simp add: tab-succ-correct subtype-rel-def*)

lemma *of-type1-impl*: *of-type1* $pt \text{ } T \longleftrightarrow \text{dfs-reachable } (\text{tab-succ } (\text{map } \text{subtype-edge } (\text{types } D))) ((=)pt) (\text{primitives } T)$
by (*simp add: subtype-rel-impl of-type1-def dfs-reachable-tab-succ-correct tab-succ-correct*)

lemma *of-type-impl-correct*: *of-type-impl* $STG \text{ } oT \text{ } T \longleftrightarrow \text{of-type } oT \text{ } T$
unfolding *of-type1-impl* *STG-def* *of-type-impl-def* *of-type-refine1* **..**

definition *mp-constT* :: (*object, type*) *mapping* **where**
mp-constT = *Mapping.of-alist (consts D)*

lemma *mp-objT-correct[simp]*: *Mapping.lookup mp-constT* = *constT*
unfolding *mp-constT-def* *constT-def*
by *transfer (simp add: Map-To-Mapping.map-apply-def)*

Lifting the subtype-graph through wf-checker

context

```

fixes ty-ent :: 'ent  $\rightarrow$  type — Entity's type, None if invalid
begin

definition is-of-type' stg v T  $\longleftrightarrow$  (
  case ty-ent v of
    Some vT  $\Rightarrow$  of-type-impl stg vT T
  | None  $\Rightarrow$  False)

lemma is-of-type'-correct: is-of-type' STG v T = is-of-type ty-ent v T
unfolding is-of-type'-def is-of-type-def of-type-impl-correct ..

fun wf-pred-atom' where wf-pred-atom' stg (p,vs)  $\longleftrightarrow$ 
  (case sig p of
    None  $\Rightarrow$  False
  | Some Ts  $\Rightarrow$  list-all2 (is-of-type' stg) vs Ts)

lemma wf-pred-atom'-correct: wf-pred-atom' STG pvs = wf-pred-atom ty-ent
pvs
by (cases pvs) (auto simp: is-of-type'-correct[abs-def] split:option.split)

fun wf-func-args' where wf-func-args' stg (f,args)  $\longleftrightarrow$ 
  (case func-sig f of
    None  $\Rightarrow$  False
  | Some Ts  $\Rightarrow$  list-all2 (is-of-type' stg) args Ts)

lemma wf-func-args'-correct: wf-func-args' STG fargs = wf-func-args ty-ent
fargs
by (cases fargs) (auto simp: is-of-type'-correct[abs-def] split:option.split)

fun wf-atom' :: -  $\Rightarrow$  'ent atom  $\Rightarrow$  bool where
  wf-atom' stg (atom.predAtm p vs)  $\longleftrightarrow$  wf-pred-atom' stg (p,vs)
  | wf-atom' stg (atom.eqAtm a b) = (ty-ent a  $\neq$  None  $\wedge$  ty-ent b  $\neq$  None)

lemma wf-atom'-correct: wf-atom' STG a = wf-atom ty-ent a
by (cases a) (auto simp: wf-pred-atom'-correct is-of-type'-correct[abs-def] split:option.splits)

fun wf-fmla' :: -  $\Rightarrow$  ('ent atom) formula  $\Rightarrow$  bool where
  wf-fmla' stg (Atom a)  $\longleftrightarrow$  wf-atom' stg a
  | wf-fmla' stg  $\perp$   $\longleftrightarrow$  True
  | wf-fmla' stg ( $\varphi1 \wedge \varphi2$ )  $\longleftrightarrow$  (wf-fmla' stg  $\varphi1 \wedge$  wf-fmla' stg  $\varphi2$ )
  | wf-fmla' stg ( $\varphi1 \vee \varphi2$ )  $\longleftrightarrow$  (wf-fmla' stg  $\varphi1 \wedge$  wf-fmla' stg  $\varphi2$ )
  | wf-fmla' stg ( $\varphi1 \rightarrow \varphi2$ )  $\longleftrightarrow$  (wf-fmla' stg  $\varphi1 \wedge$  wf-fmla' stg  $\varphi2$ )
  | wf-fmla' stg ( $\neg \varphi$ )  $\longleftrightarrow$  wf-fmla' stg  $\varphi$ 

lemma wf-fmla'-correct: wf-fmla' STG  $\varphi$   $\longleftrightarrow$  wf-fmla ty-ent  $\varphi$ 
by (induction  $\varphi$  rule: wf-fmla.induct) (auto simp: wf-atom'-correct)

fun wf-fmla-atom1' where

```

$wf\text{-}fmla\text{-}atom1' \text{ stg } (Atom \ (predAtm \ p \ vs)) \longleftrightarrow wf\text{-}pred\text{-}atom' \text{ stg } (p, vs)$
 $| \ wf\text{-}fmla\text{-}atom1' \text{ stg } - \longleftrightarrow False$

lemma $wf\text{-}fmla\text{-}atom1'\text{-}correct$: $wf\text{-}fmla\text{-}atom1' \text{ STG } \varphi = wf\text{-}fmla\text{-}atom \text{ ty-ent}$
 φ

by (cases φ rule: $wf\text{-}fmla\text{-}atom.cases$) (auto
simp: $wf\text{-}atom'\text{-}correct$ $is\text{-}of\text{-}type'\text{-}correct[abs\text{-}def]$ $split$: $option.splits$)

fun $wf\text{-}effect'$ **where**
 $wf\text{-}effect' \text{ stg } (Effect \ a \ d) \longleftrightarrow$
 $(\forall ae \in set \ a. \ wf\text{-}fmla\text{-}atom1' \text{ stg } ae)$
 $\wedge (\forall de \in set \ d. \ wf\text{-}fmla\text{-}atom1' \text{ stg } de)$

lemma $wf\text{-}effect'\text{-}correct$: $wf\text{-}effect' \text{ STG } e = wf\text{-}effect \text{ ty-ent } e$
by (cases e) (auto simp: $wf\text{-}fmla\text{-}atom1'\text{-}correct$)

fun $wf\text{-}duration\text{-}const'$:: $- \Rightarrow 'ent \text{ duration-constraint} \Rightarrow bool$ **where**
 $wf\text{-}duration\text{-}const' \text{ stg } No\text{-}Const \longleftrightarrow True$
 $| \ wf\text{-}duration\text{-}const' \text{ stg } (Time\text{-}Const \ op \ d) \longleftrightarrow d \geq 0$
 $| \ wf\text{-}duration\text{-}const' \text{ stg } (Func\text{-}Const \ op \ f \ vs) \longleftrightarrow$
 $(case \ func\text{-}sig \ f \ of$
 $\quad None \Rightarrow False$
 $\quad | \ Some \ Ts \Rightarrow list\text{-}all2 \ (is\text{-}of\text{-}type' \text{ stg }) \ vs \ Ts)$

lemma $wf\text{-}duration\text{-}const'\text{-}correct$: $wf\text{-}duration\text{-}const' \text{ STG } d = wf\text{-}duration\text{-}const \text{ ty-ent } d$
by (cases d) (auto simp: $is\text{-}of\text{-}type'\text{-}correct[abs\text{-}def]$ $split$: $option.split$)

end — Context fixing $ty\text{-}ent$

fun $wf\text{-}action\text{-}schema'$:: $- \Rightarrow - \Rightarrow ast\text{-}action\text{-}schema \Rightarrow bool$ **where**
 $wf\text{-}action\text{-}schema' \text{ stg } conT \ (Simple\text{-}Action\text{-}Schema \ n \ params \ pre \ eff) \longleftrightarrow ($
 $\quad let$
 $\quad \quad tyv = ty\text{-}term' \ (map\text{-}of \ params) \ conT$
 $\quad in$
 $\quad \quad distinct \ (map \ fst \ params)$
 $\quad \wedge \ wf\text{-}fmla' \ tyv \text{ stg } pre$
 $\quad \wedge \ wf\text{-}effect' \ tyv \text{ stg } eff)$
 $| \ wf\text{-}action\text{-}schema' \text{ stg } conT \ (Durative\text{-}Action\text{-}Schema \ n \ params \ d \ cond \ eff) \longleftrightarrow$
 $($
 $\quad let$
 $\quad \quad tyv = ty\text{-}term' \ (map\text{-}of \ params) \ conT$
 $\quad in$
 $\quad \quad distinct \ (map \ fst \ params)$
 $\quad \wedge (\forall (t, c) \in set \ cond. \ wf\text{-}fmla' \ tyv \text{ stg } c)$
 $\quad \wedge (\forall (t, e) \in set \ eff. \ wf\text{-}effect' \ tyv \text{ stg } e \wedge t \neq Over\text{-}All)$
 $\quad \wedge \ wf\text{-}duration\text{-}const' \ tyv \text{ stg } d)$

lemma $wf\text{-}action\text{-}schema'\text{-}correct$: $wf\text{-}action\text{-}schema' \text{ STG } mp\text{-}constT \ s = wf\text{-}action\text{-}schema$

s

by (*cases s*)
 (*auto simp: wf-fmla'-correct wf-effect'-correct wf-duration-const'-correct Let-def*
split: option.splits)

definition *wf-domain'* :: $- \Rightarrow - \Rightarrow \text{bool}$ **where**
wf-domain' stg conT $\equiv$
wf-types
 $\wedge \text{distinct } (\text{map } (\text{predicate-decl.pred}) (\text{predicates } D))$
 $\wedge (\forall p \in \text{set } (\text{predicates } D). \text{wf-predicate-decl } p)$
 $\wedge \text{distinct } (\text{map } (\text{function-decl.func}) (\text{functions } D))$
 $\wedge (\forall f \in \text{set } (\text{functions } D). \text{wf-function-decl } f)$
 $\wedge \text{distinct } (\text{map fst } (\text{consts } D))$
 $\wedge (\forall (n, T) \in \text{set } (\text{consts } D). \text{wf-type } T)$
 $\wedge \text{distinct } (\text{map ast-action-schema.name } (\text{actions } D))$
 $\wedge (\forall a \in \text{set } (\text{actions } D). \text{wf-action-schema' stg conT } a)$

lemma *wf-domain'-correct*: *wf-domain' STG mp-constT = wf-domain*
unfolding *wf-domain-def wf-domain'-def*
by (*auto simp: wf-action-schema'-correct*)

end — Context of *ast-domain*

2.2.2 Ground Action Interference & Application of Effects

context *ast-problem* **begin**

Implementation of executable refinements for *acts-non-intrf* & *apply-happ*

a list of simultaneous & non-interfering grounded actions are all applied at once

fun *apply-happ-exec* :: *happening* $\Rightarrow$ *world-model* $\Rightarrow$ *world-model* **where**
apply-happ-exec (*t_i*, *A_i*) *s* =
 (*let*
adds = (*concat o (map (adds o effect))*) *A_i*;
dels = (*concat o (map (dels o effect))*) *A_i* *in*
fold Set.insert adds (fold Set.remove dels s))

lemma *fold-set-remove*: *fold Set.remove xs s = s - set xs*
by (*induction xs arbitrary: s*) *auto*

lemma *fold-set-insert*: *fold Set.insert xs s = s $\cup$ set xs*
by (*induction xs arbitrary: s*) *auto*

Justification of refinement for *apply-happ*

lemma *apply-happ-exec-refine*: *apply-happ-exec h s = apply-happ h s*

by (*cases h*) (*auto simp: fold-set-remove fold-set-insert*)

end — Context of *ast-problem*

2.2.3 Well-Formedness

context *ast-problem* **begin**

We start by defining a mapping from objects to types. The container framework will generate efficient, red-black tree based code for that later.

type-synonym *objT* = (*object*, *type*) *mapping*

definition *mp-objT* :: (*object*, *type*) *mapping* **where**
mp-objT = *Mapping.of-alist* (*consts D @ objects P*)

lemma *mp-objT-correct[simp]*: *Mapping.lookup mp-objT = objT*
unfolding *mp-objT-def objT-alt*
by *transfer* (*simp add: Map-To-Mapping.map-apply-def*)

We refine the typecheck to use the mapping

definition *is-obj-of-type-impl stg mp n T* = (
case Mapping.lookup mp n of None $\Rightarrow$ False | Some oT $\Rightarrow$ of-type-impl stg oT
T
)

lemma *is-obj-of-type-impl-correct[simp]*:
is-obj-of-type-impl STG mp-objT = is-obj-of-type
apply (*intro ext*)
apply (*auto simp: is-obj-of-type-impl-def is-obj-of-type-def of-type-impl-correct*
split: option.split)
done

fun *wf-func-assign'* :: *objT* $\Rightarrow$ - $\Rightarrow$ *object atom formula* $\Rightarrow$ *bool* **where**
wf-func-assign' ot stg (*Atom* (*eqAtm* (*FuncEnt f args*) (*TimeEnt d*)))
 $\longleftrightarrow$ *wf-func-args'* (*Mapping.lookup ot*) *stg* (*f, args*)
| *wf-func-assign' ot stg* - $\longleftrightarrow$ *False*

lemma *wf-func-assign'-correct[simp]*: *wf-func-assign' mp-objT STG f = wf-func-assign*
f
by (*cases f rule: wf-func-assign.cases*) (*auto simp: wf-func-args'-correct[abs-def]*)

We refine the well-formedness checks to use the mapping

definition *wf-fact'* :: *objT* $\Rightarrow$ - $\Rightarrow$ *fact* $\Rightarrow$ *bool* **where**
wf-fact' ot stg $\equiv$ *wf-pred-atom'* (*Mapping.lookup ot*) *stg*

lemma *wf-fact'-correct[simp]*: *wf-fact' mp-objT STG = wf-fact*
by (*auto simp: wf-fact'-def wf-fact-def wf-pred-atom'-correct[abs-def]*)

definition *wf-fmla-atom2'* mp stg f
 = (case f of formula.Atom (predAtm p vs) $\Rightarrow$ (wf-fact' mp stg (p,vs)) | - $\Rightarrow$ False)

lemma *wf-fmla-atom2'-correct*[simp]:
wf-fmla-atom2' mp-objT STG φ = *wf-fmla-atom* objT φ
apply (cases φ rule: *wf-fmla-atom.cases*)
by (auto simp: *wf-fmla-atom2'-def* *wf-fact-def* split: option.splits)

definition *wf-problem'* stg conT mp $\equiv$
wf-domain' stg conT
 $\wedge$ distinct (map fst (objects P) @ map fst (consts D))
 $\wedge$ ($\forall (n,T) \in \text{set } (\text{objects } P).$ wf-type T)
 $\wedge$ distinct (init P)
 $\wedge$ ($\forall f \in \text{set } (\text{init } P).$ *wf-fmla-atom2'* mp stg f $\vee$ *wf-func-assign'* mp stg f)
 $\wedge$ *wf-fmla'* (Mapping.lookup mp) stg (goal P)

lemma *wf-problem'-correct*:
wf-problem' STG mp-constT mp-objT = *wf-problem*
unfolding *wf-problem-def* *wf-problem'-def* *wf-world-model-def*
by (auto simp: *wf-domain'-correct* *wf-fmla'-correct*)

Instantiating actions will yield well-founded effects. Corollary of $\llbracket \text{action-params-match} \text{ (Simple-Action-Schema } ?n \text{ ?params ?pre ?eff) ?args; wf-action-schema (Simple-Action-Schema } ?n \text{ ?params ?pre ?eff)} \rrbracket \Longrightarrow \text{wf-ground-action (instantiate-action-schema (Simple-Action-Schema } ?n \text{ ?params ?pre ?eff) ?args)}$ and $\llbracket \text{action-params-match} \text{ (Durative-Action-Schema } ?n \text{ ?params ?d ?cond ?eff) ?args; wf-action-schema (Durative-Action-Schema } ?n \text{ ?params ?d ?cond ?eff)} \rrbracket \Longrightarrow \text{wf-ground-action (inst-snap-action (Durative-Action-Schema } ?n \text{ ?params ?d ?cond ?eff) ?args ?ta)}$.

lemma *wf-effect-inst-weak*:
fixes n params pre eff
defines $a_{\text{schema}} \equiv \text{Simple-Action-Schema } n \text{ params pre eff}$
assumes $a = \text{instantiate-action-schema } a_{\text{schema}} \text{ args}$
and *action-params-match* a_{schema} args
and *wf-action-schema* a_{schema}
shows *wf-effect-inst* (effect a)
using *assms wf-inst-action-schema*
by (cases a) (auto simp: *wf-effect-inst-alt* Let-def)

lemma *wf-effect-durative-inst-weak*:
fixes n params d cond eff
defines $a_{\text{schema}} \equiv \text{Durative-Action-Schema } n \text{ params d cond eff}$
assumes $a = \text{inst-snap-action } a_{\text{schema}} \text{ args ta}$
and *action-params-match* a_{schema} args
and *wf-action-schema* a_{schema}
shows *wf-effect-inst* (effect a)
using *assms wf-inst-durative-action-schema*

by (*cases a*) (*auto simp: wf-effect-inst-alt Let-def*)

end — Context of *ast-problem*

2.2.4 Happening Execution

context *ast-domain* **begin**

We first lift action schema lookup into the error monad.

definition *resolve-action-schemaE* $n \equiv$
 lift-opt
 $(\text{resolve-action-schema } n)$
 $(\text{ERR } (\text{shows "No such action schema " o shows } n))$

lemma *resolve-action-schemaE-return-iff*[*return-iff*]:
 $(\text{resolve-action-schemaE } n = \text{Inr } a) \longleftrightarrow (\text{resolve-action-schema } n = \text{Some } a)$
using *resolve-action-schemaE-def* **by** (*simp add: lift-opt-return-iff*)

end — Context of *ast-domain*

context *ast-problem* **begin**

We define a function to determine whether a formula holds in a world model

definition *holds* $M F \equiv (\text{valuation } M) \models F$

Justification of this function

lemma *holds-for-wf-fmlas*:
assumes *wm-basic s*
shows $\text{holds } s F \longleftrightarrow \text{close-world } s \models F$
unfolding *holds-def* **using** *assms valuation-iff-close-world* **by** *blast*

The first refinement summarizes the enabledness check and the application of a happening. Moreover, we implement the precondition evaluation by our *holds* function. This way, we can eliminate redundant resolving and instantiation of the action.

definition *en-exE* $:: \text{happening} \Rightarrow \text{world-model} \Rightarrow \text{+world-model}$ **where**
 $\text{en-exE} \equiv \lambda(t_i, A_i) \Rightarrow \lambda s. \text{do } \{$
 $\text{check-allm } (\lambda a. \text{check } (\text{holds } s (\text{precondition } a)) (\text{ERRS "Precondition not satisfied"})) A_i;$
 $\text{check-pairwise } (\lambda a b. \text{check } (a \neq b \longrightarrow \text{acts-non-intrf } a b) (\text{ERRS "Actions in happening interfering"})) A_i;$
 $\text{Error-Monad.return } (\text{apply-happ-exec } (t_i, A_i) s)$
 $\}$

lemma *symmetric-pred-check-pairwise*:
assumes $\forall x_1 x_2. A x_1 x_2 \longleftrightarrow A x_2 x_1$
and *isOK* ($\text{check-pairwise } (\lambda x_1 x_2. \text{check } (x_1 \neq x_2 \longrightarrow A x_1 x_2) e) as$)
shows $\forall x_1 \in \text{set } as. \forall x_2 \in \text{set } as. x_1 \neq x_2 \longrightarrow A x_1 x_2$

using *assms* **by** (*auto simp: return-iff*) (*smt in-set-conv-nth linorder-neqE-nat*)

Justification of implementation.

lemma (**in** *wf-ast-problem*) *en-exE-return-iff*:
assumes *wm-basic s*
and $\forall a \in \text{set } A_i. \text{wf-ground-action } a$
shows *en-exE* (t_i, A_i) $s = \text{Inr } s' \iff \text{happ-enabled } (t_i, A_i) s \wedge s' = \text{apply-happ}$
 $(t_i, A_i) s$
unfolding *en-exE-def*
using *assms holds-for-wf-fmlas*[*OF* $\langle \text{wm-basic } s \rangle$]
symmetric-pred-check-pairwise[*OF* *acts-non-intrf-symmetric*]
apply-happ-exec-refine
by *auto*

Next, we use the efficient implementation *is-obj-of-type-impl* for the type check, and omit the well-formedness check, as effects obtained from instantiating well-formed action schemas are always well-formed (*wf-effect-inst-weak*).

abbreviation *action-params-match2* *stg mp a args*
 $\equiv \text{list-all2 } (\text{is-obj-of-type-impl } \text{stg } \text{mp})$
 $\text{args } (\text{map } \text{snd } (\text{ast-action-schema.parameters } a))$

end — Context of *ast-problem*

2.2.5 Construction of Induced Happening Sequence

context *ast-problem* **begin**

implemented own version of *insort-insert*, because *insort-insert* requires sort type; *rat* is not of sort type

fun *insort-htp* :: *time* $\Rightarrow$ *time list* $\Rightarrow$ *time list* **where**
insort-htp $x \ [] = [x]$
 $| \text{insort-htp } x (y \# ys) =$
 $(\text{if } x < y \text{ then } x \# y \# ys$
 $\text{else if } x = y \text{ then } y \# ys$
 $\text{else } y \# (\text{insort-htp } x \text{ } ys))$

Basic properties text of *insort-htp*: preserves sorted-ness & distinct-ness, inserts element

lemma *insort-htps-insort-insert*: *sorted xs* $\implies \text{insort-htp } x \text{ } xs = \text{insort-insert } x$
 xs

by (*induction x xs rule: insort-htp.induct*) (*auto simp: insort-insert-key-def*)

lemma *sorted-insort-htps*: *sorted xs* $\implies \text{sorted } (\text{insort-htp } x \text{ } xs)$
using *insort-htps-insort-insert sorted-insort-insert* **by** *auto*

lemma *set-insort-htps*: *set (insort-htp x xs)* = *insert x (set xs)*
by (*induction x xs rule: insort-htp.induct*) *auto*

lemma *distinct-insort-htps*: $\text{sorted } xs \implies \text{distinct } xs \implies \text{distinct } (\text{insort-htp } x \text{ } xs)$

using *set-insort-htps* **by** (*induction* $x \text{ } xs$ *rule*: *insort-htp.induct*) *auto*

lemma *insort-htps-non-Nil*: $\text{insort-htp } x \text{ } xs \neq []$

by (*induction* $x \text{ } xs$ *rule*: *insort-htp.induct*) *auto*

Implementation of executable refinement for happening time points. Produces a strictly sorted sequence of all *is-htp*

fun *htps-exec* :: *plan* $\Rightarrow$ *time list* **where**

htps-exec [] = []

| *htps-exec* $((t_\pi, \text{Simple-Plan-Action } n \text{ } args) \# \pi s) =$
 $\text{insort-htp } t_\pi (\text{htps-exec } \pi s)$

| *htps-exec* $((t_\pi, \text{Durative-Plan-Action } n \text{ } args \text{ } d) \# \pi s) =$
 $\text{insort-htp } t_\pi (\text{insort-htp } (t_\pi + d) (\text{htps-exec } \pi s))$

Basic properties of *htps-exec*

lemma *htps-exec-Nil*: $\text{htps-exec } \pi s = [] \longleftrightarrow \pi s = []$

by (*induction* πs *rule*: *htps-exec.induct*) (*auto simp*: *insort-htps-non-Nil split: if-splits*)

lemma *htps-exec-sorted*: $\text{sorted } (\text{htps-exec } \pi s)$

using *sorted-insort-htps* **by** (*induction* πs *rule*: *htps-exec.induct*) *auto*

lemma *htps-exec-distinct*: $\text{distinct } (\text{htps-exec } \pi s)$

using *htps-exec-sorted distinct-insort-htps sorted-insort-htps*

by (*induction* πs *rule*: *htps-exec.induct*) *auto*

Correctness proof for *htps-exec*

Every time point in *htps-exec* is a happening time point.

lemma *htps-exec-is-htps*: $t_i \in \text{set } (\text{htps-exec } \pi s) \implies \text{is-htp } \pi s \text{ } t_i$

proof (*induction* πs)

case *Nil*

then show *?case* **by** *simp*

next

case (*Cons* $\pi \text{ } \pi s$)

obtain $t_\pi \pi'$ **where** $\pi = (t_\pi, \pi')$

by (*meson surj-pair*)

then show *?case*

using $\langle \pi = (t_\pi, \pi') \rangle$ *set-insort-htps Cons.IH Cons.prem*s

by (*cases* π') (*auto simp*: *is-htp-def durative-acts-def is-act-simple-def*)

qed

lemma *htps-exec-Cons-subset*: $\text{set } (\text{htps-exec } \pi s) \subseteq \text{set } (\text{htps-exec } (\pi \# \pi s))$

by (*induction* $\pi \# \pi s$ *rule*: *htps-exec.induct*) (*auto simp*: *set-insort-htps*)

lemma *htps-exec-app-subset*: $\text{set } (\text{htps-exec } \pi s_2) \subseteq \text{set } (\text{htps-exec } (\pi s_1 @ \pi s_2))$

using *htps-exec-Cons-subset*

by (induction πs_1 arbitrary: πs_2 rule: *https-exec.induct*) (auto simp: *set-insort-https*)

Every happening time point is included in *https-exec*.

```

lemma is-https-https-exec: is-htp  $\pi s$   $t_i \implies t_i \in \text{set } (\text{https-exec } \pi s)$ 
proof (induction  $\pi s$ )
  case Nil
  then show ?case
    unfolding is-htp-def durative-acts-def is-act-simple-def by simp
  next
    case (Cons  $\pi$   $\pi s$ )
    have  $(\exists \pi'. (t_i, \pi') \in \text{set } (\pi \# \pi s)) \vee (\exists (t_\pi, \pi') \in \text{durative-acts } (\pi \# \pi s). t_i = t_\pi + (\text{duration } \pi'))$ 
    using Cons.prems unfolding is-htp-def by simp
    then show ?case
    proof
      assume  $\exists \pi'. (t_i, \pi') \in \text{set } (\pi \# \pi s)$ 
      then obtain  $\pi'$  where  $(t_i, \pi') \in \text{set } (\pi \# \pi s)$ 
      by auto
      then obtain  $\pi s_1$   $\pi s_2$  where  $(\pi \# \pi s) = \pi s_1 @ ((t_i, \pi') \# \pi s_2)$ 
      by (meson split-list)
      then have  $t_i \in \text{set } (\text{https-exec } ((t_i, \pi') \# \pi s_2))$ 
      using set-insort-https by (cases  $\pi'$ ) auto
      then show ?case
      using  $\langle (\pi \# \pi s) = \pi s_1 @ ((t_i, \pi') \# \pi s_2) \rangle$  https-exec-app-subset by auto
    next
      assume  $\exists (t_\pi, \pi') \in \text{durative-acts } (\pi \# \pi s). t_i = t_\pi + (\text{duration } \pi')$ 
      then obtain  $t_\pi$   $\pi'$  where  $(t_\pi, \pi') \in \text{durative-acts } (\pi \# \pi s) \wedge t_i = t_\pi + (\text{duration } \pi')$ 
      by auto
      then obtain  $\pi s_1$   $\pi s_2$  where  $(\pi \# \pi s) = \pi s_1 @ ((t_\pi, \pi') \# \pi s_2)$ 
      using dur-acts-subset by (meson split-list subsetD)
      then have  $t_i \in \text{set } (\text{https-exec } ((t_\pi, \pi') \# \pi s_2))$ 
      using set-insort-https  $\langle (t_\pi, \pi') \in \text{durative-acts } (\pi \# \pi s) \wedge t_i = t_\pi + (\text{duration } \pi') \rangle$ 
      unfolding durative-acts-def is-act-simple-def by (cases  $\pi'$ ) auto
      then show ?case
      using  $\langle (\pi \# \pi s) = \pi s_1 @ ((t_\pi, \pi') \# \pi s_2) \rangle$  https-exec-app-subset by auto
    qed
  qed

```

```

lemma is-htp-iff-https-exec:  $\forall t. \text{is-htp } \pi s t \iff t \in \text{set } (\text{https-exec } \pi s)$ 
using https-exec-is-https is-https-https-exec by blast

```

```

definition consec-https-in-interval :: time list  $\Rightarrow$  time  $\Rightarrow$  time  $\Rightarrow$  (time  $\times$  time)
list where
  consec-https-in-interval https  $t_i$   $t_j =$ 
    filter  $(\lambda(t, t'). t_i \leq t \wedge t' \leq t_j)$  (zip (butlast https) (tl https))

```

Construct all ground actions for a given plan action

```

fun simplify-action :: time list  $\Rightarrow$  (time  $\times$  plan-action)  $\Rightarrow$  (time  $\times$  ground-action)
list where
  simplify-action https (t, Simple-Plan-Action n args) = (
    let a = the (resolve-action-schema n) in
    [(t, instantiate-action-schema a args)]
  )
| simplify-action https (t, Durative-Plan-Action n args d) = (
  let a = the (resolve-action-schema n) in
  (t, inst-snap-action a args At-Start) #
  (t+d, inst-snap-action a args At-End) #
  (map ( $\lambda(t_i, t_j).((t_i+t_j) / 2, \text{inst-snap-action } a \text{ args Over-All})) (\text{consec-https-in-interval}$ 
    https t (t+d)))
  )

```

Auxiliary lemmas to prove properties about 'zip (butlast xs) (tl xs)', which are needed to ultimately prove properties about *consec-https*.

```

lemma zip-butlast-tl-hd:
  assumes distinct (x1 # xs)
    and (x1, x2)  $\in$  set (zip (butlast (x1 # xs)) (tl (x1 # xs)))
  shows xs  $\neq$  []  $\wedge$  x2 = hd xs
  using assms
  apply (induction xs)
  apply (auto split: if-splits)
  apply (meson in-set-butlastD in-set-zipE set-ConsD)
  done

```

```

lemma zip-butlast-sublist:
  assumes distinct (xs1 @ x1 # xs2)
    and (x1, x2)  $\in$  set (zip (butlast (xs1 @ x1 # xs2)) (tl (xs1 @ x1 # xs2)))
  shows (x1, x2)  $\in$  set (zip (butlast (x1 # xs2)) (tl (x1 # xs2)))
  using assms
  apply (induction xs1)
  apply (auto split: if-splits)
  using set-zip-leftD apply fastforce
  apply (smt append-is-Nil-conv list.collapse list.discI old.prod.inject set-ConsD
    set-zip-leftD zip-Cons-Cons)
  done

```

```

lemma zip-butlast-tl-app1:
  assumes distinct xs
    and (x1, x2)  $\in$  set (zip (butlast xs) (tl xs))
  shows  $\exists$  xs1 xs2. xs = xs1 @ x1 # x2 # xs2
  using assms
proof (induction xs)
  case Nil
  then show ?case
    by simp
next
  case (Cons x xs)

```

```

have  $x_1 \in \text{set } (x \# xs)$ 
  by (meson Cons.prem s in-set-butlastD set-zip-leftD)
then obtain  $xs_1 \ xs_2$  where  $x \# xs = xs_1 @ x_1 \# xs_2$ 
  by (meson split-list)
then have  $\text{distinct } (xs_1 @ x_1 \# xs_2)$ 
  using Cons.prem s by auto
then have  $\text{distinct } (x_1 \# xs_2)$ 
  using Cons.prem s by auto
have  $(x_1, x_2) \in \text{set } (\text{zip } (\text{butlast } (xs_1 @ x_1 \# xs_2)) (\text{tl } (xs_1 @ x_1 \# xs_2)))$ 
  using  $\langle x \# xs = xs_1 @ x_1 \# xs_2 \rangle$  Cons.prem s by auto
then have  $(x_1, x_2) \in \text{set } (\text{zip } (\text{butlast } (x_1 \# xs_2)) (\text{tl } (x_1 \# xs_2)))$ 
  using zip-butlast-sublist[OF  $\langle \text{distinct } (xs_1 @ (x_1 \# xs_2)) \rangle$ ] by auto
obtain  $xs_2'$  where  $xs_2 = x_2 \# xs_2'$ 
  using zip-butlast-tl-hd[OF  $\langle \text{distinct } (x_1 \# xs_2) \rangle$ 
     $\langle (x_1, x_2) \in \text{set } (\text{zip } (\text{butlast } (x_1 \# xs_2)) (\text{tl } (x_1 \# xs_2))) \rangle$ ]
    list.exhaust-sel by blast
then show ?case
  using  $\langle x \# xs = xs_1 @ x_1 \# xs_2 \rangle$ 
  by auto
qed

```

lemma zip-butlast-tl-app2:

$(x_1, x_2) \in \text{set } (\text{zip } (\text{butlast } (xs_1 @ x_1 \# x_2 \# xs_2)) (\text{tl } (xs_1 @ x_1 \# x_2 \# xs_2)))$

proof –

```

let ?xs= $xs_1 @ x_1 \# x_2 \# xs_2$ 
let ?butlast= $\text{butlast } ?xs$ 
let ?tl= $\text{tl } ?xs$ 
have  $x_1 = ?xs ! (\text{length } xs_1)$  and  $(\text{length } xs_1) < \text{length } ?butlast$ 
  using nth-append-length by auto
then have  $x_1 = ?butlast ! (\text{length } xs_1)$ 
  using nth-butlast by fastforce
have  $x_2 = ?xs ! (\text{length } xs_1 + 1)$  and  $?xs \neq []$ 
  using nth-append-length[where  $xs=xs_1 @ [x_1]$ ] by auto
then have  $x_2 = ?tl ! (\text{length } xs_1)$ 
  using nth-tl[OF  $\langle ?xs \neq [] \rangle$ ] by auto
have  $(\text{length } xs_1) < \text{length } ?butlast$  and  $(\text{length } xs_1) < \text{length } ?tl$ 
  by auto
then show ?thesis
  using  $\langle x_1 = ?butlast ! (\text{length } xs_1) \rangle \langle x_2 = ?tl ! (\text{length } xs_1) \rangle$  in-set-zip
  by (metis fst-conv snd-conv)

```

qed

lemma zip-butlast-tl-app:

assumes $\text{distinct } xs$

shows $(x_1, x_2) \in \text{set } (\text{zip } (\text{butlast } xs) (\text{tl } xs)) \longleftrightarrow (\exists xs_1 \ xs_2. xs = xs_1 @ x_1 \# x_2 \# xs_2)$

using zip-butlast-tl-app1[OF $\langle \text{distinct } xs \rangle$] zip-butlast-tl-app2 by fastforce

lemma sorted-distinct-zip-le:

```

    assumes strict-sorted xs
      and  $(x_1, x_2) \in \text{set } (\text{zip } (\text{butlast } xs) (\text{tl } xs))$ 
    shows  $x_1 < x_2$ 
    using assms
  proof (induction xs)
    case Nil
    then show ?case
      by simp
  next
    case (Cons x xs)
    obtain  $xs_1\ xs_2$  where  $x \# xs = xs_1 @ (x_1 \# x_2 \# xs_2)$ 
      by (meson Cons.prem1 zip-butlast-tl-app strict-sorted-iff)
    then show ?case
      using Cons.prem1 by (auto simp: sorted-append strict-sorted-iff)
  qed

```

```

lemma strict-sorted-zip-butlast-tl-leq-or-geq:
  assumes strict-sorted xs
    and  $(x_1, x_2) \in \text{set } (\text{zip } (\text{butlast } xs) (\text{tl } xs))$ 
    and  $x \in \text{set } xs$ 
  shows  $x \leq x_1 \vee x_2 \leq x$ 
  proof -
    have distinct xs
      using  $\langle \text{strict-sorted } xs \rangle$  by (auto simp: strict-sorted-iff)
    then obtain  $xs_1\ xs_2$  where  $xs = xs_1 @ x_1 \# x_2 \# xs_2$ 
      using assms zip-butlast-tl-app[OF  $\langle \text{distinct } xs \rangle$ ] by auto
    then have  $x \in \text{set } xs_1 \vee x = x_1 \vee x = x_2 \vee x \in \text{set } xs_2$ 
      using assms by auto
    then show ?thesis
  proof (elim disjE)
    assume  $x \in \text{set } xs_1$ 
    then show ?thesis
      using assms  $\langle xs = xs_1 @ x_1 \# x_2 \# xs_2 \rangle$ 
      by (auto simp: sorted-wrt-append)
  next
    assume  $x = x_1$ 
    then show ?thesis
      by auto
  next
    assume  $x = x_2$ 
    then show ?thesis
      by auto
  next
    assume  $x \in \text{set } xs_2$ 
    then show ?thesis
      using assms  $\langle xs = xs_1 @ x_1 \# x_2 \# xs_2 \rangle$ 
      by (auto simp: sorted-wrt-append)
  qed
qed

```

```

lemma consec-https-in-zip-butlast-tl:
  assumes  $https = https\_exec\ \pi s$ 
    and  $consec\_https\ \pi s\ t_i\ t_j$ 
    shows  $(t_i, t_j) \in set\ (zip\ (butlast\ https)\ (tl\ https))$ 
proof –
  have strict-sorted https
    using assms https-exec-sorted https-exec-distinct by (simp add: strict-sorted-iff)
  have  $\forall t \in set\ https. t \leq t_i \vee t_j \leq t$  and  $t_i \in set\ https$  and  $t_j \in set\ https$  and
 $t_i < t_j$ 
    using assms is-http-iff-https-exec unfolding consec-https-def by auto
  then obtain  $https_1\ https_2'$  where  $https = https_1 @ t_i \# https_2'$ 
    by (meson split-list)
  then have  $t_j \in set\ https_2'$ 
    using  $\langle strict\_sorted\ https \rangle\ \langle t_j \in set\ https \rangle\ \langle t_i < t_j \rangle$ 
    by (auto simp: sorted-wrt-append)
  then obtain  $https_2\ https_3$  where  $https_2' = https_2 @ t_j \# https_3$ 
    by (meson split-list)
  then have  $\forall t \in set\ https_2. t \leq t_i \vee t_j \leq t$ 
    using  $\langle \forall t \in set\ https. t \leq t_i \vee t_j \leq t \rangle\ \langle https = https_1 @ t_i \# https_2' \rangle$  by auto
  then have  $https_2 = []$ 
    using  $\langle strict\_sorted\ https \rangle\ \langle https = https_1 @ t_i \# https_2' \rangle\ \langle https_2' = https_2 @ t_j \# https_3 \rangle$ 
 $\# https_3$ 
    by (induction https_2) (auto simp: sorted-wrt-append)
  then show ?thesis
    using  $\langle https = https_1 @ t_i \# https_2' \rangle\ \langle https_2' = https_2 @ t_j \# https_3 \rangle$ 
    by (auto simp: zip-butlast-tl-app2)
qed

```

Executable refinement for *consec-https*.

```

lemma happ-time-pts-exec-iff-consec-https:
   $(t_i, t_j) \in set\ (zip\ (butlast\ (https\_exec\ \pi s))\ (tl\ (https\_exec\ \pi s))) \longleftrightarrow consec\_https\ \pi s\ t_i\ t_j$ 
proof –
  have strict-sorted (https-exec  $\pi s$ )
    using https-exec-sorted https-exec-distinct by (simp add: strict-sorted-iff)
  then show ?thesis
    using is-http-iff-https-exec sorted-distinct-zip-le
      strict-sorted-zip-butlast-tl-leq-or-geq consec-https-in-zip-butlast-tl
    by (metis consec-https-def in-set-butlastD in-set-tlD in-set-zipE)
qed

```

Justification for *consec-https-in-interval*.

```

lemma consec-https-in-interval-iff:
  assumes  $t_i \leq t_j$ 
  shows  $\forall t\ t'. (consec\_https\ \pi s\ t\ t' \wedge t_i \leq t \wedge t' \leq t_j) \longleftrightarrow (t, t') \in set\ (consec\_https\_in\_interval\ (https\_exec\ \pi s)\ t_i\ t_j)$ 
  unfolding consec-https-in-interval-def
  using assms happ-time-pts-exec-iff-consec-https is-https-https-exec

```

by (auto simp: consec-https-def)

All returned actions from *simplify-action* are correct instantiations of a *plan-action* for an induced simple plan.

```

lemma (in wf-ast-problem) simp-act-inst-of-plan-act:
  assumes wf-plan  $\pi s$ 
    and  $(t_\pi, \pi) \in \text{set } \pi s$ 
    and  $(t_a, a) \in \text{set } (\text{simplify-action } (\text{https-exec } \pi s) (t_\pi, \pi))$ 
  shows inst-of-plan-action  $\pi s (t_\pi, \pi) (t_a, a)$ 
  using assms
proof (cases  $\pi$ )
  case (Simple-Plan-Action n args)
  then show ?thesis
    using  $\langle (t_a, a) \in \text{set } (\text{simplify-action } (\text{https-exec } \pi s) (t_\pi, \pi)) \rangle$  by auto
next
  case (Durative-Plan-Action n args d)
  let ?https = https-exec  $\pi s$ 
  let ?a_schema = the (resolve-action-schema n)
  let ?a_start = inst-snap-action ?a_schema args At-Start
  let ?a_end = inst-snap-action ?a_schema args At-End
  let ?a_inv = inst-snap-action ?a_schema args Over-All
  let ?t_inv =  $\exists t_i t_j. (t_i, t_j) \in \text{set } (\text{consec-https-in-interval } ?https t_\pi (t_\pi + d)) \wedge t_a$ 
  =  $(t_i + t_j) / 2$ 
  have  $(a = ?a_{start} \wedge t_\pi = t_a) \vee (a = ?a_{end} \wedge t_a = t_\pi + d) \vee (a = ?a_{inv} \wedge$ 
  ?t_inv)
  using  $\langle (t_a, a) \in \text{set } (\text{simplify-action } ?https (t_\pi, \pi)) \rangle$  Durative-Plan-Action
  by (auto simp: Let-def) blast+
  then show ?thesis
proof (elim disjE)
  assume  $a = ?a_{start} \wedge t_\pi = t_a$ 
  then show ?thesis
    using Durative-Plan-Action by auto
next
  assume  $a = ?a_{end} \wedge t_a = t_\pi + d$ 
  then show ?thesis
    using Durative-Plan-Action by auto
next
  assume  $a = ?a_{inv} \wedge ?t_{inv}$ 
  then have res-inst-snap-action  $\pi$  Over-All = Some a
    using Durative-Plan-Action by auto
  obtain  $t_i t_j$  where  $(t_i, t_j) \in \text{set } (\text{consec-https-in-interval } ?https t_\pi (t_\pi + d))$ 
    and  $t_a = (t_i + t_j) / 2$ 
  using  $\langle a = ?a_{inv} \wedge ?t_{inv} \rangle$  by auto
  have  $t_\pi \leq t_\pi + \text{duration } \pi$ 
  using assms Durative-Plan-Action unfolding wf-plan-def
  by (auto simp: Let-def split: option.splits ast-action-schema.splits)
  then have consec-https  $\pi s t_i t_j \wedge t_\pi \leq t_i \wedge t_j \leq t_\pi + d$ 
  using Durative-Plan-Action  $\langle (t_i, t_j) \in \text{set } (\text{consec-https-in-interval } ?https t_\pi$ 
   $(t_\pi + d)) \rangle$ 

```



```

      consec-https-in-interval-iff
    by (auto simp: consec-https-def)
  then have  $t_i < t_a \wedge t_a < t_j$ 
    using  $\langle t_a = (t_i + t_j) / 2 \rangle$ 
    by (auto simp: consec-https-def)
  then show ?thesis
    using Durative-Plan-Action  $\langle \text{res-inst-snap-action } \pi \text{ Over-All} = \text{Some } a \rangle$ 
       $\langle \text{consec-https } \pi s \ t_i \ t_j \wedge t_\pi \leq t_i \wedge t_j \leq t_\pi + d \rangle$ 
    by auto
qed
qed

```

Function to insert a happening into a happening sequence.

```

fun insort-happ :: happening  $\Rightarrow$  happening list  $\Rightarrow$  happening list where
  insort-happ  $(t_i, A_i)$  [] = [( $t_i, A_i$ )]
| insort-happ  $(t_i, A_i)$  (( $t_j, A_j$ )#hs) =
  (if  $t_i < t_j$  then ( $t_i, A_i$ )#( $t_j, A_j$ )#hs
   else if  $t_i = t_j$  then ( $t_j, A_i$  @  $A_j$ )#hs
   else ( $t_j, A_j$ )#(insort-happ  $(t_i, A_i)$  hs))

```

Properties of *insort-happ*

lemma *insort-happ-insort-insert*:

```

  sorted (map fst hs)  $\implies$  insort-insert (fst h) (map fst hs) = map fst (insort-happ h hs)
  by (induction h hs rule: insort-happ.induct) (auto simp: insort-insert-key-def split: if-splits)

```

insort-happ preserves sorted-ness.

lemma *insort-happ-distinct*: $\text{strict-sorted } (\text{map fst } hs) \implies \text{distinct } (\text{map fst } (\text{insort-happ } h \text{ } hs))$

```

  using strict-sorted-iff
  by (metis insort-happ-insort-insert distinct-insort-insert)

```

lemma *insort-happ-not-Nil*:

```

  assumes  $A_i \neq []$  and  $\forall (t_j, A_j) \in \text{set } hs. A_j \neq []$ 
  shows  $\forall (t_j, A_j) \in \text{set } (\text{insort-happ } (t_i, A_i) \text{ } hs). A_j \neq []$ 
  using assms by (induction  $(t_i, A_i)$  hs rule: insort-happ.induct) auto

```

lemma *insort-happ-action-subset*:

```

 $\forall (t_a, A) \in \text{set } hs \cup \{h\}. \exists A'. (t_a, A') \in \text{set } (\text{insort-happ } h \text{ } hs) \wedge \text{set } A \subseteq \text{set } A'$ 
  by (induction h hs rule: insort-happ.induct) auto+

```

lemma *insort-happ2-action-subset*:

```

 $\exists A'. (t_2, A') \in \text{set } (\text{insort-happ } (t_1, A_1) (\text{insort-happ } (t_2, A_2) \text{ } hs)) \wedge \text{set } A_2 \subseteq \text{set } A'$ 
  using insort-happ-action-subset
  by (induction  $(t_2, A_2)$  hs rule: insort-happ.induct) auto

```

lemma *insort-happ-sorted*: $\text{sorted } (\text{map fst } hs) \implies \text{sorted } (\text{map fst } (\text{insort-happ } h \text{ } hs))$

by (*metis insort-happ-insort-insert sorted-insort-insert*)

lemma *insort-happ-strict-sorted*:

$\text{strict-sorted } (\text{map fst } hs) \implies \text{strict-sorted } (\text{map fst } (\text{insort-happ } h \text{ } hs))$

using *insort-happ-sorted insort-happ-distinct*

by (*simp add: strict-sorted-iff*)

Function to insert multiple happening into a happening sequence.

fun *insort-mult-happs* :: $\text{happening list} \Rightarrow \text{happening list} \Rightarrow \text{happening list}$ **where**

insort-mult-happs [] $hs_2 = hs_2$

| *insort-mult-happs* ($h \# hs_1$) $hs_2 = \text{insort-happ } h \text{ } (\text{insort-mult-happs } hs_1 \text{ } hs_2)$

fun *insort-timed-ground-acts* :: $(\text{time} \times \text{ground-action}) \text{ list} \Rightarrow \text{happening list} \Rightarrow \text{happening list}$ **where**

insort-timed-ground-acts [] $hs = hs$

| *insort-timed-ground-acts* ($(t_a, a) \# as$) $hs =$

insort-happ ($t_a, [a]$) (*insort-timed-ground-acts* $as \text{ } hs$)

lemma *insort-timed-ground-acts-correct*:

insort-mult-happs ($\text{map } (\lambda(t, a). (t, [a])) \text{ } as$) $hs = \text{insort-timed-ground-acts } as \text{ } hs$

by (*induction as hs rule: insort-timed-ground-acts.induct*) *auto*

Properties of *insort-mult-happs*

lemma *insort-mult-happs-strict-sorted*:

$\text{strict-sorted } (\text{map fst } hs_2) \implies \text{strict-sorted } (\text{map fst } (\text{insort-mult-happs } hs_1 \text{ } hs_2))$

by (*induction hs₁ arbitrary: hs₂*) (*auto simp: insort-happ-strict-sorted*)

lemma *insort-mult-happs-not-Nil*:

assumes $\forall (t_a, A) \in \text{set } xs. A \neq []$ **and** $\forall (t_a, A) \in \text{set } ys. A \neq []$

shows $\forall (t_a, A) \in \text{set } (\text{insort-mult-happs } xs \text{ } ys). A \neq []$

using *assms*

proof (*induction xs ys rule: insort-mult-happs.induct*)

case (*1 ys*)

then show *?case* **by** *simp*

next

case (*2 x xs ys*)

then show *?case*

proof (*cases x*)

case (*Pair t_a A*)

then have $\forall (t_a, A) \in \text{set } xs. A \neq []$ **and** $A \neq []$

using $\langle \forall (t_a, A) \in \text{set } (x \# xs). A \neq [] \rangle$

by *auto*

then show *?thesis*

using *Pair 2.IH[OF $\langle \forall (t_a, A) \in \text{set } xs. A \neq [] \rangle \langle \forall (t_a, A) \in \text{set } ys. A \neq [] \rangle$]*

insort-happ-not-Nil[OF $\langle A \neq [] \rangle$] **by** *auto*

qed

qed

lemma *insort-mult-happs-action-subset1:*

$(t_a, A) \in \text{set } hs_1 \implies \exists as'. (t_a, as') \in \text{set } (\text{insort-mult-happs } hs_1 \text{ } hs_2) \wedge \text{set } A \subseteq \text{set } as'$

proof (*induction* hs_1)

case *Nil*

then show *?case*

by *auto*

next

case (*Cons* h hs_1)

have $(t_a, A) = h \vee (t_a, A) \neq h$

by *simp*

then show *?case*

proof

assume $(t_a, A) = h$

then show *?case*

using *insort-happ-action-subset* **by** *auto*

next

assume $(t_a, A) \neq h$

then have $(t_a, A) \in \text{set } hs_1$

using *Cons.prem*s **by** *auto*

then obtain A' **where** $(t_a, A') \in \text{set } (\text{insort-mult-happs } hs_1 \text{ } hs_2)$ **and** $\text{set } A \subseteq \text{set } A'$

using *Cons.IH* **by** *auto*

then obtain A'' **where** $(t_a, A'') \in \text{set } (\text{insort-happ } h \text{ } (\text{insort-mult-happs } hs_1 \text{ } hs_2))$

and $\text{set } A' \subseteq \text{set } A''$

using *insort-happ-action-subset* [**where** $h=h$ **and** $hs=\text{insort-mult-happs } hs_1$ hs_2] **by** *auto*

then show *?case*

using $\langle \text{set } A \subseteq \text{set } A' \rangle$ **by** *auto*

qed

qed

lemma *insort-mult-happs-action-subset2:*

assumes $(t_a, A) \in \text{set } hs_2$

shows $\exists A'. (t_a, A') \in \text{set } (\text{insort-mult-happs } hs_1 \text{ } hs_2) \wedge \text{set } A \subseteq \text{set } A'$

proof (*induction* hs_1)

case *Nil*

then show *?case*

using *assms* **by** *auto*

next

case (*Cons* h hs_1)

obtain A' **where** $(t_a, A') \in \text{set } (\text{insort-mult-happs } hs_1 \text{ } hs_2)$ **and** $\text{set } A \subseteq \text{set } A'$

using *assms* *Cons.IH* **by** *auto*

then obtain A'' **where** $(t_a, A'') \in \text{set } (\text{insort-happ } h \text{ } (\text{insort-mult-happs } hs_1 \text{ } hs_2))$

and $\text{set } A' \subseteq \text{set } A''$

```

    using insert-happ-action-subset[where  $h=h$  and  $hs=insert-mult-happs\ hs_1$ 
 $hs_2$ ] by auto
    then show ?case
    using  $\langle set\ A \subseteq set\ A' \rangle$  by auto
qed

lemma insert-mult-happs-action-subset:
 $\forall (t_a, A) \in set\ hs_1 \cup set\ hs_2. \exists A'. (t_a, A') \in set\ (insert-mult-happs\ hs_1\ hs_2) \wedge$ 
 $set\ A \subseteq set\ A'$ 
    using insert-mult-happs-action-subset1 insert-mult-happs-action-subset2 by
fastforce

lemma insert-happ2-insert-mult-happs-action-subset-aux:
    assumes  $(t_a, A) \in set\ hs_1$ 
    shows  $\exists A'. (t_a, A') \in set\ (insert-happ\ (t_1, A_1)\ (insert-happ\ (t_2, A_2)\ (insert-mult-happs\$ 
 $hs_1\ hs_2))) \wedge set\ A \subseteq set\ A'$ 
    proof -
    have  $\exists A'. (t_a, A') \in set\ (insert-mult-happs\ hs_1\ hs_2) \wedge set\ A \subseteq set\ A'$ 
    using assms insert-mult-happs-action-subset[where  $hs_1=hs_1$  and  $hs_2=hs_2$ ]
    by (auto split: prod.splits)
    then obtain  $A'$  where  $(t_a, A') \in set\ (insert-mult-happs\ hs_1\ hs_2)$  and  $set\ A \subseteq$ 
 $set\ A'$ 
    by auto
    then obtain  $A''$  where  $(t_a, A'') \in set\ (insert-happ\ (t_2, A_2)\ (insert-mult-happs\$ 
 $hs_1\ hs_2))$ 
    and  $set\ A' \subseteq set\ A''$ 
    using insert-happ-action-subset[where  $h=(t_2, A_2)$  and  $hs=insert-mult-happs\$ 
 $hs_1\ hs_2$ ] by auto
    then obtain  $A'''$  where  $(t_a, A''') \in set\ (insert-happ\ (t_1, A_1)\ (insert-happ\ (t_2, A_2)\$ 
 $(insert-mult-happs\ hs_1\ hs_2)))$ 
    and  $set\ A'' \subseteq set\ A'''$ 
    using insert-happ-action-subset[where  $h=(t_1, A_1)$ 
    and  $hs=insert-happ\ (t_2, A_2)\ (insert-mult-happs\ hs_1$ 
 $hs_2)$ ]
    by auto
    show ?thesis
    using  $\langle (t_a, A) \in set\ hs_1 \rangle$ 
     $\langle (t_a, A''') \in set\ (insert-happ\ (t_1, A_1)\ (insert-happ\ (t_2, A_2)\ (insert-mult-happs\$ 
 $hs_1\ hs_2))) \rangle$ 
     $\langle set\ A \subseteq set\ A' \rangle \langle set\ A' \subseteq set\ A'' \rangle \langle set\ A'' \subseteq set\ A''' \rangle$  by auto
qed

lemma insert-happ2-insert-mult-happs-action-subset:
 $\forall (t_a, A) \in set\ hs_1. \exists A'. (t_a, A') \in set\ (insert-happ\ (t_1, A_1)\ (insert-happ\ (t_2, A_2)\$ 
 $(insert-mult-happs\ hs_1\ hs_2))) \wedge set\ A \subseteq set\ A'$ 
    using insert-happ2-insert-mult-happs-action-subset-aux by auto

```

We can define a function to construct a simplified plan for a given temporal

plan.

```

fun simplify-plan :: time list  $\Rightarrow$  plan  $\Rightarrow$  happening list where
  simplify-plan https [] = []
| simplify-plan https ( $\pi \# \pi s$ ) =
  insert-mult-happs (map ( $\lambda(t_a, a). (t_a, [a])$ ) (simplify-action https  $\pi$ )) (simplify-plan
https  $\pi s$ )

```

Definition of subset on happening sequences. This is used to simplify some proofs.

```

definition happ-seq-member :: happening  $\Rightarrow$  happening list  $\Rightarrow$  bool (infix  $\in_{hs}$ 
53) where
  happ-seq-member h hs  $\longleftrightarrow$  (case h of ( $t_a, A$ )  $\Rightarrow \exists A_2. \text{set } A \subseteq \text{set } A_2 \wedge (t_a, A_2) \in \text{set } hs$ )

```

```

definition happ-seq-subset :: happening list  $\Rightarrow$  happening list  $\Rightarrow$  bool (infix  $\subseteq_{hs}$ 
53) where
  happ-seq-subset hs1 hs2  $\longleftrightarrow (\forall h_1 \in \text{set } hs_1. h_1 \in_{hs} hs_2)$ 

```

Properties of ($\subseteq_{hs}$)

```

lemma happ-seq-subset-Nil: []  $\subseteq_{hs}$  hs
unfolding happ-seq-subset-def by auto

```

```

lemma happ-seq-subset-refl: hs  $\subseteq_{hs}$  hs
unfolding happ-seq-subset-def happ-seq-member-def by auto

```

```

lemma happ-seq-subset-trans: hs1  $\subseteq_{hs}$  hs2  $\wedge$  hs2  $\subseteq_{hs}$  hs3  $\implies$  hs1  $\subseteq_{hs}$  hs3
unfolding happ-seq-subset-def happ-seq-member-def
by (auto split: prod.splits) (metis (full-types) dual-order.trans)

```

```

lemma insert-happ-subset: hs  $\subseteq_{hs}$  insert-happ h hs
proof –

```

```

  have hs = []  $\vee$  hs  $\neq$  []
  by simp

```

```

  then show ?thesis

```

```

proof

```

```

  assume hs = []

```

```

  then show ?thesis

```

```

    unfolding happ-seq-subset-def by auto

```

```

next

```

```

  assume hs  $\neq$  []

```

```

  then show ?thesis

```

```

    unfolding happ-seq-subset-def happ-seq-member-def

```

```

    using insert-happ-action-subset by blast

```

```

  qed

```

```

qed

```

```

lemma insert-mult-happs-subset: hs1  $\subseteq_{hs}$  insert-mult-happs hs2 hs1
proof (induction hs2 hs1 rule: insert-mult-happs.induct)

```

```

    case (1 hs)
    then show ?case
      using happ-seq-subset-refl by simp
  next
    case (2 a A hs)
    have insert-mult-happs (a # A) hs = insert-happ a (insert-mult-happs A hs)
      by simp
    then have insert-mult-happs A hs  $\subseteq_{hs}$  insert-mult-happs (a # A) hs
      using insert-happ-subset by auto
    then show ?case
      using 2.IH happ-seq-subset-trans by blast
  qed

```

Happening sequence subset for simplify plan.

```

lemma simplify-plan-happs-subset-Cons: simplify-plan https  $\pi s \subseteq_{hs}$  simplify-plan
https ( $\pi \# \pi s$ )
  using insert-mult-happs-subset happ-seq-subset-refl
  by (induction  $\pi s$  arbitrary:  $\pi$ ) auto

```

```

lemma simplify-plan-happs-subset-app: simplify-plan https  $\pi s_2 \subseteq_{hs}$  simplify-plan
https ( $\pi s_1 @ \pi s_2$ )
  using simplify-plan-happs-subset-Cons
  apply (induction  $\pi s_1$  arbitrary:  $\pi s_2$ )
  apply (auto simp: happ-seq-subset-refl)
  using happ-seq-subset-trans apply blast
done

```

Properties of *simplify-plan*

All happenings in a simplified plan are always non-*Nil*.

```

lemma simp-plan-happ-not-Nil:
  fixes  $\pi s$ 
  defines https  $\equiv$  https-exec  $\pi s$ 
  shows  $\forall (t_a, A) \in \text{set } (\text{simplify-plan https } \pi s). A \neq []$ 
proof (induction  $\pi s$ )
  case Nil
  then show ?case by simp
next
  case (Cons  $\pi \pi s$ )
  have  $\forall (t_a, A) \in \text{set } (\text{map } (\lambda(t_a, a). (t_a, [a])) (\text{simplify-action https } \pi)). A \neq []$ 
    by auto
  then show ?case
    using Cons.IH insert-mult-happs-not-Nil simplify-plan.simps(2)
    by presburger
qed

```

A simplified plan produced by *simplify-plan* is strictly sorted.

```

lemma simp-plan-strict-sorted: strict-sorted (map fst (simplify-plan https  $\pi s$ ))
  by (induction  $\pi s$ ) (auto simp: insert-mult-happs-strict-sorted)

```

A simplified plan contains all grounded actions for every *Simple-Plan-Action*.

lemma *simpl-plan-corr-simp-acts*:

assumes $hs = \text{simplify-plan } hps \ \pi s$

and $(t_\pi, \pi) \in \text{simple-acts } \pi s$

shows $\exists A. (t_\pi, A) \in \text{set } hs \wedge \text{the } (\text{res-inst } \pi) \in \text{set } A$

using *assms*

proof –

obtain $\pi s_1 \ \pi s_2$ **where** $\pi s = \pi s_1 @ ((t_\pi, \pi) \# \pi s_2)$

using *assms simple-acts-subset* **by** (*meson split-list subsetD*)

then obtain hs' **where** $hs' = \text{simplify-plan } hps ((t_\pi, \pi) \# \pi s_2)$

by *simp*

have $(t_\pi, \pi) = \text{hd } ((t_\pi, \pi) \# \pi s_2)$

by *auto*

have $(t_\pi, \pi) \in \text{simple-acts } ((t_\pi, \pi) \# \pi s_2)$

using *assms* $\langle \pi s = \pi s_1 @ ((t_\pi, \pi) \# \pi s_2) \rangle$

unfolding *simple-acts-def is-act-simple-def* **by** (*auto split: plan-action.splits*)

have $\exists as. (t_\pi, as) \in \text{set } hs' \wedge \text{the } (\text{res-inst } \pi) \in \text{set } as$

using $\langle hs' = \text{simplify-plan } hps ((t_\pi, \pi) \# \pi s_2) \rangle$

$\langle (t_\pi, \pi) = \text{hd } ((t_\pi, \pi) \# \pi s_2) \rangle$

$\langle (t_\pi, \pi) \in \text{simple-acts } ((t_\pi, \pi) \# \pi s_2) \rangle$

insort-happ-action-subset

unfolding *simple-acts-def is-act-simple-def*

apply (*induction* πs)

apply (*auto split: plan-action.splits*)

apply (*meson list.set-intros(1) subsetD*)

done

have $hs' \subseteq_{hs} hs$

using *assms simplify-plan-happs-subset-app*

$\langle hs' = \text{simplify-plan } hps ((t_\pi, \pi) \# \pi s_2) \rangle$

$\langle \pi s = \pi s_1 @ ((t_\pi, \pi) \# \pi s_2) \rangle$

by *blast*

show *?thesis*

using $\langle \exists as. (t_\pi, as) \in \text{set } hs' \wedge \text{the } (\text{res-inst } \pi) \in \text{set } as \rangle$

$\langle hs' \subseteq_{hs} hs \rangle$

unfolding *happ-seq-subset-def happ-seq-member-def* **by** *blast*

qed

lemma *simpl-plan-corr-dur-acts-start-aux*:

assumes $hs = \text{simplify-plan } hps \ \pi s$

and $(t_\pi, \pi) = \text{hd } \pi s$

and $(t_\pi, \pi) \in \text{durative-acts } \pi s$

shows $(\exists A. (t_\pi, A) \in \text{set } hs \wedge (\text{the } (\text{res-inst-snap-action } \pi \text{ At-Start})) \in \text{set } A)$

using *assms insort-happ-action-subset*

unfolding *durative-acts-def is-act-simple-def*

apply (*induction* πs)

apply (*auto simp: Let-def split: plan-action.splits*)

apply (*meson list.set-intros(1) subsetD*)

done

A simplified plan contains all *At-Start*-snap-actions.

```

lemma simp-plan-corr-dur-acts-start:
  fixes  $\pi$ 
  defines  $a_{start} \equiv \text{the } (res-inst-snap-action \ \pi \ At-Start)$ 
  assumes  $hs = \text{simplify-plan } hps \ \pi s$ 
    and  $(t_\pi, \pi) \in \text{durative-acts } \pi s$ 
  shows  $\exists A. (t_\pi, A) \in \text{set } hs \wedge a_{start} \in \text{set } A$ 
  using assms
proof –
  obtain  $\pi s_1 \ \pi s_2$  where  $\pi s = \pi s_1 @ ((t_\pi, \pi) \# \pi s_2)$ 
    using assms dur-acts-subset by (meson split-list subsetD)
  then obtain  $hs'$  where  $hs' = \text{simplify-plan } hps \ ((t_\pi, \pi) \# \pi s_2)$ 
    by simp
  have  $(t_\pi, \pi) = hd \ ((t_\pi, \pi) \# \pi s_2)$ 
    by auto
  have  $(t_\pi, \pi) \in \text{durative-acts } ((t_\pi, \pi) \# \pi s_2)$ 
    using assms  $\langle \pi s = \pi s_1 @ ((t_\pi, \pi) \# \pi s_2) \rangle$ 
  unfolding durative-acts-def is-act-simple-def by (auto split: plan-action.splits)
  have  $\exists as. (t_\pi, as) \in \text{set } hs' \wedge a_{start} \in \text{set } as$ 
    using assms simp-plan-corr-dur-acts-start-aux  $[OF \ \langle hs' = \text{simplify-plan } hps \ ((t_\pi, \pi) \# \pi s_2) \rangle$ 
       $\langle (t_\pi, \pi) = hd \ ((t_\pi, \pi) \# \pi s_2) \rangle$ 
       $\langle (t_\pi, \pi) \in \text{durative-acts } ((t_\pi, \pi) \# \pi s_2) \rangle]$ 
    by simp
  have  $hs' \subseteq_{hs} hs$ 
    using assms simplify-plan-happs-subset-app
       $\langle hs' = \text{simplify-plan } hps \ ((t_\pi, \pi) \# \pi s_2) \rangle$ 
       $\langle \pi s = \pi s_1 @ ((t_\pi, \pi) \# \pi s_2) \rangle$ 
    by blast
  show ?thesis
    using  $\langle \exists as. (t_\pi, as) \in \text{set } hs' \wedge a_{start} \in \text{set } as \rangle$ 
       $\langle hs' \subseteq_{hs} hs \rangle$ 
    unfolding happ-seq-subset-def happ-seq-member-def by blast
qed

```

```

lemma simp-plan-corr-dur-acts-end-aux:
  assumes  $hs = \text{simplify-plan } hps \ \pi s$ 
    and  $(t_\pi, \pi) = hd \ \pi s$ 
    and  $(t_\pi, \pi) \in \text{durative-acts } \pi s$ 
  shows  $(\exists A. (t_\pi + (\text{duration } \pi), A) \in \text{set } hs \wedge (\text{the } (res-inst-snap-action \ \pi \ At-End)) \in \text{set } A)$ 
  using assms insort-happ2-action-subset
  unfolding durative-acts-def is-act-simple-def
  apply (induction \pi s)
  apply (auto simp: Let-def split: plan-action.splits)
  apply (meson list.set-intros(1) subsetD)
done

```

A simplified plan contains all *At-End*-snap-actions.

lemma *simp-plan-corr-dur-acts-end*:
fixes π
defines $a_{end} \equiv \text{the } (\text{res-inst-snap-action } \pi \text{ At-End})$
assumes $hs = \text{simplify-plan } htps \ \pi s$
and $(t_\pi, \pi) \in \text{durative-acts } \pi s$
shows $\exists A. (t_\pi + (\text{duration } \pi), A) \in \text{set } hs \wedge a_{end} \in \text{set } A$
using *assms*
proof –
obtain $\pi s_1 \ \pi s_2$ **where** $\pi s = \pi s_1 @ ((t_\pi, \pi) \# \pi s_2)$
using *assms dur-acts-subset* **by** (*meson split-list subsetD*)
then obtain hs' **where** $hs' = \text{simplify-plan } htps \ ((t_\pi, \pi) \# \pi s_2)$
by *simp*
have $(t_\pi, \pi) = \text{hd } ((t_\pi, \pi) \# \pi s_2)$
by *auto*
have $(t_\pi, \pi) \in \text{durative-acts } ((t_\pi, \pi) \# \pi s_2)$
using *assms* $\langle \pi s = \pi s_1 @ ((t_\pi, \pi) \# \pi s_2) \rangle$
unfolding *durative-acts-def is-act-simple-def* **by** (*auto split: plan-action.splits*)
have $\exists as. (t_\pi + (\text{duration } \pi), as) \in \text{set } hs' \wedge a_{end} \in \text{set } as$
using *assms simp-plan-corr-dur-acts-end-aux* [*OF* $\langle hs' = \text{simplify-plan } htps \ ((t_\pi, \pi) \# \pi s_2) \rangle$
 $\langle (t_\pi, \pi) = \text{hd } ((t_\pi, \pi) \# \pi s_2) \rangle$
 $\langle (t_\pi, \pi) \in \text{durative-acts } ((t_\pi, \pi) \# \pi s_2) \rangle$]
by *simp*
have $hs' \subseteq_{hs} hs$
using *assms simplify-plan-happs-subset-app*
 $\langle hs' = \text{simplify-plan } htps \ ((t_\pi, \pi) \# \pi s_2) \rangle$
 $\langle \pi s = \pi s_1 @ ((t_\pi, \pi) \# \pi s_2) \rangle$
by *blast*
show *?thesis*
using $\langle \exists as. (t_\pi + (\text{duration } \pi), as) \in \text{set } hs' \wedge a_{end} \in \text{set } as \rangle$
 $\langle hs' \subseteq_{hs} hs \rangle$
unfolding *happ-seq-subset-def happ-seq-member-def* **by** *blast*
qed

lemma *member-map*: $a \in \text{set } A \implies f a \in \text{set } (\text{map } f A)$

by *simp*

lemma *simp-plan-corr-dur-acts-overall-aux*:

assumes *strict-sorted htps*

and $hs = \text{simplify-plan } htps \ \pi s$

and $(t_\pi, \pi) = \text{hd } \pi s$

and $(t_\pi, \pi) \in \text{durative-acts } \pi s$

shows $\forall t_i \ t_j. (t_i, t_j) \in \text{set } (\text{consec-htps-in-interval } htps \ t_\pi \ (t_\pi + \text{duration } \pi))$

→

$(\exists (t', A) \in \text{set } hs. t_i < t' \wedge t' < t_j \wedge (\text{the } (\text{res-inst-snap-action } \pi \text{ Over-All})) \in \text{set } A)$

using *assms*

unfolding *durative-acts-def is-act-simple-def*

```

proof (induction  $\pi s$ )
  case Nil
  then show ?case
    by simp
next
  case (Cons  $\pi$   $\pi s$ )
  fix  $t_i$   $t_j$ 
  obtain  $t_\pi$   $\pi'$  where  $\pi = (t_\pi, \pi')$ 
    by (meson surj-pair)
  then show ?case
  proof (cases  $\pi'$ )
    case (Simple-Plan-Action  $n$  args)
    then show ?thesis
      using Cons.prems  $\langle \pi = (t_\pi, \pi') \rangle$  by simp
    next
      case (Durative-Plan-Action  $n$  args  $d$ )
      let ?to-happ =  $\lambda(t_a, a). (t_a, [a])$ 
      let ?snap-act = inst-snap-action (the (resolve-action-schema  $n$ )) args
      let ?chtps-in-intv = consec-https-in-interval https  $t_\pi$  ( $t_\pi + \text{duration } \pi'$ )
      let ?hs1 = map ( $\lambda(t_i, t_j). ((t_i + t_j) / 2, ?\text{snap-act Over-All})$ ) ?chtps-in-intv
      have 1:  $\forall (t_a, A) \in \text{set } (\text{map } ?\text{to-happ } ?\text{hs}_1). \exists A'. (t_a, A') \in \text{set } (\text{simplify-plan}$ 
https ( $\pi \# \pi s$ ))  $\wedge \text{set } A \subseteq \text{set } A'$ 
      using  $\langle \pi = (t_\pi, \pi') \rangle$  Durative-Plan-Action insort-happ2-insort-mult-happs-action-subset
      by (auto simp: Let-def)
      have  $\forall (t_i, t_j) \in \text{set } ?\text{chtps-in-intv}. ((t_i + t_j) / 2, ?\text{snap-act Over-All}) \in \text{set}$ 
?hs1
      by auto
      then have  $\forall (t_i, t_j) \in \text{set } ?\text{chtps-in-intv}.$ 
         $?\text{to-happ } ((t_i + t_j) / 2, ?\text{snap-act Over-All}) \in \text{set } (\text{map } ?\text{to-happ}$ 
?hs1)
      by (smt member-map case-prodI2 case-prod-conv)
      then have  $\forall (t_i, t_j) \in \text{set } ?\text{chtps-in-intv}.$ 
         $(\exists A. ((t_i + t_j) / 2, A) \in \text{set } \text{hs} \wedge$ 
 $t_i < (t_i + t_j) / 2 \wedge (t_i + t_j) / 2 < t_j \wedge$ 
 $(\text{the } (\text{res-inst-snap-action } \pi' \text{ Over-All})) \in \text{set } A)$ 
      using 1 Durative-Plan-Action  $\langle \text{hs} = \text{simplify-plan } \text{https } (\pi \# \pi s) \rangle$ 
sorted-distinct-zip-le[OF  $\langle \text{strict-sorted } \text{https} \rangle$ 
by (auto simp: consec-https-in-interval-def)
      then have  $\forall t_i t_j. (t_i, t_j) \in \text{set } ?\text{chtps-in-intv} \longrightarrow$ 
 $(\exists (t', A) \in \text{set } \text{hs}. t_i < t' \wedge t' < t_j \wedge$ 
 $(\text{the } (\text{res-inst-snap-action } \pi' \text{ Over-All})) \in \text{set } A)$ 
      by fastforce
      then show ?thesis
        using Cons.prems  $\langle \pi = (t_\pi, \pi') \rangle$  Durative-Plan-Action by simp
  qed
qed

```

A simplified plan contains all *Over-All*-snap-actions.

lemma (*in wf-ast-problem*) *simp-plan-corr-dur-acts-overall*:

```

fixes  $\pi \ \pi s$ 
defines  $https \equiv https\text{-}exec \ \pi s$ 
and  $a_{inv} \equiv the \ (res\text{-}inst\text{-}snap\text{-}action \ \pi \ Over\text{-}All)$ 
assumes  $wf\text{-}plan \ \pi s$ 
and  $hs = simplify\text{-}plan \ https \ \pi s$ 
and  $(t_\pi, \pi) \in durative\text{-}acts \ \pi s$ 
shows  $\forall t_i \ t_j. (consec\text{-}https \ \pi s \ t_i \ t_j \wedge t_\pi \leq t_i \wedge t_j \leq t_\pi + duration \ \pi) \longrightarrow$ 
 $(\exists (t', A) \in set \ hs. t_i < t' \wedge t' < t_j \wedge a_{inv} \in set \ A)$ 

using  $assms$ 
proof –
have  $strict\text{-}sorted \ https$ 
using  $assms \ https\text{-}exec\text{-}sorted \ https\text{-}exec\text{-}distinct$  by  $(simp \ add: \ strict\text{-}sorted\text{-}iff)$ 
have  $t_\pi \leq t_\pi + duration \ \pi$ 
using  $assms \ unfolding \ wf\text{-}plan\text{-}def \ durative\text{-}acts\text{-}def \ is\text{-}act\text{-}simple\text{-}def$ 
by  $(auto \ simp: \ Let\text{-}def \ split: \ option.splits \ plan\text{-}action.splits \ ast\text{-}action\text{-}schema.splits)$ 
obtain  $\pi s_1 \ \pi s_2$  where  $\pi s = \pi s_1 \ @ \ ((t_\pi, \pi) \# \pi s_2)$ 
using  $assms \ dur\text{-}acts\text{-}subset$  by  $(meson \ split\text{-}list \ subsetD)$ 
then have  $(t_\pi, \pi) = hd \ ((t_\pi, \pi) \# \pi s_2)$  and  $(t_\pi, \pi) \in durative\text{-}acts \ ((t_\pi, \pi) \# \pi s_2)$ 
using  $assms \ unfolding \ durative\text{-}acts\text{-}def \ is\text{-}act\text{-}simple\text{-}def$  by  $(auto \ split: \ plan\text{-}action.splits)$ 
obtain  $hs'$  where  $hs' = simplify\text{-}plan \ https \ ((t_\pi, \pi) \# \pi s_2)$ 
by  $simp$ 
then have  $hs' \subseteq_{hs} hs$ 
using  $assms \ simplify\text{-}plan\text{-}happs\text{-}subset\text{-}app \ \langle \pi s = \pi s_1 \ @ \ ((t_\pi, \pi) \# \pi s_2) \rangle$  by
 $blast$ 
have  $\forall t_i \ t_j. (t_i, t_j) \in set \ (consec\text{-}https\text{-}in\text{-}interval \ https \ t_\pi \ (t_\pi + (duration \ \pi)))$ 
 $\longrightarrow (\exists (t', A) \in set \ hs'. t_i < t' \wedge t' < t_j \wedge a_{inv} \in set \ A)$ 
using  $assms \ simp\text{-}plan\text{-}corr\text{-}dur\text{-}acts\text{-}overall\text{-}aux[OF \ \langle strict\text{-}sorted \ https \rangle$ 
 $\langle hs' = simplify\text{-}plan \ https \ ((t_\pi, \pi) \# \pi s_2) \rangle$ 
 $\langle (t_\pi, \pi) = hd \ ((t_\pi, \pi) \# \pi s_2) \rangle$ 
 $\langle (t_\pi, \pi) \in durative\text{-}acts \ ((t_\pi, \pi) \# \pi s_2) \rangle]$ 
by  $auto$ 
then have  $\forall t_i \ t_j. (consec\text{-}https \ \pi s \ t_i \ t_j \wedge t_\pi \leq t_i \wedge t_j \leq t_\pi + (duration \ \pi))$ 
 $\longrightarrow (\exists (t', as) \in set \ hs'. t_i < t' \wedge t' < t_j \wedge a_{inv} \in set \ as)$ 
using  $https\text{-}def \ consec\text{-}https\text{-}in\text{-}interval\text{-}iff[OF \ \langle t_\pi \leq t_\pi + (duration \ \pi) \rangle]$  by
 $auto$ 
then show  $?thesis$ 
using  $\langle hs' \subseteq_{hs} hs \rangle$  unfolding  $happ\text{-}seq\text{-}subset\text{-}def \ happ\text{-}seq\text{-}member\text{-}def$  by
 $blast$ 
qed

```

Flatten a happening into individual timed grounded actions. This is used to simplify proofs.

```

fun  $set\text{-}happ\text{-}acts :: happening \ list \Rightarrow (time \times ground\text{-}action) \ set$  where
 $set\text{-}happ\text{-}acts \ [] = \{\}$ 
 $| set\text{-}happ\text{-}acts \ ((t_a, A) \# hs) = \{(t_a', a'). t_a = t_a' \wedge a' \in set \ A\} \cup set\text{-}happ\text{-}acts \ hs$ 

```

lemma *set-happ-acts-refine*:

$(\exists A. (t_a, A) \in \text{set } hs \wedge a \in \text{set } A) \longleftrightarrow ((t_a, a) \in \text{set-happ-acts } hs)$

by (*induction* *hs*) *auto*

lemma *set-happ-acts-insort-happ*:

$\text{set-happ-acts } (\text{insort-happ } h \text{ } hs) = \text{set-happ-acts } [h] \cup \text{set-happ-acts } hs$

by (*induction* *h* *hs* *rule*: *insort-happ.induct*) *auto*

lemma *set-happ-acts-insort-mult-happs*:

$\text{set-happ-acts } (\text{insort-mult-happs } hs_1 \text{ } hs_2) = \text{set-happ-acts } hs_1 \cup \text{set-happ-acts } hs_2$

*hs*₂

using *set-happ-acts-insort-happ*

by (*induction* *hs*₁) *auto*

lemma *set-happ-acts-to-happ*: $\text{set-happ-acts } (\text{map } (\lambda(t_a, a). (t_a, [a])) \text{ } xs) = \text{set } xs$

proof (*induction* *xs*)

case *Nil*

then show *?case*

by *auto*

next

case (*Cons* *x* *xs*)

then show *?case*

by (*cases* *x*) (*auto split*: *prod.splits*)

qed

lemma *simp-plan-corr-inst-plan-act-aux*:

assumes *strict-sorted* *htps*

and *hs* = *simplify-plan* *htps* *πs*

shows $\forall (t_a, a) \in \text{set-happ-acts } hs. \exists (t_\pi, \pi) \in \text{set } \pi s. (t_a, a) \in \text{set } (\text{simplify-action } htps (t_\pi, \pi))$

using *assms* *set-happ-acts-insort-mult-happs* *set-happ-acts-to-happ*

by (*induction* *πs* *arbitrary*: *hs*) *auto*

Every ground action in a simplified plan is the instantiation of a plan action.

lemma (*in* *wf-ast-problem*) *simp-plan-corr-inst-plan-act*:

fixes *πs*

defines *htps* $\equiv$ *htps-exec* *πs*

assumes *wf-plan* *πs*

and *hs* = *simplify-plan* *htps* *πs*

and $(t_a, A) \in \text{set } hs$

and $a \in \text{set } A$

shows $\exists (t_\pi, \pi) \in \text{set } \pi s. \text{inst-of-plan-action } \pi s (t_\pi, \pi) (t_a, a)$

proof –

have *strict-sorted* *htps*

using *assms* *htps-exec-sorted* *htps-exec-distinct* **by** (*simp* *add*: *strict-sorted-iff*)

have $\exists (t_\pi, \pi) \in \text{set } \pi s. (t_a, a) \in \text{set } (\text{simplify-action } htps (t_\pi, \pi))$

using *assms*

simp-plan-corr-inst-plan-act-aux[*OF* $\langle \text{strict-sorted } htps \rangle \langle hs = \text{simplify-plan } htps \pi s \rangle$]

```

      set-happ-acts-refine by blast
    then obtain  $t_\pi \pi$  where  $(t_\pi, \pi) \in \text{set } \pi s$  and  $(t_a, a) \in \text{set } (\text{simplify-action } https$ 
 $(t_\pi, \pi))$ 
      by auto
    show ?thesis
      using assms  $\langle (t_\pi, \pi) \in \text{set } \pi s \rangle \langle (t_a, a) \in \text{set } (\text{simplify-action } https (t_\pi, \pi)) \rangle$ 
      simp-act-inst-of-plan-act by blast
  qed

```

Finally the correctness proof for *simplify-plan*

```

lemma (in wf-ast-problem) simplify-plan-correct:
  fixes  $\pi s$ 
  defines  $https \equiv https-exec \pi s$ 
  assumes wf-plan  $\pi s$ 
  shows ind-happ-seq  $\pi s$  (simplify-plan  $https \pi s$ )
  unfolding ind-happ-seq-def
  using assms
    simp-plan-happ-not-Nil
    simp-plan-strict-sorted
    simp-plan-corr-simp-acts
    simp-plan-corr-dur-acts-start
    simp-plan-corr-dur-acts-end
    simp-plan-corr-dur-acts-overall
    simp-plan-corr-inst-plan-act
  by auto

```

```

lemma (in wf-ast-problem)
  assumes wf-plan  $\pi s$ 
  shows ind-happ-seq  $\pi s$  (simplify-plan (https-exec  $\pi s$ )  $\pi s$ )
  using assms by (rule simplify-plan-correct)

```

With the correctness proof of *simplify-plan*. We can easily proof that for every plan there exists an induced happening sequence.

```

lemma (in wf-ast-problem) ind-happ-seq-exists:
  assumes wf-plan  $\pi s$ 
  shows  $\exists hs. \text{ind-happ-seq } \pi s hs$ 
  using assms simplify-plan-correct by auto

```

```

lemma (in wf-ast-problem) simplify-plan-wf-ground-action-aux:
  fixes  $\pi s$ 
  defines  $https \equiv https-exec \pi s$ 
  assumes wf-plan  $\pi s$ 
    and  $hs = \text{simplify-plan } https \pi s$ 
    and  $(t_a, A) \in \text{set } hs$ 
    and  $a \in \text{set } A$ 
  shows wf-ground-action  $a$ 
proof -
  obtain  $t_\pi \pi$  where  $(t_\pi, \pi) \in \text{set } \pi s$  and inst-of-plan-action  $\pi s (t_\pi, \pi) (t_a, a)$ 

```

```

    using assms simp-plan-corr-inst-plan-act by blast
  then show ?thesis
    using assms wf-inst-of-plan-action by simp
qed

```

```

lemma (in wf-ast-problem) simplify-plan-wf-ground-action:
  fixes  $\pi s$ 
  defines  $https \equiv https-exec \pi s$ 
  assumes wf-plan  $\pi s$ 
    and  $hs = simplify-plan https \pi s$ 
  shows  $\forall (t_a, A) \in set\ hs. \forall a \in set\ A. wf-ground-action\ a$ 
  using assms simplify-plan-wf-ground-action-aux by (induction  $hs$ ) auto

```

Recursive refinement for *valid-happ-seq*.

```

fun valid-happ-seq-from :: world-model  $\Rightarrow$  happening list  $\Rightarrow$  bool where
  valid-happ-seq-from  $s \ [] \longleftrightarrow close-world\ s \models (goal\ P)$ 
| valid-happ-seq-from  $s\ (h\#\hs) \longleftrightarrow$ 
  (happ-enabled  $h\ s \wedge valid-happ-seq-from\ (apply-happ-exec\ h\ s)\ hs)$ 

```

Justification of the refinement for *valid-happ-seq*.

```

lemma valid-happ-seq-from-refine:
  assumes wf-happ-seq  $hs$ 
  shows valid-happ-seq-from  $M\ hs \longleftrightarrow (\exists M'. valid-happ-seq\ M\ hs\ M' \wedge M' \stackrel{c}{\models} (goal\ P))$ 
  using assms apply-happ-exec-refine
  by (induction  $hs$  arbitrary:  $M$ ) (auto split: prod.splits simp: pairwise-def)

```

```

definition mp-Fevl :: (object, rat) mapping where
  mp-Fevl = Mapping.of-alist (fold ( $\lambda a\ fev. case\ a\ of\ Atom\ (eqAtm\ f\ (TimeEnt\ t)) \Rightarrow (f, t)\#fev \mid - \Rightarrow fev)$  (init  $P$ )  $[]$ )

```

```

lemma mp-func-eval-correct[simp]: Mapping.lookup mp-Fevl = func-eval
unfolding mp-Fevl-def func-eval-def by transfer (simp add: Map-To-Mapping.map-apply-def)

```

```

fun duration-matches' :: (object, rat) mapping  $\Rightarrow$  time  $\Rightarrow$  term duration-constraint
 $\Rightarrow$  (variable  $\times$  type) list  $\Rightarrow$  object list  $\Rightarrow$  bool where
  duration-matches' mp-fe  $d\ No-Const\ params\ args \longleftrightarrow d \geq 0$ 
| duration-matches' mp-fe  $d\ (Time-Const\ op\ d')\ params\ args \longleftrightarrow$ 
  (case  $op$  of
    LEQ  $\Rightarrow d \leq d'$ 
  | EQ  $\Rightarrow d = d'$ 
  | GEQ  $\Rightarrow d \geq d'$ )
| duration-matches' mp-fe  $d\ (Func-Const\ op\ f\ vs)\ params\ args \longleftrightarrow$ 
  (let  $tsubst = subst-term\ (the\ o\ (map-of\ (zip\ (map\ fst\ params)\ args)))$  in
    case Mapping.lookup mp-fe  $(FuncEnt\ f\ (map\ tsubst\ vs))$  of
      None  $\Rightarrow False$ 
  | Some  $d' \Rightarrow$  (case  $op$  of
    LEQ  $\Rightarrow d \leq d'$ 
  | EQ  $\Rightarrow d = d'$ 

```

| $GEQ \Rightarrow d \geq d'$)

lemma (in *wf-ast-problem*) *duration-matches'-correct*:

duration-matches' mp-Fevl d dc params args $\longleftrightarrow$ *duration-matches d dc params args*

by (*cases dc*) (*auto split: option.splits*)

fun *consec-https-in-interval-exec* :: *time list* $\Rightarrow$ *time* $\Rightarrow$ *time* $\Rightarrow$ (*time* $\times$ *time*) *list*
where

consec-https-in-interval-exec ($t \# t' \# ts$) t_i t_j =
 (if $t < t_i$ then *consec-https-in-interval-exec* ($t' \# ts$) t_i t_j
 else if $t_i \leq t \wedge t' \leq t_j$ then (t, t')#(*consec-https-in-interval-exec* ($t' \# ts$) t_i t_j)
 else $\square$)

| *consec-https-in-interval-exec* ts t_i t_j = $\square$

lemma *consec-https-in-interval-exec-correct*:

assumes *strict-sorted ts*

shows *consec-https-in-interval* ts t_i t_j = *consec-https-in-interval-exec* ts t_i t_j

unfolding *consec-https-in-interval-def*

using *assms*

proof (*induction ts* t_i t_j *rule: consec-https-in-interval-exec.induct*)

case ($1\ t\ t'\ ts\ t_i\ t_j$)

have $t < t_i \vee (t_i \leq t \wedge t' \leq t_j) \vee (t_i \leq t \wedge t_j < t')$

by *auto*

then show *?case*

proof (*elim disjE*)

assume $t < t_i$

then show *?case*

using *1.prem 1.IH* **by** *auto*

next

assume $t_i \leq t \wedge t' \leq t_j$

then show *?case*

using *1.prem 1.IH* **by** *auto*

next

assume $t_i \leq t \wedge t_j < t'$

then have $\forall (t, t') \in \text{set } (\text{zip } (\text{butlast } (t \# t' \# ts)) (\text{tl } (t \# t' \# ts))).\ t_j < t'$

using $\langle \text{strict-sorted } (t \# t' \# ts) \rangle$

apply (*induction ts*)

apply *auto*

apply (*metis dual-order.antisym less-eq-rat-def order-trans set-zip-rightD*)

done

then have $\neg(\exists (t, t') \in \text{set } (\text{zip } (\text{butlast } (t \# t' \# ts)) (\text{tl } (t \# t' \# ts))).\ t_i \leq t \wedge t' \leq t_j)$

by *fastforce*

then have *filter* $(\lambda(t, t').\ t_i \leq t \wedge t' \leq t_j)\ (\text{zip } (t' \# \text{butlast } ts)\ ts) = \square$

by (*induction ts*) *auto*

then show *?case*

using *1.prem 1.IH*

by *auto*

```

qed
next
  case (2-1  $t_i t_j$ )
  then show ?case by auto
next
  case (2-2  $v t_i t_j$ )
  then show ?case by auto
qed

```

```

fun place-inv-snap-acts :: time list  $\Rightarrow$  time  $\Rightarrow$  time  $\Rightarrow$  ground-action  $\Rightarrow$  (time  $\times$ 
ground-action) list where
  place-inv-snap-acts ( $t \# t' \# ts$ )  $t_i t_j a_{inv}$  =
    (if  $t < t_i$  then place-inv-snap-acts ( $t' \# ts$ )  $t_i t_j a_{inv}$ 
     else if  $t_i \leq t \wedge t' \leq t_j$  then  $((t+t') \text{ div } 2, a_{inv}) \# (\text{place-inv-snap-acts } (t' \# ts) t_i$ 
 $t_j a_{inv})$ 
     else [])
  | place-inv-snap-acts  $ts t_i t_j a_{inv}$  = []

```

```

lemma place-inv-snap-acts-correct:
assumes strict-sorted  $ts$ 
shows place-inv-snap-acts  $ts t_1 t_2 a_{inv}$  = map ( $\lambda(t_i, t_j). ((t_i + t_j) \text{ div } 2, a_{inv}))$ 
(consec-https-in-interval  $ts t_1 t_2$ )
using assms consec-https-in-interval-exec-correct
by (induction  $ts t_1 t_2 a_{inv}$  rule: place-inv-snap-acts.induct) auto

```

Lift *simplify-action* into an Error Monad.

```

fun simplify-actionE :: -  $\Rightarrow$  (object, type) mapping  $\Rightarrow$  (object, rat) mapping  $\Rightarrow$ 
time list  $\Rightarrow$  (time  $\times$  plan-action)  $\Rightarrow$  -(time  $\times$  ground-action) list where
  simplify-actionE stg mp mp-fe https ( $t_\pi$ , Simple-Plan-Action  $n$  args) = do {
    check ( $t_\pi \geq 0$ ) (ERRS "Invalid starting time point");
     $a \leftarrow$  resolve-action-schemaE  $n$ ;
    check (case  $a$  of Simple-Action-Schema - - -  $\Rightarrow$  True | -  $\Rightarrow$  False) (ERRS
"Invalid Plan action type");
    check (action-params-match2 stg mp  $a$  args) (ERRS "Parameter mismatch");
    Error-Monad.return [( $t_\pi$ , instantiate-action-schema  $a$  args)]
  }
  | simplify-actionE stg mp mp-fe https ( $t_\pi$ , Dulative-Plan-Action  $n$  args  $d$ ) = do {
    check ( $t_\pi \geq 0$ ) (ERRS "Invalid starting time point");
     $a \leftarrow$  resolve-action-schemaE  $n$ ;
    check (case  $a$  of Dulative-Action-Schema - - - -  $\Rightarrow$  True | -  $\Rightarrow$  False) (ERRS
"Invalid Plan action type");
    check (action-params-match2 stg mp  $a$  args) (ERRS "Parameter mismatch");
    check ( $d \geq 0$ ) (ERRS "Invalid duration (negative)");
    check (duration-matches' mp-fe  $d$  (duration-constraint  $a$ ) (parameters  $a$ ) args)
(ERRS "Duration constraint not satisfied");
    let  $a_{inv}$  = inst-snap-action  $a$  args Over-All in
    Error-Monad.return (
      ( $t_\pi$ , inst-snap-action  $a$  args At-Start) #

```



```

      (tπ+d, inst-snap-action a args At-End) #
      (place-inv-snap-acts htps tπ (tπ+d) ainv)
    )
  }

```

Justification of refinement for *simplify-action*.

```

lemma (in wf-ast-problem) simplify-actionE-return-iff:
  assumes wf-domain and strict-sorted htps
  shows simplify-actionE STG mp-objT mp-Fevl htps (tπ, π) = Inr as
     $\longleftrightarrow$  (wf-plan-action π  $\wedge$  tπ  $\geq$  0  $\wedge$  simplify-action htps (tπ, π) = as)
proof
  assume assm1: simplify-actionE STG mp-objT mp-Fevl htps (tπ, π) = Inr as
  obtain aschema where aschema-obt: Some aschema = resolve-action-schema
    (name π)
    using assm1 resolve-action-schema-def
    unfolding wf-domain-def by (cases π) (auto simp: return-iff)
    then have wf-act-schema-π: wf-action-schema (the (resolve-action-schema
      (name π)))
      using assms aschema-obt
        resolve-action-schema-def
        index-by-eq-SomeD[where n=name π and x=aschema]
      unfolding wf-domain-def by (metis option.sel)
    then show wf-plan-action π  $\wedge$  tπ  $\geq$  0  $\wedge$  simplify-action htps (tπ, π) = as
    proof (cases π)
      case case-π: (Simple-Plan-Action n args)
        obtain a where a-obt: a = instantiate-action-schema aschema args  $\wedge$  as =
          [(tπ, a)]
          using assm1 aschema-obt case-π by (auto simp: return-iff)
        have wf-p-act: wf-plan-action π  $\wedge$  tπ  $\geq$  0
        using assm1 wf-act-schema-π case-π action-params-match-def wf-effect-inst-weak
          apply (auto simp: return-iff split: option.splits ast-action-schema.splits)
          apply (meson ast-action-schema.exhaust)+
          done
        show ?thesis
          using a-obt aschema-obt case-π wf-p-act
          by (auto split: option.splits ast-action-schema.splits)
      next
      case case-π: (Durative-Plan-Action n args d)
        show ?thesis
          using assm1 wf-act-schema-π case-π action-params-match-def
            wf-effect-durative-inst-weak
            duration-matches'-correct
            place-inv-snap-acts-correct[OF  $\langle$ strict-sorted htps $\rangle$ ]
          apply (auto simp: return-iff split: ast-action-schema.splits)
          apply (auto simp: return-iff split: option.splits)
          apply (meson ast-action-schema.exhaust)+
          done
    qed
  next

```

```

assume assm2: wf-plan-action  $\pi \wedge t_\pi \geq 0 \wedge \text{simplify-action } \text{https } (t_\pi, \pi) = \text{as}$ 
obtain aschema where aschema-obt: Some aschema = resolve-action-schema
(name  $\pi$ )
  using assm2 by (cases  $\pi$ ) (auto split: option.splits)
then show simplify-actionE STG mp-objT mp-Fevl https ( $t_\pi, \pi$ ) = Inr as
  using assm2 action-params-match-def
    duration-matches'-correct
    place-inv-snap-acts-correct[OF  $\langle \text{strict-sorted https} \rangle$ ]
  by (cases  $\pi$ ) (auto simp: return-iff split: option.splits ast-action-schema.splits)
qed

```

Lift *simplify-plan* into an Error Monad.

```

fun simplify-planE :: -  $\Rightarrow$  (object, type) mapping  $\Rightarrow$  (object, rat) mapping  $\Rightarrow$  time
list  $\Rightarrow$  plan  $\Rightarrow$  -+happening list where
  simplify-planE stg mp mp-fe https [] = do {
    Error-Monad.return []
  }
| simplify-planE stg mp mp-fe https ( $\pi \# \pi s$ ) = do {
  as  $\leftarrow$  simplify-actionE stg mp mp-fe https  $\pi$ ;
  hs  $\leftarrow$  simplify-planE stg mp mp-fe https  $\pi s$ ;
  Error-Monad.return (insort-timed-ground-acts as hs)
}

```

Justification of refinement for *simplify-plan*

```

lemma (in wf-ast-problem) simplify-planE-return-iff[return-iff]:
  assumes wf-domain and strict-sorted https
  shows simplify-planE STG mp-objT mp-Fevl https  $\pi s = \text{Inr } hs$ 
     $\longleftrightarrow$  (simplify-plan https  $\pi s = hs \wedge \text{wf-plan } \pi s$ )
using assms simplify-actionE-return-iff wf-plan-def insort-timed-ground-acts-correct
by (induction  $\pi s$  arbitrary: hs) (auto simp: return-iff)

```

type-synonym *'a tree-ht* = (*'a* $\times$ *nat*) *tree*

```

fun avl-height :: happening tree-ht  $\Rightarrow$  nat where
  avl-height  $\langle \rangle$  = 0
| avl-height  $\langle l, (t, as), ht, r \rangle$  = ht

```

```

definition avl-node :: happening tree-ht  $\Rightarrow$  happening  $\Rightarrow$  happening tree-ht  $\Rightarrow$ 
happening tree-ht where
  avl-node l h r =  $\langle l, (h, (\max (\text{avl-height } l) (\text{avl-height } r) + 1)), r \rangle$ 

```

```

definition avl-balL :: happening tree-ht  $\Rightarrow$  happening  $\Rightarrow$  happening tree-ht  $\Rightarrow$ 
happening tree-ht where
  avl-balL l a r =
    (if avl-height l = avl-height r + 2 then
      case l of  $\langle bl, (b, -), br \rangle \Rightarrow$ 
        (if avl-height bl < avl-height br then
          case br of  $\langle cl, (c, -), cr \rangle \Rightarrow \text{avl-node } (\text{avl-node } bl \ b \ cl) \ c \ (\text{avl-node } cr \ a \ r)$ 

```

else avl-node bl b (avl-node br a r)
 else avl-node l a r)

definition avl-balR :: happening tree-ht $\Rightarrow$ happening $\Rightarrow$ happening tree-ht $\Rightarrow$ happening tree-ht **where**

avl-balR l a r =
 (if avl-height r = avl-height l + 2 then
 case r of <bl,(b,-),br> $\Rightarrow$
 if avl-height bl > avl-height br then
 case bl of <cl,(c,-),cr> $\Rightarrow$ avl-node (avl-node l a cl) c (avl-node cr b br)
 else avl-node (avl-node l a bl) b br
 else avl-node l a r)

fun insert-happ-to-tree :: happening $\Rightarrow$ happening tree-ht $\Rightarrow$ happening tree-ht **where**

insert-happ-to-tree (t_i,A_i) <> = avl-node <> (t_i,A_i) <>
 | insert-happ-to-tree (t_i,A_i) <l,((t_j,A_j),h),r> = (
 if t_i < t_j then avl-balL (insert-happ-to-tree (t_i,A_i) l) (t_j,A_j) r
 else if t_i = t_j then avl-node l (t_j,A_i @ A_j) r
 else avl-balR l (t_j,A_j) (insert-happ-to-tree (t_i,A_i) r))

fun insert-timed-ground-acts-to-tree :: (time $\times$ ground-action) list $\Rightarrow$ happening tree-ht $\Rightarrow$ happening tree-ht **where**

insert-timed-ground-acts-to-tree [] htree = htree
 | insert-timed-ground-acts-to-tree ((t_a,a)#as) htree =
 insert-happ-to-tree (t_a,[a]) (insert-timed-ground-acts-to-tree as htree)

definition avl-inorder :: happening tree-ht $\Rightarrow$ happening list **where**
 avl-inorder htree = map fst (inorder htree)

fun simplify-planE-avl' :: - $\Rightarrow$ (object, type) mapping $\Rightarrow$ (object, rat) mapping $\Rightarrow$ time list $\Rightarrow$ plan $\Rightarrow$ -+happening tree-ht **where**

simplify-planE-avl' G mp mp-fe https [] = do {
 Error-Monad.return <>
 }
 | simplify-planE-avl' G mp mp-fe https (π#πs) = do {
 as $\leftarrow$ simplify-actionE G mp mp-fe https π;
 htree $\leftarrow$ simplify-planE-avl' G mp mp-fe https πs;
 Error-Monad.return (insert-timed-ground-acts-to-tree as htree)
 }

fun simplify-planE-avl :: - $\Rightarrow$ (object, type) mapping $\Rightarrow$ (object, rat) mapping $\Rightarrow$ time list $\Rightarrow$ plan $\Rightarrow$ -+happening list **where**

simplify-planE-avl G mp mp-fe https πs = do {
 htree $\leftarrow$ simplify-planE-avl' G mp mp-fe https πs;
 Error-Monad.return (avl-inorder htree)
 }

lemma insort-happ-consL:

assumes $\forall (t_j, A_j) \in \text{set } \text{hs}_1. t_j < t_i$
shows $\text{hs}_1 @ (\text{insort-happ } (t_i, A_i) \text{ hs}_2) = \text{insort-happ } (t_i, A_i) (\text{hs}_1 @ \text{hs}_2)$
using *assms* **by** (*induction* hs_1 *arbitrary*: $t_i A_i \text{hs}_2$) *auto*

lemma *insort-happ-consR*:

assumes $\forall (t_j, A_j) \in \text{set } \text{hs}_2. t_i < t_j$
shows $(\text{insort-happ } (t_i, A_i) \text{ hs}_1) @ \text{hs}_2 = \text{insort-happ } (t_i, A_i) (\text{hs}_1 @ \text{hs}_2)$
using *assms*
proof (*induction* hs_2 *arbitrary*: $t_i A_i \text{hs}_1$)
 case *Nil*
 then show ?*case* **by** *auto*
next
 case (*Cons* $h \text{hs}_2$)
 then show ?*case* **by** (*induction* hs_1) *auto*
qed

lemma *inorder-avl-balL*: $\text{avl-inorder } (\text{avl-balL } l \ a \ r) = \text{avl-inorder } l @ a \#$
avl-inorder r
unfolding *avl-inorder-def avl-node-def avl-balL-def* **by** (*auto split: tree.splits*)

lemma *inorder-avl-balR*: $\text{avl-inorder } (\text{avl-balR } l \ a \ r) = \text{avl-inorder } l @ a \#$
avl-inorder r
unfolding *avl-inorder-def avl-node-def avl-balR-def* **by** (*auto split: tree.splits*)

lemma *strict-sorted-appl*: $\text{strict-sorted } (a @ [y]) \implies \forall x \in \text{set } a. x < y$
by (*induction* a) *auto*

lemma *strict-sorted-consg*: $\text{strict-sorted } (y \# a) \implies \forall x \in \text{set } a. y < x$
by (*induction* a) *auto*

lemma *strict-sorted-avl-inorder-l*:

assumes $\text{strict-sorted } (\text{map fst } (\text{avl-inorder } \langle l, ((t_i, A_i), h), r \rangle))$
shows $\forall (t_j, A_j) \in \text{set } (\text{avl-inorder } l). t_j < t_i$
proof –
 have $\text{strict-sorted } (\text{map fst } (\text{avl-inorder } l) @ [t_i])$
 using *assms strict-sorted-app-iff* [**where** $a = \text{map fst } (\text{avl-inorder } l) @ [t_i]$]
 by (*auto simp: avl-inorder-def*)
 then show ?*thesis*
 using *strict-sorted-appl* [*OF* $\langle \text{strict-sorted } (\text{map fst } (\text{avl-inorder } l) @ [t_i]) \rangle$] **by**
auto
qed

lemma *strict-sorted-avl-inorder-r*:

assumes $\text{strict-sorted } (\text{map fst } (\text{avl-inorder } \langle l, ((t_i, A_i), h), r \rangle))$
shows $\forall (t_j, A_j) \in \text{set } (\text{avl-inorder } r). t_i < t_j$
proof –
 have $\text{strict-sorted } (t_i \# \text{map fst } (\text{avl-inorder } r))$

```

    using assms strict-sorted-app-iff[where  $b=t_i \# \text{map fst (avl-inorder r)}$ ]
    by (auto simp: avl-inorder-def)
  then show ?thesis
    using strict-sorted-consg[OF  $\langle \text{strict-sorted } (t_i \# \text{map fst (avl-inorder r)}) \rangle$ ] by
auto
qed

lemma insert-happ-to-tree-set:
  assumes strict-sorted (map fst (avl-inorder htree))
  shows set (map fst (avl-inorder (insert-happ-to-tree (ti, Ai) htree))) = set (map
fst (avl-inorder htree))  $\cup \{t_i\}$ 
  using assms
proof (induction (ti, Ai) htree rule: insert-happ-to-tree.induct)
  case 1
  then show ?case by (auto simp: avl-inorder-def avl-node-def)
next
  case (2 l tj Aj h r)
  then have strict-sorted (map fst (avl-inorder  $\langle l, ((t_j, A_j), h), r \rangle$ ))
    by auto
  then have strict-sorted (map fst (avl-inorder l @ (tj, Aj) # avl-inorder r))
    unfolding avl-inorder-def by auto
  then have strict-sorted (map fst (avl-inorder l) @ tj # map fst (avl-inorder
r))
    and strict-sorted (map fst (avl-inorder l) @ [tj] @ map fst (avl-inorder r))
    by auto
  then have strict-sorted (map fst (avl-inorder l))
    and strict-sorted (map fst (avl-inorder r))
    using strict-sorted-app-iff[where  $a=\text{map fst (avl-inorder l)}$  and  $b=t_j \# \text{map}$ 
fst (avl-inorder r)]
    strict-sorted-app-iff[where  $a=\text{map fst (avl-inorder l) @ [t_j]}$  and  $b=\text{map}$ 
fst (avl-inorder r)]
    by auto
  have  $t_i < t_j \vee t_i = t_j \vee t_j < t_i$ 
    by auto
  then show ?case
  proof (elim disjE)
    assume  $t_i < t_j$ 
    then show ?case
      using inorder-avl-balL  $\langle \text{strict-sorted (map fst (avl-inorder l))} \rangle$  2.hyps by
(auto simp: avl-inorder-def)
  next
    assume  $t_i = t_j$ 
    then show ?case
      by (auto simp: avl-inorder-def avl-node-def)
  next
    assume  $t_j < t_i$ 
    then show ?case
      using inorder-avl-balR  $\langle \text{strict-sorted (map fst (avl-inorder r))} \rangle$  2.hyps by
(auto simp: avl-inorder-def)

```

```

qed
qed

lemma insert-timed-ground-act-to-tree-strict-sorted:
  assumes strict-sorted (map fst (avl-inorder htree))
  shows strict-sorted (map fst (avl-inorder (insert-happ-to-tree h htree)))
  using assms
proof (induction h htree rule: insert-happ-to-tree.induct)
  case (1 ti Ai)
  then show ?case
  by (auto simp: avl-node-def avl-inorder-def)
next
  case (2 ti Ai l tj Aj h r)
  then have strict-sorted (map fst (avl-inorder ⟨l, ((tj, Aj), h), r⟩))
    by auto
  then have strict-sorted (map fst (avl-inorder l @ (tj, Aj) # avl-inorder r))
    unfolding avl-inorder-def by auto
  then have strict-sorted (map fst (avl-inorder l) @ tj # map fst (avl-inorder
r))
    and strict-sorted (map fst (avl-inorder l) @ [tj] @ map fst (avl-inorder r))
    by auto
  then have strict-sorted (map fst (avl-inorder l))
    and strict-sorted (map fst (avl-inorder l @ [(tj, Aj)]))
    and strict-sorted (map fst (avl-inorder r))
    and strict-sorted (map fst ((tj, Aj) # avl-inorder r))
    using strict-sorted-app-iff[where a=map fst (avl-inorder l) and b=tj # map
fst (avl-inorder r)]
    strict-sorted-app-iff[where a=map fst (avl-inorder l) @ [tj] and b=map
fst (avl-inorder r)]
    by auto
  have ti < tj ∨ ti = tj ∨ tj < ti
    by auto
  then show ?case
proof (elim disjE)
  assume ti < tj
  then have strict-sorted (map fst (avl-inorder (insert-happ-to-tree (ti, Ai) l)))
    using ⟨strict-sorted (map fst (avl-inorder l))⟩ 2.IH by auto

  have strict-sorted (map fst (avl-inorder l) @ map fst ((tj, Aj) # avl-inorder
r))
    using ⟨strict-sorted (map fst (avl-inorder l @ (tj, Aj) # avl-inorder r))⟩ by
auto
  then have ∀ x ∈ set (map fst (avl-inorder l)). ∀ y ∈ set (map fst ((tj, Aj) #
avl-inorder r)). x < y
    using strict-sorted-app-iff[where a=map fst (avl-inorder l) and b=map fst
((tj, Aj) # avl-inorder r)]
    by auto
  then have ∀ x ∈ set (map fst (avl-inorder l)) ∪ {ti}. ∀ y ∈ set (map fst ((tj,
Aj) # avl-inorder r)). x < y

```

```

      using ⟨ $t_i < t_j$ ⟩ strict-sorted-avl-inorder-r[OF ⟨strict-sorted (map fst
(avl-inorder ⟨ $l$ , (( $t_j$ ,  $A_j$ ),  $h$ ),  $r$ )))⟩] by auto
      then have  $\forall x \in \text{set (map fst (avl-inorder (insert-happ-to-tree (t_i, A_i) l)))}$ .  $\forall y$ 
 $\in \text{set (map fst ((t_j, A_j) \# \text{avl-inorder } r))}$ .  $x < y$ 
      using insert-happ-to-tree-set[OF ⟨strict-sorted (map fst (avl-inorder l))⟩] by
      auto
      then have strict-sorted (map fst (avl-inorder (insert-happ-to-tree (t_i, A_i) l) @
(t_j, A_j) \# \text{avl-inorder } r))
      using ⟨strict-sorted (map fst (avl-inorder (insert-happ-to-tree (t_i, A_i) l)))⟩
      ⟨strict-sorted (map fst ((t_j, A_j) \# \text{avl-inorder } r))⟩
      strict-sorted-app-iff[where  $a = \text{map fst (avl-inorder (insert-happ-to-tree$ 
( $t_i, A_i$ )  $l$ )) and  $b = \text{map fst ((t_j, A_j) \# \text{avl-inorder } r)}$ ]
      by auto
      then have strict-sorted (map fst (avl-inorder (avl-balL (insert-happ-to-tree
(t_i, A_i) l) (t_j, A_j) r)))
      using inorder-avl-balL by auto
      then have strict-sorted (map fst (avl-inorder (insert-happ-to-tree (t_i, A_i) ⟨ $l$ ,
((t_j, A_j),  $h$ ),  $r$ )))
      using ⟨ $t_i < t_j$ ⟩ by auto
      then show ?case
      by auto
    next
      assume  $t_i = t_j$ 
      then show ?case
      using 2.prem by (auto simp: avl-inorder-def avl-node-def)
    next
      assume  $t_j < t_i$ 
      then have strict-sorted (map fst (avl-inorder (insert-happ-to-tree (t_i, A_i) r)))
      using ⟨strict-sorted (map fst (avl-inorder r))⟩ 2.IH by auto

      have strict-sorted (map fst (avl-inorder l @ [(t_j, A_j)]) @ map fst (avl-inorder
r))
      using ⟨strict-sorted (map fst (avl-inorder l @ (t_j, A_j) \# \text{avl-inorder } r))⟩ by
      auto
      then have  $\forall x \in \text{set (map fst (avl-inorder l @ [(t_j, A_j)]))}$ .  $\forall y \in \text{set (map fst$ 
(avl-inorder r)).  $x < y$ 
      using strict-sorted-app-iff[where  $a = \text{map fst (avl-inorder l @ [(t_j, A_j)]}$ ) and
 $b = \text{map fst (avl-inorder r)}$ ]
      by auto
      then have  $\forall x \in \text{set (map fst (avl-inorder l @ [(t_j, A_j)]))}$ .  $\forall y \in \text{set (map fst$ 
(avl-inorder r))  $\cup \{t_i\}$ .  $x < y$ 
      using ⟨ $t_j < t_i$ ⟩ strict-sorted-avl-inorder-l[OF ⟨strict-sorted (map fst
(avl-inorder ⟨ $l$ , ((t_j, A_j),  $h$ ),  $r$ )))⟩] by auto
      then have  $\forall x \in \text{set (map fst (avl-inorder l @ [(t_j, A_j)]))}$ .  $\forall y \in \text{set (map fst$ 
(avl-inorder (insert-happ-to-tree (t_i, A_i) r)).  $x < y$ 
      using insert-happ-to-tree-set[OF ⟨strict-sorted (map fst (avl-inorder r))⟩]
      by auto
      then have strict-sorted (map fst (avl-inorder l @ (t_j, A_j) \# \text{avl-inorder}
(insert-happ-to-tree (t_i, A_i) r)))

```

```

using ⟨strict-sorted (map fst (avl-inorder l @ [(tj, Aj)]))⟩
      ⟨strict-sorted (map fst (avl-inorder (insert-happ-to-tree (ti, Ai) r)))⟩
      strict-sorted-app-iff[where a=map fst (avl-inorder l @ [(tj, Aj)]) and
b=map fst (avl-inorder (insert-happ-to-tree (ti, Ai) r))]
by auto
then have strict-sorted (map fst (avl-inorder (avl-balR l (tj, Aj) (insert-happ-to-tree
(ti, Ai) r))))
using inorder-avl-balR by auto
then have strict-sorted (map fst (avl-inorder (insert-happ-to-tree (ti, Ai) ⟨l,
((tj, Aj), h), r)⟩))
using ⟨tj < ti⟩ by auto
then show ?case
by auto
qed
qed

```

```

lemma insert-timed-ground-acts-to-tree-strict-sorted:
assumes strict-sorted (map fst (avl-inorder htree))
shows strict-sorted (map fst (avl-inorder (insert-timed-ground-acts-to-tree as
htree)))
using assms insert-timed-ground-act-to-tree-strict-sorted
by (induction as) auto

```

```

lemma insert-happ-to-tree-equiv:
assumes strict-sorted (map fst (avl-inorder htree))
shows avl-inorder (insert-happ-to-tree (ti, Ai) htree) = insert-happ (ti, Ai)
(avl-inorder htree)
using assms
proof (induction htree)
case Leaf
then show ?case
by (auto simp: avl-node-def avl-inorder-def)
next
case (Node l a r)
show ?case
proof (cases a)
case (Pair a' ht)
then have ⟨l, a, r⟩ = ⟨l, (a', ht), r⟩
by auto
show ?thesis
proof (cases a')
case (Pair tj Aj)
then have ⟨l, a, r⟩ = ⟨l, ((tj, Aj), ht), r⟩
using ⟨⟨l, a, r⟩ = ⟨l, (a', ht), r⟩⟩ by auto
then have avl-inorder ⟨l, a, r⟩ = avl-inorder l @ (tj, Aj) # avl-inorder r
unfolding avl-inorder-def by auto
then have strict-sorted (map fst (avl-inorder l @ (tj, Aj) # avl-inorder
r))

```



```

      using Node.premis by auto
      then have strict-sorted (map fst (avl-inorder l @ [(tj, Aj)]) @ map fst
(avl-inorder r))
        and strict-sorted (map fst (avl-inorder l) @ map fst ((tj, Aj) #
avl-inorder r))
      by auto
      then have strict-sorted (map fst (avl-inorder l @ [(tj, Aj)]))
        and strict-sorted (map fst (avl-inorder l))
        and strict-sorted (map fst ((tj, Aj) # avl-inorder r))
        and strict-sorted (map fst (avl-inorder r))
      using strict-sorted-app-iff by blast+
      then have strict-sorted (map fst (avl-inorder l) @ [tj])
        and strict-sorted (tj # map fst (avl-inorder r))
      by auto
      then have ∀ (tk, Ak) ∈ set (avl-inorder l). tk < tj
        and ∀ (tk, Ak) ∈ set (avl-inorder r). tj < tk
        using strict-sorted-appl[OF ‹strict-sorted (map fst (avl-inorder l) @
[tj])›]
          strict-sorted-consg[OF ‹strict-sorted (tj # map fst (avl-inorder
r))›]
      by auto

have ti < tj ∨ ti = tj ∨ tj < ti
  by auto
then show ?thesis
proof (elim disjE)
  assume ti < tj
  then have ∀ (tk, Ak) ∈ set ((tj, Aj) # avl-inorder r). ti < tk
    using ‹∀ (tk, Ak) ∈ set (avl-inorder r). tj < tk› by auto
  then show ?thesis
    using ‹⟨l, a, r⟩ = ⟨l, ((tj, Aj), ht), r⟩› ‹ti < tj› ‹strict-sorted (map fst
(avl-inorder l))›
      Node.IH inorder-avl-balL insert-happ-consR
    by (auto simp: avl-inorder-def)
next
  assume ti = tj
  have avl-inorder (insert-happ-to-tree (ti, Ai) ⟨l, a, r⟩) = avl-inorder
(avl-node l (tj, Ai @ Aj) r)
    using ‹⟨l, a, r⟩ = ⟨l, ((tj, Aj), ht), r⟩› ‹ti = tj› by auto
  also have ... = avl-inorder l @ (tj, Ai @ Aj) # avl-inorder r
    unfolding ‹⟨l, a, r⟩ = ⟨l, ((tj, Aj), ht), r⟩› ‹ti = tj› avl-node-def by (auto
simp: avl-inorder-def)

  also have ... = insert-happ (ti, Ai) (avl-inorder l @ [(tj, Aj)]) @
avl-inorder r
    using ‹ti = tj› insert-happ-consL[OF ‹∀ (tk, Ak) ∈ set (avl-inorder
l). tk < tj›, where hs2 = [(tj, Aj)]]
    by auto
  also have ... = insert-happ (ti, Ai) (avl-inorder ⟨l, a, r⟩)

```

```

    using ⟨ $t_i = t_j$ ⟩ ⟨ $\langle l, a, r \rangle = \langle l, ((t_j, A_j), ht), r \rangle$ ⟩
      insert-happ-consR[OF ⟨ $\forall (t_k, A_k) \in \text{set } (avl\text{-inorder } r). t_j < t_k$ ⟩]
    by (auto simp: avl-inorder-def)
  finally show ?thesis
    by auto
next
  assume  $t_j < t_i$ 
  then have  $\forall (t_k, A_k) \in \text{set } (avl\text{-inorder } l @ [(t_j, A_j)]) . t_k < t_i$ 
    using ⟨ $\forall (t_k, A_k) \in \text{set } (avl\text{-inorder } l). t_k < t_j$ ⟩ by auto
  then show ?thesis
    using ⟨ $\langle l, a, r \rangle = \langle l, ((t_j, A_j), ht), r \rangle$ ⟩ ⟨ $t_j < t_i$ ⟩ ⟨strict-sorted (map fst
(avl-inorder r))⟩
      Node.IH inorder-avl-balR
      insert-happ-consL[OF ⟨ $\forall (t_k, A_k) \in \text{set } (avl\text{-inorder } l @ [(t_j, A_j)]) .$ 
 $t_k < t_i$ ⟩]
    by (auto simp: avl-inorder-def)
  qed
qed
qed
qed

lemma insert-timed-ground-acts-to-tree-equiv:
  assumes strict-sorted (map fst (avl-inorder htree))
  shows insert-timed-ground-acts as (avl-inorder htree)
    = avl-inorder (insert-timed-ground-acts-to-tree as htree)
  using assms
proof (induction as arbitrary: htree)
  case Nil
  then show ?case by auto
next
  case (Cons a' as)
  then show ?case
    using insert-happ-to-tree-equiv[OF insert-timed-ground-acts-to-tree-strict-sorted[OF
Cons.prem]]
    by (cases a') auto
  qed

lemma simplify-planE-avl'-strict-sorted:
  assumes simplify-planE-avl' STG mp-objT mp-Fevl https  $\pi s = \text{Inr htree}$ 
  shows strict-sorted (map fst (avl-inorder htree))
  using assms
proof (induction  $\pi s$  arbitrary: htree)
  case Nil
  then show ?case
    by (auto simp: avl-inorder-def)
next
  case (Cons  $\pi \pi s$ )
  then obtain htree' where simplify-planE-avl' STG mp-objT mp-Fevl https  $\pi s$ 
    = Inr htree'

```

by (auto simp: return-iff)
 then show ?case
 using Cons.premis Cons.IH insert-timed-ground-acts-to-tree-strict-sorted by
 auto
 qed

lemma *simplify-planE-avl'-equiv*:
 assumes *simplify-planE STG mp-objT mp-Fevl htps $\pi s = \text{Inr } hs$*
 and *simplify-planE-avl' STG mp-objT mp-Fevl htps $\pi s = \text{Inr } htree$*
 shows *hs = avl-inorder htree*
 using *assms unfolding avl-inorder-def*
proof (*induction πs arbitrary: hs htree*)
 case Nil
 then show ?case by auto
 next
 case (Cons $\pi \pi s$)
 then obtain *hs' htree'* where *simplify-planE STG mp-objT mp-Fevl htps $\pi s = \text{Inr } hs'$*
 and *simplify-planE-avl' STG mp-objT mp-Fevl htps $\pi s = \text{Inr } htree'$*
 by (auto simp: return-iff)
 then have *hs' = map fst (inorder htree')*
 using Cons.premis Cons.IH by auto
 then show ?case
 using Cons.premis
 insert-timed-ground-acts-to-tree-equiv[OF *simplify-planE-avl'-strict-sorted*]
 ‹*simplify-planE STG mp-objT mp-Fevl htps $\pi s = \text{Inr } hs'$* ›
 ‹*simplify-planE-avl' STG mp-objT mp-Fevl htps $\pi s = \text{Inr } htree'$* ›
 by (auto simp: avl-inorder-def)
 qed

lemma *simplify-planE-avl-equiv-Inr*:
 shows $(\exists hs. \text{simplify-planE STG mp-objT mp-Fevl htps } \pi s = \text{Inr } hs)$
 $\longleftrightarrow (\exists htree. \text{simplify-planE-avl' STG mp-objT mp-Fevl htps } \pi s = \text{Inr } htree)$
 by (*induction πs*) (auto simp: return-iff)

lemma *simplify-planE-avl-equiv*:
 shows *simplify-planE STG mp-objT mp-Fevl htps $\pi s = \text{Inr } hs$*
 $\longleftrightarrow \text{simplify-planE-avl STG mp-objT mp-Fevl htps } \pi s = \text{Inr } hs$
 using *simplify-planE-avl'-equiv simplify-planE-avl-equiv-Inr* by fastforce

lemma (*in wf-ast-problem*)
 assumes *wf-domain*
 and *simplify-planE-avl STG mp-objT mp-Fevl (https-exec πs) $\pi s = \text{Inr } hs$*
 shows *ind-happ-seq πs hs* and *wf-plan πs*
 using *assms simplify-planE-avl-equiv simplify-planE-return-iff*
https-exec-sorted https-exec-distinct simplify-plan-correct

by (*auto simp: strict-sorted-iff*)

Lift *valid-happ-seq-from* into an Error Monad, and use efficient happening execution *en-exE*

```
fun valid-happ-seq-fromE :: nat  $\Rightarrow$  world-model  $\Rightarrow$  happening list  $\Rightarrow$   $\rightarrow$ unit
where
  valid-happ-seq-fromE si s [] =
    check (holds s (goal P)) (ERRS "Postcondition does not hold")
  | valid-happ-seq-fromE si s (h#hs) = do {
    s  $\leftarrow$  en-exE h s <+ ? ( $\lambda e$  -. shows "at step " o shows si o shows ": " o e ());
    valid-happ-seq-fromE (si+1) s hs
  }
```

For the refinement, we need to show that the world models only contain atoms, i.e., containing only atoms is an invariant under execution of well-formed happenings.

```
lemma (in wf-ast-problem) wf-happ-only-add-atoms:
assumes wm-basic s
and  $\forall a \in \text{set } as. \text{wf-ground-action } a$ 
shows wm-basic (apply-happ-exec (t,as) s)
using wf-problem wf-domain
unfolding wf-problem-def wf-domain-def
using assms
  apply-happ-exec-refine
  wf-fmla-atom-alt
  wf-ground-action-wf-fmla-atom
by (auto simp: wm-basic-def) fastforce
```

```
lemma (in wf-ast-problem) wf-happ-only-add-atoms':
assumes wf-world-model s
and  $\forall a \in \text{set } as. \text{wf-ground-action } a$ 
shows wf-world-model (apply-happ-exec (t,as) s)
using wf-problem wf-domain
unfolding wf-problem-def wf-domain-def
using assms
  apply-happ-exec-refine
  wf-fmla-atom-alt
  wf-ground-action-wf-fmla-atom
by (auto simp: wf-world-model-def)
```

Justification for refinement of *valid-happ-seq-from*.

```
lemma (in wf-ast-problem) valid-happ-seq-fromE-return-iff[return-iff]:
assumes wm-basic s
and  $\forall (t,as) \in \text{set } hs. \forall a \in \text{set } as. \text{wf-ground-action } a$ 
shows valid-happ-seq-from s hs  $\longleftrightarrow$  valid-happ-seq-fromE k s hs = Inr ()
using assms
  holds-for-wf-fmlas
  wf-happ-only-add-atoms
```

apply-happ-exec-refine
by (*induction* *hs* *arbitrary*: *s* *k*) (*auto simp*: *return-iff en-exE-return-iff*)

Now we can define a refinement for *valid-plan-from2*.

```
fun valid-plan-from2-exec :: world-model  $\Rightarrow$  plan  $\Rightarrow$  bool where
  valid-plan-from2-exec s  $\pi s \longleftrightarrow$ 
    (wf-plan  $\pi s \wedge$  valid-happ-seq-from s (simplify-plan (https-exec  $\pi s$ )  $\pi s$ ))
```

With $\llbracket \text{wf-ast-problem } ?P; \text{ast-problem.ind-happ-seq } ?P \text{ ?}\pi s \text{ ?}hs; \text{ast-problem.ind-happ-seq } ?P \text{ ?}\pi s \text{ ?}hs'; \text{ast-problem.wf-plan } ?P \text{ ?}\pi s; \text{valid-happ-seq } ?M_0 \text{ ?}hs \text{ ?}M_n \rrbracket \implies \text{valid-happ-seq } ?M_0 \text{ ?}hs' \text{ ?}M_n$ we can prove, that if there exists a valid induced happening sequence, then simplify plan is also valid.

```
lemma (in wf-ast-problem) simplify-plan-valid-exists1:
  fixes  $\pi s$ 
  defines https  $\equiv$  https-exec  $\pi s$ 
  assumes wf-plan  $\pi s$ 
  and  $\exists hs. \text{ind-happ-seq } \pi s \text{ } hs \wedge \text{valid-happ-seq } M \text{ } hs \text{ } M' \wedge M' \text{ }^c \models \text{(goal } P)$ 
  shows valid-happ-seq M (simplify-plan https  $\pi s$ )  $M' \wedge M' \text{ }^c \models \text{(goal } P)$ 
  using assms simplify-plan-correct ind-happ-seq-unique-final-state' by blast
```

Other way round is easy, since *simplify-plan* is an induced happening sequence.

```
lemma (in wf-ast-problem) simplify-plan-valid-exists2:
  fixes  $\pi s$ 
  defines https  $\equiv$  https-exec  $\pi s$ 
  assumes wf-plan  $\pi s$ 
  and valid-happ-seq M (simplify-plan https  $\pi s$ )  $M' \wedge M' \text{ }^c \models \text{(goal } P)$ 
  shows  $\exists hs. \text{ind-happ-seq } \pi s \text{ } hs \wedge \text{valid-happ-seq } M \text{ } hs \text{ } M' \wedge M' \text{ }^c \models \text{(goal } P)$ 
  using assms simplify-plan-correct by blast
```

Justification for the refinement of *valid-plan-from2*.

```
lemma (in wf-ast-problem) valid-plan-from2-exec-refine:
  valid-plan-from2 s  $\pi s \longleftrightarrow$  valid-plan-from2-exec s  $\pi s$ 
  using simplify-plan-valid-exists1
    simplify-plan-valid-exists2
    valid-happ-seq-from-refine[OF inst-wf-plan-wf-happ-seq]
    simplify-plan-correct
  unfolding valid-plan-from2-def plan-happ-path-def
  by (meson valid-plan-from2-exec.simps)
```

Now we can also lift *valid-plan-from2-exec* into an Error Monad, using *simplify-planE* and *valid-happ-seq-fromE*

```
fun valid-plan-from2E :: -  $\Rightarrow$  (object, type) mapping  $\Rightarrow$  (object, rat) mapping  $\Rightarrow$ 
  world-model  $\Rightarrow$  plan  $\Rightarrow$  -+unit where
  valid-plan-from2E stg mp mp-fe s  $\pi s =$  do {
    hs  $\leftarrow$  simplify-planE-avl stg mp mp-fe (https-exec  $\pi s$ )  $\pi s$ ;
    valid-happ-seq-fromE 1 s hs
```

}

```

lemma (in wf-ast-problem) valid-plan-fromE-wf-plan:
  assumes valid-plan-from2E STG mp-objT mp-Fevl s  $\pi s = \text{Inr } ()$ 
  shows wf-plan  $\pi s$ 
proof –
  have strict-sorted (https-exec  $\pi s$ )
    using https-exec-distinct https-exec-sorted by (simp add: strict-sorted-iff)
  then show ?thesis
    using assms wf-problem simplify-planE-return-iff simplify-planE-avl-equiv
    by (auto simp: return-iff wf-problem-def)
qed

```

Refinement lemma for our plan checking algorithm

```

lemma (in wf-ast-problem) valid-plan-from2E-return-iff:
  assumes wf-world-model s
  shows valid-plan-from2E STG mp-objT mp-Fevl s  $\pi s = \text{Inr } () \longleftrightarrow \text{valid-plan-from2E}$ 
 $s \pi s$ 
proof –
  have wf-domain
    using wf-problem unfolding wf-problem-def by simp
  have wm-basic s
    using  $\langle \text{wf-world-model } s \rangle$  wf-fmla-atom-alt wf-func-assign-is-eqAtom
    unfolding wf-world-model-def wm-basic-def by auto
  have strict-sorted (https-exec  $\pi s$ )
    using https-exec-distinct https-exec-sorted by (simp add: strict-sorted-iff)
  show ?thesis
    using  $\langle \text{wf-domain} \rangle \langle \text{strict-sorted (https-exec } \pi s) \rangle$  valid-plan-from2-exec-refine
    valid-happ-seq-fromE-return-iff[OF  $\langle \text{wm-basic } s \rangle$  simplify-plan-wf-ground-action]

    simplify-planE-return-iff[OF  $\langle \text{wf-domain} \rangle \langle \text{strict-sorted (https-exec } \pi s) \rangle$ ]
    simplify-planE-avl-equiv
  apply (simp add: valid-plan-fromE-wf-plan)
  apply (auto simp: return-iff)
  applymetis+
  done
qed

```

```

lemma (in wf-ast-problem)
  assumes wf-world-model s
  shows valid-plan-from2E STG mp-objT mp-Fevl s  $\pi s = \text{Inr } () \longleftrightarrow \text{valid-plan-from2E}$ 
 $s \pi s$ 
  using assms by (rule valid-plan-from2E-return-iff)

lemmas valid-plan-from2E-return-iff'[return-iff]

```

$= \text{wf-ast-problem.valid-plan-from2E-return-iff}[of\ P, OF\ \text{wf-ast-problem.intro}]$

end — Context of *ast-problem*

2.3 Executable Plan Checker

We obtain the main plan checker by combining the well-formedness check and executability check.

definition *check-all-list* $P\ l\ msg\ msgf \equiv$
 $\text{forallM } (\lambda x. \text{check } (P\ x) (\lambda :: \text{unit}. \text{shows } msg\ o\ \text{shows } ": " o\ msgf\ x))\ l\ <+?\$
 snd

lemma *check-all-list-return-iff*[*return-iff*]: $\text{check-all-list } P\ l\ msg\ msgf = \text{Inr } () \longleftrightarrow$
 $(\forall x \in \text{set } l. P\ x)$

unfolding *check-all-list-def* **by** (*induction l*) (*auto*)

definition *check-wf-types* $D \equiv \text{do } \{$
 $\text{check-all-list } (\lambda(-,t). t = \text{"object"} \vee t \in \text{fst'set } (\text{types } D)) (\text{types } D) \text{"Undeclared"}$
 $\text{supertype"} (\text{shows } o\ \text{snd})$
 $\}$

lemma *check-wf-types-return-iff*[*return-iff*]: $\text{check-wf-types } D = \text{Inr } () \longleftrightarrow \text{ast-domain.wf-types } D$

unfolding *ast-domain.wf-types-def* *check-wf-types-def* **by** (*force simp: return-iff*)

definition *check-wf-domain* $D\ stg\ conT \equiv \text{do } \{$
 $\text{check-wf-types } D;$
 $\text{check } (\text{distinct } (\text{map } (\text{predicate-decl.pred}) (\text{predicates } D))) (\text{ERRS } \text{"Duplicate predicate declaration"})$
 $\text{check-all-list } (\text{ast-domain.wf-predicate-decl } D) (\text{predicates } D) \text{"Malformed predicate declaration"} (\text{shows } o\ \text{predicate.name } o\ \text{predicate-decl.pred});$
 $\text{check } (\text{distinct } (\text{map } (\text{function-decl.func}) (\text{functions } D))) (\text{ERRS } \text{"Duplicate function declaration"})$
 $\text{check-all-list } (\text{ast-domain.wf-function-decl } D) (\text{functions } D) \text{"Malformed function declaration"} (\text{shows } o\ \text{func.name } o\ \text{function-decl.func});$
 $\text{check } (\text{distinct } (\text{map } \text{fst } (\text{consts } D))) (\text{ERRS } \text{"Duplicate constant declaration"})$
 $\text{check } (\forall (n,T) \in \text{set } (\text{consts } D). \text{ast-domain.wf-type } D\ T) (\text{ERRS } \text{"Malformed type"})$
 $\text{check } (\text{distinct } (\text{map } \text{ast-action-schema.name } (\text{actions } D))) (\text{ERRS } \text{"Duplicate action name"})$
 $\text{check-all-list } (\text{ast-domain.wf-action-schema' } D\ stg\ conT) (\text{actions } D) \text{"Malformed action"} (\text{shows } o\ \text{ast-action-schema.name})$
 $\}$

lemma *check-wf-domain-return-iff*[*return-iff*]:
 $check_wf_domain\ D\ stg\ conT = Inr\ () \longleftrightarrow ast_domain.wf_domain'\ D\ stg\ conT$
proof –
interpret *ast-domain* *D* .
show ?thesis
unfolding *check-wf-domain-def* *wf-domain'-def* **by** (auto simp: *return-iff*)
qed

definition *prepend-err-msg* *msg* *e* $\equiv \lambda::unit. shows\ msg\ o\ shows\ '' : ''\ o\ e\ ()$

definition *check-wf-problem* *P* *stg* *conT* *mp* $\equiv do\ \{$
 $let\ D = ast_problem.domain\ P;$
 $check_wf_domain\ D\ stg\ conT <+? prepend_err_msg\ "Domain\ not\ well-formed";$
 $check\ (distinct\ (map\ fst\ (objects\ P)\ @\ map\ fst\ (consts\ D)))\ (ERRS\ "Duplicate\ object\ declaration");$
 $check\ ((\forall (n,T) \in set\ (objects\ P). ast_domain.wf_type\ D\ T))\ (ERRS\ "Malformed\ type");$
 $check\ (distinct\ (init\ P))\ (ERRS\ "Duplicate\ fact\ in\ initial\ state");$
 $check\ (\forall f \in set\ (init\ P). ast_problem.wf_fmula_atom2'\ P\ mp\ stg\ f \vee ast_problem.wf_func_assign'\ P\ mp\ stg\ f)\ (ERRS\ "Malformed\ formula\ in\ initial\ state");$
 $check\ (ast_domain.wf_fmula'\ D\ (Mapping.lookup\ mp)\ stg\ (goal\ P))\ (ERRS\ "Malformed\ goal\ formula")$
 $\}$

lemma *check-wf-problem-return-iff*[*return-iff*]:
 $check_wf_problem\ P\ stg\ conT\ mp = Inr\ () \longleftrightarrow ast_problem.wf_problem'\ P\ stg\ conT\ mp$
proof –
interpret *ast-problem* *P* .
show ?thesis
unfolding *check-wf-problem-def* *wf-problem'-def* **by** (auto simp: *return-iff*)
qed

definition *check-plan* *P* $\pi s \equiv do\ \{$
 $let\ stg = ast_domain.STG\ (ast_problem.domain\ P);$
 $let\ conT = ast_domain.mp_constT\ (ast_problem.domain\ P);$
 $let\ mp = ast_problem.mp_objT\ P;$
 $let\ mp_fe = ast_problem.mp_Fevl\ P;$
 $check_wf_problem\ P\ stg\ conT\ mp;$
 $ast_problem.valid_plan_from2E\ P\ stg\ mp\ mp_fe\ (ast_problem.I\ P)\ \pi s$
 $\} <+? (\lambda e. String.implode\ (e\ ()\ '''))$

Correctness theorem of the plan checker: It returns *Inr* () if and only if the problem is well-formed and the plan is valid.

theorem *check-plan-return-iff*[*return-iff*]:
 $check_plan\ P\ \pi s = Inr\ () \longleftrightarrow ast_problem.wf_problem\ P \wedge ast_problem.valid_plan2\ P\ \pi s$
proof –
interpret *ast-problem* *P* .


```

show ?thesis
  unfolding check-plan-def ast-problem.valid-plan2-def
  using valid-plan-from2E-return-iff' wf-ast-problem-def wf-ast-problem.wf-I
  by (fastforce simp: wf-problem'-correct return-iff)
qed

find-theorems ast-problem.valid-plan-from2

```

2.4 Code Setup

In this section, we set up the code generator to generate verified code for our plan checker.

2.4.1 Code Equations

We first register the code equations for the functions of the checker. Note that we not necessarily register the original code equations, but also optimized ones.

```

lemmas wf-domain-code =
  ast-domain.sig-def
  ast-domain.wf-types-def
  ast-domain.wf-type.simps
  ast-domain.wf-predicate-decl.simps
  ast-domain.STG-def
  ast-domain.is-of-type'-def
  ast-domain.wf-atom'.simps
  ast-domain.wf-pred-atom'.simps
  ast-domain.wf-fmla'.simps
  ast-domain.wf-fmla-atom1'.simps
  ast-domain.wf-effect'.simps
  ast-domain.wf-action-schema'.simps
  ast-domain.wf-domain'-def
  ast-domain.subst-term.simps
  ast-domain.mp-constT-def

  ast-domain.wf-function-decl.simps
  ast-domain.func-sig-def
  ast-domain.wf-func-args'.simps
  ast-domain.wf-duration-const'.simps
  ast-domain.acts-non-intrf-def

```

```

declare wf-domain-code[code]

```

```

lemmas wf-problem-code =
  ast-problem.wf-problem'-def
  ast-problem.wf-fact'-def
  ast-problem.is-obj-of-type-alt

```

ast-problem.wf-fact-def
ast-problem.wf-plan-action.simps
ast-domain.subtype-edge.simps

declare *wf-problem-code*[code]

lemmas *check-code* =
ast-problem.valid-plan2-def
ast-problem.valid-plan-from2E.simps
ast-problem.res-inst.simps
ast-domain.resolve-action-schema-def
ast-domain.resolve-action-schemaE-def
ast-problem.I-def
ast-domain.instantiate-action-schema.simps
ast-problem.holds-def
ast-problem.mp-objT-def
ast-problem.is-obj-of-type-impl-def
ast-problem.wf-fmla-atom2'-def
valuation-def

ast-problem.valid-happ-seq-fromE.simps
ast-problem.htps-exec.simps
ast-problem.insort-htp.simps
ast-problem.simplify-planE.simps
ast-problem.simplify-actionE.simps
ast-problem.insort-happ.simps
ast-domain.inst-snap-action.simps
ast-domain.filter-time-spec-def
ast-domain.conjunct-effects-def

ast-problem.apply-happ-exec.simps
ast-problem.en-exE-def
ast-problem.insort-mult-happs.simps
ast-problem.duration-matches'.simps
ast-problem.wf-func-assign'.simps

ast-problem.mp-Fevl-def
ast-problem.place-inv-snap-acts.simps

ast-problem.insort-timed-ground-acts.simps
ast-problem.insert-happ-to-tree.simps
ast-problem.insert-timed-ground-acts-to-tree.simps
ast-problem.avl-inorder-def
ast-problem.simplify-planE-avl.simps
ast-problem.simplify-planE-avl'.simps
ast-problem.avl-height.simps
ast-problem.avl-node-def
ast-problem.avl-balR-def
ast-problem.avl-balL-def

declare *check-code*[*code*]

2.4.2 Setup for Containers Framework

derive (*linorder*) *compare rat*

derive (*eq*) *ceq predicate func atom object formula*
derive *ccompare predicate func atom object formula*
derive (*no*) *cenum atom object formula*
derive (*rbt*) *set-impl func atom object formula*

derive (*rbt*) *mapping-impl object*

derive *linorder predicate func object atom object atom formula*

2.4.3 More Efficient Distinctness Check for Linorders

fun *no-stutter* :: '*a list* $\Rightarrow$ *bool* **where**
 no-stutter [] = *True*
 | *no-stutter* [-] = *True*
 | *no-stutter* (*a#b#l*) = (*a#b* $\wedge$ *no-stutter* (*b#l*))

lemma *sorted-no-stutter-eq-distinct*: *sorted l $\Rightarrow$ no-stutter l $\longleftrightarrow$ distinct l*
apply (*induction l rule: no-stutter.induct*)
apply (*auto simp:*)
done

definition *distinct-ds* :: '*a::linorder list* $\Rightarrow$ *bool*
where *distinct-ds l* $\equiv$ *no-stutter (quicksort l)*

lemma [*code-unfold*]: *distinct = distinct-ds*
apply (*intro ext*)
unfolding *distinct-ds-def*
apply (*auto simp: sorted-no-stutter-eq-distinct*)
done

2.4.4 Parsing Rational Numbers

type-synonym *digit* = *nat*

Well-formedness conditions for a digit and a sequence of digits.

definition *wf-digit d* $\longleftrightarrow$ *d < 10*

A sequence of digits is well-formed iff it is normalized and only contains well-formed digits.

definition *wf-digits ds* $\longleftrightarrow$ ($\forall d \in \text{set } ds. \text{wf-digit } d$)

Functions to trim leading and trailing zeros.

fun *trim-ld-zs* :: *digit list* $\Rightarrow$ *digit list* **where**

trim-ld-zs [] = []
 | *trim-ld-zs* (d#ds) =
 (if d = 0 then *trim-ld-zs* ds else d#ds)

lemma *tlz-hd-neq-z*: *trim-ld-zs* ds = ds' $\implies$ hd ds' $\neq$ 0 $\vee$ ds' = []
by (induction ds) auto

lemma *trim-ld-zs-idem*: *trim-ld-zs* (*trim-ld-zs* ds) = *trim-ld-zs* ds
by (induction ds) auto

lemma *trim-ld-zs-wf*: *wf-digits* ds $\implies$ *wf-digits* (*trim-ld-zs* ds)
unfolding *wf-digits-def* **by** (induction ds) auto

lemma *trim-ld-zs-app*: *trim-ld-zs* (ds1 @ ds2) = (*trim-ld-zs* ds1) @ ds2 $\vee$ ($\forall d \in$
set ds1. d = 0)
by (induction ds1) auto

fun *trim-tr-zs'* :: *digit list* $\Rightarrow$ *digit list* $\Rightarrow$ *digit list* **where**

trim-tr-zs' [] zs = []
 | *trim-tr-zs'* (d#ds) zs =
 (if d = 0 then *trim-tr-zs'* ds (0#zs)
 else zs @ d # (*trim-tr-zs'* ds []))

fun *trim-tr-zs* :: *digit list* $\Rightarrow$ *digit list* **where**
trim-tr-zs ds = *trim-tr-zs'* ds []

lemma *ttz-last-neq-z-aux*:
assumes *trim-tr-zs'* ds zs = ds'
shows last ds' $\neq$ 0 $\vee$ ds' = []
using *assms*
proof (induction ds arbitrary: ds' zs)
 case Nil
 then show ?case **by** auto
next
 case (Cons d ds)
 then show ?case
 by (cases d = 0) force+
qed

lemma *ttz-last-neq-z*: *trim-tr-zs* ds = ds' $\implies$ last ds' $\neq$ 0 $\vee$ ds' = []
using *ttz-last-neq-z-aux* **by** (auto simp: Let-def split: if-splits)

lemma *trim-tr-zs'-wf*: *wf-digits* ds $\wedge$ *wf-digits* zs $\implies$ *wf-digits* (*trim-tr-zs'* ds zs)
unfolding *wf-digits-def* **by** (induction ds zs rule: *trim-tr-zs'.induct*) auto

lemma *trim-tr-zs-wf*: *wf-digits* ds $\implies$ *wf-digits* (*trim-tr-zs* ds)
using *trim-tr-zs'-wf* [where zs=[]] **by** (auto simp: *wf-digits-def*)

Functions to convert between integers and a sequence of digits.

```

fun int-of-digits :: digit list  $\Rightarrow$  int  $\Rightarrow$  int where
  int-of-digits [] acc = acc
| int-of-digits (d#ds) acc = int-of-digits ds (acc * 10 + d)

fun digits-of-int :: int  $\Rightarrow$  digit list where
  digits-of-int i =
    (if i  $\leq$  0 then []
     else if i < 10 then [nat i]
     else digits-of-int (i div 10) @ [nat (i mod 10)])

value digits-of-int (int-of-digits [1,2,3,4] 0)
value int-of-digits (digits-of-int 1234) 0

value digits-of-int (int-of-digits [1] 123)

lemma digits-of-int-int-of-digits-aux:
  assumes acc > 0 and  $\forall d \in \text{set } ds. \text{wf-digit } d$ 
  shows digits-of-int (int-of-digits ds acc) = digits-of-int acc @ ds
  using assms
  by (induction ds acc arbitrary: acc rule: int-of-digits.induct) (auto simp: wf-digit-def)

```

Proof for correctness of function *int-of-digits* and *digits-of-int*.

```

lemma digits-of-int-int-of-digits:
  assumes wf-digits ds
  shows digits-of-int (int-of-digits ds 0) = trim-ld-zs ds
  using assms digits-of-int-int-of-digits-aux
proof (induction ds)
  case Nil
  then show ?case by auto
next
  case (Cons d ds)
  then show ?case
    by (cases d = 0) (auto simp: wf-digits-def wf-digit-def)
qed

```

Function to convert from a sequences of digits to a rational number.

```

primrec rat-of-digits-pair :: digit list  $\times$  digit list  $\Rightarrow$  rat where
  rat-of-digits-pair (ds1, ds2) = (
    let ds1' = trim-ld-zs ds1; ds2' = trim-tr-zs ds2 in
    Rat.Fract (int-of-digits (ds1' @ ds2') 0) (10 ^ length ds2')
  )

```

For proofs about the function *rat-of-digits-pair* see `Rat_Parsing.thy`.

2.4.5 Code Generation

```

export-code
  check-plan

```

```

  nat-of-integer integer-of-nat int-of-integer integer-of-int Inl Inr Rat.Fract Rat.of-int
  rat-of-digits-pair
  predAtm eqAtm predicate Pred Func Either Var Obj PredDecl FuncDecl BigAnd
  BigOr
  formula.Not formula.Bot Effect No-Const Time-Const Func-Const LEQ EQ GEQ
  Simple-Action-Schema Durative-Action-Schema At-Start At-End Over-All
  map-atom Domain Problem Simple-Plan-Action Durative-Plan-Action
  term.CONST term.VAR
  String.explode String.implode
in SML
module-name TEMPORAL-PDDL-Checker-Exported
file code/TEMPORAL-PDDL-Checker-Exported.sml

```

end — Theory

3 Connecting the Semantics to State-Transition Based Semantics

```

theory TEMPORAL-PDDL-Semantics-Alt
imports
  TEMPORAL-PDDL-Semantics
  TEMPORAL-PDDL-Checker
begin

```

Auxiliary lemma with about *dropWhile*, *takeWhile* with *le* and *leq*

```

lemma strict-sorted-drop-le-take-leq-comm:
  assumes  $t_i \leq t_j$ 
  and strict-sorted (map fst hs)
  shows dropWhile (le  $t_i$ ) (takeWhile (leq  $t_j$ ) hs) = takeWhile (leq  $t_j$ ) (dropWhile
  (le  $t_i$ ) hs)
  using assms
  by (induction hs) (auto simp: le-def leq-def)

lemma strict-sorted-drop-leq-take-le-comm:
  assumes  $t_i < t_j$ 
  and strict-sorted (map fst hs)
  shows dropWhile (leq  $t_i$ ) (takeWhile (le  $t_j$ ) hs) = takeWhile (le  $t_j$ ) (dropWhile
  (leq  $t_i$ ) hs)
  using assms
  by (induction hs) (auto simp: le-def leq-def)

lemma strict-sorted-drop-leq-take-leq-comm:
  assumes  $t_i \leq t_j$ 

```

and *strict-sorted* (*map fst hs*)
shows *dropWhile* (*leq t_i*) (*takeWhile* (*leq t_j*) *hs*) = *takeWhile* (*leq t_j*) (*dropWhile* (*leq t_i*) *hs*)
using *assms* **by** (*induction hs*) (*auto simp: leq-def*)

lemma *strict-sorted-drop-le-take-leq*:
assumes *strict-sorted* (*map fst hs*)
and (*t_a*, *A*) ∈ *set hs*
shows (*t_a*, *A*) ∈ *set* (*dropWhile* (*le t_i*) (*takeWhile* (*leq t_j*) *hs*)) $\longleftrightarrow t_i \leq t_a \wedge t_a \leq t_j$
unfolding *leq-def le-def*
using *assms*
apply (*induction hs*)
apply (*auto simp: leq-def le-def*)
apply (*meson set-dropWhileD set-takeWhileD*)
apply *auto*[1]
using *set-takeWhileD* **apply** *fastforce*
apply (*meson set-dropWhileD*)
done

lemma *strict-sorted-drop-leq-take-leq*:
assumes *strict-sorted* (*map fst hs*)
and (*t_a*, *A*) ∈ *set hs*
shows (*t_a*, *A*) ∈ *set* (*dropWhile* (*leq t_i*) (*takeWhile* (*leq t_j*) *hs*)) $\longleftrightarrow t_i < t_a \wedge t_a \leq t_j$
unfolding *leq-def*
using *assms*
apply (*induction hs*)
apply *auto*
apply (*meson set-dropWhileD set-takeWhileD*)
apply *auto*[1]
using *set-takeWhileD* **apply** *fastforce*
apply (*meson set-dropWhileD*)
done

lemma *strict-sorted-drop-leq-take-le*:
assumes *strict-sorted* (*map fst hs*)
and (*t_a*, *A*) ∈ *set hs*
shows (*t_a*, *A*) ∈ *set* (*dropWhile* (*leq t_i*) (*takeWhile* (*le t_j*) *hs*)) $\longleftrightarrow t_i < t_a \wedge t_a < t_j$
unfolding *leq-def le-def*
using *assms*
apply (*induction hs*)
apply *auto*
apply (*meson set-dropWhileD set-takeWhileD*)
apply *auto*[1]
using *set-takeWhileD* **apply** *fastforce*
apply (*meson set-dropWhileD*)
done

lemma *split-list2-le*:

assumes *strict-sorted xs*

and $x_1 \in \text{set } xs$

and $x_2 \in \text{set } xs$

shows $x_1 < x_2 \longleftrightarrow (\exists xs_1 \ xs_2 \ xs_3. xs = xs_1 @ x_1 \# xs_2 @ x_2 \# xs_3)$

proof

assume $x_1 < x_2$

obtain $xs_2' \ xs_3$ **where** $xs = xs_2' @ x_2 \# xs_3$

using $\langle x_2 \in \text{set } xs \rangle$

by (*meson split-list*)

then have $\forall x \in \text{set } xs_3. x_2 < x$

using $\langle \text{strict-sorted } xs \rangle$ *sorted-wrt-append*[**where** $xs = xs_2'$ **and** $ys = x_2 \# xs_3$]

by (*simp add: $\langle xs = xs_2' @ x_2 \# xs_3 \rangle$*)

then have $x_1 \in \text{set } xs_2'$

using $\langle x_1 < x_2 \rangle \ \langle x_1 \in \text{set } xs \rangle \ \langle xs = xs_2' @ x_2 \# xs_3 \rangle$ **by** *auto*

then obtain $xs_1 \ xs_2$ **where** $xs_2' = xs_1 @ x_1 \# xs_2$

by (*meson split-list*)

then show $\exists xs_1 \ xs_2 \ xs_3. xs = xs_1 @ x_1 \# xs_2 @ x_2 \# xs_3$

using $\langle xs = xs_2' @ x_2 \# xs_3 \rangle \ \langle xs_2' = xs_1 @ x_1 \# xs_2 \rangle$ **by** *auto*

next

assume $\exists xs_1 \ xs_2 \ xs_3. xs = xs_1 @ x_1 \# xs_2 @ x_2 \# xs_3$

then obtain $xs_1 \ xs_2 \ xs_3$ **where** $xs = xs_1 @ x_1 \# xs_2 @ x_2 \# xs_3$

by *auto*

then show $x_1 < x_2$

using $\langle \text{strict-sorted } xs \rangle$

sorted-wrt-append[**where** $xs = xs_1 @ x_1 \# xs_2$ **and** $ys = x_2 \# xs_3$]

by (*simp add: $\langle xs = xs_1 @ x_1 \# xs_2 @ x_2 \# xs_3 \rangle$*)

qed

lemma *drop-take-leq_i-leq-leq_j-aux*:

assumes $t_k \leq t_j$

and *strict-sorted (map fst hs)*

shows $\text{takeWhile } (\text{leq } t_k) \ hs @ \text{takeWhile } (\text{leq } t_j) \ (\text{dropWhile } (\text{leq } t_k) \ hs) = \text{takeWhile } (\text{leq } t_j) \ hs$

using *assms* **by** (*induction hs*) (*auto simp: leq-def*)

lemma *drop-take-leq_i-leq-leq_j-simp*:

assumes $t_i \leq t_k$ **and** $t_k \leq t_j$

and *strict-sorted (map fst hs)*

shows $\text{dropWhile } (\text{leq } t_i) \ (\text{takeWhile } (\text{leq } t_k) \ hs) @ \text{dropWhile } (\text{leq } t_k) \ (\text{takeWhile } (\text{leq } t_j) \ hs)$

$= \text{dropWhile } (\text{leq } t_i) \ (\text{takeWhile } (\text{leq } t_j) \ hs)$

using $\langle t_i \leq t_k \rangle$ *drop-take-leq_i-leq-leq_j-aux*[*OF* $\langle t_k \leq t_j \rangle \ \langle \text{strict-sorted } (\text{map fst } hs) \rangle$]

strict-sorted-drop-leq-take-leq-comm[*OF* $\langle t_k \leq t_j \rangle \ \langle \text{strict-sorted } (\text{map fst } hs) \rangle$]

by (*induction hs*) (*auto simp: leq-def*)

3.1 State-Transition Based Semantics

context *ast-problem* **begin**

fun *inst-cond* :: *ast-action-schema* $\Rightarrow$ *object list* $\Rightarrow$ *temporal-annotation* $\Rightarrow$ (*object atom*) *formula* **where**

inst-cond (*Durative-Action-Schema* *n* *params* *d* *cond* *eff*) *args* *ts* =
 (let *tsubst* = *subst-term* (*the* *o* (*map-of* (*zip* (*map* *fst* *params*) *args*))) in
 (*map-formula* *o* *map-atom*) *tsubst* (*BigAnd* (*filter-time-spec* *ts* *cond*)))

fun *res-inst-inv* :: *plan-action* $\Rightarrow$ (*object atom*) *formula* *option* **where**

res-inst-inv (*Simple-Plan-Action* *n* *args*) = *None*

| *res-inst-inv* (*Durative-Plan-Action* *n* *args* *d*) =
Some (*inst-cond* (*the* (*resolve-action-schema* *n*)) *args* *Over-All*)

lemma *res-inst-inv-refine*:

assumes *wf-plan-action* π

and *Some* *a* = *res-inst-snap-action* π *Over-All*

shows *Some* (*precondition* *a*) = *res-inst-inv* π

using *assms*

by (*cases* π) (*auto simp: Let-def split: option.splits ast-action-schema.splits*)

Set of all ground actions that, are executed at a given time point in the induced happening sequence.

definition *acts-of-plan-at* :: *time* $\Rightarrow$ *plan* $\Rightarrow$ *ground-action set* **where**

acts-of-plan-at t_i π s =

$\{a_\pi. \exists \pi. (t_i, \pi) \in \text{simple-acts } \pi \text{ s} \wedge \text{Some } a_\pi = \text{res-inst } \pi\}$

$\cup \{a_{start}. \exists \pi. (t_i, \pi) \in \text{durative-acts } \pi \text{ s} \wedge \text{Some } a_{start} = \text{res-inst-snap-action } \pi \text{ At-Start}\}$

$\cup \{a_{end}. \exists t' \pi. (t', \pi) \in \text{durative-acts } \pi \text{ s} \wedge t_i = t' + \text{duration } \pi \wedge \text{Some } a_{end} = \text{res-inst-snap-action } \pi \text{ At-End}\}$

Set of all invariant that have to hold in between consecutive happening time points $t' < t$.

definition *invs-of-plan-at* :: *time* $\Rightarrow$ *plan* $\Rightarrow$ (*object atom*) *formula set* **where**

invs-of-plan-at t_i π s =

$\{inv. \exists t_\pi \pi. (t_\pi, \pi) \in \text{durative-acts } \pi \text{ s} \wedge t_\pi < t_i \wedge t_i \leq t_\pi + (\text{duration } \pi) \wedge$

Some *inv* = *res-inst-inv* $\pi\}$

Predicate for sequence of happening time points.

fun *apply-eff* :: *ground-action set* $\Rightarrow$ *world-model* $\Rightarrow$ *world-model* **where**

apply-eff A_i *M* = (*M* - $\bigcup$ (*set* 'dels' *effect* 'A_i')) $\cup$ $\bigcup$ (*set* 'adds' *effect* 'A_i')

Instead of explicitly defining an induced happening sequence. Here the happening sequence is only implicitly constructed. Only happening time points are considered for the valid execution, invariants are treated like preconditions of an entire happening.

fun *valid-state-seq* :: *world-model* $\Rightarrow$ *time list* $\Rightarrow$ *plan* $\Rightarrow$ *world-model* $\Rightarrow$ *bool*
where
valid-state-seq $M \sqcup \pi s M' \longleftrightarrow (M = M')$
| *valid-state-seq* $M (t_i \# ts) \pi s M' \longleftrightarrow$
(*let* $A_i = \text{acts-of-plan-at } t_i \pi s$ *in*
 $(\forall i \in \text{invs-of-plan-at } t_i \pi s. M \stackrel{c}{\models} i)$
 $\wedge (\forall a \in A_i. M \stackrel{c}{\models} \text{precondition } a)$
 $\wedge (\forall a \in A_i. \forall b \in A_i. a \neq b \longrightarrow \text{acts-non-intrf } a \ b)$
 $\wedge \text{valid-state-seq } (\text{apply-eff } A_i \ M) \ ts \ \pi s \ M'$)

definition *valid-plan-from* :: *world-model* $\Rightarrow$ *plan* $\Rightarrow$ *bool* **where**
valid-plan-from $M \ \pi s \longleftrightarrow (\text{wf-plan } \pi s \wedge (\exists \text{https } M'. \text{https-seq } \pi s \ \text{https} \wedge$
valid-state-seq $M \ \text{https } \pi s \ M' \wedge M' \stackrel{c}{\models} (\text{goal } P)))$

definition *valid-plan* :: *plan* $\Rightarrow$ *bool*
where *valid-plan* $\equiv \text{valid-plan-from } I$

Concise definition used in paper:

lemma *valid-plan* $\pi s \equiv \text{wf-plan } \pi s \wedge (\exists \text{https } M'. \text{https-seq } \pi s \ \text{https} \wedge \text{valid-state-seq}$
 $I \ \text{https } \pi s \ M' \wedge M' \stackrel{c}{\models} (\text{goal } P))$
unfolding *valid-plan-def* *valid-plan-from-def* **by** *auto*

3.2 Connecting the State-Transition Based Semantics to Happening Sequence Based Semantics

Justification for Semantic refinement

lemma *apply-happ* $(t, A) \ M = \text{apply-eff } (\text{set } A) \ M$
by *auto*

lemma *valid-state-seq-app-iff*:
valid-state-seq $M_i (ts_1 @ ts_2) \pi s M_k$
 $\longleftrightarrow (\exists M_j. \text{valid-state-seq } M_i \ ts_1 \ \pi s M_j \wedge \text{valid-state-seq } M_j \ ts_2 \ \pi s M_k)$
by (*induction* ts_1 *arbitrary*: $ts_2 \ M_i \ M_k$) (*auto simp: Let-def*)

lemma *valid-state-seq-state-unique*:
assumes *valid-state-seq* $M_0 \ ts \ \pi s \ M_i$ **and** *valid-state-seq* $M_0 \ ts \ \pi s \ M_i'$
shows $M_i = M_i'$
using *assms* **by** (*induction* $M_0 \ ts \ \pi s \ M_i$ *rule: valid-state-seq.induct*) (*auto simp: Let-def*)

acts-of-plan-at returns exactly the set of actions that would be executed for a happening time point in the induced happening sequence

lemma *ind-happ-seq-htp-acts-of-plan-at*:
assumes *ind-happ-seq* $\pi s \ hs$
and *is-htp* $\pi s \ t$
and $(t, A) \in \text{set } hs$
shows $\text{set } A = \text{acts-of-plan-at } t \ \pi s$
proof

```

show set A  $\subseteq$  acts-of-plan-at t  $\pi$ s
proof
  fix a
  assume a  $\in$  set A
  then have  $\exists (t_\pi, \pi) \in$  set  $\pi$ s. inst-of-plan-action  $\pi$ s (t $\pi$ ,  $\pi$ ) (t, a)
    using assms by (auto simp: ind-happ-seq-def)
  then obtain t $\pi$   $\pi$  where (t $\pi$ ,  $\pi$ )  $\in$  set  $\pi$ s and inst-of-plan-action  $\pi$ s (t $\pi$ ,  $\pi$ )
(t, a)
    by auto
  let ?case1 = res-inst  $\pi$  = Some a  $\wedge$  t $\pi$  = t
  let ?case2 = res-inst-snap-action  $\pi$  At-Start = Some a  $\wedge$  t $\pi$  = t
  let ?case3 = res-inst-snap-action  $\pi$  At-End = Some a  $\wedge$  t $\pi$  + duration  $\pi$  = t
  let ?case4 = res-inst-snap-action  $\pi$  Over-All = Some a  $\wedge$ 
    ( $\exists t_i t_j$ . consec-htps  $\pi$ s t $i$  t $j$   $\wedge$  t $\pi$   $\leq$  t $i$   $\wedge$  t $j$   $\leq$  t $\pi$  + (duration  $\pi$ )  $\wedge$  t $i$  < t
 $\wedge$  t < t $j$ )
  have ?case1  $\vee$  ?case2  $\vee$  ?case3  $\vee$  ?case4
    using  $\langle$ inst-of-plan-action  $\pi$ s (t $\pi$ ,  $\pi$ ) (t, a) $\rangle$  by simp
  then show a  $\in$  acts-of-plan-at t  $\pi$ s
    proof (elim disjE)
      assume ?case1
      then show ?thesis
        unfolding acts-of-plan-at-def simple-acts-def is-act-simple-def
        using  $\langle$ (t $\pi$ ,  $\pi$ )  $\in$  set  $\pi$ s $\rangle$  by (cases  $\pi$ ) auto
    next
      assume ?case2
      then show ?thesis
        unfolding acts-of-plan-at-def durative-acts-def is-act-simple-def
        using  $\langle$ (t $\pi$ ,  $\pi$ )  $\in$  set  $\pi$ s $\rangle$  by (cases  $\pi$ ) auto
    next
      assume ?case3
      then show ?thesis
        unfolding acts-of-plan-at-def durative-acts-def is-act-simple-def
        using  $\langle$ (t $\pi$ ,  $\pi$ )  $\in$  set  $\pi$ s $\rangle$  by (cases  $\pi$ ) fastforce+
    next
      assume ?case4
      then show ?thesis
        using assms by (auto simp: consec-htps-def)
    qed
  qed
next
show set A  $\supseteq$  acts-of-plan-at t  $\pi$ s
proof
  fix a
  let ?case1 = a  $\in$  {a $\pi$ .  $\exists \pi$ . (t,  $\pi$ )  $\in$  simple-acts  $\pi$ s  $\wedge$  Some a $\pi$  = res-inst  $\pi$ }
  let ?case2 = a  $\in$  {a $_{start}$ .  $\exists \pi$ . (t,  $\pi$ )  $\in$  durative-acts  $\pi$ s  $\wedge$  Some a $_{start}$  =
res-inst-snap-action  $\pi$  At-Start}
  let ?case3 = a  $\in$  {a $_{end}$ .  $\exists t'$   $\pi$ . (t',  $\pi$ )  $\in$  durative-acts  $\pi$ s  $\wedge$  t = t' + duration  $\pi$ 
 $\wedge$  Some a $_{end}$  = res-inst-snap-action  $\pi$  At-End}
  assume a  $\in$  acts-of-plan-at t  $\pi$ s

```

```

then have ?case1  $\vee$  ?case2  $\vee$  ?case3
  unfolding acts-of-plan-at-def by simp
then show  $a \in \text{set } A$ 
proof (elim disjE)
  assume ?case1
  then obtain  $\pi$  where  $(t, \pi) \in \text{simple-acts } \pi s$  and the  $(\text{res-inst } \pi) = a$ 
    by auto (metis option.sel)
  then have  $\exists as. (t, as) \in \text{set } hs \wedge a \in \text{set } as$ 
    using assms unfolding ind-happ-seq-def by auto
  then obtain  $A'$  where  $(t, A') \in \text{set } hs$  and  $a \in \text{set } A'$ 
    by auto
  have  $A = A'$ 
    using assms  $\langle (t, A') \in \text{set } hs \rangle$  unfolding ind-happ-seq-def
    by (auto simp: strict-sorted-iff distinct-map-fstD)
  then show ?thesis
    using  $\langle a \in \text{set } A' \rangle$  by auto
next
  assume ?case2
  then obtain  $\pi$  where  $(t, \pi) \in \text{durative-acts } \pi s$ 
    and the  $(\text{res-inst-snap-action } \pi \text{ At-Start}) = a$ 
    by auto (metis option.sel)
  then have  $\exists as. (t, as) \in \text{set } hs \wedge a \in \text{set } as$ 
    using assms unfolding ind-happ-seq-def by auto
  then obtain  $A'$  where  $(t, A') \in \text{set } hs$  and  $a \in \text{set } A'$ 
    by auto
  have  $A = A'$ 
    using assms  $\langle (t, A') \in \text{set } hs \rangle$  unfolding ind-happ-seq-def
    by (auto simp: strict-sorted-iff distinct-map-fstD)
  then show ?thesis
    using  $\langle a \in \text{set } A' \rangle$  by auto
next
  assume ?case3
  then obtain  $t' \pi$  where  $(t', \pi) \in \text{durative-acts } \pi s$  and  $t = t' + \text{duration}$ 
    and the  $(\text{res-inst-snap-action } \pi \text{ At-End}) = a$ 
    by auto (metis option.sel)
  then have  $\exists as. (t, as) \in \text{set } hs \wedge a \in \text{set } as$ 
    using assms unfolding ind-happ-seq-def by auto
  then obtain  $A'$  where  $(t, A') \in \text{set } hs$  and  $a \in \text{set } A'$ 
    by auto
  have  $A = A'$ 
    using assms  $\langle (t, A') \in \text{set } hs \rangle$  unfolding ind-happ-seq-def
    by (auto simp: strict-sorted-iff distinct-map-fstD)
  then show ?thesis
    using  $\langle a \in \text{set } A' \rangle$  by auto
qed
qed
qed
end

```

context *wf-ast-problem*
begin

The precondition of every invariant ground action in between two consecutive happening time points is included in *invs-of-plan-at*.

lemma *ind-happ-seq-invs-of-plan-at*:
assumes *wf-plan* πs
and *consec-htps* $\pi s t_i t_j$
and *ind-happ-seq* $\pi s hs$
and $(t, A) \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs))$
and $a \in \text{set } A$
shows $(\text{precondition } a) \in \text{invs-of-plan-at } t_j \pi s$
proof –
let $?hs' = \text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs)$
have *strict-sorted* $(\text{map } \text{fst } hs)$
using $\langle \text{ind-happ-seq } \pi s hs \rangle$ **unfolding** *ind-happ-seq-def* **by** *auto*
have $(t, A) \in \text{set } hs$
using *assms* **by** $(\text{meson } \text{set-dropWhileD } \text{set-takeWhileD})$
then have $\exists (t_\pi, \pi) \in \text{set } \pi s. \text{inst-of-plan-action } \pi s (t_\pi, \pi) (t, a)$
using *assms* **by** $(\text{auto simp: ind-happ-seq-def})$
then obtain $t_\pi \pi$ **where** $(t_\pi, \pi) \in \text{set } \pi s$ **and** *inst-of-plan-action* $\pi s (t_\pi, \pi)$
 (t, a)
by *auto*
then have *is-htp* $\pi s t_\pi$
unfolding *is-htp-def* **by** *auto*
have $\neg \text{is-htp } \pi s t$

thm *sorted-take* $[OF \text{ sorted-drop}]$
thm *wf-ast-problem.strict-sorted-drop-leq-take-le'*

using *assms* *strict-sorted-drop-leq-take-le'* $[OF \langle \text{strict-sorted } (\text{map } \text{fst } hs) \rangle]$
unfolding *consec-htps-def* **by** *auto*
let $?case1 = \text{res-inst } \pi = \text{Some } a \wedge t_\pi = t$
let $?case2 = \text{res-inst-snap-action } \pi \text{ At-Start} = \text{Some } a \wedge t_\pi = t$
let $?case3 = \text{res-inst-snap-action } \pi \text{ At-End} = \text{Some } a \wedge t_\pi + \text{duration } \pi = t$
let $?case4 = \text{res-inst-snap-action } \pi \text{ Over-All} = \text{Some } a \wedge$
 $(\exists t_i t_j. \text{consec-htps } \pi s t_i t_j \wedge t_\pi \leq t_i \wedge t_j \leq t_\pi + (\text{duration } \pi) \wedge t_i < t$
 $\wedge t < t_j)$
have $?case1 \vee ?case2 \vee ?case3 \vee ?case4$
using $\langle \text{inst-of-plan-action } \pi s (t_\pi, \pi) (t, a) \rangle$ **by** *simp*
then show *?thesis*
proof $(\text{elim } \text{disjE})$
assume $?case1$
then show *?thesis*
using *assms* $\langle \neg \text{is-htp } \pi s t \rangle \langle \text{is-htp } \pi s t_\pi \rangle$ **by** *auto*
next
assume $?case2$
then show *?thesis*

```

    using assms ⟨¬is-htp π s t⟩ ⟨is-htp π s tπ⟩ by auto
next
  assume ?case3
  then have (tπ, π) ∈ durative-acts π s
    unfolding durative-acts-def is-act-simple-def
    using ⟨(tπ, π) ∈ set π s⟩ by (cases π) auto
  then have is-htp π s (tπ + duration π)
    unfolding is-htp-def by auto
  then show ?thesis
    using ⟨?case3⟩ ⟨¬is-htp π s t⟩ by auto
next
  assume ?case4
  obtain ti' tj' where consec-htps π s ti' tj'
    and tπ ≤ ti' ∧ ti' < tπ + duration π
    and ti' < t ∧ t < tj'
  using ⟨?case4⟩ by auto
  have (tπ, π) ∈ durative-acts π s
    unfolding durative-acts-def is-act-simple-def
    using ⟨?case4⟩ ⟨(tπ, π) ∈ set π s⟩ by (cases π) auto
  then have is-htp π s (tπ + duration π)
    unfolding is-htp-def by auto
  have ti < t ∧ t < tj
    using assms
    strict-sorted-drop-leq-take-le'[OF ⟨strict-sorted (map fst hs)⟩]
    by simp
  then have (ti ≤ ti' ∧ ti' ≤ tj) ∨ (ti' ≤ ti ∧ ti ≤ tj')
    using ⟨ti' < t ∧ t < tj'⟩ by auto
  then have ti' = ti and tj' = tj
    using ⟨consec-htps π s ti tj⟩ ⟨consec-htps π s ti' tj'⟩ ⟨ti < t ∧ t < tj⟩ ⟨ti'
    < t ∧ t < tj'⟩
    unfolding consec-htps-def by auto (metis dual-order.strict-trans eq-iff
    leD)+
  then have tπ ≤ ti ∧ ti < tπ + duration π
    using ⟨tπ ≤ ti' ∧ ti' < tπ + duration π⟩ by simp
  then have tπ < tj ∧ tj ≤ tπ + (duration π)
    using assms ⟨is-htp π s (tπ + duration π)⟩ unfolding consec-htps-def by
    auto

  have Some (precondition a) = res-inst-inv π
    using assms ⟨(tπ, π) ∈ set π s⟩ ⟨?case4⟩ res-inst-inv-refine unfolding
    wf-plan-def by auto

  then show ?thesis
    unfolding invs-of-plan-at-def
    using ⟨(tπ, π) ∈ durative-acts π s⟩
    ⟨tπ < tj ∧ tj ≤ tπ + (duration π)⟩
    ⟨Some (precondition a) = res-inst-inv π⟩
    by auto
qed

```

qed

Every invariant from *invs-of-plan-at* is placed as an invariant ground action in between two consecutive happening time points in the induced happening sequence.

lemma *invs-of-plan-at-ind-happ-seq*:
assumes *wf-plan* πs
and *consec-htps* $\pi s t_i t_j$
and *ind-happ-seq* $\pi s hs$
and $i \in \text{invs-of-plan-at } t_j \pi s$
shows $\exists (t, A) \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs)). \exists a \in \text{set } A. i$
 $= \text{precondition } a$
proof –
let $?hs' = \text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs)$
have *strict-sorted* $(\text{map fst } hs)$
using $\langle \text{ind-happ-seq } \pi s hs \rangle$ **unfolding** *ind-happ-seq-def* **by** *auto*
obtain $t_\pi \pi$ **where** $(t_\pi, \pi) \in \text{durative-acts } \pi s$ **and** $t_\pi < t_j \wedge t_j \leq t_\pi + (\text{duration } \pi)$
 $\pi)$
and *Some* $i = \text{res-inst-inv } \pi$
using *assms* **unfolding** *invs-of-plan-at-def* **by** *auto*
then have *is-htp* $\pi s t_\pi$
unfolding *is-htp-def* **by** $(\text{auto simp: durative-acts-def})$
then have $t_\pi \leq t_i \wedge t_j \leq t_\pi + (\text{duration } \pi)$
using *assms* $\langle t_\pi < t_j \wedge t_j \leq t_\pi + (\text{duration } \pi) \rangle$ **unfolding** *consec-htps-def*
by *auto*
then have $\exists (t', as) \in \text{set } hs. t_i < t' \wedge t' < t_j \wedge \text{the } (\text{res-inst-snap-action } \pi$
 $\text{Over-All}) \in \text{set } as$
using *assms* $\langle (t_\pi, \pi) \in \text{durative-acts } \pi s \rangle$ **unfolding** *ind-happ-seq-def* **by** *auto*
then obtain $t A$ **where** $(t, A) \in \text{set } hs$ **and** $t_i < t \wedge t < t_j$
and *the* $(\text{res-inst-snap-action } \pi \text{ Over-All}) \in \text{set } A$
by *auto*
obtain a **where** *Some* $a = \text{res-inst-snap-action } \pi \text{ Over-All}$
using $\langle \text{Some } i = \text{res-inst-inv } \pi \rangle$
by $(\text{metis option.discI res-inst-inv.simps(1) res-inst-snap-action.elims})$
then have $a \in \text{set } A$
using $\langle \text{the } (\text{res-inst-snap-action } \pi \text{ Over-All}) \in \text{set } A \rangle$ **by** $(\text{metis option.sel})$
have $(t, A) \in \text{set } ?hs'$
using $\langle t_i < t \wedge t < t_j \rangle \langle (t, A) \in \text{set } hs \rangle$
 $\text{strict-sorted-drop-leq-take-le}[OF \langle \text{strict-sorted } (\text{map fst } hs) \rangle]$
by *auto*
have *wf-plan-action* π
using *assms* $\langle (t_\pi, \pi) \in \text{durative-acts } \pi s \rangle$ **unfolding** *wf-plan-def*
by $(\text{auto simp: durative-acts-def})$
then have *Some* $(\text{precondition } a) = \text{res-inst-inv } \pi$
using *assms* $\langle \text{Some } a = \text{res-inst-snap-action } \pi \text{ Over-All} \rangle$ *res-inst-inv-refine*
by *auto*
then have $i = \text{precondition } a$

```

    using ⟨Some i = res-inst-inv π⟩
    by (metis option.sel)
  show ?thesis
    using ⟨(t,A) ∈ set ?hs'⟩ ⟨a ∈ set A⟩ ⟨i = precondition a⟩ by auto
qed

```

In an induced happening sequence all ground actions between two consecutive happening time points have empty effect. Those ground actions are the instantiated durative action for the (`over all ...`) time specifier.

```

lemma ind-happ-seq-leqi-lej-empty-eff:
  assumes consec-htps π s ti tj
    and wf-plan π s
    and ind-happ-seq π s hs
    and (ta,A) ∈ set (dropWhile (leq ti) (takeWhile (le tj) hs))
    and a ∈ set A
  shows effect a = Effect [] []
proof -
  let ?hs' = dropWhile (leq ti) (takeWhile (le tj) hs)
  have strict-sorted (map fst hs)
    using ⟨ind-happ-seq π s hs⟩ unfolding ind-happ-seq-def by auto
  then have (ta,A) ∈ set hs
    using ⟨(ta,A) ∈ set ?hs'⟩ by (meson set-dropWhileD set-takeWhileD)
  have ti < ta ∧ ta < tj
    using ⟨(ta,A) ∈ set ?hs'⟩
      strict-sorted-drop-leq-take-le[OF ⟨strict-sorted (map fst hs)⟩ ⟨(ta,A) ∈ set
hs⟩]
    by auto
  have ¬is-htp π s ta
    using ⟨ti < ta ∧ ta < tj⟩ ⟨consec-htps π s ti tj⟩ unfolding consec-htps-def
  by auto
  have A ≠ []
    using ⟨ind-happ-seq π s hs⟩ ⟨(ta,A) ∈ set hs⟩ unfolding ind-happ-seq-def by
  auto
  have ∃ (tπ,π) ∈ set π s. inst-of-plan-action π s (tπ,π) (ta,a)
    using ⟨ind-happ-seq π s hs⟩ ⟨(ta,A) ∈ set hs⟩ ⟨a ∈ set A⟩ unfolding
ind-happ-seq-def by auto
  then obtain tπ π where (tπ,π) ∈ set π s and inst-of-plan-action π s (tπ,π)
(ta,a)
    by auto
  then have is-htp π s tπ
    unfolding is-htp-def by auto
  let ?case1 = res-inst π = Some a ∧ tπ = ta
  let ?case2 = res-inst-snap-action π At-Start = Some a ∧ tπ = ta
  let ?case3 = res-inst-snap-action π At-End = Some a ∧ tπ + (duration π) = ta
  let ?case4 = res-inst-snap-action π Over-All = Some a ∧
    (∃ ti tj. consec-htps π s ti tj ∧ tπ ≤ ti ∧ tj ≤ tπ + (duration π) ∧ ti < ta
    ∧ ta < tj)
  have ?case1 ∨ ?case2 ∨ ?case3 ∨ ?case4
    using ⟨inst-of-plan-action π s (tπ,π) (ta,a)⟩ by simp

```



```

then show ?thesis
proof (elim disjE)
  assume ?case1
  then show ?thesis
    using ⟨¬is-htp π s ta⟩ ⟨is-htp π s tπ⟩ by auto
next
  assume ?case2
  then show ?thesis
    using ⟨¬is-htp π s ta⟩ ⟨is-htp π s tπ⟩ by auto
next
  assume ?case3
  then have (tπ, π) ∈ durative-acts π s
    unfolding durative-acts-def is-act-simple-def
    using ⟨(tπ, π) ∈ set π s⟩ by (cases π) auto
  then have is-htp π s (tπ + (duration π))
    unfolding is-htp-def by auto
  then show ?thesis
    using ⟨?case3⟩ ⟨¬is-htp π s ta⟩ ⟨is-htp π s (tπ + (duration π))⟩ by auto
next
  assume ?case4
  let ?ainv = the (res-inst-snap-action π Over-All)
  have wf-plan-action π
    using ⟨wf-plan π s⟩ ⟨(tπ, π) ∈ set π s⟩
    unfolding wf-plan-def by auto
  obtain aschema where aschema-def: Some aschema = resolve-action-schema
    (name π)
    using ⟨wf-plan-action π⟩ by (cases π) (auto split: option.splits)
  then have aschema ∈ set (actions D)
    unfolding resolve-action-schema-def by (metis index-by-eq-SomeD)
  then have wf-action-schema aschema
    using wf-domain
    unfolding wf-domain-def by auto
  then show ?thesis
    using ⟨?case4⟩ wf-over-all-empty-eff[OF ⟨wf-plan-action π⟩
      ⟨wf-action-schema aschema⟩ aschema-def]
    by auto
qed
qed

```

Applying a happening containing only action with empty effect, doesn't change the current state.

lemma *apply-happ-empty-effect*:

```

assumes ∀ (ta, A) ∈ set hs. ∀ a ∈ set A. effect a = Effect [] []
shows ∀ h ∈ set hs. apply-happ h M = M
using assms by (induction hs) auto

```

Combining $\llbracket wf\text{-ast-problem } ?P; consec\text{-htps } ?\pi s \text{ } ?t_i \text{ } ?t_j; ast\text{-problem}.wf\text{-plan } ?P \text{ } ?\pi s; ast\text{-problem}.ind\text{-happ-seq } ?P \text{ } ?\pi s \text{ } ?hs; (?t_a, ?A) \in set (dropWhile (leq ?t_i) (takeWhile (le ?t_j) ?hs)); ?a \in set ?A \rrbracket \implies ground\text{-action}.effect \text{ } ?a$

$= \text{Effect } [] []$ and $\forall (t_a, A) \in \text{set } ?hs. \forall a \in \text{set } A. \text{ground-action.effect } a = \text{Effect } [] [] \implies \forall h \in \text{set } ?hs. \text{apply-happ } h \text{ } ?M = ?M$: Applying the happening in between two consecutive happening time points doesn't change the current state.

lemma *ind-happ-seq-leq_i-le_j*:
assumes *consec-htps* $\pi s \ t_i \ t_j$
and *wf-plan* πs
and *ind-happ-seq* $\pi s \ hs$
shows $\forall h \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) \ hs)). \text{apply-happ } h \ M$
 $= M$
using *assms apply-happ-empty-effect ind-happ-seq-leq_i-le_j-empty-eff*
by *auto*

lemma *valid-happ-seq-M_i-M_i-precond*:
assumes $\forall h \in \text{set } hs. \text{apply-happ } h \ M_i = M_i$
and *valid-happ-seq* $M_i \ hs \ M_i$
shows $\forall (t_a, as) \in \text{set } hs. \forall a \in \text{set } as. M_i \models_{\text{c}} \text{precondition } a$
using *assms* **by** (*induction* $M_i \ hs \ M_i$ *rule: valid-happ-seq.induct*) *auto*

lemma *valid-happ-seq-leq_i-le_j-aux1*:
assumes *consec-htps* $\pi s \ t_i \ t_j$
and *wf-plan* πs
and *ind-happ-seq* $\pi s \ hs$
and *valid-happ-seq* $M_i (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) \ hs)) \ M_j$
shows $\forall (t_a, as) \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) \ hs)). \forall a \in \text{set } as. M_i \models_{\text{c}} \text{precondition } a$

proof –
let $?hs' = \text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) \ hs)$
have $\forall h \in \text{set } ?hs'. \text{apply-happ } h \ M_i = M_i$
using *assms ind-happ-seq-leq_i-le_j* **by** *simp*
have *valid-happ-seq* $M_i \ ?hs' \ M_i$
using *assms valid-happ-seq-M_j-is-M_i[OF $\forall h \in \text{set } ?hs'. \text{apply-happ } h \ M_i = M_i$]* **by** *auto*
then show *?thesis*
using *assms $\forall h \in \text{set } ?hs'. \text{apply-happ } h \ M_i = M_i$ valid-happ-seq-M_i-M_i-precond*
by *simp*
qed

lemma *valid-happ-seq-leq_i-le_j-aux2*:
assumes $\forall h \in \text{set } hs. \text{apply-happ } h \ M_i = M_i$
and $\forall (t_a, A) \in \text{set } hs. \forall a \in \text{set } A. M_i \models_{\text{c}} \text{precondition } a$
and $\forall (t_a, A) \in \text{set } hs. \forall (t_a', A') \in \text{set } hs. \forall a \in \text{set } A. \forall a' \in \text{set } A'. \text{acts-non-intrf } a \ a'$
shows *valid-happ-seq* $M_i \ hs \ M_i$
using *assms*
by (*induction* hs) (*auto split: prod.splits simp: pairwise-def*)

end — Context *wf-ast-problem*

context *ast-problem*
begin

All invariants from *invs-of-plan-at* are satisfied iff an induced happening sequence is valid in between two consecutive happening time points.

lemma (in *wf-ast-problem*) *valid-happ-seq-leq_i-le_j-invs-of-plan-at*:
assumes *wf-plan* πs
and *consec-htps* πs t_i t_j
and *ind-happ-seq* πs hs
shows *valid-happ-seq* M (*dropWhile* (*leq* t_i) (*takeWhile* (*le* t_j) hs)) M
 $\longleftrightarrow (\forall i \in \text{invs-of-plan-at } t_j \pi s. M \models^c i)$
proof
let $?hs' = \text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs)$
assume *valid-happ-seq* M $?hs'$ M
then show $\forall i \in \text{invs-of-plan-at } t_j \pi s. M \models^c i$
using *assms* *invs-of-plan-at-ind-happ-seq*[*OF* *assms*]
valid-happ-seq-leq_i-le_j-aux1
by *fastforce*
next
let $?hs' = \text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs)$
assume $\forall i \in \text{invs-of-plan-at } t_j \pi s. M \models^c i$
have 1: $\forall h \in \text{set } ?hs'. \text{apply-happ } h M = M$
using *assms* *ind-happ-seq-leq_i-le_j* **by** *simp*
have 2: $\forall (t_a, A) \in \text{set } ?hs'. \forall a \in \text{set } A. M \models^c \text{precondition } a$
using $\langle \forall i \in \text{invs-of-plan-at } t_j \pi s. M \models^c i \rangle$ *ind-happ-seq-invs-of-plan-at*[*OF* *assms*] **by** *auto*
have $\forall (t_a, A) \in \text{set } ?hs'. \forall a \in \text{set } A. \text{effect } a = \text{Effect } [] []$
using *assms* *ind-happ-seq-leq_i-le_j-empty-eff* **by** *auto*
then have 3: $\forall (t_a, A) \in \text{set } ?hs'. \forall (t_a', A') \in \text{set } ?hs'.$
 $\forall a \in \text{set } A. \forall a' \in \text{set } A'. \text{acts-non-intrf } a a'$
unfolding *acts-non-intrf-def* **by** (*auto simp: Let-def*)
show *valid-happ-seq* M $?hs'$ M
using *valid-happ-seq-leq_i-le_j-aux2*[*OF* 1 2 3] **by** *auto*
qed

lemma *drop-take-leq_i-le-leq_j-aux*:
assumes $t_k \leq t_j$
and *strict-sorted* (*map fst* hs)
shows *takeWhile* (*le* t_k) hs @ *takeWhile* (*leq* t_j) (*dropWhile* (*le* t_k) hs) =
takeWhile (*leq* t_j) hs
using *assms* **by** (*induction* hs) (*auto simp: le-def leq-def*)

We can split up a happening sequence via *dropWhile* and *takeWhile*

lemma *drop-take-leq_i-le-leq_j-split-app*:
assumes $t_i < t_k$ **and** $t_k \leq t_j$
and *strict-sorted* (*map fst* hs)
shows *dropWhile* (*leq* t_i) (*takeWhile* (*le* t_k) hs) @ *dropWhile* (*le* t_k) (*takeWhile* (*leq* t_j) hs)
 $= \text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{leq } t_j) hs)$

using $\langle t_i < t_k \rangle$ *drop-take-leq_i-le-leq_j-aux*[*OF* $\langle t_k \leq t_j \rangle$ *strict-sorted* (*map fst* *hs*) $\rangle$]
strict-sorted-drop-le-take-leq-comm[*OF* $\langle t_k \leq t_j \rangle$ *strict-sorted* (*map fst* *hs*) $\rangle$]
by (*induction* *hs*) (*auto simp: le-def leq-def*)

Some properties of *valid-happ-seq* that are needed later.

lemma *valid-happ-seq-app-iff*:
valid-happ-seq M_0 (*hs*₁@*hs*₂) $M_j \longleftrightarrow (\exists M_i. \text{valid-happ-seq } M_0 \text{ } hs_1 \text{ } M_i \wedge \text{valid-happ-seq } M_i \text{ } hs_2 \text{ } M_j)$
by (*induction* *hs*₁ *arbitrary: hs*₂ $M_0 \text{ } M_j$) *auto*

lemma *valid-happ-seq-split-iff*:
valid-happ-seq M_0 *hs* $M_j \longleftrightarrow (\exists M_i. \text{valid-happ-seq } M_0 \text{ (takeWhile } f \text{ } hs) \text{ } M_i \wedge \text{valid-happ-seq } M_i \text{ (dropWhile } f \text{ } hs) \text{ } M_j)$
using *valid-happ-seq-app-iff* **by** *force*

Every happening sequence contains a happening at every happening time point.

lemma *ind-happ-seq-happ-at-htps*:
assumes *is-htp* $\pi s \text{ } t_i$
and *ind-happ-seq* $\pi s \text{ } hs$
shows $\exists A_i. (t_i, A_i) \in \text{set } hs$
proof –
have $(\exists \pi. (t_i, \pi) \in \text{set } \pi s) \vee (\exists (t_\pi, \pi) \in \text{durative-acts } \pi s. t_i = t_\pi + (\text{duration } \pi))$
using $\langle \text{is-htp } \pi s \text{ } t_i \rangle$ **unfolding** *is-htp-def* **by** *simp*
then show *?thesis*
proof
assume $\exists \pi. (t_i, \pi) \in \text{set } \pi s$
then obtain π **where** $(t_i, \pi) \in \text{set } \pi s$
by *auto*
then show *?thesis*
using $\langle \text{ind-happ-seq } \pi s \text{ } hs \rangle$
unfolding *ind-happ-seq-def durative-acts-def simple-acts-def*
apply (*cases* π) **by** *auto blast+*
next
assume $\exists (t_\pi, \pi) \in \text{durative-acts } \pi s. t_i = t_\pi + (\text{duration } \pi)$
then obtain $t_\pi \text{ } \pi$ **where** $(t_\pi, \pi) \in \text{durative-acts } \pi s$ **and** $t_i = t_\pi + (\text{duration } \pi)$
by *auto*
then show *?thesis*
using $\langle \text{ind-happ-seq } \pi s \text{ } hs \rangle$
unfolding *ind-happ-seq-def durative-acts-def simple-acts-def*
apply (*cases* π) **by** *auto force+*
qed
qed

lemma *distinct-single-set-list*: $set\ xs = \{x\} \implies distinct\ xs \implies xs = [x]$
by (*induction xs*) *auto*

An induced happening sequence contain exactly one happening for every happening time point.

lemma *ind-happ-seq-le_i-leq_i*:
assumes *is-htp* $\pi s\ t_i$
and *ind-happ-seq* $\pi s\ hs$
shows $\exists A_i. dropWhile\ (le\ t_i)\ (takeWhile\ (leq\ t_i)\ hs) = [(t_i, A_i)]$
proof –
have *strict-sorted* $(map\ fst\ hs)$
using $\langle ind-happ-seq\ \pi s\ hs \rangle$
unfolding *ind-happ-seq-def* **by** *auto*
let $?hs' = dropWhile\ (le\ t_i)\ (takeWhile\ (leq\ t_i)\ hs)$
have $set\ ?hs' \subseteq set\ hs$
using *set-dropWhileD set-takeWhileD* **by** *fastforce*
have *distinct* hs
using $\langle strict-sorted\ (map\ fst\ hs) \rangle\ distinct-mapI$ **by** (*auto simp: strict-sorted-iff*)
then have *distinct* $?hs'$
using *distinct-dropWhile distinct-takeWhile* **by** *auto*
obtain A_i **where** $(t_i, A_i) \in set\ hs$
using *ind-happ-seq-happ-at-htps* [*OF* $\langle is-htp\ \pi s\ t_i \rangle\ \langle ind-happ-seq\ \pi s\ hs \rangle]$ **by** *auto*
then have $(t_i, A_i) \in set\ ?hs'$
unfolding *le-def leq-def*
using $\langle strict-sorted\ (map\ fst\ hs) \rangle$ **by** (*induction hs*) *auto*
have $\forall (t_a, A) \in set\ ?hs'. t_a = t_i$
using $\langle set\ ?hs' \subseteq set\ hs \rangle\ strict-sorted-drop-le-take-leq$ [*OF* $\langle strict-sorted\ (map\ fst\ hs) \rangle]$
by *auto*
have $\forall (t_1, A_1) \in set\ hs. \forall (t_2, A_2) \in set\ hs. t_1 = t_2 \longrightarrow A_1 = A_2$
using $\langle strict-sorted\ (map\ fst\ hs) \rangle$ **by** (*auto simp: strict-sorted-iff distinct-map-fstD*)
then have $\forall (t_1, A_1) \in set\ ?hs'. \forall (t_2, A_2) \in set\ ?hs'. t_1 = t_2 \longrightarrow A_1 = A_2$
using $\langle set\ ?hs' \subseteq set\ hs \rangle$ **by** *blast*
then have $\forall (t_a, as') \in set\ ?hs'. t_a = t_i \wedge as' = A_i$
using $\langle (t_i, A_i) \in set\ ?hs' \rangle\ \langle \forall (t_a, as) \in set\ ?hs'. t_a = t_i \rangle$ **by** *auto force*
then have $set\ ?hs' = \{(t_i, A_i)\}$
using $\langle (t_i, A_i) \in set\ ?hs' \rangle$ **by** *blast*
then show *?thesis*
using *distinct-single-set-list* [*OF* $\langle set\ ?hs' = \{(t_i, A_i)\} \rangle\ \langle distinct\ ?hs' \rangle]$ **by** *simp*
qed

invs-of-plan-at is valid for a single happening time point iff any induced happening sequence is valid from the previous happening time point to the happening time point.

lemma (*in wf-ast-problem*) *valid-happ-seq-valid-state-seq-consec-htps*:
assumes *wf-plan* πs
and *consec-htps* $\pi s\ t_i\ t_j$
and *ind-happ-seq* $\pi s\ hs$

shows $\text{valid-happ-seq } M \text{ (dropWhile (leq } t_i) \text{ (takeWhile (leq } t_j) \text{ } hs)) } M'$
 $\longleftrightarrow \text{valid-state-seq } M [t_j] \pi s M'$

proof

let $?hs' = \text{dropWhile (leq } t_i) \text{ (takeWhile (leq } t_j) \text{ } hs)$
assume $\text{valid-happ-seq } M ?hs' M'$

have $\text{is-htp } \pi s t_j$ **and** $t_i < t_j$ **and** $t_j \leq t_j$
using $\langle \text{consec-htps } \pi s t_i t_j \rangle$ **unfolding** consec-htps-def **by** *auto*
have $\text{strict-sorted (map fst } hs)$
using $\langle \text{ind-happ-seq } \pi s hs \rangle$ **unfolding** ind-happ-seq-def **by** *auto*
then have $?hs'$
 $= \text{dropWhile (leq } t_i) \text{ (takeWhile (le } t_j) \text{ } hs) @ \text{dropWhile (le } t_j) \text{ (takeWhile (le } t_j) \text{ } hs)}$
using $\text{drop-take-leq}_i\text{-le-leq}_j\text{-split-app}[OF \langle t_i < t_j \rangle \langle t_j \leq t_j \rangle]$ **by** *fastforce*
then obtain M'' **where** $\text{valid-happ-seq } M \text{ (dropWhile (leq } t_i) \text{ (takeWhile (le } t_j) \text{ } hs)) } M''$
and $\text{valid-happ-seq } M'' \text{ (dropWhile (le } t_j) \text{ (takeWhile (leq } t_j) \text{ } hs)) } M'$
using $\text{valid-happ-seq-app-iff } \langle \text{valid-happ-seq } M ?hs' M' \rangle$ **by** *auto*
then have $M = M''$
using $\text{assms valid-happ-seq-}M_j\text{-is-}M_i[OF \text{ind-happ-seq-leq}_i\text{-le}_j]$ **by** *auto*

have $\text{is-htp } \pi s t_j$
using $\text{assms unfolding consec-htps-def}$ **by** *auto*
obtain A_j **where** $\text{dropWhile (le } t_j) \text{ (takeWhile (leq } t_j) \text{ } hs) = [(t_j, A_j)]$
using $\text{assms ind-happ-seq-le}_i\text{-leq}_i[OF \langle \text{is-htp } \pi s t_j \rangle]$ **by** *auto*
then have $(t_j, A_j) \in \text{set } hs$
by $(\text{metis list.set-intros}(1) \text{ set-dropWhileD set-takeWhileD})$
then have $\text{set } A_j = \text{acts-of-plan-at } t_j \pi s$
using $\text{assms } \langle \text{is-htp } \pi s t_j \rangle \text{ ind-happ-seq-htp-acts-of-plan-at}$ **by** *auto*
let $?as = \text{acts-of-plan-at } t_j \pi s$
have 1: $\forall i \in \text{invs-of-plan-at } t_j \pi s. M \stackrel{c}{\models} i$
and 2: $\forall a \in ?as. M \stackrel{c}{\models} \text{precondition } a$
and 3: $\forall a \in ?as. \forall b \in ?as. a \neq b \longrightarrow \text{acts-non-intrf } a b$
and 4: $\text{apply-eff } ?as M = M'$
using $\text{valid-happ-seq-leq}_i\text{-le}_j\text{-invs-of-plan-at}[OF \text{assms}]$
 $\langle \text{valid-happ-seq } M \text{ (dropWhile (leq } t_i) \text{ (takeWhile (le } t_j) \text{ } hs)) } M'' \rangle$
 $\langle \text{valid-happ-seq } M'' \text{ (dropWhile (le } t_j) \text{ (takeWhile (leq } t_j) \text{ } hs)) } M' \rangle$
 $\langle M = M'' \rangle$
 $\langle \text{dropWhile (le } t_j) \text{ (takeWhile (leq } t_j) \text{ } hs) = [(t_j, A_j)] \rangle$
 $\langle \text{set } A_j = \text{acts-of-plan-at } t_j \pi s \rangle$
by $(\text{auto simp: pairwise-def})$
then show $\text{valid-state-seq } M [t_j] \pi s M'$
by *auto*

next

let $?hs' = \text{dropWhile (leq } t_i) \text{ (takeWhile (leq } t_j) \text{ } hs)$
assume $\text{valid-state-seq } M [t_j] \pi s M'$
let $?A_j = \text{acts-of-plan-at } t_j \pi s$
have 1: $\forall i \in \text{invs-of-plan-at } t_j \pi s. M \stackrel{c}{\models} i$
and 2: $\forall a \in ?A_j. M \stackrel{c}{\models} \text{precondition } a$

```

and 3:  $\forall a \in ?A_j. \forall b \in ?A_j. a \neq b \longrightarrow \text{acts-non-intrf } a \ b$ 
and 4: apply-eff  $?A_j \ M = M'$ 
using  $\langle \text{valid-state-seq } M \ [t_j] \ \pi s \ M' \rangle$ 
by (induction  $M \ [t_j] \ \pi s \ M'$  rule: valid-state-seq.induct) (auto simp: Let-def)

have valid-happ-seq  $M \ (\text{dropWhile } (\text{leq } t_i) \ (\text{takeWhile } (\text{le } t_j) \ hs)) \ M$ 
using 1 valid-happ-seq-leqi-lej-invs-of-plan-at[OF assms] by auto

have is-htp  $\pi s \ t_j$ 
using assms unfolding consec-htps-def by auto
obtain  $A_j$  where  $\text{dropWhile } (\text{le } t_j) \ (\text{takeWhile } (\text{leq } t_j) \ hs) = [(t_j, A_j)]$ 
using assms ind-happ-seq-lei-leqi[OF  $\langle \text{is-htp } \pi s \ t_j \rangle$ ] by auto
then have  $(t_j, A_j) \in \text{set } hs$ 
by (metis list.set-intros(1) set-dropWhileD set-takeWhileD)
then have  $\text{set } A_j = \text{acts-of-plan-at } t_j \ \pi s$ 
using assms  $\langle \text{is-htp } \pi s \ t_j \rangle$  ind-happ-seq-htp-acts-of-plan-at by auto
then have valid-happ-seq  $M \ (\text{dropWhile } (\text{le } t_j) \ (\text{takeWhile } (\text{leq } t_j) \ hs)) \ M'$ 
using 2 3 4  $\langle \text{dropWhile } (\text{le } t_j) \ (\text{takeWhile } (\text{leq } t_j) \ hs) = [(t_j, A_j)] \rangle$  by (auto simp: pairwise-def)

have is-htp  $\pi s \ t_j$  and  $t_i < t_j$  and  $t_j \leq t_j$ 
using  $\langle \text{consec-htps } \pi s \ t_i \ t_j \rangle$  unfolding consec-htps-def by auto
have strict-sorted (map fst hs)
using  $\langle \text{ind-happ-seq } \pi s \ hs \rangle$  unfolding ind-happ-seq-def by auto
then have  $?hs'$ 
 $= \text{dropWhile } (\text{leq } t_i) \ (\text{takeWhile } (\text{le } t_j) \ hs) \ @ \ \text{dropWhile } (\text{le } t_j) \ (\text{takeWhile } (\text{leq } t_j) \ hs)$ 
using drop-take-leqi-le-leqj-split-app[OF  $\langle t_i < t_j \rangle \ \langle t_j \leq t_j \rangle$ ] by fastforce
then show valid-happ-seq  $M \ ?hs' \ M'$ 
using  $\langle \text{valid-happ-seq } M \ (\text{dropWhile } (\text{leq } t_i) \ (\text{takeWhile } (\text{le } t_j) \ hs)) \ M \rangle$ 
 $\langle \text{valid-happ-seq } M \ (\text{dropWhile } (\text{le } t_j) \ (\text{takeWhile } (\text{leq } t_j) \ hs)) \ M' \rangle$ 
valid-happ-seq-app-iff
by auto
qed

lemma cons-consec-htps:
assumes htps-seq  $\pi s \ htps$ 
and  $htps = ts_1 \ @ \ t_i \ \# \ t_j \ \# \ ts_2$ 
shows consec-htps  $\pi s \ t_i \ t_j$ 
proof –
obtain  $ts'$  where  $htps = ts_1 \ @ \ t_i \ \# \ ts' \ @ \ t_j \ \# \ ts_2$  and  $ts' = []$ 
using assms by auto
then have  $\forall t \in \text{set } htps. t_i < t \wedge t < t_j \longrightarrow t \in \text{set } ts'$ 
using assms unfolding htps-seq-def by (auto simp: sorted-append strict-sorted-iff)
then have  $\neg(\exists t \in \text{set } htps. t_i < t \wedge t < t_j)$ 
using  $\langle ts' = [] \rangle$  by auto
then have  $\forall t. t \in \text{set } htps \longrightarrow t \leq t_i \vee t_j \leq t$ 
by auto

```

```

then show ?thesis
  using assms unfolding consec-https-def https-seq-def
  by (auto simp: sorted-append strict-sorted-iff)
qed

lemma (in wf-ast-problem) valid-happ-seq-valid-state-seq-i-to-j-aux:
  fixes  $\pi s$ 
  assumes https-seq  $\pi s$  https
    and https =  $ts_1 @ t_i \# ts_2 @ t_j \# ts_3$ 
    and wf-plan  $\pi s$ 
    and ind-happ-seq  $\pi s$  hs
  shows valid-happ-seq  $M$  (dropWhile (leq  $t_i$ ) (takeWhile (leq  $t_j$ ) hs))  $M'$ 
     $\longleftrightarrow$  valid-state-seq  $M$  ( $ts_2 @ [t_j]$ )  $\pi s$   $M'$ 
  using assms
proof (induction  $ts_2$  arbitrary:  $ts_1$   $t_i$   $ts_3$   $M$ )
  case Nil
  let ?hs' = dropWhile (leq  $t_i$ ) (takeWhile (leq  $t_j$ ) hs)
  have consec-https  $\pi s$   $t_i$   $t_j$ 
    using Nil.premis cons-consec-https by auto
  then show ?case
    using Nil.premis valid-happ-seq-valid-state-seq-consec-https by auto
  next
  case (Cons  $t_i' ts_2$ )
  have strict-sorted https
    using assms unfolding https-seq-def by auto
  then have consec-https  $\pi s$   $t_i$   $t_i'$ 
    using Cons.premis cons-consec-https by auto
  have  $t_i \in \text{set } https$  and  $t_i' \in \text{set } https$  and  $t_j \in \text{set } https$ 
    using Cons.premis by auto
  have  $t_i < t_j$ 
    using Cons.premis split-list2-le[OF <strict-sorted https> < $t_i \in \text{set } https$ > < $t_j \in \text{set } https$ >]
    by blast
  have  $t_i \leq t_i'$ 
    using <consec-https  $\pi s$   $t_i$   $t_i'$ > unfolding consec-https-def by auto
  have https = ( $ts_1 @ [t_i]$ ) @  $t_i' \# ts_2 @ t_j \# ts_3$ 
    using Cons.premis by auto
  then have  $t_i' \leq t_j$ 
    using Cons.premis split-list2-le[OF <strict-sorted https> < $t_i' \in \text{set } https$ > < $t_j \in \text{set } https$ >]
    less-eq-rat-def
    by blast

  let ?hs' = dropWhile (leq  $t_i$ ) (takeWhile (leq  $t_j$ ) hs)
  have strict-sorted (map fst hs)
    using <ind-happ-seq  $\pi s$  hs> unfolding ind-happ-seq-def by auto
  then have hs'-app: ?hs' = dropWhile (leq  $t_i$ ) (takeWhile (leq  $t_i'$ ) hs)
    @ dropWhile (leq  $t_i'$ ) (takeWhile (leq  $t_j$ ) hs)
    using drop-take-leqi-leqj-simp[OF < $t_i \leq t_i'$ > < $t_i' \leq t_j$ >] by fastforce

```



```

show valid-happ-seq  $M$   $?hs'$   $M' \longleftrightarrow \text{valid-state-seq } M ((t_i' \# ts_2) @ [t_j]) \pi s$ 
 $M'$ 
proof
  assume valid-happ-seq  $M$   $?hs'$   $M'$ 
  then obtain  $M''$  where  $ii'$ : valid-happ-seq  $M$  (dropWhile (leq  $t_i$ ) (takeWhile
(leq  $t_i'$ )  $hs$ ))  $M''$ 
    and  $i'j$ : valid-happ-seq  $M''$  (dropWhile (leq  $t_i'$ ) (takeWhile (leq
 $t_j$ )  $hs$ ))  $M'$ 
    using hs'-app valid-happ-seq-app-iff by auto
    then have 1: valid-state-seq  $M$   $[t_i'] \pi s$   $M''$ 
    using Cons.premis  $\langle \text{consec-https } \pi s \ t_i \ t_i' \rangle$  valid-happ-seq-valid-state-seq-consec-https
by blast
    have 2: valid-state-seq  $M''$  ( $ts_2 @ [t_j]$ )  $\pi s$   $M'$ 
    using assms Cons.IH  $\langle \text{https} = (ts_1 @ [t_i]) @ t_i' \# ts_2 @ t_j \# ts_3 \rangle$   $i'j$  by
blast
    then show valid-state-seq  $M$   $((t_i' \# ts_2) @ [t_j]) \pi s$   $M'$ 
    using 1 2 valid-state-seq-app-iff [where  $ts_1 = [t_i']$ ] by auto
next
  assume valid-state-seq  $M$   $((t_i' \# ts_2) @ [t_j]) \pi s$   $M'$ 
  then obtain  $M''$  where  $i'$ : valid-state-seq  $M$   $[t_i'] \pi s$   $M''$ 
    and  $ts_2j$ : valid-state-seq  $M''$  ( $ts_2 @ [t_j]$ )  $\pi s$   $M'$ 
    using valid-state-seq-app-iff [where  $ts_1 = [t_i']$ ] by auto
    then have 1: valid-happ-seq  $M$  (dropWhile (leq  $t_i$ ) (takeWhile (leq  $t_i'$ )  $hs$ ))
 $M''$ 
    using Cons.premis  $\langle \text{consec-https } \pi s \ t_i \ t_i' \rangle$  valid-happ-seq-valid-state-seq-consec-https
    by blast
    have 2: valid-happ-seq  $M''$  (dropWhile (leq  $t_i'$ ) (takeWhile (leq  $t_j$ )  $hs$ ))  $M'$ 
    using assms Cons.IH  $\langle \text{https} = (ts_1 @ [t_i]) @ t_i' \# ts_2 @ t_j \# ts_3 \rangle$   $ts_2j$  by
blast
    then show valid-happ-seq  $M$   $?hs'$   $M'$ 
    using hs'-app 1 2 valid-happ-seq-app-iff by auto
qed
qed

```

lemma *prod-list-Nil*: $\forall x_1 \ x_2. (x_1, x_2) \notin \text{set } xs \implies xs = []$
by (*induction xs*) *auto*

An induced happening sequence is only Nil iff the original plan was Nil

lemma *ind-happ-seq-Nil*: *ind-happ-seq* πs $hs \implies \pi s = [] \longleftrightarrow hs = []$
unfolding *ind-happ-seq-def simple-acts-def durative-acts-def is-act-simple-def*
by (*auto simp: prod-list-Nil*)

lemma *https-seq-Nil-iff*:
assumes *https-seq* πs $https$
shows $\pi s = [] \longleftrightarrow https = []$
proof
assume $\pi s = []$
then show $https = []$

```

    using assms dur-acts-subset unfolding htps-seq-def is-htp-def by fastforce
next
  assume htps = []
  show  $\pi s = []$ 
  proof (rule ccontr)
    assume  $\pi s \neq []$ 
    then obtain  $t_\pi \pi$  where  $(t_\pi, \pi) \in \text{set } \pi s$ 
      by (meson prod-list-Nil)
    then have  $t_\pi \in \text{set } \textit{htps}$ 
      using assms unfolding htps-seq-def is-htp-def by auto
    then show False
      using  $\langle \textit{htps} = [] \rangle$  by simp
  qed
qed

lemma non-Nil-sorted-last-max:
  assumes  $(xs::('a::\text{linorder}) \text{ list}) \neq []$ 
    and sorted xs
    and  $x_{\text{max}} = \text{last } xs$ 
  shows  $\forall x \in \text{set } xs. x \leq x_{\text{max}}$ 
  using assms by (induction xs) auto

```

Proof for the existence of a sequence of happening time point for every plan.
 Needed later in the proof for the completeness of the semantics.

```

lemma htps-seq-exists:  $\exists \textit{htps}. \textit{htps-seq } \pi s \textit{ htps}$ 
proof (induction  $\pi s$ )
  case Nil
  then have htps-seq [] []
    unfolding htps-seq-def is-htp-def using dur-acts-subset by fastforce
  then show ?case
    by auto
next
  case (Cons  $\pi \pi s$ )
  then show ?case
  proof (cases  $\pi$ )
    case (Pair  $t_\pi \pi'$ )
    obtain htps' where htps-seq  $\pi s \textit{ htps}'$ 
      using Cons.IH by auto
    then have sorted htps' and distinct htps'
      unfolding htps-seq-def by (auto simp: strict-sorted-iff)

    have  $\forall t. t \in \text{set } \textit{htps}' \longleftrightarrow \textit{is-htp } \pi s t$ 
      using  $\langle \textit{htps-seq } \pi s \textit{ htps}' \rangle$  unfolding htps-seq-def by auto
    then have  $\forall t. t \in \text{set } \textit{htps}' \longrightarrow \textit{is-htp } (\pi \# \pi s) t$ 
      unfolding is-htp-def durative-acts-def is-act-simple-def by auto
    have is-htp  $(\pi \# \pi s) t_\pi$ 
      unfolding is-htp-def using Pair by auto
    show ?thesis
  proof (cases  $\pi'$ )

```

```

case (Simple-Plan-Action n args)
then obtain https where https = insert-insert  $t_\pi$  https'
  by auto
then have strict-sorted https
  using sorted-insert-insert[OF <sorted https'>]
    distinct-insert-insert[OF <distinct https'>]
  by (auto simp: strict-sorted-iff)
have  $\forall t. \text{is-htp } \pi s t \longrightarrow t \in \text{set https}$  and  $t_\pi \in \text{set https}$ 
  using < $\forall t. t \in \text{set https}' \longleftrightarrow \text{is-htp } \pi s t$ >
    <https = insert-insert  $t_\pi$  https'>
  by (auto simp: set-insert-insert)
then have  $\forall t. \text{is-htp } (\pi \# \pi s) t \longrightarrow t \in \text{set https}$ 
  unfolding is-htp-def durative-acts-def is-act-simple-def
  using Pair Simple-Plan-Action by auto
have  $\forall t. t \in \text{set https} \longrightarrow \text{is-htp } (\pi \# \pi s) t$ 
  using < $\forall t. t \in \text{set https}' \longrightarrow \text{is-htp } (\pi \# \pi s) t$ >
    <https = insert-insert  $t_\pi$  https'>
    <is-htp  $(\pi \# \pi s) t_\pi$ >
  by (auto simp: set-insert-insert)
show ?thesis
  using <strict-sorted https>
    < $\forall t. \text{is-htp } (\pi \# \pi s) t \longrightarrow t \in \text{set https}$ >
    < $\forall t. t \in \text{set https} \longrightarrow \text{is-htp } (\pi \# \pi s) t$ >
  by (auto simp: https-seq-def)
next
case (Durative-Plan-Action n args d)
then have is-htp  $(\pi \# \pi s) (t_\pi + d)$ 
  unfolding is-htp-def durative-acts-def is-act-simple-def using Pair by
auto
obtain https where https = insert-insert  $t_\pi$  (insert-insert  $(t_\pi + d)$  https')
  by auto
then have strict-sorted https
  using <sorted https'> <distinct https'>
  by (auto simp: sorted-insert-insert distinct-insert-insert strict-sorted-iff)
have  $\forall t. \text{is-htp } \pi s t \longrightarrow t \in \text{set https}$  and  $t_\pi \in \text{set https}$  and  $(t_\pi + d) \in \text{set}$ 
https
  using < $\forall t. t \in \text{set https}' \longleftrightarrow \text{is-htp } \pi s t$ >
    <https = insert-insert  $t_\pi$  (insert-insert  $(t_\pi + d)$  https')>
  by (auto simp: set-insert-insert)
then have  $\forall t. \text{is-htp } (\pi \# \pi s) t \longrightarrow t \in \text{set https}$ 
  unfolding is-htp-def durative-acts-def is-act-simple-def
  using Pair Durative-Plan-Action by auto
have  $\forall t. t \in \text{set https} \longrightarrow \text{is-htp } (\pi \# \pi s) t$ 
  using < $\forall t. t \in \text{set https}' \longrightarrow \text{is-htp } (\pi \# \pi s) t$ >
    <https = insert-insert  $t_\pi$  (insert-insert  $(t_\pi + d)$  https')>
    <is-htp  $(\pi \# \pi s) t_\pi$ > <is-htp  $(\pi \# \pi s) (t_\pi + d)$ >
  by (auto simp: set-insert-insert)
show ?thesis
  using <strict-sorted https>

```

```

      ⟨∀ t. is-htp (π#πs) t ⟶ t ∈ set https⟩
      ⟨∀ t. t ∈ set https ⟶ is-htp (π#πs) t⟩
    by (auto simp: https-seq-def)
  qed
qed
qed

end

context wf-ast-problem
begin

Two induced happening sequences contain the same ground actions in between two consecutive happening time points.

lemma ind-happ-seqs-same-acts-leqi-lej:
  assumes consec-https πs ti tj
    and ind-happ-seq πs hs1
    and ind-happ-seq πs hs2
    and (ta, A) ∈ set (dropWhile (leq ti) (takeWhile (le tj) hs1))
    and a ∈ set A
  shows ∃ ta' A'. (ta', A') ∈ set (dropWhile (leq ti) (takeWhile (le tj) hs2)) ∧ a
    ∈ set A'
  proof -
    let ?hs1' = dropWhile (leq ti) (takeWhile (le tj) hs1)
    let ?hs2' = dropWhile (leq ti) (takeWhile (le tj) hs2)
    have strict-sorted (map fst hs1) and strict-sorted (map fst hs2)
      using ⟨ind-happ-seq πs hs1⟩ ⟨ind-happ-seq πs hs2⟩ unfolding ind-happ-seq-def
    by auto
    have (ta, A) ∈ set hs1
      using ⟨(ta, A) ∈ set ?hs1'⟩ by (meson set-dropWhileD set-takeWhileD)
    have ti < ta ∧ ta < tj
      using ⟨(ta, A) ∈ set ?hs1'⟩
      strict-sorted-drop-leq-take-le[OF ⟨strict-sorted (map fst hs1)⟩ ⟨(ta, A) ∈
set hs1⟩]
    by auto
    have ∃ (tπ, π) ∈ set πs. inst-of-plan-action πs (tπ, π) (ta, a)
      using ⟨ind-happ-seq πs hs1⟩ ⟨(ta, A) ∈ set hs1⟩ ⟨a ∈ set A⟩
      unfolding ind-happ-seq-def by auto
    then obtain tπ π where (tπ, π) ∈ set πs and inst-of-plan-action πs (tπ, π)
      (ta, a)
    by auto
    let ?case1 = res-inst π = Some a ∧ tπ = ta
    let ?case2 = res-inst-snap-action π At-Start = Some a ∧ tπ = ta
    let ?case3 = res-inst-snap-action π At-End = Some a ∧ tπ + (duration π) = ta
    let ?case4 = res-inst-snap-action π Over-All = Some a ∧
      (∃ ti tj. consec-https πs ti tj ∧ tπ ≤ ti ∧ tj ≤ tπ + (duration π) ∧ ti < ta
      ∧ ta < tj)
    have ?case1 ∨ ?case2 ∨ ?case3 ∨ ?case4
      using ⟨inst-of-plan-action πs (tπ, π) (ta, a)⟩ by simp

```

```

then show ?thesis
proof (elim disjE)
  assume ?case1
  then have  $(t_\pi, \pi) \in \text{simple-acts } \pi s$ 
    unfolding simple-acts-def is-act-simple-def using  $\langle (t_\pi, \pi) \in \text{set } \pi s \rangle$  by
(cases  $\pi$ ) auto
  then have  $\exists as. (t_\pi, as) \in \text{set } hs_2 \wedge \text{the } (res-inst \ \pi) \in \text{set } as$ 
    using  $\langle ind-happ-seq \ \pi s \ hs_2 \rangle$  unfolding ind-happ-seq-def by auto
  then obtain  $A'$  where  $(t_\pi, A') \in \text{set } hs_2$  and  $\text{the } (res-inst \ \pi) \in \text{set } A'$ 
    by auto
  then have  $(t_a, A') \in \text{set } hs_2$ 
    using  $\langle ?case1 \rangle$  by auto
  show ?thesis
    using  $\langle ?case1 \rangle \langle t_i < t_a \wedge t_a < t_j \rangle \langle \text{the } (res-inst \ \pi) \in \text{set } A' \rangle$ 
      strict-sorted-drop-leq-take-le[OF  $\langle \text{strict-sorted } (map \ fst \ hs_2) \rangle \langle (t_a, A') \in$ 
set  $hs_2 \rangle$ ]
    by auto
next
  assume ?case2
  let  $?a_{start} = \text{the } (res-inst-snap-action \ \pi \ At-Start)$ 
  have  $(t_\pi, \pi) \in \text{durative-acts } \pi s$ 
    unfolding durative-acts-def is-act-simple-def
    using  $\langle ?case2 \rangle \langle (t_\pi, \pi) \in \text{set } \pi s \rangle$  by (cases  $\pi$ ) auto
  then have  $\exists as. (t_\pi, as) \in \text{set } hs_2 \wedge ?a_{start} \in \text{set } as$ 
    using  $\langle ind-happ-seq \ \pi s \ hs_2 \rangle$  unfolding ind-happ-seq-def by auto
  then obtain  $A'$  where  $(t_\pi, A') \in \text{set } hs_2$  and  $?a_{start} \in \text{set } A'$ 
    by auto
  then have  $(t_a, A') \in \text{set } hs_2$ 
    using  $\langle ?case2 \rangle$  by auto
  show ?thesis
    using  $\langle ?case2 \rangle \langle t_i < t_a \wedge t_a < t_j \rangle \langle ?a_{start} \in \text{set } A' \rangle$ 
      strict-sorted-drop-leq-take-le[OF  $\langle \text{strict-sorted } (map \ fst \ hs_2) \rangle \langle (t_a, A') \in$ 
set  $hs_2 \rangle$ ]
    by auto
next
  assume ?case3
  let  $?a_{end} = \text{the } (res-inst-snap-action \ \pi \ At-End)$ 
  have  $(t_\pi, \pi) \in \text{durative-acts } \pi s$ 
    unfolding durative-acts-def is-act-simple-def
    using  $\langle ?case3 \rangle \langle (t_\pi, \pi) \in \text{set } \pi s \rangle$  by (cases  $\pi$ ) auto
  then have  $\exists as. (t_\pi + (\text{duration } \pi), as) \in \text{set } hs_2 \wedge ?a_{end} \in \text{set } as$ 
    using  $\langle ind-happ-seq \ \pi s \ hs_2 \rangle$  unfolding ind-happ-seq-def by auto
  then obtain  $A'$  where  $(t_\pi + (\text{duration } \pi), A') \in \text{set } hs_2$  and  $?a_{end} \in \text{set } A'$ 
    by auto
  then have  $(t_a, A') \in \text{set } hs_2$ 
    using  $\langle ?case3 \rangle$  by auto
  show ?thesis
    using  $\langle ?case3 \rangle \langle t_i < t_a \wedge t_a < t_j \rangle \langle ?a_{end} \in \text{set } A' \rangle$ 
      strict-sorted-drop-leq-take-le[OF  $\langle \text{strict-sorted } (map \ fst \ hs_2) \rangle \langle (t_a, A') \in$ 

```

```

set hs2>]
  by auto
next
  assume ?case4
  let ?ainv=the (res-inst-snap-action  $\pi$  Over-All)
  obtain  $t_i' t_j'$  where consec-htps  $\pi s t_i' t_j'$ 
    and  $t_\pi \leq t_i' \wedge t_j' \leq t_\pi + \text{duration } \pi$ 
    and  $t_i' < t_a \wedge t_a < t_j'$ 
  using <?case4> by auto
  have  $(t_\pi, \pi) \in \text{durative-acts } \pi s$ 
  unfolding durative-acts-def is-act-simple-def
  using <?case4>  $\langle (t_\pi, \pi) \in \text{set } \pi s \rangle$  by (cases  $\pi$ ) auto

  then have  $\exists (t', as) \in \text{set } hs_2. t_i' < t' \wedge t' < t_j' \wedge ?a_{inv} \in \text{set } as$ 
  using <ind-happ-seq  $\pi s hs_2$ > unfolding ind-happ-seq-def
  using <consec-htps  $\pi s t_i' t_j'$ >  $\langle t_\pi \leq t_i' \wedge t_j' \leq t_\pi + \text{duration } \pi \rangle$  by auto
  then obtain  $t_a' A'$  where  $(t_a', A') \in \text{set } hs_2$  and  $t_i' < t_a' \wedge t_a' < t_j'$  and
  ?ainv  $\in \text{set } A'$ 
  by auto
  then have  $(t_a', A') \in \text{set } hs_2$ 
  using <?case4> by auto
  have  $t_i < t_a' \wedge t_a' < t_j$ 
  using assms <consec-htps  $\pi s t_i' t_j'$ >  $\langle t_i < t_a \wedge t_a < t_j \rangle$   $\langle t_i' < t_a \wedge t_a < t_j' \rangle$ 
   $\langle t_i' < t_a' \wedge t_a' < t_j' \rangle$  unfolding consec-htps-def by auto
  show ?thesis
  using <?case4>  $\langle ?a_{inv} \in \text{set } A' \rangle$   $\langle t_i < t_a' \wedge t_a' < t_j \rangle$ 
  strict-sorted-drop-leq-take-le[OF <strict-sorted (map fst  $hs_2$ )>]  $\langle (t_a', A') \in \text{set } hs_2 \rangle$ 
  by auto
qed
qed

```

If one induced happening sequence is a valid happening sequence in between two consecutive happening time points, then every other induced happening sequence is also valid in between those consecutive happening time point.

lemma *valid-happ-seq-leq_i-le_j*:

assumes consec-htps $\pi s t_i t_j$

and wf-plan πs

and ind-happ-seq $\pi s hs_1$

and ind-happ-seq $\pi s hs_2$

and valid-happ-seq M_i (dropWhile (leq t_i) (takeWhile (le t_j) hs_1)) M_j

shows valid-happ-seq M_i (dropWhile (leq t_i) (takeWhile (le t_j) hs_2)) M_j

proof –

let $?hs_1' = \text{dropWhile (leq } t_i) (\text{takeWhile (le } t_j) hs_1)$

let $?hs_2' = \text{dropWhile (leq } t_i) (\text{takeWhile (le } t_j) hs_2)$

have $\forall h \in \text{set } ?hs_1'. \text{apply-happ } h M_i = M_i$

using assms ind-happ-seq-leq_i-le_j by simp

have $M_i = M_j$

using valid-happ-seq- M_j -is- M_i [OF < $\forall h \in \text{set } ?hs_1'. \text{apply-happ } h M_i = M_i$ >]

$\langle \text{valid-happ-seq } M_i \text{ ?hs}_1' M_j \rangle$ **by** *simp*
have $\forall (t_a, A) \in \text{set ?hs}_1'. \forall a \in \text{set } A. M_i \text{ }^c\| = \text{precondition } a$
using *assms valid-happ-seq-leq_i-le_j-aux1* $\langle \text{valid-happ-seq } M_i \text{ ?hs}_1' M_j \rangle$ **by**
simp
have 1: $\forall h \in \text{set ?hs}_2'. \text{apply-happ } h \text{ } M_i = M_i$
using *assms ind-happ-seq-leq_i-le_j* **by** *simp*
have 2: $\forall (t_a, A) \in \text{set ?hs}_2'. \forall a \in \text{set } A. M_i \text{ }^c\| = \text{precondition } a$
using *assms ind-happ-seq-same-acts-leq_i-le_j*
 $\langle \forall (t_a, A) \in \text{set ?hs}_1'. \forall a \in \text{set } A. M_i \text{ }^c\| = \text{precondition } a \rangle$ **by** *blast*
have $\forall (t_a, A) \in \text{set ?hs}_2'. \forall a \in \text{set } A. \text{effect } a = \text{Effect } []$
using *assms ind-happ-seq-leq_i-le_j-empty-eff* **by** *auto*
then have 3: $\forall (t_a, A) \in \text{set ?hs}_2'. \forall (t_a', A') \in \text{set ?hs}_2'.$
 $\forall a \in \text{set } A. \forall a' \in \text{set } A'. \text{acts-non-intrf } a \text{ } a'$
unfolding *acts-non-intrf-def* **by** (*auto simp: Let-def*)
then show *?thesis*
using *valid-happ-seq-leq_i-le_j-aux2*[*OF* 1 2 3] $\langle M_i = M_j \rangle$ **by** *auto*
qed

An induced happening sequence contain exactly one happening for every happening time point.

lemma *ind-happ-seq-le_i-le_i*:
assumes *is-htp* $\pi s \text{ } t_i$
and *ind-happ-seq* $\pi s \text{ } hs$
shows $\exists A_i. \text{dropWhile } (le \text{ } t_i) (\text{takeWhile } (leq \text{ } t_i) \text{ } hs) = [(t_i, A_i)]$
proof –
have *strict-sorted* $(\text{map fst } hs)$
using $\langle \text{ind-happ-seq } \pi s \text{ } hs \rangle$
unfolding *ind-happ-seq-def* **by** *auto*
let $?hs' = \text{dropWhile } (le \text{ } t_i) (\text{takeWhile } (leq \text{ } t_i) \text{ } hs)$
have $\text{set ?hs}' \subseteq \text{set } hs$
using *set-dropWhileD set-takeWhileD* **by** *fastforce*
have *distinct* hs
using $\langle \text{strict-sorted } (\text{map fst } hs) \rangle$ *distinct-mapI* **by** (*auto simp: strict-sorted-iff*)
then have *distinct* $?hs'$
using *distinct-dropWhile distinct-takeWhile* **by** *auto*
obtain A_i **where** $(t_i, A_i) \in \text{set } hs$
using *assms*(1) *assms*(2) *ast-problem.ind-happ-seq-happ-at-htps*
by *blast*
then have $(t_i, A_i) \in \text{set ?hs}'$
unfolding *le-def leq-def*
using $\langle \text{strict-sorted } (\text{map fst } hs) \rangle$ **by** (*induction* hs) *auto*
have $\forall (t_a, A) \in \text{set ?hs}'. t_a = t_i$
using $\langle \text{set ?hs}' \subseteq \text{set } hs \rangle$ *strict-sorted-drop-le-take-leq*[*OF* $\langle \text{strict-sorted } (\text{map fst } hs) \rangle$]
by *auto*
have $\forall (t_1, A_1) \in \text{set } hs. \forall (t_2, A_2) \in \text{set } hs. t_1 = t_2 \longrightarrow A_1 = A_2$
using $\langle \text{strict-sorted } (\text{map fst } hs) \rangle$ **by** (*auto simp: strict-sorted-iff distinct-map-fstD*)
then have $\forall (t_1, A_1) \in \text{set ?hs}'. \forall (t_2, A_2) \in \text{set ?hs}'. t_1 = t_2 \longrightarrow A_1 = A_2$
using $\langle \text{set ?hs}' \subseteq \text{set } hs \rangle$ **by** *blast*

then have $\forall (t_a, as') \in \text{set } ?hs'. t_a = t_i \wedge as' = A_i$
 using $\langle (t_i, A_i) \in \text{set } ?hs' \rangle \langle \forall (t_a, as) \in \text{set } ?hs'. t_a = t_i \rangle$ by auto force
 then have $\text{set } ?hs' = \{(t_i, A_i)\}$
 using $\langle (t_i, A_i) \in \text{set } ?hs' \rangle$ by blast
 then show ?thesis
 using distinct-single-set-list[OF $\langle \text{set } ?hs' = \{(t_i, A_i)\} \rangle \langle \text{distinct } ?hs' \rangle$] by simp
 qed

Two induced happening sequences contain the same ground actions for every happening time point.

lemma *ind-happ-seqs-same-acts-le_i-leq_i*:
 assumes *is-htp* $\pi s t_i$
 and *ind-happ-seq* $\pi s hs_1$
 and *ind-happ-seq* $\pi s hs_2$
 and $(t_a, A) \in \text{set } (\text{dropWhile } (le t_i) (\text{takeWhile } (leq t_i) hs_1))$
 and $a \in \text{set } A$
 shows $\exists t_a' A'. (t_a', A') \in \text{set } (\text{dropWhile } (le t_i) (\text{takeWhile } (leq t_i) hs_2)) \wedge a \in \text{set } A'$
proof –
 have *strict-sorted* (map fst hs_1) and *strict-sorted* (map fst hs_2)
 using $\langle \text{ind-happ-seq } \pi s hs_1 \rangle \langle \text{ind-happ-seq } \pi s hs_2 \rangle$
 unfolding *ind-happ-seq-def* by auto
 let $?hs_1' = \text{dropWhile } (le t_i) (\text{takeWhile } (leq t_i) hs_1)$
 let $?hs_2' = \text{dropWhile } (le t_i) (\text{takeWhile } (leq t_i) hs_2)$
 have $(t_a, A) \in \text{set } hs_1$
 using $\langle (t_a, A) \in \text{set } ?hs_1' \rangle$ by (meson *set-dropWhileD* *set-takeWhileD*)
 have $t_i = t_a$
 using $\langle (t_a, A) \in \text{set } ?hs_1' \rangle$
 strict-sorted-drop-le-take-leq[OF $\langle \text{strict-sorted } (\text{map fst } hs_1) \rangle \langle (t_a, A) \in \text{set } hs_1 \rangle$]
 by auto
 have $\exists (t_\pi, \pi) \in \text{set } \pi s. \text{inst-of-plan-action } \pi s (t_\pi, \pi) (t_a, a)$
 using $\langle \text{ind-happ-seq } \pi s hs_1 \rangle \langle (t_a, A) \in \text{set } hs_1 \rangle \langle a \in \text{set } A \rangle$
 unfolding *ind-happ-seq-def* by auto
 then obtain $t_\pi \pi$ where $(t_\pi, \pi) \in \text{set } \pi s$ and *inst-of-plan-action* $\pi s (t_\pi, \pi) (t_a, a)$
 by auto
 let $?case1 = \text{res-inst } \pi = \text{Some } a \wedge t_\pi = t_a$
 let $?case2 = \text{res-inst-snap-action } \pi \text{ At-Start} = \text{Some } a \wedge t_\pi = t_a$
 let $?case3 = \text{res-inst-snap-action } \pi \text{ At-End} = \text{Some } a \wedge t_\pi + (\text{duration } \pi) = t_a$
 let $?case4 = \text{res-inst-snap-action } \pi \text{ Over-All} = \text{Some } a \wedge$
 $(\exists t_i t_j. \text{consec-htps } \pi s t_i t_j \wedge t_\pi \leq t_i \wedge t_j \leq t_\pi + (\text{duration } \pi) \wedge t_i < t_a \wedge t_a < t_j)$
 have $?case1 \vee ?case2 \vee ?case3 \vee ?case4$
 using $\langle \text{inst-of-plan-action } \pi s (t_\pi, \pi) (t_a, a) \rangle$ by simp
 then show ?thesis
 proof (elim disjE)
 assume ?case1
 then have $(t_\pi, \pi) \in \text{simple-acts } \pi s$


```

      unfolding simple-acts-def is-act-simple-def using  $\langle (t_\pi, \pi) \in \text{set } \pi s \rangle$  by
(cases  $\pi$ ) auto
    then have  $\exists as. (t_\pi, as) \in \text{set } hs_2 \wedge \text{the } (res-inst \ \pi) \in \text{set } as$ 
      using  $\langle ind-happ-seq \ \pi s \ hs_2 \rangle$  unfolding ind-happ-seq-def by auto
    then obtain  $A'$  where  $(t_\pi, A') \in \text{set } hs_2$  and  $\text{the } (res-inst \ \pi) \in \text{set } A'$ 
      by auto
    then have  $(t_a, A') \in \text{set } hs_2$ 
      using  $\langle ?case1 \rangle$  by auto
    show ?thesis
      using  $\langle t_i = t_a \rangle \langle ?case1 \rangle \langle \text{the } (res-inst \ \pi) \in \text{set } A' \rangle$ 
        strict-sorted-drop-le-take-leq[OF  $\langle strict-sorted \ (\text{map } fst \ hs_2) \rangle \langle (t_a, A') \in$ 
set  $hs_2 \rangle$ ]
      by auto
  next
    assume ?case2
    let  $?a_{start} = \text{the } (res-inst-snap-action \ \pi \ At-Start)$ 
    have  $(t_\pi, \pi) \in \text{durative-acts } \pi s$ 
      unfolding durative-acts-def is-act-simple-def
      using  $\langle ?case2 \rangle \langle (t_\pi, \pi) \in \text{set } \pi s \rangle$  by (cases  $\pi$ ) auto
    then have  $\exists as. (t_\pi, as) \in \text{set } hs_2 \wedge ?a_{start} \in \text{set } as$ 
      using  $\langle ind-happ-seq \ \pi s \ hs_2 \rangle$  unfolding ind-happ-seq-def by auto
    then obtain  $A'$  where  $(t_\pi, A') \in \text{set } hs_2$  and  $?a_{start} \in \text{set } A'$ 
      by auto
    then have  $(t_a, A') \in \text{set } hs_2$ 
      using  $\langle ?case2 \rangle$  by auto
    show ?thesis
      using  $\langle t_i = t_a \rangle \langle ?case2 \rangle \langle ?a_{start} \in \text{set } A' \rangle$ 
        strict-sorted-drop-le-take-leq[OF  $\langle strict-sorted \ (\text{map } fst \ hs_2) \rangle \langle (t_a, A') \in$ 
set  $hs_2 \rangle$ ]
      by auto
  next
    assume ?case3
    let  $?a_{end} = \text{the } (res-inst-snap-action \ \pi \ At-End)$ 
    have  $(t_\pi, \pi) \in \text{durative-acts } \pi s$ 
      unfolding durative-acts-def is-act-simple-def
      using  $\langle ?case3 \rangle \langle (t_\pi, \pi) \in \text{set } \pi s \rangle$  by (cases  $\pi$ ) auto
    then have  $\exists as. (t_\pi + (\text{duration } \pi), as) \in \text{set } hs_2 \wedge ?a_{end} \in \text{set } as$ 
      using  $\langle ind-happ-seq \ \pi s \ hs_2 \rangle$  unfolding ind-happ-seq-def by auto
    then obtain  $A'$  where  $(t_\pi + (\text{duration } \pi), A') \in \text{set } hs_2$  and  $?a_{end} \in \text{set } A'$ 
      by auto
    then have  $(t_a, A') \in \text{set } hs_2$ 
      using  $\langle ?case3 \rangle$  by auto
    show ?thesis
      using  $\langle t_i = t_a \rangle \langle ?case3 \rangle \langle ?a_{end} \in \text{set } A' \rangle$ 
        strict-sorted-drop-le-take-leq[OF  $\langle strict-sorted \ (\text{map } fst \ hs_2) \rangle \langle (t_a, A') \in$ 
set  $hs_2 \rangle$ ]
      by auto
  next

```

```

assume  $?case4$ 
obtain  $t_i' t_j'$  where  $consec-htps \pi s t_i' t_j'$ 
    and  $t_\pi \leq t_i' \wedge t_i' < t_\pi + duration \pi$ 
    and  $t_i' < t_a \wedge t_a < t_j'$ 
    using  $\langle ?case4 \rangle$  by auto
have  $is-htp \pi s t_a$ 
    using  $\langle is-htp \pi s t_i \rangle \langle t_i = t_a \rangle$  by simp
show  $?thesis$ 
    using  $\langle is-htp \pi s t_a \rangle \langle t_i' < t_a \wedge t_a < t_j' \rangle \langle consec-htps \pi s t_i' t_j' \rangle$ 
    unfolding  $consec-htps-def$  by auto
qed
qed

```

With $\llbracket is-htp \pi s t_i; ind-happ-seq \pi s ?hs_1; ind-happ-seq \pi s ?hs_2; (?t_a, ?A) \in set (dropWhile (le ?t_i) (takeWhile (leq ?t_i) ?hs_1)); ?a \in set ?A \rrbracket \implies \exists t_a' A'. (t_a', A') \in set (dropWhile (le ?t_i) (takeWhile (leq ?t_i) ?hs_2)) \wedge ?a \in set A'$ we can prove: If we have one induced happening sequence that is a valid happening sequence for a single happening time point, then every other induced happening sequence is also valid for that happening time point.

```

lemma  $valid-happ-seq-le_i-leq_i$ :
  assumes  $is-htp \pi s t_i$ 
    and  $ind-happ-seq \pi s hs_1$ 
    and  $ind-happ-seq \pi s hs_2$ 
    and  $valid-happ-seq M_i (dropWhile (le t_i) (takeWhile (leq t_i) hs_1)) M_j$ 
  shows  $valid-happ-seq M_i (dropWhile (le t_i) (takeWhile (leq t_i) hs_2)) M_j$ 
proof –
  obtain  $A_1$  where  $dropWhile (le t_i) (takeWhile (leq t_i) hs_1) = [(t_i, A_1)]$ 
    using  $assms ind-happ-seq-le_i-leq_i[OF \langle is-htp \pi s t_i \rangle \langle ind-happ-seq \pi s hs_1 \rangle]$  by
auto
  obtain  $A_2$  where  $dropWhile (le t_i) (takeWhile (leq t_i) hs_2) = [(t_i, A_2)]$ 
    using  $assms ind-happ-seq-le_i-leq_i[OF \langle is-htp \pi s t_i \rangle \langle ind-happ-seq \pi s hs_2 \rangle]$  by
auto
  have  $set A_1 = set A_2$ 
    using  $ind-happ-seqs-same-acts-le_i-leq_i[OF \langle is-htp \pi s t_i \rangle \langle ind-happ-seq \pi s hs_1 \rangle$ 
       $\langle ind-happ-seq \pi s hs_2 \rangle$ 
       $\langle dropWhile (le t_i) (takeWhile (leq t_i) hs_1) = [(t_i, A_1)] \rangle$ 
       $ind-happ-seqs-same-acts-le_i-leq_i[OF \langle is-htp \pi s t_i \rangle \langle ind-happ-seq \pi s hs_2 \rangle$ 
       $\langle ind-happ-seq \pi s hs_1 \rangle$ 
       $\langle dropWhile (le t_i) (takeWhile (leq t_i) hs_2) = [(t_i, A_2)] \rangle]$ 
    by auto
  have  $valid-happ-seq M_i [(t_i, A_1)] M_j$ 
    using  $\langle valid-happ-seq M_i (dropWhile (le t_i) (takeWhile (leq t_i) hs_1)) M_j \rangle$ 
       $\langle dropWhile (le t_i) (takeWhile (leq t_i) hs_1) = [(t_i, A_1)] \rangle$  by simp
  then have  $valid-happ-seq M_i [(t_i, A_2)] M_j$ 
    using  $\langle set A_1 = set A_2 \rangle$  by auto
  then show  $?thesis$ 
    using  $\langle dropWhile (le t_i) (takeWhile (leq t_i) hs_2) = [(t_i, A_2)] \rangle$  by auto
qed

```

Putting together $\llbracket wf\text{-}ast\text{-}problem\ ?P; consec\text{-}htps\ ?\pi s\ ?t_i\ ?t_j; ast\text{-}problem.wf\text{-}plan\ ?P\ ?\pi s; ast\text{-}problem.ind\text{-}happ\text{-}seq\ ?P\ ?\pi s\ ?hs_1; ast\text{-}problem.ind\text{-}happ\text{-}seq\ ?P\ ?\pi s\ ?hs_2; valid\text{-}happ\text{-}seq\ ?M_i\ (dropWhile\ (leq\ ?t_i)\ (takeWhile\ (le\ ?t_j)\ ?hs_1))\ ?M_j \rrbracket \implies valid\text{-}happ\text{-}seq\ ?M_i\ (dropWhile\ (leq\ ?t_i)\ (takeWhile\ (le\ ?t_j)\ ?hs_2))\ ?M_j$ and $\llbracket is\text{-}htp\ ?\pi s\ ?t_i; ind\text{-}happ\text{-}seq\ ?\pi s\ ?hs_1; ind\text{-}happ\text{-}seq\ ?\pi s\ ?hs_2; valid\text{-}happ\text{-}seq\ ?M_i\ (dropWhile\ (le\ ?t_i)\ (takeWhile\ (leq\ ?t_i)\ ?hs_1))\ ?M_j \rrbracket \implies valid\text{-}happ\text{-}seq\ ?M_i\ (dropWhile\ (le\ ?t_i)\ (takeWhile\ (leq\ ?t_i)\ ?hs_2))\ ?M_j$. If we have one induced happening sequence that is valid from a happening time point to the next consecutive happening time point, that every other induced happening sequence is also valid for that interval.

lemma *valid-happ-seq-consec-htps*:

assumes *consec-htps* $\pi s\ t_i\ t_j$

and *wf-plan* πs

and *ind-happ-seq* $\pi s\ hs_1$

and *ind-happ-seq* $\pi s\ hs_2$

and *valid-happ-seq* $M_i\ (dropWhile\ (leq\ t_i)\ (takeWhile\ (leq\ t_j)\ hs_1))\ M_j$

shows *valid-happ-seq* $M_i\ (dropWhile\ (leq\ t_i)\ (takeWhile\ (leq\ t_j)\ hs_2))\ M_j$

using *assms*

proof –

have *is-htp* $\pi s\ t_j$ **and** $t_i < t_j$ **and** $t_j \leq t_j$

using $\langle consec\text{-}htps\ \pi s\ t_i\ t_j \rangle$ **unfolding** *consec-htps-def* **by** *auto*

have *strict-sorted* $(map\ fst\ hs_1)$ **and** *strict-sorted* $(map\ fst\ hs_2)$

using $\langle ind\text{-}happ\text{-}seq\ \pi s\ hs_1 \rangle$ $\langle ind\text{-}happ\text{-}seq\ \pi s\ hs_2 \rangle$ **unfolding** *ind-happ-seq-def*

by *auto*

have *dropWhile* $(leq\ t_i)\ (takeWhile\ (leq\ t_j)\ hs_1)$

$= dropWhile\ (leq\ t_i)\ (takeWhile\ (le\ t_j)\ hs_1) @ dropWhile\ (le\ t_j)\ (takeWhile\ (leq\ t_j)\ hs_1)$

using *drop-take-leq_i-le-leq_j-split-app* $[OF\ \langle t_i < t_j \rangle\ \langle t_j \leq t_j \rangle\ \langle strict\text{-}sorted\ (map\ fst\ hs_1) \rangle]$

by *simp*

then obtain M_i' **where** *valid-happ-seq* $M_i\ (dropWhile\ (leq\ t_i)\ (takeWhile\ (le\ t_j)\ hs_1))\ M_i'$

and *valid-happ-seq* $M_i'\ (dropWhile\ (le\ t_j)\ (takeWhile\ (leq\ t_j)\ hs_1))\ M_j$

using *valid-happ-seq-app-iff*

$\langle valid\text{-}happ\text{-}seq\ M_i\ (dropWhile\ (leq\ t_i)\ (takeWhile\ (leq\ t_j)\ hs_1))\ M_j \rangle$ **by** *auto*

have 1: *valid-happ-seq* $M_i\ (dropWhile\ (leq\ t_i)\ (takeWhile\ (le\ t_j)\ hs_2))\ M_i'$

using *valid-happ-seq-leq_i-le_j*

$[OF\ \langle consec\text{-}htps\ \pi s\ t_i\ t_j \rangle\ \langle wf\text{-}plan\ \pi s \rangle\ \langle ind\text{-}happ\text{-}seq\ \pi s\ hs_1 \rangle\ \langle ind\text{-}happ\text{-}seq\ \pi s\ hs_2 \rangle]$

$\langle valid\text{-}happ\text{-}seq\ M_i\ (dropWhile\ (leq\ t_i)\ (takeWhile\ (le\ t_j)\ hs_1))\ M_i' \rangle$ **by** *auto*

have 2: *valid-happ-seq* $M_i'\ (dropWhile\ (le\ t_j)\ (takeWhile\ (leq\ t_j)\ hs_2))\ M_j$

using *valid-happ-seq-le_i-leq_i* $[OF\ \langle is\text{-}htp\ \pi s\ t_j \rangle\ \langle ind\text{-}happ\text{-}seq\ \pi s\ hs_1 \rangle\ \langle ind\text{-}happ\text{-}seq\ \pi s\ hs_2 \rangle]$

$\langle valid\text{-}happ\text{-}seq\ M_i'\ (dropWhile\ (le\ t_j)\ (takeWhile\ (leq\ t_j)\ hs_1))\ M_j \rangle$ **by** *auto*

```

have dropWhile (leq ti) (takeWhile (leq tj) hs2)
    = dropWhile (leq ti) (takeWhile (le tj) hs2) @ dropWhile (le tj) (takeWhile
(leq tj) hs2)
    using drop-take-leqi-le-leqj-split-app[OF ‹ti < tj› ‹tj ≤ tj› ‹strict-sorted (map
fst hs2)›]
    by simp
then show ?thesis
    using valid-happ-seq-app-iff 1 2 by auto
qed

```

```

lemma https-state-trace-unique-i-to-j-aux:
  fixes π s
  assumes https-seq π s https
    and https = ts1 @ ti # ts2 @ tj # ts3
    and wf-plan π s
    and ind-happ-seq π s hs1
    and ind-happ-seq π s hs2
    and valid-happ-seq Mi (dropWhile (leq ti) (takeWhile (leq tj) hs1)) Mj
  shows valid-happ-seq Mi (dropWhile (leq ti) (takeWhile (leq tj) hs2)) Mj
  using assms
proof (induction ts2 arbitrary: ts1 ti ts3 Mi)
  case Nil
  then have consec-https π s ti tj
    using cons-consec-https by auto
  then show ?case
    using valid-happ-seq-consec-https
      [OF ‹consec-https π s ti tj› ‹wf-plan π s› ‹ind-happ-seq π s hs1› ‹ind-happ-seq
π s hs2›]
      ‹valid-happ-seq Mi (dropWhile (leq ti) (takeWhile (leq tj) hs1)) Mj›
    by (simp add: leq-def)
  next
  case (Cons ti' ts2)
  then have strict-sorted https and distinct https
    using assms unfolding https-seq-def by (auto simp: strict-sorted-iff)
  have consec-https π s ti ti'
    using Cons.prem1 cons-consec-https by auto
  have ti ∈ set https and ti' ∈ set https and tj ∈ set https
    using ‹https = ts1 @ ti # (ti' # ts2) @ tj # ts3› by auto
  have ti < tj
    using split-list2-le[OF ‹strict-sorted https› ‹ti ∈ set https› ‹tj ∈ set https›]
      ‹https = ts1 @ ti # (ti' # ts2) @ tj # ts3› by blast
  have ti ≤ ti'
    using ‹consec-https π s ti ti'› unfolding consec-https-def by auto
  have https = (ts1 @ [ti]) @ ti' # ts2 @ tj # ts3
    using Cons.prem1 by auto
  then have ti' ≤ tj
    using Cons.prem1 split-list2-le[OF ‹strict-sorted https› ‹ti' ∈ set https› ‹tj ∈
set https›]

```

```

      less-eq-rat-def by blast
    have strict-sorted (map fst hs1) and strict-sorted (map fst hs2)
      using ⟨ind-happ-seq π s hs1⟩ ⟨ind-happ-seq π s hs2⟩ unfolding ind-happ-seq-def
    by auto
    have ∃ Mi'. valid-happ-seq Mi (dropWhile (leq ti) (takeWhile (leq ti') hs1)) Mi'
      ∧ valid-happ-seq Mi' (dropWhile (leq ti') (takeWhile (leq tj) hs1)) Mj
      using drop-take-leqi-leq-leqj-simp[OF ⟨ti ≤ ti'⟩ ⟨ti' ≤ tj⟩ ⟨strict-sorted (map
fst hs1)⟩]
      valid-happ-seq-app-iff
      ⟨valid-happ-seq Mi (dropWhile (leq ti) (takeWhile (leq tj) hs1)) Mj⟩
      by (metis (no-types, lifting))
    then obtain Mi' where valid-happ-seq Mi (dropWhile (leq ti) (takeWhile (leq
ti') hs1)) Mi'
      and valid-happ-seq Mi' (dropWhile (leq ti') (takeWhile (leq tj) hs1))
Mj
    by auto
    then have valid-happ-seq Mi (dropWhile (leq ti) (takeWhile (leq ti') hs2)) Mi'
      using valid-happ-seq-consec-htps
      [OF ⟨consec-htps π s ti ti'⟩ ⟨wf-plan π s⟩ ⟨ind-happ-seq π s hs1⟩ ⟨ind-happ-seq
π s hs2⟩]
      ⟨valid-happ-seq Mi (dropWhile (leq ti) (takeWhile (leq ti') hs1)) Mi'⟩ by
simp
    have htps = (ts1 @ [ti]) @ ti' # ts2 @ tj # ts3
      using Cons.prems by simp
    then have valid-happ-seq Mi' (dropWhile (leq ti') (takeWhile (leq tj) hs2)) Mj
      using assms Cons.IH
      ⟨valid-happ-seq Mi' (dropWhile (leq ti') (takeWhile (leq tj) hs1)) Mj⟩ by
fastforce
    then show ?case
      using drop-take-leqi-leq-leqj-simp[OF ⟨ti ≤ ti'⟩ ⟨ti' ≤ tj⟩ ⟨strict-sorted (map
fst hs2)⟩]
      valid-happ-seq-app-iff
      ⟨valid-happ-seq Mi (dropWhile (leq ti) (takeWhile (leq ti') hs2)) Mi'⟩
      ⟨valid-happ-seq Mi' (dropWhile (leq ti') (takeWhile (leq tj) hs2)) Mj⟩
      by (metis (no-types, lifting))
  qed

```

With $\llbracket \text{wf-ast-problem } ?P; \text{consec-htps } ?\pi s \text{ } ?t_i \text{ } ?t_j; \text{ast-problem.wf-plan } ?P \text{ } ?\pi s; \text{ast-problem.ind-happ-seq } ?P \text{ } ?\pi s \text{ } ?hs_1; \text{ast-problem.ind-happ-seq } ?P \text{ } ?\pi s \text{ } ?hs_2; \text{valid-happ-seq } ?M_i \text{ (dropWhile (leq } ?t_i \text{) (takeWhile (leq } ?t_j \text{) } ?hs_1)) \text{ } ?M_j \rrbracket \implies \text{valid-happ-seq } ?M_i \text{ (dropWhile (leq } ?t_i \text{) (takeWhile (leq } ?t_j \text{) } ?hs_2)) \text{ } ?M_j$ we can now proof that if one induced happening sequence is valid form a happening time point to another happening time point (not just consecutive), than every other induced happening sequence is valid for between those two happening time points.

lemma *valid-happ-seq-htp-i-to-htp-j*:
assumes $t_i < t_j$

```

    and is-htp  $\pi s$   $t_i$ 
    and is-htp  $\pi s$   $t_j$ 
    and wf-plan  $\pi s$ 
    and ind-happ-seq  $\pi s$   $hs_1$ 
    and ind-happ-seq  $\pi s$   $hs_2$ 
    and valid-happ-seq  $M_i$  ( $\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{leq } t_j) hs_1)$ )  $M_j$ 
  shows valid-happ-seq  $M_i$  ( $\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{leq } t_j) hs_2)$ )  $M_j$ 
proof –
  obtain  $htps$  where  $htps\text{-seq } \pi s$   $htps$ 
  using  $htps\text{-seq-exists}$  by auto
  then have strict-sorted  $htps$  and  $t_i \in \text{set } htps$  and  $t_j \in \text{set } htps$ 
  unfolding  $htps\text{-seq-def}$  using  $assms$  by ( $\text{auto simp: strict-sorted-iff}$ )
  then obtain  $ts_1$   $ts_2$   $ts_3$  where  $htps = ts_1 @ t_i \# ts_2 @ t_j \# ts_3$ 
  using  $\langle t_i < t_j \rangle$   $\text{split-list2-le}[OF \langle \text{strict-sorted } htps \rangle \langle t_i \in \text{set } htps \rangle \langle t_j \in \text{set } htps \rangle]$  by auto
  then show ?thesis
  using  $assms$   $\langle htps\text{-seq } \pi s$   $htps \rangle$   $htps\text{-state-trace-unique-i-to-j-aux}$  by metis
qed

lemma happ-tp-leq- $t_{min}$ - $t_{max}$ :
  assumes  $(t_\pi, \pi) \in \text{set } \pi s$ 
  and inst-of-plan-action  $\pi s$   $(t_\pi, \pi)$   $(t_a, a)$ 
  shows  $(\exists t_j. \text{is-htp } \pi s$   $t_j \wedge t_j \leq t_a) \wedge (\exists t_j. \text{is-htp } \pi s$   $t_j \wedge t_a \leq t_j)$ 
proof –
  let ?case1 =  $\text{res-inst } \pi = \text{Some } a \wedge t_\pi = t_a$ 
  let ?case2 =  $\text{res-inst-snap-action } \pi$   $\text{At-Start} = \text{Some } a \wedge t_\pi = t_a$ 
  let ?case3 =  $\text{res-inst-snap-action } \pi$   $\text{At-End} = \text{Some } a \wedge t_\pi + (\text{duration } \pi) = t_a$ 
  let ?case4 =  $\text{res-inst-snap-action } \pi$   $\text{Over-All} = \text{Some } a \wedge$ 
     $(\exists t_i$   $t_j. \text{consec-htps } \pi s$   $t_i$   $t_j \wedge t_\pi \leq t_i \wedge t_j \leq t_\pi + (\text{duration } \pi) \wedge t_i < t_a$ 
     $\wedge t_a < t_j)$ 
  have ?case1  $\vee$  ?case2  $\vee$  ?case3  $\vee$  ?case4
  using  $\langle \text{inst-of-plan-action } \pi s$   $(t_\pi, \pi)$   $(t_a, a) \rangle$  by simp
  then show ?thesis
  proof (elim disjE)
    assume ?case1
    then show ?thesis
    using  $\langle (t_\pi, \pi) \in \text{set } \pi s \rangle$  by ( $\text{auto simp: is-htp-def}$ )
  next
    assume ?case2
    then show ?thesis
    using  $\langle (t_\pi, \pi) \in \text{set } \pi s \rangle$  by ( $\text{auto simp: is-htp-def}$ )
  next
    assume ?case3
    then show ?thesis
    unfolding is-htp-def durative-acts-def is-act-simple-def
    using  $\langle (t_\pi, \pi) \in \text{set } \pi s \rangle$   $\langle ?case3 \rangle$  by ( $\text{cases } \pi$ ) fastforce+
  next
    assume ?case4
    then obtain  $t_i$   $t_j$  where consec-htps  $\pi s$   $t_i$   $t_j$  and  $t_\pi \leq t_i \wedge t_i < t_\pi +$ 

```

```

(duration  $\pi$ )
    and  $t_i < t_a \wedge t_a < t_j$ 
    by auto
    then show ?thesis
    using  $\langle \text{consec-https } \pi s \ t_i \ t_j \rangle$  unfolding consec-https-def by auto
qed
qed

```

Proof that every happening of a induced happening sequence is in between the smallest and largest happening time point.

```

lemma ind-happ-seq-https-leq- $t_{min}$ - $t_{max}$ :
  fixes  $\pi s$ 
  assumes ind-happ-seq  $\pi s$  ( $hs'@hs$ )
    and  $\forall t_i. \text{is-htp } \pi s \ t_i \longrightarrow t_{min} \leq t_i$ 
    and  $\forall t_i. \text{is-htp } \pi s \ t_i \longrightarrow t_i \leq t_{max}$ 
  shows  $\forall (t_a, A) \in \text{set } hs. t_{min} \leq t_a \wedge t_a \leq t_{max}$ 
  using assms
proof (induction  $hs$  arbitrary:  $hs'$ )
  case Nil
  then show ?case
  by simp
next
  case (Cons  $h$   $hs$ )
  then show ?case
  proof (cases  $h$ )
    case (Pair  $t_a$   $A$ )
    then obtain  $a$  where  $a \in \text{set } A$ 
    using  $\langle \text{ind-happ-seq } \pi s \ (hs'@h\#hs) \rangle$  unfolding ind-happ-seq-def by (cases
  A) auto
    then have  $\exists (t_\pi, \pi) \in \text{set } \pi s. \text{inst-of-plan-action } \pi s \ (t_\pi, \pi) \ (t_a, a)$ 
    using  $\langle \text{ind-happ-seq } \pi s \ (hs'@h\#hs) \rangle$  Pair unfolding ind-happ-seq-def by
  auto
    then obtain  $t_\pi \ \pi$  where  $(t_\pi, \pi) \in \text{set } \pi s$  and  $\text{inst-of-plan-action } \pi s \ (t_\pi, \pi)$ 
  ( $t_a, a$ )
    by auto
    obtain  $hs''$  where  $hs'' = hs'@h$ 
    by simp
    then have ind-happ-seq  $\pi s$  ( $hs''@hs$ )
    using  $\langle \text{ind-happ-seq } \pi s \ (hs'@h\#hs) \rangle$  by simp
    show ?thesis
    using assms Pair Cons.IH[OF  $\langle \text{ind-happ-seq } \pi s \ (hs''@hs) \rangle$ ]
      happ-tp-leq- $t_{min}$ - $t_{max}$ [OF  $\langle (t_\pi, \pi) \in \text{set } \pi s \rangle \langle \text{inst-of-plan-action } \pi s$ 
  ( $t_\pi, \pi$ ) ( $t_a, a$ ) $\rangle$ ]
    by auto
  qed
qed

```

```

lemma valid-happ-seq-state-unique:
  assumes valid-happ-seq  $M_0$   $hs$   $M_i$  and valid-happ-seq  $M_0$   $hs$   $M_i'$ 

```

shows $M_i = M_i'$
using *assms* **by** (*induction* M_0 *hs* M_i *rule: valid-happ-seq.induct*) *auto*

lemma *valid-happ-seq-iff-valid-state-seq*:
fixes πs
assumes *https-seq* πs *https*
and *wf-plan* πs
and *ind-happ-seq* πs *hs*
shows *valid-happ-seq* M_0 *hs* $M_n \longleftrightarrow$ *valid-state-seq* M_0 *https* πs M_n
proof (*cases* $\pi s = []$)
case *True*
then have $hs = []$ **and** $https = []$
using *assms* *ind-happ-seq-Nil* *https-seq-Nil-iff* **by** *auto*
then show *?thesis*
by *auto*
next
case *False*

have *strict-sorted* (*map fst hs*)
using $\langle \text{ind-happ-seq } \pi s \text{ } hs \rangle$ **unfolding** *ind-happ-seq-def* **by** *auto*
have $https \neq []$ **and** *sorted* *https* **and** *distinct* *https* **and** *strict-sorted* *https*
using *assms* $\langle \pi s \neq [] \rangle$ *https-seq-Nil-iff* **unfolding** *https-seq-def*
by (*auto simp: strict-sorted-iff*)

Obtain the smallest happening time point.

obtain t_{min} **where** $t_{min} = hd \text{ } https$
by *auto*
then have $t_{min} \in set \text{ } https$
using $\langle https \neq [] \rangle$ **by** *auto*
have *is-htp* πs t_{min} **and** $\forall t_k. \text{is-htp } \pi s \text{ } t_k \longrightarrow t_{min} \leq t_k$
using *assms* $\langle https \neq [] \rangle$ $\langle \text{sorted } https \rangle$ $\langle t_{min} = hd \text{ } https \rangle$ *sorted-hd-min*
by (*auto simp: https-seq-def*)
have $\forall (t_\pi, \pi) \in set \text{ } \pi s. t_{min} \leq t_\pi$
using $\langle \forall t_k. \text{is-htp } \pi s \text{ } t_k \longrightarrow t_{min} \leq t_k \rangle$ **by** (*auto simp: is-htp-def*)

Obtain the largest happening time point.

obtain t_{max} **where** $t_{max} = last \text{ } https$
by *auto*
then have $t_{max} \in set \text{ } https$
using $\langle https \neq [] \rangle$ **by** *auto*
then have *is-htp* πs t_{max} **and** $\forall t_k. \text{is-htp } \pi s \text{ } t_k \longrightarrow t_k \leq t_{max}$
using *assms* *non-Nil-sorted-last-max* [*OF* $\langle https \neq [] \rangle$ $\langle \text{sorted } https \rangle$ $\langle t_{max} = last \text{ } https \rangle$]
by (*auto simp: https-seq-def*)

have $\forall (t_a, A) \in set \text{ } hs. t_{min} \leq t_a \wedge t_a \leq t_{max}$
using *assms* $\langle \forall t_k. \text{is-htp } \pi s \text{ } t_k \longrightarrow t_k \leq t_{max} \rangle$ $\langle \forall t_k. \text{is-htp } \pi s \text{ } t_k \longrightarrow t_{min} \leq t_k \rangle$
ind-happ-seq-https-leq- t_{min} - t_{max} [**where** $hs' = []$]

by auto

have takeWhile (leq t_{max}) hs = hs
 using $\langle \forall (t_a, A) \in \text{set } hs. t_{\min} \leq t_a \wedge t_a \leq t_{\max} \rangle$ by (auto simp: leq-def)
 have dropWhile (le t_{min}) (takeWhile (leq t_{min}) hs) = takeWhile (leq t_{min}) hs
 using $\langle \forall (t_a, A) \in \text{set } hs. t_{\min} \leq t_a \wedge t_a \leq t_{\max} \rangle$ $\langle \text{strict-sorted (map fst } hs) \rangle$
 by (induction hs) (auto simp: le-def)

obtain as_{min} where dropWhile (le t_{min}) (takeWhile (leq t_{min}) hs) = [(t_{min}, as_{min})]
 using assms ind-happ-seq-le_i-leq_i[OF $\langle \text{is-htp } \pi s \text{ } t_{\min} \rangle$] by auto
 then have (t_{min}, as_{min}) ∈ set hs
 by (metis list.set-intros(1) set-dropWhileD set-takeWhileD)
 then have set as_{min} = acts-of-plan-at t_{min} π s
 using assms $\langle \text{is-htp } \pi s \text{ } t_{\min} \rangle$ ind-happ-seq-htp-acts-of-plan-at by auto
 let ?as = acts-of-plan-at t_{min} π s

have t_{min} ≤ t_{max}
 using $\langle \text{is-htp } \pi s \text{ } t_{\min} \rangle$ by (simp add: $\langle \forall t_k. \text{is-htp } \pi s \text{ } t_k \longrightarrow t_k \leq t_{\max} \rangle$)

show valid-happ-seq M₀ hs M_n $\longleftrightarrow$ valid-state-seq M₀ htps π s M_n

proof

assume valid-happ-seq M₀ hs M_n

Remove first happening from the happening sequence.

then have $\exists M_1. \text{valid-happ-seq } M_0 \text{ (takeWhile (leq } t_{\min}) \text{ } hs) \text{ } M_1$
 $\wedge \text{valid-happ-seq } M_1 \text{ (dropWhile (leq } t_{\min}) \text{ } hs) \text{ } M_n$
 using valid-happ-seq-app-iff[where hs₁ = takeWhile (leq t_{min}) hs
 and hs₂ = dropWhile (leq t_{min}) hs]
 by auto
 then obtain M₁ where valid-happ-seq M₀ (dropWhile (le t_{min}) (takeWhile
 (leq t_{min}) hs)) M₁
 and valid-happ-seq M₁ (dropWhile (leq t_{min}) (takeWhile (leq
 t_{max}) hs)) M_n
 using $\langle \text{dropWhile (le } t_{\min}) \text{ (takeWhile (leq } t_{\min}) \text{ } hs) = takeWhile (leq } t_{\min})$
 hs $\rangle$
 $\langle \text{takeWhile (leq } t_{\max}) \text{ } hs = hs \rangle$ [symmetric]
 by auto

have invs-of-plan-at t_{min} π s = {}
 unfolding invs-of-plan-at-def using $\langle \forall (t_\pi, \pi) \in \text{set } \pi s. t_{\min} \leq t_\pi \rangle$
 by (auto simp: durative-acts-def)
 then have valid-state-seq M₀ [t_{min}] π s M₁
 using $\langle \text{valid-happ-seq } M_0 \text{ (dropWhile (le } t_{\min}) \text{ (takeWhile (leq } t_{\min}) \text{ } hs))} \rangle$
 M₁ $\rangle$
 $\langle \text{dropWhile (le } t_{\min}) \text{ (takeWhile (leq } t_{\min}) \text{ } hs) = [(t_{\min}, \text{as}_{\min})] \rangle$
 $\langle \text{set as}_{\min} = \text{acts-of-plan-at } t_{\min} \text{ } \pi s \rangle$
 by (auto simp add: pairwise-def)

have t_{min} = t_{max} ∨ t_{min} < t_{max}

```

    using  $\langle t_{min} \leq t_{max} \rangle$  by auto
  then show valid-state-seq  $M_0$  https  $\pi s$   $M_n$ 
proof
  assume  $t_{min} = t_{max}$ 
  then have valid-happ-seq  $M_0$  hs  $M_1$ 
    using  $\langle \text{dropWhile } (le\ t_{min})\ (\text{takeWhile } (leq\ t_{min})\ hs) = \text{takeWhile } (leq\ t_{min})\ hs \rangle$ 
     $\langle \text{valid-happ-seq } M_0\ (\text{dropWhile } (le\ t_{min})\ (\text{takeWhile } (leq\ t_{min})\ hs))$ 
     $M_1 \rangle$ 
    by (simp add:  $\langle \text{takeWhile } (leq\ t_{max})\ hs = hs \rangle$ )
  then have  $M_1 = M_n$ 
  using valid-happ-seq-state-unique[OF  $\langle \text{valid-happ-seq } M_0\ hs\ M_n \rangle \langle \text{valid-happ-seq } M_0\ hs\ M_1 \rangle$ ]
    by auto
  have https =  $[t_{min}]$ 
    using  $\langle https \neq [] \rangle \langle \text{strict-sorted } https \rangle \langle t_{min} = \text{hd } https \rangle \langle t_{max} = \text{last } https \rangle$ 
     $\langle t_{min} = t_{max} \rangle$ 
    by (induction https) (auto split: if-splits)
  then show ?thesis
    using  $\langle \text{valid-state-seq } M_0\ [t_{min}]\ \pi s\ M_1 \rangle \langle M_1 = M_n \rangle$  by auto
next
  assume  $t_{min} < t_{max}$ 
  then have  $\exists \text{https}'.\ https = [] @ t_{min} \# \text{https}' @ t_{max} \# []$ 
    using  $\langle https \neq [] \rangle \langle t_{min} = \text{hd } https \rangle \langle t_{max} = \text{last } https \rangle$ 
    apply (induction https)
    apply (auto split: if-splits)
    apply (metis append-butlast-last-id)
    done
  then obtain https' where https =  $[] @ t_{min} \# \text{https}' @ t_{max} \# []$ 
    by auto
  then have https-seq  $\pi s$  ( $[] @ t_{min} \# \text{https}' @ t_{max} \# []$ )
    using assms by auto
  then have valid-state-seq  $M_1$  ( $\text{https}' @ [t_{max}]$ )  $\pi s$   $M_n$ 
    using assms  $\langle \text{valid-happ-seq } M_1\ (\text{dropWhile } (leq\ t_{min})\ (\text{takeWhile } (leq\ t_{max})\ hs))\ M_n \rangle$ 
    valid-happ-seq-valid-state-seq-i-to-j-aux
    [OF  $\langle \text{https-seq } \pi s\ ([] @ t_{min} \# \text{https}' @ t_{max} \# []) \rangle$ ]
    by blast
  then show ?thesis
    using  $\langle \text{valid-state-seq } M_0\ [t_{min}]\ \pi s\ M_1 \rangle \langle https = [] @ t_{min} \# \text{https}' @ t_{max} \# [] \rangle$ 
    #  $[] \rangle$ 
    valid-state-seq-app-iff[where  $ts_1=[t_{min}]$  and  $ts_2=https' @ [t_{max}]$ ]
    by auto
qed
next
  assume valid-state-seq  $M_0$  https  $\pi s$   $M_n$ 

```

Remove first happening from the happening sequence.

```

  then obtain  $M_1$  where valid-state-seq  $M_0\ [t_{min}]\ \pi s\ M_1$  and valid-state-seq

```

```

M1 (tl https) πs Mn
  using ⟨tmin = hd https⟩ valid-state-seq-app-iff[where ts1=[tmin] and ts2=tl
https]
  by (auto simp: ⟨https ≠ []⟩)
then have valid-happ-seq M0 (dropWhile (le tmin) (takeWhile (leq tmin) hs))
M1
  using ⟨valid-state-seq M0 [tmin] πs M1⟩
    ⟨dropWhile (le tmin) (takeWhile (leq tmin) hs) = [(tmin, asmin)]⟩
    ⟨set asmin = acts-of-plan-at tmin πs⟩
  by (auto simp: Let-def pairwise-def)

have tmin = tmax ∨ tmin < tmax
  using ⟨tmin ≤ tmax⟩ by auto
then show valid-happ-seq M0 hs Mn
proof
  assume tmin = tmax
  have https = [tmin]
    using ⟨https ≠ []⟩ ⟨strict-sorted https⟩ ⟨tmin = hd https⟩ ⟨tmax = last https⟩
  ⟨tmin = tmax⟩
  by (induction https) (auto split: if-splits)
  then have M1 = Mn
    using ⟨valid-state-seq M0 [tmin] πs M1⟩
      valid-state-seq-state-unique [OF ⟨valid-state-seq M0 https πs Mn⟩]
    by (auto simp add: ⟨takeWhile (leq tmax) hs = hs⟩)
  then show ?thesis
    using ⟨dropWhile (le tmin) (takeWhile (leq tmin) hs) = takeWhile (leq
tmin) hs⟩
      ⟨valid-happ-seq M0 (dropWhile (le tmin) (takeWhile (leq tmin) hs))
M1⟩
      ⟨tmin = tmax⟩
    by (simp add: ⟨takeWhile (leq tmax) hs = hs⟩)
next
  assume tmin < tmax
  then have ∃ https'. https = [] @ tmin # https' @ tmax # []
    using ⟨https ≠ []⟩ ⟨tmin = hd https⟩ ⟨tmax = last https⟩
  apply (induction https)
  apply (auto split: if-splits)
  apply (metis append-butlast-last-id)
  done
  then obtain https' where https = [] @ tmin # https' @ tmax # []
    by auto
  then have https-seq πs ([] @ tmin # https' @ tmax # [])
    using assms by auto
  then have valid-happ-seq M1 (dropWhile (leq tmin) (takeWhile (leq tmax)
hs)) Mn
    using assms ⟨valid-state-seq M1 (tl https) πs Mn⟩ ⟨https = [] @ tmin #
https' @ [tmax]]⟩
      valid-happ-seq-valid-state-seq-i-to-j-aux
      [OF ⟨https-seq πs ([] @ tmin # https' @ tmax # [])⟩]

```

```

    by fastforce
  then show ?thesis
    using valid-happ-seq-app-iff[where  $hs_1 = \text{takeWhile } (\text{leq } t_{min}) \text{ } hs$ 
      and  $hs_2 = \text{dropWhile } (\text{leq } t_{min}) \text{ } hs$ ]
      ⟨valid-happ-seq  $M_0$  ( $\text{dropWhile } (\text{le } t_{min}) (\text{takeWhile } (\text{leq } t_{min}) \text{ } hs)$ )  $M_1$ ⟩
      ⟨takeWhile  $(\text{leq } t_{max}) \text{ } hs = hs$ ⟩
      ⟨dropWhile  $(\text{le } t_{min}) (\text{takeWhile } (\text{leq } t_{min}) \text{ } hs) = \text{takeWhile } (\text{leq } t_{min})$ 
 $hs$ ⟩
    by auto
  qed
qed
qed

```

```

lemma (in wf-ast-problem) plan-happ-path-iff-valid-state-seq:
  assumes wf-plan  $\pi s$  and https-seq  $\pi s$  https
  shows plan-happ-path  $M \pi s M' \longleftrightarrow \text{valid-state-seq } M \text{ https } \pi s M'$ 
  unfolding plan-happ-path-def
  using assms ind-happ-seq-exists valid-happ-seq-iff-valid-state-seq by blast

```

```

lemmas plan-happ-path-iff-valid-state-seq-iff'
= wf-ast-problem.plan-happ-path-iff-valid-state-seq[of  $P$ , OF wf-ast-problem.intro]

```

```

lemma strict-sorted  $xs \implies \text{strict-sorted } ys \implies \text{set } xs = \text{set } ys \implies xs = ys$ 
  by (simp add: strict-sorted-iff sorted-distinct-set-unique)

```

Proof that a sequence of happening time points is unique for every plan

```

lemma https-seq-unique:
  assumes https-seq  $\pi s$  https1
    and https-seq  $\pi s$  https2
  shows https1 = https2
proof -
  have set https1 = set https2
    using assms unfolding https-seq-def by auto
  then show ?thesis
    using assms unfolding https-seq-def by (simp add: strict-sorted-iff sorted-distinct-set-unique)
qed

```

```

lemma valid-plan-from2-iff:
  assumes wf-problem
  shows valid-plan-from2  $I \pi s \longleftrightarrow \text{valid-plan-from } I \pi s$ 
  unfolding valid-plan-from-def valid-plan-from2-def
  using https-seq-exists https-seq-unique
    plan-happ-path-iff-valid-state-seq-iff'[OF assms]
  by blast

```

```

lemma
  assumes wf-problem
  shows valid-plan2  $\pi s \longleftrightarrow \text{valid-plan } \pi s$ 

```

unfolding *valid-plan-def valid-plan2-def*
using *valid-plan-from2-iff*[*OF assms*] **by** *blast*

end — Context of *ast-problem*

end — Theory

References

- [1] M. Abdulaziz and L. Koller. Formal semantics and formally verified validation for temporal planning. In *The 36th AAAI Conference on Artificial Intelligence (AAAI)*, 2022.
- [2] M. Abdulaziz and P. Lammich. AI Planning Languages Semantics. *Arch. Formal Proofs*, 2020.